

TCP/IP Illustrated, Volume 1

The Protocols
SECOND EDITION

Kevin R. Fall
W. Richard Stevens



The Echo Request message is sent to the address 10.0.0.127. The IPv4 implementation determines this to be the subnet-directed broadcast address by consulting information in the local routing table and interface configuration information, and it sends the datagram using the link-layer broadcast address ff:ff:ff:ff:ff:ff, so no ARP request is needed to determine the MAC addresses for each destination. In fact, the sender is unaware of what hosts will respond until they do. It knows only that 10.0.0.127 is a broadcast address and that it should therefore use a broadcast link-layer destination address when sending. The source addresses at both the IP and link layers are entirely conventional unicast; multicast addresses are used only as destination addresses.

In this particular example, notice that each of the responses generated is directed at 10.0.0.13, the unicast address of the original sender, and that each response includes the IPv4 address of the responder: 10.0.0.6, 10.0.0.113, and 10.0.0.120. This is a simple example of a more general principle: broadcast addressing (and multicast addressing, as we shall see shortly) can be used to discover systems or services that are otherwise unknown. In this example, the outgoing broadcast ping request discovered three hosts that are willing to respond to broadcast Echo Request messages.

9.2.2 Sending Broadcast Datagrams

Generally speaking, applications using broadcast use the UDP protocol (or ICMPv4 protocol) and invoke an ordinary set of API calls to send traffic. The only exception is that when invoking the API calls, a special flag (SO_BROADCAST) is used in some operating systems to indicate that the application really does intend to send broadcast datagrams. For example, in Linux, failing to use the `-b` flag when attempting to do a broadcast ping causes the following output:

```
Linux% ping 10.0.0.127
Do you want to ping broadcast? Then -b
```

This error is caused because the SO_BROADCAST flag is provided through the API only when the `-b` option is provided in the command line. This helps to avoid accidentally generating broadcast traffic that could temporarily congest a network.

To determine which interfaces are used for broadcasting, the IPv4 forwarding table (called `routing table` here) is consulted. The following is an example of a Windows Vista routing table (later versions of Windows use an identical format) showing the interface list and broadcast-related routing information (other information has been removed for clarity):

```
C:\> netstat -rn
=====
Interface List
 10 ...02 00 4c 4f 50 ..... Microsoft Loopback Adapter
   9 ...00 13 02 20 b9 18 ..... Intel(R) PRO/Wireless 3945ABG Network
                               Connection
```


15.7 Attacks Involving Window Management

The window management procedures for TCP have been the subject of various attacks, primarily forms of resource exhaustion. In essence, advertising a small window slows a TCP transfer, tying up resources such as memory for a potentially long time. This has been used as a form of attack on bad traffic (i.e., worms). The LaBrea tarpit [L01], for example, arranges to complete the TCP three-way handshake and then either does nothing or produces minimal responses that simply cause the sending TCP to continually slow down. This keeps the sending TCP busy and essentially slows down worm propagation. Tarpits are thus attacks on attacking traffic.

A more recent attack was published in 2009 [I09], based on a known vulnerability of the persist timer. It uses a client-side variety of the “SYN cookies” technique (see Chapter 13). All the necessary connection state can thus be offloaded onto the victim machine, minimizing the amount of resources consumed at the attacker’s machine. The attack itself is similar to the LaBrea idea, except it focuses specifically on the persist timer. Multiple such attacks can be mounted on the same server, which can lead to resource exhaustion (e.g., running out of system memory). The “solution” to this attack, as suggested by [C723308], is to allow some other process to terminate TCP connections when resource exhaustion appears to be taking place.

15.8 Summary

Interactive data is normally transmitted in segments smaller than the SMSS. Delayed acknowledgments may be used by the receiver of these small segments to see if the acknowledgment can be piggybacked along with data going back to the sender. This often reduces the number of segments, especially for interactive traffic, where the server is echoing the characters typed at the client. However, it may introduce additional delay.

On connections with relatively large round-trip times, such as WANs, the Nagle algorithm is often used to reduce the number of small segments. This algorithm limits the sender to a single small packet of unacknowledged data at any time. While this can reduce the number of high-overhead small packets in the network and reduce the total number of packets carried during a connection, it adds delay that is sometimes unacceptable to applications. In addition, the interaction between delayed ACKs and the Nagle algorithm can lead to an undesirable form of temporary deadlock. Because of these issues, the Nagle algorithm can be disabled by applications, and most interactive applications take advantage of this capability.

TCP implements flow control by including a window advertisement on every ACK it sends. Such window advertisements signal the peer TCP how much buffer space is left at the endpoint that sent the window advertisement ACK. The

maximum window advertisement is 65,535 bytes unless the Window Scale TCP option is used. In that case, the maximum window advertisement can be much larger (about 1GB).

The window advertisement can be as small as 0 bytes, indicating that the receiver is completely full. When this happens, the sender stops sending data and instead begins probing the closed window using a retransmission interval with a backoff scheme similar to timer-based retransmissions (see Chapter 14). This probing of the closed window continues indefinitely, until either an ACK is returned indicating a larger window or the receiver sends an unsolicited window advertisement (a window update) because buffer space has become available. This indefinite behavior has been used to create a resource exhaustion attack against TCP.

During the development of TCP, a curious phenomenon was observed. When a small window was advertised, the sender would immediately fill it. This behavior, which causes the connection to use a large number of high-overhead small packets, would continue until the connection became idle and was dubbed “silly window syndrome.” Techniques were created to avoid it, applying to both the TCP send and receive logic. The sender avoids sending small segments when a small window is advertised; receivers try to avoid ever sending small window advertisements.

The size of the receiver's window is limited by the size of the receiver's buffer. Historically, applications that failed to specify their receive buffers would be allocated a relatively small buffer that would cause throughput performance to suffer over network paths with high bandwidth and high delay. In more recent operating systems, auto-tuning sets the buffer size allocated automatically in an efficient way, causing such concerns to largely be a thing of the past.

15.9 References

[C723308] US-CERT Vulnerability Note VU#723308, Nov. 2009.

[F03] C. Fraleigh et al., “Packet-Level Traffic Measurements from the Sprint IP Backbone,” IEEE Network Magazine, Nov./Dec. 2003.

[I09] F. Hantzis (ithilgore), “Exploiting TCP and the Persist Timer Infiniteness,” Phrack, 66(9), June 2009.

[L01] T. Liston, “LaBrea: ‘Sticky’ Honeypot and IDS,” <http://labrea.sourceforge.net>

[MM01] J. Mogul and G. Minshall, “Rethinking the TCP Nagle Algorithm,” ACM Computer Communication Review, 31(6), Jan. 2001.

[MMQ] <http://technet.microsoft.com/en-us/library/cc730960.aspx>

- [MMSV99] G. Minshall, J. Mogul, Y. Saito, and B. Verghese, "Application Performance Pitfalls and TCP's Nagle Algorithm," Proc. Workshop on Internet Server Performance, May 1999.
- [P05] R. Pang, M. Allman, M. Bennett, J. Lee, V. Paxson, and B. Tierney, "A First Look at Modern Enterprise Traffic," Proc. Internet Measurement Conference, Oct. 2005.
- [RFC0813] D. Clark, "Window and Acknowledgment Strategy in TCP," Internet RFC 0813, July 1982.
- [RFC0896] J. Nagle, "Congestion Control in IP/TCP Internetworks," Internet RFC 0896, Jan. 1984.
- [RFC1122] R. Braden, ed., "Requirements for Internet Hosts—Communication Layers," Internet RFC 1122/STD 0003, Oct. 1989.
- [RFC2675] D. Borman, S. Deering, and R. Hinden, "IPv6 Jumbograms," Internet RFC 2675, Aug. 1999.
- [RFC4251] T. Ylonen and C. Lonvick, "The Secure Shell (SSH) Protocol Architecture," Internet RFC 4251, Jan. 2006.
- [RFC4254] T. Ylonen and C. Lonvick, ed., "The Secure Shell (SSH) Connection Protocol," Internet RFC 4254, Jan. 2006.
- [RFC6093] F. Gont and A. Yourtchenko, "On the Implementation of the TCP Urgent Mechanism," Internet RFC 6093, Jan. 2011.
- [S98] J. Semke, J. Mahdavi, and M. Mathis, "Automatic TCP Buffer Tuning," Proc. ACM SIGCOMM, Oct. 1998.
- [W100] <http://www.web100.org>

This page intentionally left blank

16

TCP Congestion Control

16.1 Introduction

In this chapter we investigate how TCP approaches the issue of congestion control, which is most important in the context of bulk data transfers. Congestion control is a set of behaviors determined by algorithms that each TCP implements in an attempt to prevent the network from being overwhelmed by too large an aggregate offered traffic load. The basic approach is to have TCP slow down when it has reason to believe the network is about to be congested (or is already so congested that routers are discarding packets). The challenge is to determine exactly when and how TCP should slow down, and when it can speed up again.

TCP is a protocol designed to provide reliable delivery of data from one system to another. We have already seen in Chapter 15 how a sending TCP can be made to slow down if its peer (receiving) TCP cannot keep up. This is accomplished by TCP's procedures for flow control and is realized by a sender adapting its sending rate based on the advertised Window Size field provided by a receiver in its ACKs. This provides explicit information about the state of the receiver back to the sender and allows it to avoid overrunning the receiver's buffers.

Consider what happens when the network between a collection of senders and receivers is asked to carry more traffic than it can handle. Either the senders must slow down or the network must ultimately throw some data away (or some combination thereof). This fact arises from the most basic observation from queuing theory as applied at a router: even if the router can store some data, if the long-term data arrival rate exceeds the long-term departure rate, any amount of intermediate storage will grow without bound. Stated more simply, if a router receives more data per unit time than it can send out, it must store that data. If this situation persists, eventually the storage will run out and the router will be forced to drop some of the data.

This situation, when a router is forced to discard data because it cannot handle the arriving traffic rate, is called *congestion*. The router is said to be *congested* when it is in this state, and even a single connection can drive one or more routers into

congestion. Left unaddressed, congestion can cause the performance of a network to be reduced so badly that it becomes unusable. In the very worst cases, it is said to be in a state of congestion collapse. To either avoid or at least react effectively to mitigate this situation, each TCP implements congestion control procedures. Different versions or variants of TCP (and the operating systems that host the TCP/IP stack) have somewhat different procedures and behaviors. We will discuss most of the better-known ones in this chapter.

16.1.1 Detection of Congestion in TCP

As we have seen, the primary mechanism TCP has available to combat packet loss is retransmission, induced either by a retransmission timer expiring, or by the fast retransmit algorithm (see Chapter 14). Consider, for a moment, the consequence of many TCP connections that share an Internet path simply retransmitting more packets while the network is in a state of congestion collapse. As you can imagine, this only makes the situation worse. It has been called the analog of pouring gasoline on a fire and is something to be avoided.

In order to deal with congestion, we would like to have sending TCPs slow down when congestion is present (or about to be) and, if the congestion has subsided, detect and use an appropriate amount of new bandwidth when it becomes available. In the Internet, this can be quite challenging, as there has traditionally been no explicit way for a sending TCP to learn about the state of the intermediate routers. In other words, there is no explicit signaling about congestion. Instead, if a typical TCP is to react somehow to congestion, it must first conclude that congestion is occurring. This is usually accomplished by detecting that one or more packets have been lost. In TCP, an assumption is made that a lost packet is an indicator of congestion, and that some response (i.e., slowing down in some way) is required. We shall see that TCP has been this way since the late 1980s. Other methods for detecting congestion, including measuring delay and network-supported Explicit Congestion Notification (ECN), which we discuss in Section 16.11, allow TCP to learn about congestion even before it has become so bad as to cause dropped packets. We discuss these approaches after studying the “classic” algorithms.

Note

In today’s wired networks, packet loss is caused primarily by congestion in routers or switches. With wireless networks, transmission and reception errors become a significant cause of packet loss. Determining whether loss is due to congestion or transmission errors has been an active research topic since the mid-1990s when wireless networks started to attain widespread use.

In Chapter 14 we saw how TCP can use timers, acknowledgments, and selective acknowledgments to detect and recover from dropped packets. When packets are detected as lost, it is TCP’s responsibility to resend them. We are now concerned

with what else TCP does when it observes a lost packet. In particular, we are interested in how it interprets this as a signal that congestion has occurred, and that it should slow down. Just how it slows down and when (and how it speeds back up again) are the main subjects of the following sections. We begin with the classic algorithm used on a new connection to establish the base data rate and continue with another classic algorithm that is used by TCP during its steady-state operation when performing large data transfers. We will also incorporate the recommended variations on these algorithms into the discussion and discuss other modifications that have been made over the years. We will also examine an extended trace in detail. We conclude with a discussion of some of the security issues related to TCP congestion control and summarize the most important points. The area of congestion control has been a fertile area for networking researchers [RFC6077], and several new papers on this subject tend to appear each year.

16.1.2 Slowing Down a TCP Sender

One detail we need to address right away is just how to slow down a TCP sender. We saw in Chapter 15 that the Window Size field in the TCP header is used to signal a sender to adjust its window based on the availability of buffer space at the receiver. We can go a step further and arrange for the sender to slow down if either the receiver is too slow or the network is too slow. This is accomplished by introducing a window control variable at the sender that is based on an estimate of the network's capacity and ensuring that the sender's window size never exceeds the minimum of the two. In effect, a sending TCP then sends at a rate equal to what the receiver or the network can handle, whichever is less.

The new value used to hold the estimate of the network's available capacity is called the congestion window, written more compactly as simply `cwnd`. The sender's actual (usable) window `W` is then written as the minimum of the receiver's advertised window `awnd` and the congestion window:

$$W = \min(\text{cwnd}, \text{awnd})$$

With this relationship, the TCP sender is not permitted to have more than `W` unacknowledged packets or bytes outstanding in the network. The total amount of data a sender has introduced into the network for which it has not yet received an acknowledgment is sometimes called the flight size, which is always less than or equal to `W`. In general, `W` can be maintained in either packet or byte units.

Note

When TCP does not make use of selective acknowledgment, the restriction on `W` means that the sender is not permitted to send a segment with a sequence number greater than the sum of the highest acknowledged sequence number and the value of `W`. A SACK TCP sender treats `W` somewhat differently, using it as an overall limit to the flight size.

This all seems logical but is far from the whole story. Because both the state of the network and the state of the receiver change with time, the values of both `awnd` and `cwnd` change over time. In addition, because of the lack of explicit signals (see the preceding section), the “correct” value of `cwnd` is generally not directly available to the sending TCP. Thus, all of the values `W`, `cwnd`, and `awnd` must be empirically determined and dynamically updated. In addition, as we said before, we do not want `W` to be too big or too small—we want it to be set to about the bandwidth-delay product (BDP) of the network path, also called the optimal window size. This is the amount of data that can be stored in the network in transit to the receiver. It is equal to the product of the RTT and the capacity of the lowest capacity (“bottleneck”) link on the path from sender to receiver. Generally, the sending strategy is to keep the network busy by arranging to have an amount of data at least as large as the BDP in the network. Using an outstanding limit that substantially exceeds the BDP, however, is usually undesirable as it can lead to unwanted delays (see Section 16.10). On the Internet, determining the BDP for a connection can be challenging, given that routes, delay, and the level of statistical multiplexing (i.e., sharing of capacity) change as a function of time.

Note

Although handling congestion at the TCP sender is our primary area of interest, work has been done on handling the cases where congestion occurs on the reverse path, because of ACKs. In [RFC5690] a method is introduced to inform a TCP receiver of the ACK ratio it should use (i.e., how many packets it should receive before sending an ACK).

16.2 The Classic Algorithms

When a new TCP connection first starts out, it usually has no idea what the initial value for `cwnd` should be, as it has no idea how much network capacity is available for it to send its data. (There are some exceptions, such as systems that cache performance values that were determined earlier. These were called destination metrics in Chapter 14.) TCP learns the value for `awnd` with one packet exchange to the receiver, but without any explicit signaling, the only obvious way it has to learn a good value for `cwnd` is to try sending data at faster and faster rates until it experiences a packet drop (or other congestion indicator). This could be accomplished by either sending immediately at the maximum rate it can (subject to the value of `awnd`), or it could start more slowly. Because of the detrimental effects on the performance of other TCP connections sharing the same network path that could be experienced when starting at full rate, a TCP generally uses one algorithm to avoid starting so fast when it starts up to get to steady state. It uses a different one once it is in steady state.

The operation of TCP congestion control at a sender is driven or “clocked” by the receipt of ACKs. If a TCP is operating at steady state (with an appropriate value

of $cwnd$), receipt of an ACK indicates that one or more packets have been removed from the network, and consequently that an opportunity to send more has arisen. Following this line of reasoning, the TCP congestion behavior in steady state attempts to achieve a conservation of packets in the network (see Figure 16-1). The term conservation here is used in the sense it is in physics—that some quantity (e.g., momentum, energy) going into a system does not simply disappear or appear but rather can be found as long as proper accounting is performed.

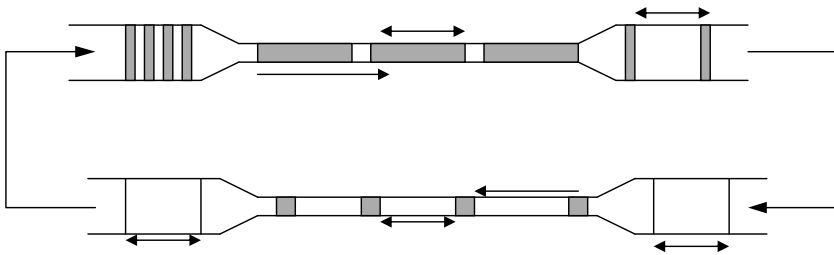


Figure 16-1 TCP congestion control operates on a principle of conservation of packets. Packets P_i are “stretched out” in time as they are sent from sender to receiver over links with constrained capacity. As they are received at the receiver spaced apart P_r , ACKs are generated (A_r), which return to the sender. ACKs traveling from receiver to sender become spaced out (A_b) in relation to the inter-packet spacing of the packets. When ACKs reach the sender (A_s), their arrivals provide a signal or “ACK clock,” used to tell the sender it is time to send more. In steady state, the overall system is said to be “self-clocked.” The figure is adapted from [J88] and copied from S. Seshan’s CMU Lecture Notes dated March 22, 2005.

This idea is illustrated in Figure 16-1. We shall call the top and bottom objects “funnels.” The top funnel holds (larger) data packets traveling along the path from the sender to the receiver. The comparatively narrow width of the funnel depicts how packets are “stretched out” in time as they travel through a relatively slow link. The ends of the funnels (at sender and receiver) show the queues where packets are held before or after they travel along the path. The bottom funnel holds the ACKs sent by the receiver back to the sender that correspond to the data packets in the top funnel. When operating efficiently at steady state, there are no bunches of packets in the top or bottom funnels. In addition, there is no significant extra space between packets in the top funnel. Note that an arrival of an ACK at the sender “liberates” another data packet to be sent into the top funnel, and that this happens at just the right time (i.e., when the network is able to accept another packet). This relationship is sometimes called self-clocking, because the arrival of an ACK, called the ACK clock, triggers the system to take the action of sending another packet.

We now turn to the main two algorithms of TCP: slow start and congestion avoidance. These algorithms, based on the principles of packet conservation and ACK clocking, were first formally described in the classic paper by Jacobson [J88].

An update to the congestion avoidance algorithm was given by Jacobson a couple of years later [J90]. These algorithms do not operate at the same time—TCP executes only one at any given time, but it may switch back and forth between the two. We now explore these in more detail and examine what determines when each of them is used. We also look at how they have been modified and extended since they were initially implemented. Each TCP connection is able to individually execute these algorithms.

16.2.1 Slow Start

The slow start algorithm is executed when a new TCP connection is created or when a loss has been detected due to a retransmission timeout (RTO). It may also be invoked after a sending TCP has gone idle for some time. The purpose of slow start is to help TCP find a value for `cwnd` before probing for more available bandwidth using congestion avoidance and to establish the ACK clock. Typically, a TCP begins a new connection in slow start, eventually drops a packet, and then settles into steady-state operation using the congestion avoidance algorithm (See [Section 16.2.2](#)). To quote from [RFC5681]:

Beginning transmission into a network with unknown conditions requires TCP to slowly probe the network to determine the available capacity, in order to avoid congesting the network with an inappropriately large burst of data. The slow start algorithm is used for this purpose at the beginning of a transfer, or after repairing loss detected by the retransmission timer.

A TCP begins in slow start by sending a certain number of segments (after the SYN exchange), called the initial window (IW). The value of IW was originally one SMSS, although with [RFC5681] it is allowed to be larger. The formula works as follows:

$IW = 2 * (SMSS)$ and not more than 2 segments (if $SMSS > 2190$ bytes)

$IW = 3 * (SMSS)$ and not more than 3 segments (if $2190 < SMSS < 1095$ bytes)

$IW = 4 * (SMSS)$ and not more than 4 segments (otherwise)

While this assignment for IW may allow several packets (e.g., three or four) in the initial window, we shall discuss the case where $IW = 1$ SMSS for simplicity. A TCP just starting out begins its connection, then, with `cwnd` = 1 SMSS, meaning the initial usable window `W` is also equal to SMSS. Note that in most cases SMSS is equal to the smaller of the receiver's MSS and the path MTU (less header sizes).

Assuming no packets are lost and each packet causes an ACK to be sent in response, an ACK is returned for the first segment, allowing the sending TCP to send another segment. However, slow start operates by incrementing `cwnd` by $\min(N, SMSS)$ for each good ACK received, where N is the number of previously

unacknowledged bytes ACKed by the received “good ACK.” A good ACK is one that returns a higher ACK number than has been seen so far.

Note

The number of bytes ACKed is used to support *Appropriate Byte Counting* (ABC) [RFC3465], an experimental specification recommended by [RFC5681]. It can be used to counter an “ACK division” attack, described in Section 16.12, where many small ACKs are used in an attempt to cause a TCP sender to send faster than normal. Linux uses the Boolean system configuration variable `net.ipv4.tcp_abc` to determine if ABC is enabled (default no). In recent versions of Windows, ABC defaults to on.

Thus, after one segment is ACKed, the `cwnd` value is ordinarily increased to 2, and two segments are sent. If each of those causes new good ACKs to be returned, 2 increases to 4, 4 to 8, and so on. In general, assuming no loss and an ACK for every packet, the value of W after k round-trip exchanges is $W = 2^k$. Rewriting, we can say that $k = \log_2 W$ RTTs are required to reach an operating window of W . This growth seems quite “fast” (increasing as an exponential function) but is still “slower” than what TCP would do if it were allowed to send immediately a window of packets equal in size to the receiver’s advertised window. (Recall that W is still never allowed to exceed `awnd`.)

If we imagine a TCP connection where the receiver’s advertised window is very large (say, infinitely large), `cwnd` is the primary governor of the sending rate (provided there is something for the sender to send). As we saw, this value grows exponentially fast in the RTT of the connection. So, eventually, `cwnd` (and thus W) could become so large that the corresponding window of packets sent overwhelms the network (recall that TCP’s throughput rate is proportional to W/RTT). When this happens, `cwnd` is reduced substantially (to half of its former value). In addition, this is the point at which TCP switches from operating in slow start to operating in congestion avoidance. The switch point is determined by the relationship between `cwnd` and a value called the slow start threshold (or `ssthresh`).

Figure 16-2 (left) illustrates the operation of slow start. The numbers are in units of the RTT of the connection. Assuming the connection starts out with one packet (top), one ACK is returned, allowing two packets to be sent during the second RTT. These packets cause two ACKs to be returned. The TCP sender increments `cwnd` by one segment for each ACK returned, so the process continues. The exponential growth of `cwnd` as a function of time is illustrated on the right. The second line shows how `cwnd` grows when every other packet is acknowledged, which is common when delayed ACKs are being used. In this case, the growth is still exponential but not as rapid. For this reason, some TCPs arrange to delay ACKs only after the connection has completed slow start. In Linux, this is called quick acknowledgments (“quickack mode”) and has been part of the basic TCP/IP stack since kernel version 2.4.4.

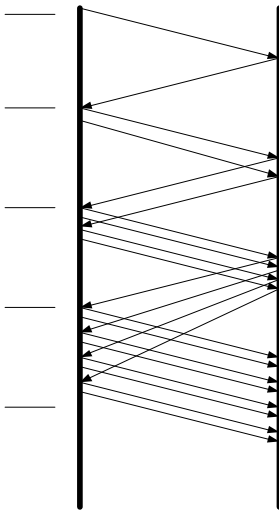


Figure 16-2 Operation of the classic slow start algorithm. In the simple case where ACKs are not delayed, every arriving good ACK allows the sender to inject two new packets (left). This leads to an exponential growth in the size of the sender's window as a function of time (right, upper line). When ACKs are delayed, such as when an ACK is produced for every other packet, the growth is still exponential but slower (right, lower line).

16.2.2 Congestion Avoidance

Slow start, just described, is used when initiating data flow across a connection or after a loss event invoked by a timeout. It increases `sswnd` fairly rapidly and helps to establish a value for `ssthresh`. Once this is achieved, there is always the possibility that more network capacity may become available for a connection. If such capacity were to be immediately used with large traffic bursts, other TCP connections with packets sharing the same queues in routers would likely experience significant packet drops, leading to overall instability in the network as many connections simultaneously experience packet drops and react with retransmissions.

To address the problem of trying to find additional capacity that may become available, but to not do so too aggressively, TCP implements the congestion avoidance algorithm. Once `ssthresh` is established and `cwnd` is at least at this level, a TCP runs the congestion avoidance algorithm, which seeks additional capacity by increasing `cwnd` by approximately one segment for each window's worth of data that is moved from sender to receiver successfully. This provides a much slower growth rate than slow start: approximately linear in terms of time, as opposed to slow start's exponential growth. More precisely, `cwnd` is usually updated as follows for each received nonduplicate ACK:

$$cwnd_{t+1} = cwnd_t + \text{SMSS} * \text{SMSS} / cwnd_t$$

Looking at this relationship briefly, assume $cwnd_0 = k \cdot \text{SMSS}$ bytes were sent into the network in k segments. After the first ACK arrives, $cwnd$ is updated to be larger by a factor of $(1/k)$:

$$cwnd_1 = cwnd_0 + \text{SMSS} \cdot \text{SMSS} / cwnd_0 = k \cdot \text{SMSS} + \text{SMSS} \cdot (\text{SMSS} / (k \cdot \text{SMSS})) = k \cdot \text{SMSS} + (1/k) \cdot \text{SMSS} = (k + (1/k)) \cdot \text{SMSS} = cwnd_0 + (1/k) \cdot \text{SMSS}$$

Because the value of $cwnd$ grows slightly with each new ACK arrival, and this value is in the denominator of the expression in the first equation above, the overall growth rate of $cwnd$ is slightly sublinear. Nonetheless, we generally think of congestion avoidance growing the window linearly with respect to time (Figure 16-3), whereas slow start grows it exponentially with respect to time (Figure 16-2). This function is also called additive increase because a particular value (about one packet in this case) is added to $cwnd$ for each successfully received window's worth of data.

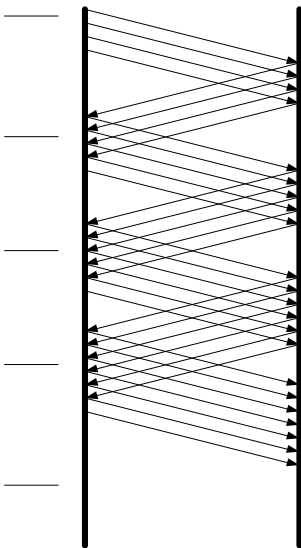


Figure 16-3 Operation of the congestion avoidance algorithm. In the simple case where ACKs are not delayed, every arriving good ACK allows the sender to inject approximately $1/W$ fraction of a new packet. This leads to approximately linear growth in the size of the sender's window as a function of time (right, upper line). When ACKs are delayed, such as when an ACK is produced for every other packet, the growth is still approximately linear but somewhat slower (right, lower line).

Figure 16-3 (left) illustrates the operation of congestion avoidance. Once again, the numbers are in units of the RTT of the connection. Assuming the connection sends four packets (top), four ACKs are returned, allowing $cwnd$ to grow slightly. By the second RTT period, the growth is enough to overcome the integer rounding and cause an increase of one SMSS to $cwnd$, allowing one additional packet to be

sent. The growth of *cwnd* as a nearly linear function of time is illustrated on the right, on a linear-linear plot. The second line to the right shows how *cwnd* grows when every other packet is acknowledged, simulating the use of delayed ACKs. In this case, the growth is still about linear, but not as rapid.

The assumption of the algorithm is that packet loss caused by bit errors is very small (much less than 1%), and therefore the loss of a packet signals congestion somewhere in the network between the source and destination. If this assumption is false, which it sometimes is for wireless networks, TCP slows down even when no congestion is present. In addition, many RTTs may be required for the value of *cwnd* to grow large, which is required for efficient use of networks with high capacity. Fixing these issues with TCP has been a popular area for research, and we discuss some of the various approaches later.

16.2.3 Selecting between Slow Start and Congestion Avoidance

In normal operations, a TCP connection is always running either the slow start or the congestion avoidance procedure, but never the two simultaneously. We now turn to the question, What determines the algorithm TCP uses at any given time? We already know that slow start is used when a new connection is created or when a timeout-based retransmission occurs. We now turn to what controls the selection between slow start and congestion avoidance.

We mentioned *ssthresh* earlier. This threshold is a limit on the value of *cwnd* that determines which algorithm is in operation, slow start or congestion avoidance. When *cwnd* < *ssthresh*, slow start is used, and when *cwnd* > *ssthresh*, congestion avoidance is used. When they are equal, either can be used. The most important distinction between slow start and congestion avoidance, as we have seen, is how each modifies the value of *cwnd* when new ACKs arrive. What makes TCP somewhat tricky and interesting is that the value of *ssthresh* is not fixed but instead varies over time. Its main purpose is to remember the last “best” estimate of the operating window when no loss was present. Said another way, it holds the lower bound on TCP’s best estimate of the optimal window size.

The initial value of *ssthresh* may be set arbitrarily high (e.g., to *awnd* or higher), which causes TCP to always start with slow start. When a retransmission occurs, caused by either a retransmission timeout or the execution of fast retransmit, *ssthresh* is updated as follows:

$$\text{ssthresh} = \max(\text{flight size}/2, 2 * \text{SMSS}) \quad [1]$$

Note

In Microsoft’s most recent (“Next Generation”) TCP/IP stack, this equation is reportedly changed to the somewhat more conservative relationship: *ssthresh* = $\max(\min(\text{cwnd}, \text{awnd})/2, 2 * \text{SMSS})$ [NB08].

Here we see that if a retransmission is required, TCP assumes that the operating window must have been too large for the network to handle. Reducing the estimate of the optimal window size is accompanied by altering `ssthresh` to be about half of what the current window size is (but not ever below twice the `SMSS`). This usually results in lowering `ssthresh`, but it can also result in increasing `ssthresh`. If we examine the congestion avoidance procedure for TCP, we recall that if an entire window's worth of data is successfully exchanged, the value of `cwnd` is allowed to increase by approximately 1 `SMSS`. Thus, if `cwnd` has grown large over a considerable amount of time, setting `ssthresh` to half of the flight size could cause it to increase. This happens when TCP has discovered more usable bandwidth. The interplay between `ssthresh` and `cwnd`, in conjunction with the operation of slow start and congestion avoidance, gives TCP its characteristic behavior in the face of congestion. We now explore the complete, combined algorithms.

16.2.4 Tahoe, Reno, and Fast Recovery

The algorithms discussed so far, slow start and congestion avoidance, constitute the first congestion control algorithms applied to TCP. They were introduced in the late 1980s with the 4.2 release of UC Berkeley's version of UNIX, called the Berkeley Software Distribution, or BSD UNIX. Thus began the convention of naming various versions of TCP after U.S. cities, especially those where gambling is permitted.

The 4.2 release of BSD (called Tahoe) included a version of TCP that started connections in slow start, and if a packet was lost, detected by either a timeout or the fast retransmit procedure, the slow start algorithm was reinitiated. Tahoe was implemented by simply reducing `cwnd` to its starting value (1 `SMSS` at that time) upon any loss, forcing the connection to slow start until `cwnd` grew to the value `ssthresh`.

One problem with this approach is that for large BDP paths, this can cause the connection to significantly underutilize the available bandwidth while the sending TCP goes through slow start to get back to the point at which it was operating before the packet loss. To address this problem, the reinitiation of slow start on any packet loss was reconsidered. Ultimately, if packet loss is detected by duplicate ACKs (invoking fast retransmit), `cwnd` is instead reset to the last value of `ssthresh` instead of only 1 `SMSS`. (Slow start is still initiated on a timeout, which is generally the case for most TCP variants.) This approach allows the TCP to slow down to half of its previous rate without reverting to slow start.

In exploring the issue of large BDP paths further and thinking back to the conservation of packets principle mentioned before, it has been observed that any ACKs that are received, even while recovering after a loss, still represent opportunities to inject new packets into the network. This became the basis of the fast recovery procedure, which was released in conjunction with the popular 4.3 BSD Reno version of BSD UNIX. Fast recovery allows `cwnd` to (temporarily) grow by 1 `SMSS` for each ACK received while recovering. The congestion window is therefore

inflated for a period of time, allowing an additional new packet to be sent for each ACK received, until a good ACK is seen. Any nonduplicate (“good”) ACK causes TCP to exit recovery and reduces the congestion back to its pre-inflated value. TCP Reno became very popular and ultimately the basis for what might reasonably be called “standard TCP.”

16.2.5 Standard TCP

Although what constitutes “standard” TCP is subject to some debate, the algorithms we have discussed so far constitute the primary procedures identified with standard TCP operation. The slow start and congestion avoidance algorithms are usually implemented together, and the baseline overall behavior is given in [RFC5681]. This specification does not require the use of these exact algorithms, but a requirement is imposed that any TCP implementation not be more aggressive than these algorithms would allow.

To summarize the combined algorithm from [RFC5681], TCP begins a connection in slow start ($cwnd = IW$) with a large value of $ssthresh$, generally at least the value of $awnd$. Upon receiving a good ACK (one that acknowledges new data), TCP updates the value of $cwnd$ as follows:

$cwnd += SMSS$	(if $cwnd < ssthresh$)	Slow start
$cwnd += SMSS * SMSS / cwnd$	(if $cwnd > ssthresh$)	Congestion avoidance

When fast retransmit is invoked because of receipt of a third duplicate ACK (or other signal, if conventional fast retransmit initiation is not used), the following actions are performed:

1. $ssthresh$ is updated to no more than the value given in equation [1].
2. The fast retransmit algorithm is performed, and $cwnd$ is set to $(ssthresh + 3 * SMSS)$.
3. $cwnd$ is temporarily increased by $SMSS$ for each duplicate ACK received.
4. When a good ACK is received, $cwnd$ is reset back to $ssthresh$.

The actions in steps 2 and 3 constitute fast recovery. Step 2 first adjusts $cwnd$, which usually causes it to be reduced to half of its former value, and then temporarily inflates it to take into account the fact that the receipt of each duplicate ACK indicates that some packet has left the network (and thus should permit another to be inserted). This step is also where multiplicative decrease occurs, as $cwnd$ is ordinarily multiplied by some value (0.5 here) to form its new value. Step 3 continues the inflation process, allowing the sender to send additional packets (assuming $awnd$ is not exceeded). In step 4, the TCP is assumed to have recovered, so the temporary inflation is removed (and so this step is sometimes called “deflation”).

Slow start is always used in two cases: when a new connection is started, and when a retransmission timeout occurs. It can also be invoked when a sender has been idle for a relatively long time or there is some other reason to suspect that `cwnd` may not accurately reflect the current network congestion state (see Section 16.3.5). In this case, the initial value of `cwnd` is set to the restart window (`RW`). In [RFC5681], the recommended value of $RW = \min(IW, cwnd)$. Other than this case, when slow start is invoked, `cwnd` is set to `IW`.

16.3 Evolution of the Standard Algorithms

The classic and standard TCP algorithms made a tremendous contribution to the operation of TCP, essentially addressing the major problem of Internet congestion collapse.

Note

The problem of Internet congestion collapse was a serious concern during the years 1986–1988. In October 1986 the NSFNET backbone, an important component of the early Internet, had been observed to operate with an effective capacity some 1000 times less than it should have (called the “NSFNET meltdown”). The primary reason for the problem was aggressive retransmissions during times of loss without any controls. This behavior drove the network into a persistently congested state where packet loss was massive (causing more retransmissions) and throughput was low. Adoption of the classic congestion control algorithms effectively eliminated this problem.

However, there remained several areas for improvement. Given TCP's popularity, a growing amount of effort was put into ensuring that TCP could be made to work well under a wider range of conditions. We now mention several of these that are found in many TCP implementations today.

16.3.1 NewReno

One problem with fast recovery is that when multiple packets are dropped in a window of data, once one packet is recovered (i.e., successfully delivered and ACKed), a good ACK can be received at the sender that causes the temporary window inflation in fast recovery to be erased before all the packets that were lost have been retransmitted. ACKs that trigger this behavior are called partial ACKs. A Reno TCP reacting to a partial ACK by reducing its inflated congestion window can go idle until a retransmission timer fires. To understand why this happens, recall that (non-SACK) TCP depends on the signal of three (`ordupthresh`) duplicate ACKs to trigger its fast retransmit procedure. If there are not enough packets in the network, it is not possible to trigger this procedure on packet loss, ultimately

leading to the expiration of the retransmission timer and invocation of the slow start procedure, which drastically impacts TCP throughput performance.

To address this problem with Reno, a modification called NewReno [RFC3782] has been developed. This procedure modifies fast recovery by keeping track of the highest sequence number from the last transmitted window of data (the recovery point, which we first saw in Chapter 14). Only when an ACK with an ACK number at least as large as the recovery point is received the inflation of fast recovery is removed. This allows a TCP to continue sending one segment for each ACK it receives while recovering and reduces the occurrence of retransmission timeouts, especially when multiple packets are dropped in a single window of data. NewReno is a popular variant of modern TCPs—it does not suffer from the problems of the original fast recovery and is significantly less complicated to implement than SACKs. With SACKs, however, a TCP can perform better than NewReno when multiple packets are lost in a window of data, but doing this requires careful attention to the congestion control procedures, which we discuss next.

16.3.2 TCP Congestion Control with SACK

With the introduction of SACKs and selective repeat to TCP, a sender is able to make better decisions about what segments to send in order to fill holes at the receiver (see Chapter 14). In filling the receiver's holes, the sender generally sends each of the missing segments, in order, until all of the retransmissions for the lost segments have been received successfully. This procedure differs from the basic fast retransmit/recovery procedure mentioned previously in a somewhat subtle way.

In the case of fast retransmit/recovery, when a packet is lost, the sending TCP transmits only the segment it believes is lost and is able to send new data if the window W allows. Because the window is inflated for each arriving ACK during fast recovery, with larger windows TCP typically is able to send some additional data after performing its retransmission. With SACK TCP, the sender can be informed of multiple missing segments and would theoretically be able to send them all immediately because they would all be in the valid window. However, this might involve sending too much data into the network at once, thereby compromising the congestion control. The following issue arises with SACK TCP: using only $cwnd$ as a bound on the sender's sliding window to indicate how many (and which) packets to send during recovery periods is not sufficient. Instead, the selection of which packets to send needs to be decoupled from the choice of when to send them. Said another way, SACK TCP underscores the need to separate the congestion management from the selection and mechanism of packet retransmission. Conventional (non-SACK) TCP mixes these together.

One way to implement this decoupling is to have a TCP keep track of how much data it has injected into the network separately from the maintenance of the window. In [RFC3517] this is called the pipe variable, an estimate of the flight size. Importantly, the pipe variable counts bytes (or packets, depending on the

implementation) of transmissions and retransmissions, provided they are not known to be lost. Assuming a large value of `awnd`, a SACK TCP is permitted to send a segment anytime the following relationship holds true: `cwnd - pipe ≤ MSS`. In other words, `cwnd` is still used to place a limit on the amount of data that can be outstanding in the network, but the amount of data estimated to be in the network is accounted for separately from the window itself. How SACK TCP using this approach to congestion control compares with conventional TCP was first explored in detail with a series of simulations in [FF96].

16.3.3 Forward Acknowledgment (FACK) and Rate Halving

For TCP variants based on Reno (including NewReno), the typical behavior is that when `cwnd` is reduced after a fast retransmit, ACKs for at least one-half of the current window's outstanding data must be received before the sending TCP is allowed to continue transmitting. This is an expected consequence of reducing the congestion window by half immediately when a loss is detected. It causes the sending TCP to wait for about half of an RTT and then send any new data during the second half of the same RTT, a more bursty behavior than is really required.

In an effort to avoid the initial pause after loss but not violate the convention of emerging from recovery with a congestion window set to half of its size on entry, forward acknowledgment (FACK) was described in [MM96]. It consists of two algorithms called “overdamping” and “rampdown.” Since the initial proposal, the authors updated their approach to form a unified and improved algorithm they call rate halving, based on earlier work by Hoe [H96]. To ensure that it works as effectively as possible, they further govern its behavior by adding bounding parameters, resulting in the complete algorithm being called Rate-Halving with Bounding Parameters (RHBP) [PSCRH].

The basic operation of RHBP allows the TCP sender to send one packet for every two duplicate ACKs it receives during one RTT. This causes the recovering TCP to have sent the appropriate amount of data by the end of the recovery period, but it spaces or paces this data evenly, rather than bunching all the transmissions into the second half of the RTT period. Avoiding the bunching or burstiness is advantageous because bursts tend to persist across multiple RTTs, stressing router buffers more than required.

To keep an accurate estimate of the flight size, RHBP uses information from SACKs to determine the FACK: the highest sequence number known to have reached the receiver, plus 1. Taking the difference between the highest sequence number about to be sent by the sender (`SND.NXT` in Figure 15-9) and the FACK gives an estimate of the flight size, not including retransmissions.

With RHBP, a distinction is made between the adjustment interval (the period when `cwnd` is modified) and the repair interval (when some segments are retransmitted). The adjustment interval is entered immediately upon a loss or congestion indicator. The final value for `cwnd` when the interval completes is half of the correctly delivered portion of the window of data in the network at the time

of detection. The following expression allows the RHBP sender to transmit, if satisfied:

$$(\text{SND.NXT} - \text{fack} + \text{retran_data} + \text{len}) < \text{cwnd}$$

This expression captures the flight size, including retransmissions, and ensures that if injecting another packet of length `len`, `cwnd` will not be exceeded. Provided all the data prior to the FACK is indeed no longer in the network (i.e., is lost or stored at the receiver), this causes the SACK sender to be appropriately controlled by `cwnd`. However, it can be overly aggressive if packets have been reordered in the network because the holes indicated by SACK have not been lost.

In Linux, FACK and rate halving are implemented and enabled by default. FACK is activated only when SACK is enabled and the Boolean configuration variable `net.ipv4.tcp_fack` is set to 1. When reordering is detected in the network, the more aggressive behavior of FACK is disabled.

Rate halving is one of several ways of pacing TCP's sending procedure to avoid or limit burstiness. Although it offers a number of benefits, it also has a few problems. In [ASA00], the authors analyze TCP pacing in some detail using simulations, concluding that in many cases it offers inferior performance to TCP Reno. Furthermore, rate-halving TCP has been known to exhibit poor performance when the connection may become limited by the receiver's advertised window [MM05].

16.3.4 Limited Transmit

In [RFC3042], the authors propose limited transmit, a small modification to TCP designed to help it perform better when the usable window is small. Recall from the experience with Reno TCP that when operating with a small window, there may not be enough packets in the network to trigger the fast retransmit/recovery algorithms when loss occurs, as these algorithms typically require three duplicate ACKs to be observed prior to initiation.

With limited transmit, a TCP with unsent data is permitted to send a new packet for each pair of consecutive duplicate ACKs it receives. Doing this helps to keep at least a minimal number of packets in the network—enough so that fast retransmit can be triggered upon packet loss. This is advantageous to TCP because waiting for an RTO (which can be a relatively large amount of time—several hundred milliseconds) can degrade throughput performance considerably. As of [RFC5681], limited transmit is now a recommended TCP behavior. Note that rate halving is one form of limited transmit.

16.3.5 Congestion Window Validation (CWV)

One of the issues with congestion management in TCP arises when the TCP sender stops sending for a period of time, either because it has no more data to send, or because it has been prevented from sending when it wants to for some

other reason. If all goes well, a sender never pauses, and it continues sending data and receiving ACKs from its peer. This continuous feedback enables it to keep a reasonably current (within one RTT) estimate of what `cwnd` and `ssthresh` should be.

If the TCP sender has been sending for some time, its `cwnd` may have grown to a substantial size. If it then fails to send for some time but resumes later, the large `cwnd` may allow the sender to inject an undesirably large number of packets (i.e., a high-rate burst) into the network without delay. Furthermore, if the pause is sufficiently long, its last `cwnd` value may no longer be appropriate for the path and congestion state.

In [RFC2861], the authors propose an experimental Congestion Window Validation (CWV) mechanism. Essentially, the sender's current value of `cwnd` decays over a period of nonuse, and `ssthresh` maintains the “memory” of it prior to the initiation of the decay. To understand the scheme, a distinction is made between an idle sender and an application-limited sender. The idle sender has stopped producing data it wants to send into the network; ACKs for all the data it has sent so far have been received. Thus, the connection is truly quiescent—no data is flowing, so no ACKs are either, except for occasional window updates (see Chapter 15). The application-limited sender does have more data to send but has been unable to for some reason. This could be because the sending computer is busy doing other tasks, or because some mechanism or protocol layer below TCP is preventing data from being sent. This case results in underutilization of the allowed congestion window, but the connection is not completely quiescent. In particular, ACKs may still be returning for previously sent data.

The CWV algorithm works as follows: Whenever a new packet is to be sent, the time since the last send event is measured to determine if it exceeds one RTO. If so,

- `ssthresh` is modified but not reduced—it is set to $\max(\text{ssthresh}, (3/4) * \text{cwnd})$.
- `cwnd` is reduced by half for each RTT of idle time but is always at least 1 SMSS.

For application-limited periods that are not idle, the following similar behavior is used:

- The amount of window actually used is stored in `W_used`.
- `ssthresh` is modified but not reduced—it is set to $\max(\text{ssthresh}, (3/4) * \text{cwnd})$.
- `cwnd` is set to the average of `cwnd` and `W_used`.

Both of these changes decay the value of `cwnd` while “remembering” it in `ssthresh`. The first case can dramatically affect `cwnd` in one operation, if the application has been idle for a long time. Handling the congestion window in this way can lead to better performance under some circumstances. As the authors report, reducing the burst of packets that can arise after an idle period eases the pressure

on potentially limited buffer space in routers, ultimately leading to fewer dropped packets. Note that because `cwnd` is decayed and `sssthresh` is not, the typical consequence of applying this algorithm is to place the sender into slow start after a long enough pause. CWV is enabled by default in Linux TCP implementations.

16.4 Handling Spurious RTOs—the Eifel Response Algorithm

As we saw in Chapter 15, when TCP encounters a large delay spike, it can experience a retransmission timeout even if no packet has been lost. Such spurious retransmissions arise in a number of circumstances relating to changes in the underlying link layer (such as cellular handoff) or sudden onset of severe congestion contributing to a large increase in RTT. When this happens, the TCP adjusts `sssthresh` and enters slow start by setting `cwnd` to 1W. If no packets have been lost, ACKs arriving after the RTO cause `cwnd` to grow relatively quickly, but TCP still sends unnecessary retransmissions and underutilizes the capacity until `cwnd` and `sssthresh` resettle.

To avoid the performance problems associated with spurious retransmissions, several methods have been proposed to detect them. We discussed some of them (e.g., DSACK, Eifel, F-RTO) in Chapter 14. Any one of these, or possibly others that may be developed, can be coupled with a response algorithm used to “undo” the changes TCP makes to its congestion control variables after a timeout. One popular (i.e., in the IETF standards track) response algorithm is the Eifel Response Algorithm [RFC4015].

Eifel comprises both a detection and a response algorithm, which are logically disjoint. Any TCP implementation using the Eifel Response Algorithm is compelled to use some detection algorithm specified in a standards-track or experimental RFC (i.e., one that is documented).

The Eifel Response Algorithm is aimed at handling the retransmission timer and congestion control state after a retransmission timer has expired. Here we discuss only the congestion-related portions of the response algorithm. It is initiated after the first timeout-based retransmission is sent. Its purpose is to undo a change to `sssthresh` when a retransmission is deemed to be spurious. In all cases, before `sssthresh` is modified as a result of the RTO, it is captured in a special variable as follows: `pipe_prev = min(flight size, sssthresh)`. Once this has been accomplished, a detection algorithm, such as one of those mentioned previously, is invoked in order to determine if the RTO is spurious. If it is, the following steps are executed when an ACK arrives after the retransmission:

1. If a received good ACK includes an ECN-Echo flag, stop (see Section 16.11).
2. `cwnd = flight size + min(bytes_acked, 1W)` (assuming `cwnd` is measured in bytes).
3. `sssthresh = pipe_prev`.

The `pipe_prev` variable is set before `ssthresh` is changed in the ordinary way. It provides a memory for `ssthresh`, so that it can be reinstantiated in step 3 if necessary. Step 1 deals with the case of an arriving ACK carrying the ECN flag. (We discuss ECN more in Section 16.11.) When this happens, it is considered unsafe to avoid undoing the reduction of `ssthresh`, so the algorithm terminates. Steps 2 and 3 constitute the important part of the algorithm (with respect to `cwnd`). Step 2 restores `cwnd` to a point where it may be able to inject some additional traffic into the network, but not more than `IW` new data. `IW` is considered a safe amount of data to inject into a network path with unknown congestion state. Step 3 restores `ssthresh` to its value before the RTO occurred, completing the undo operation.

16.5 An Extended Example

We now turn to an extended example to demonstrate most of the behaviors described in the preceding sections. Using the `sock` program, we arrange to send about 2.5MB of data from a Linux (2.6) sender to a FreeBSD (5.4) receiver over a DSL line. The DSL line is rate-limited in this direction to approximately 300Kb/s. The FreeBSD receiver is attached to a high-bandwidth connection. The minimum RTT between sender and receiver is 15.9ms, and there are 17 hops in the path. The systems are configured to use the baseline algorithms (i.e., slow start and congestion avoidance) for most of their processing. This avoids many of the operating-system-specific details. (We cover some of these later.) To set up this experiment, we run the following command at the receiver:

```
FreeBSD% sock -i -r 32768 -R 233016 -s 6666
```

This command arranges for the `sock` program to use a fairly large socket receive buffer (228KB) and perform fairly large application reads (32KB). For the path used, this is an adequate size of buffer for the receiver. At the sender we run the `sock` program in sending mode, as follows:

```
Linux% sock -n20 -i -w 131072 -S 262144 128.32.37.219 6666
```

This selects a large send buffer and sends 20*131,072 bytes (2.5MB) of data. The packet trace is captured using `tcpdump` on the sender. The command used to capture this trace is as follows:

```
Linux# tcpdump -s 128 -w sack-to-free-12.td port 6666
```

This ensures that at least 128 bytes of each packet are captured, plenty to capture all interesting TCP and IP header information. After the trace is collected, we can use the `tcptrace` tool [TCPTRACE] to get a number of useful summary statistics regarding the connection:

```
Linux% tcptrace -wl sack-to-free-12.td
```

This command requests the program to provide information on the congestion window and output using a long (verbose) format. It produces the following output:

```

1 arg remaining, starting with 'sack-to-free-12.td'
Ostermann's tcptrace -- version 6.6.7 -- Thu Nov 4, 2004

3175 packets seen, 3175 TCP packets traced
elapsed wallclock time: 0:00:00.167213, 18987 pkts/sec analyzed
trace file elapsed time: 0:01:40.475872
TCP connection info:
1 TCP connection traced:
TCP connection 1:
    host a:          adsl-63-203-72-138.dsl.snfc21.pacbell.net:1059
    host b:          dwight.CS.Berkeley.EDU:6666
    complete conn:   yes
    first packet:    Wed Sep 28 22:15:29.956897 2005
    last packet:     Wed Sep 28 22:17:10.432769 2005
    elapsed time:    0:01:40.475872
    total packets:   3175
    filename:        sack-to-free-12.td
a->b:
total packets:      1903
ack pkts sent:      1902
pure acks sent:     2
sack pkts sent:     0
dsack pkts sent:    0
max sack blks/ack:  0
unique bytes sent:  2621440
actual data pkts:   1900
actual data bytes:  2659240
rexmt data pkts:    27
rexmt data bytes:   37800
zwnd probe pkts:    0
zwnd probe bytes:   0
outoforder pkts:    0
pushed data pkts:   44
SYN/FIN pkts sent:  1/1
req 1323 ws/ts:     Y/Y
adv wind scale:     2
req sack:           Y
sacks sent:         0
urgent data pkts:   0 pkts
urgent data bytes:  0 bytes
mss requested:      1412 bytes
max segm size:      1400 bytes
min segm size:      640 bytes
avg segm size:      1399 bytes
max win adv:        5808 bytes
min win adv:        5808 bytes
zero win adv:       0 times
avg win adv:        5808 bytes
b->a:
total packets:      1272
ack pkts sent:      1272
pure acks sent:     1270
sack pkts sent:     79
dsack pkts sent:    0
max sack blks/ack:  2
unique bytes sent:   0
actual data pkts:   0
actual data bytes:   0
rexmt data pkts:    0
rexmt data bytes:   0
zwnd probe pkts:    0
zwnd probe bytes:   0
outoforder pkts:    0
pushed data pkts:   0
SYN/FIN pkts sent:  1/1
req 1323 ws/ts:     Y/Y
adv wind scale:     2
req sack:           Y
sacks sent:         79
urgent data pkts:   0 pkts
urgent data bytes:  0 bytes
mss requested:      1460 bytes
max segm size:      0 bytes
min segm size:      0 bytes
avg segm size:      0 bytes
max win adv:        233016 bytes
min win adv:        170016 bytes
zero win adv:       0 times
avg win adv:        232268 bytes

```

max owin:	137201 bytes	max owin:	1 bytes
min non-zero owin:	1 bytes	min non-zero owin:	1 bytes
avg owin:	37594 bytes	avg owin:	1 bytes
wavg owin:	33285 bytes	wavg owin:	0 bytes
initial window:	2800 bytes	initial window:	0 bytes
initial window:	2 pkts	initial window:	0 pkts
ttl stream length:	2621440 bytes	ttl stream length:	0 bytes
missed data:	0 bytes	missed data:	0 bytes
truncated data:	2556640 bytes	truncated data:	0 bytes
truncated packets:	1900 pkts	truncated packets:	0 pkts
data xmit time:	99.631 secs	data xmit time:	0.000 secs
idletime max:	7778.8 ms	idletime max:	7930.4 ms
throughput:	26090 Bps	throughput:	0 Bps

From this useful tool we can learn quite a bit about the connection. We are primarily interested in the left portion of the output (a→b). First of all, we see that 1903 packets were sent in the a→b direction and 1902 of them were ACKs. This is expected, as the very first packet is normally a SYN—the only packet without the ACK flag turned on. Pure ACKs refer to packets containing no data. The sender produces one of these early in the connection, when providing an ACK to its peer's SYN + ACK and when producing the final ACK when the connection is closed, so this is also expected. In the second column (b→a direction), we find that the receiver sent 1272 packets, all of which are ACKs. Of these, 1270 were pure ACKs, and 79 SACK packets (i.e., ACKs containing the SACK option) were sent. The two “non-pure” ACKs are the SYN + ACK and the FIN + ACK sent at the beginning and end of the connection, respectively.

The next five values indicate the proportion of data that was retransmitted. As we can see, 2,621,440 unique bytes were sent (i.e., not retransmitted), but 2,659,240 bytes were sent in total, meaning some $2,659,240 - 2,621,440 = 37,800$ bytes must have been sent more than once. The next two fields confirm this and indicate that these retransmitted bytes were contained in 27 retransmitted packets, for an average retransmitted segment size of 1399 bytes. Because this connection transferred 2,659,240 bytes in 100.476 s, its average throughput is 26,466 bytes/s (about 212Kb/s). Its average goodput, the amount of unretransmitted data transferred per unit time, is $2,621,440/100.476 = 26,090$ B/s, about 209Kb/s. As we shall see, this connection experiences a number of significant disruptions to its normal operation. We shall use Wireshark's analysis capabilities and our own analysis to follow TCP's behavior when such events occur.

To get a visual image of the trace, we can use the Statistics | TCP Stream Graph | Time-Sequence Graph (tcptrace) function in Wireshark's Statistics menu to obtain the image shown in Figure 16-4 (enhanced with arrows for the discussion that follows).

The y-axis of Figure 16-4 represents the relative TCP sequence number. Each small tick mark represents 100,000 sequence numbers. The x-axis is time, in seconds. The dark solid line comprises many smaller I-shaped line segments, each of which represents the range of sequence numbers contained in a TCP segment.

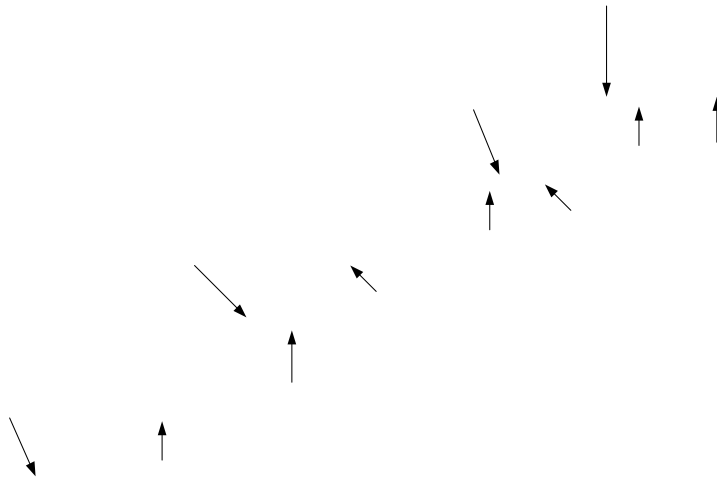


Figure 16-4 Wireshark trace of a 2.5MB file upload executed by a Linux 2.6.10 TCP sender over a DSL line rate-limited to approximately 300Kb/s. The dark line represents sent sequence numbers. The top line is the highest sequence number advertised by the receiver (its right window edge), and the lower line represents the highest segment acknowledged by the receiver so far seen at the sender. The 11 events labeled represent cases where the congestion window has been modified.

The height of the I indicates the user-data payload size, in bytes. The slope of the “line” formed by these I-shaped characters is the data rate achieved by the connection. Any movement to the lower right indicates a retransmission. The slope of the line for any given time range provides the average throughput over that time. As we can see, the highest sequence number sent was about 2600000 at time 100, which provides for a rough average goodput rate of 26,000 bytes/s, quite close to the numeric value from the preceding `tcptrace` output.

The top line is the largest sequence number the receiver is willing to accept (its highest advertised window) so far. As we can see, at the beginning of the time series, this line is at about 250000, with the actual value being 233016, as indicated in the `tcptrace` output, in the `b` column. The bottom line represents the highest ACK number received at the sender so far. As discussed previously, TCP searches for additional bandwidth while it operates, by increasing its congestion

window. It does not violate the receiver's advertised window. We see this in operation in this graph as the solid line moves from the lower line toward the upper line over time. If the upper line is never reached, either the sender or the usable network capacity is the limiting factor for the throughput of the connection. If the upper line is always reached, the receiver's window is the likely limiting factor.

16.5.1 Slow Start Behavior

We begin our analysis by observing the operation of the slow start algorithm described earlier. In Wireshark, we select the first packet of the trace and then use its Statistics | Flow graph function to illustrate the packets exchanged at the beginning of the connection (see Figure 16-5).

Figure 16-5 The Wireshark analysis shows the sequence and ACK numbers exchanged when the connection is first established. Each ACK received at the sender liberates two or three packets. This characteristic is typical of a sender in slow start.

Here we see the initial SYN and SYN + ACK exchange. The ACK at time 0.032 is a window update (see Chapter 15). The first two data packets are sent at times 0.126 and 0.127. The ACK at time 0.210 is not for a single packet. Its ACK number is 2801 and thus ACKs both of the previously sent data packets because of the cumulative property of TCP ACKs. This is an example of delayed ACKs, which are often generated for every other data packet (or more frequently, as recommended by [RFC5681]). As we shall see for this particular (FreeBSD 5.4) receiver, it alternates between ACKing one packet and two packets. This means there are two

ACKs returned for every three data packets sent on average (assuming no errors or retransmissions). We discussed delayed ACKs and window updates in Chapter 15.

An ACK arriving that covers two packets allows the sliding window at the sender to move forward by two packets and therefore permits two additional packets to be sent into the network. However, because this connection is just starting out and it is still executing slow start, the arrival of a good ACK causes the sender to increase its congestion window by one packet (this Linux TCP manages its congestion window in packet units). In this case, the `cwnd` grows from 2 to 3. This has effect of allowing three packets to be sent overall as a result of the arriving ACK. They are sent at times 0.215, 0.216, and 0.217.

The ACK arriving at time 0.264 ACKs a single packet and indicates that the receiver next expects to see sequence number 4201. That packet, however, and the one after it with sequence number 5601, have already been sent and are still out standing. Thus, the ACK arrival allows `cwnd` to grow from 3 to 4, but because two packets are already outstanding, only two more are allowed to be sent (one because the ACK slid the window forward, another because the received good ACK allowed `cwnd` to grow by one packet). They are sent at times 0.268 and 0.268 (within the same 1/1000s).

This startup behavior is typical of a sender executing slow start with a receiver delaying ACKs. The process continues in this fashion (each ACK liberating two or three packets) until something interesting occurs at about time 5.6. We now explore this further.

16.5.2 Sender Pause and Local Congestion (Event 1)

Looking at Figure 16-4, we find that after a segment is sent at time 5.512, a pause occurs until the next data segment is sent at time 6.162. This can be better seen by using Wireshark's graphical zoom-in feature as shown in Figure 16-6.

In this figure we see that the sender has stopped sending, no retransmitted packet appears to be present, yet the data rate appears to decrease after the pause. Why is this? We can investigate further with the flow trace function once again (see Figure 16-7).

The sending TCP has evidently ceased its sending demand at time 5.559. This is supported by the fact that the last transmitted data segment before the pause has the PSH flag turned on, which typically indicates that the sending buffer has been emptied. There could be several reasons for this, including the possibility that the host system is busy doing something else, preventing the sending application from initiating its next write of data into the network.

We can observe that this pause is not the beginning of a retransmission recovery period, yet the slope of the line decreases after the pause, indicating a reduced sending rate. Let us explore this behavior more closely to figure out why.

The last sequence number sent before the pause is $343001 + 1400 - 1 = 344400$, which has never been sent before and is therefore not a retransmission. After the segment is sent at time 5.486 (highlighted), this connection will have its greatest amount of outstanding data: $341,601 + 1400 - 205,801 = 137,200$ bytes (98 packets).

Figure 16-6 After starting using the slow start procedure, the connection pauses for about 512ms, at time 5.512, and then continues by sending a burst.

This tells us that the value of `cwnd` is 98 packets. The arrival of the ACK at time 5.556 indicates that two more packets have been received at the receiver. The last packet to be sent before the pause contains sequence number 344400, so 97 packets are outstanding.

While the application is paused, 11 ACKs arrive (each alternating between ACKing either one or two full-size segments as mentioned before). The last one indicates that sequence number 233800 has been received, meaning 110,600 bytes (79 packets) now remain outstanding. At this point, the sender wakes up and continues to transmit. As a result of receiving this ACK at time 6.204, it should be able to inject $98 - 79 = 19$ more packets at this point but is able to send only 8. The last sequence number it is able to send is $354201 + 1400 - 1 = 355600$ at time 6.128.

What happens to the TCP at this point is not immediately obvious from the trace. We would have expected 19 packets to be sent, but only 8 were. The reason is that the sender filled a local (lower-layer) queue with its burst of packets and the subsequent ones were unable to be sent. Using the following command in Linux, and knowing that our transfer takes place over the `ppp0` network interface, we can try to determine if some lower layer has caused TCP to have problems:

Figure 16-7 The sender pauses at time 5.559. In addition, the burst of packets at time 6.209 is limited to eight because of local congestion. Some TCP implementations such as this one limit the sending rate to avoid congesting queues on the sending host.

```
Linux% tc -s -d qdisc show dev ppp0
qdisc pfifo_fast 0: bands 3 priomap  1 2 2 2 1 2 0 0 1 1 1 1 1 1 1 1
Sent 122569547 bytes 348574 pkts (dropped 2, overlimits 0 requeues 0)
```

The `tc` program is used to administer the packet scheduling and traffic control subsystem in Linux [LARTC]. The `-s` and `-d` options provide detailed statistics. The directive `qdisc show dev ppp0` means the queuing discipline for device `ppp0` should be displayed, which is the method used to hold and prioritize the order in which packets are sent. Notice the two dropped packets. These packets were not dropped in the network but rather in the sending computer in a protocol layer below TCP. Furthermore, because they were dropped in a layer below TCP but above the layer where the packet capture facility operates, these packet transmission attempts are not visible in the trace. Dropping transmitted TCP packets at the sending system is sometimes called local congestion, and it arises because TCP is producing data faster than the underlying local queues can be emptied.

Note

The Linux traffic control subsystem and other priority or QoS features supported in routers and operating systems (e.g., Microsoft's qWave API [WQOS]) support different queuing disciplines that may order packets differently based on features in the packets (e.g., the IP DSCP value or TCP port number). Placing priority on some packets (e.g., multimedia data packets, TCP pure ACKs) may improve the user experience for interactive applications in networks that support priority. Much of the Internet does not support such priorities, but many LANs and some enterprise IP networks do.

Local congestion is one of several reasons the Linux TCP implementation may be placed in the Congestion Window Reducing (CWR) state [SK02]. It starts by setting `ssthresh` to `cwnd/2`, and by setting `cwnd` to $\min(\text{cwnd}, \text{flight size} + 1)$. In the CWR state, the sender reduces `cwnd` by one packet for every two ACKs received until `cwnd` reaches the new `ssthresh` or the CWR state is exited for some other reason such as a loss event. It is essentially the rate-halving algorithm we mentioned previously. It is also invoked when the sending TCP receives an ECN-Echo indication in the received TCP header (see Section 16.1.1).

With this knowledge, we can now understand what happened. When TCP continues after the pause, it is able to send only 8 packets. Any additional packets cannot be sent because of local congestion and instead place the TCP into the CWR state. Immediately, `ssthresh` is reduced to $98/2 = 49$ packets and `cwnd` is set to $79 + 8 = 87$ packets. It then remains in the CWR state where it reduces `cwnd` by 1 for every two ACKs it receives, leading to a reduction in sending rate, until `cwnd` reaches 66 packets at time 8.364.

The reduction in sending rate can also be observed as follows: Looking at Figure 16-6, before time 5.5 the slope of the line gives an effective data rate of approximately 500Kb/s. This is higher than the capacity of the link in the direction of the data transfer, so this extra apparent capacity is the result of one or more queues being filled up in the path, leading to an increased RTT because of queuing delay. We can use the Statistics | TCP Stream Graph | Round Trip Time Graph to visualize this effect (see Figure 16-8).

In this figure, the y-axis represents the estimated RTT in seconds and the x-axis represents the sequence number. We can see that at approximately sequence number 340000, the RTT begins to decrease. This sequence number corresponds closely to the last sequence number sent before the pause described earlier (344400). The decreasing RTT corresponds to the fact that as the sender slows down, the network is becoming less loaded (i.e., the rate at which data is draining from the network exceeds the rate at which new traffic is arriving). This causes queues within network routers to empty, leading to a smaller wait time and a consequentially lower RTT.

The sending rate reduction continues while TCP remains in the CWR state. Eventually, if this continued, the RTT would decrease to its bare minimum value of about 17ms. In general, TCP avoids allowing this to happen because it wants to

Figure 16-8 The sender's estimated connection round-trip time. Periods of increasing RTT (dense groupings of increasing values) correspond to buffers filling because of an excess of sending rate over forwarding rate at a router along the path. Decreasing RTTs represent the opposite effect, resulting from the sender slowing down and the queues draining.

“keep the pipe full” to ensure that it is using the maximum amount of network capacity currently available to it.

16.5.3 Stretch ACKs and Recovery from Local Congestion

At time 8.364, following the gradual reduction in `cwnd` initially caused by the TCP entering the CWR state, the TCP appears to start decreasing more quickly. This is a consequence of a change in the relationship of `wnd` and the amount of outstanding data indicated by the ACK at time 8.362 (highlighted in Figure 16-9).

The ACK at time 8.362 is for sequence number 317801, but the previously received ACK is for sequence number 313601, meaning this new ACK is for $317,801 - 313,601 = 4,200$ bytes (three packets). This is commonly called a stretch ACK, meaning it ACKs more than twice the largest segment sent so far. It could be caused by a number of possibilities, the simplest of which is a lost ACK. It is usually difficult to determine with certainty the cause of the stretch ACK, but the precise reason

Figure 16-9 A “stretch ACK” acknowledges three packets’ worth of sequence numbers. Such ACKs can cause the sender to act in a bursty manner and can occur when other ACKs are lost in transit.

is not usually important. In this example, we can assume that an earlier ACK was lost and continue to investigate how the sender behaves. Its arrival causes `sssthresh` to drop from 68 to 66.

The Linux TCP implementation attempts to revise its estimate of the number of outstanding packets whenever it receives an ACK. (It also attempts to validate the congestion window whenever it sends segments, according to the Congestion Window Validation algorithm described previously, but this does not have an effect here.) When in CWR state, if the outstanding packet count estimate is reduced for some reason, as it is here after receiving the stretch ACK, `sssthresh` is adjusted to be the estimate plus 1. Note that this is in addition to its ordinary behavior in CWR, where it reduces `sssthresh` by 1 for each pair of ACKs received. Generally, `sssthresh` is reduced by either 1 or 0 for each ACK, and then `sssthresh` is set to $\min(\text{flight size} + 1, [\text{possibly reduced}] \text{sssthresh})$. The CWR state remains operating until `sssthresh` reaches `sssthresh` or some other event, such as a loss and retransmission, occurs.

Prior to receiving the stretch ACK, at time 8.258, $407,401 + 1400 - 313,601 = 95,200$ bytes (68 packets) are outstanding. After the stretch ACK is received, the number of outstanding packets is reduced to 65 and `sssthresh` is set to 66.

Because the flight size estimate and `sssthresh` are closely coupled in the CWR state, and the TCP receiver in this example delays ACKs, the result of a pair of ACKs arriving is to reduce `sssthresh` by 2 and to liberate one packet. The reason for

this is as follows: Assume that before the arrival of any ACKs, $cwnd$ is c_0 and the flight size estimate is $f_0 = c_0$. When the first ACK arrives (i.e., for one packet), $f_1 = f_0 - 1$ and $cwnd$ is updated to $c_1 = \min(c_0 - 1, f_1 + 1) = c_0 - 1$. When the second ACK arrives (for two packets, because of delayed ACKs), $f_2 = f_1 - 2 = c_0 - 3$ and $cwnd$ is set to $c_2 = \min(c_1, f_2 + 1) = \min(c_0 - 1, c_0 - 2) = c_0 - 2$. Because the congestion window has shrunk by two packets, but three packets have been ACKed during this period, a single packet is liberated after the receipt of the second ACK.

The sender exits the CWR state at time 9.37 when $cwnd$ reaches $ssthresh$ at 49 packets. TCP now returns to normal behavior and continues in congestion avoidance (see Figures 16-10 and 16-11).

Figure 16-10 By time 9.369, the sender reverts to normal and sends either one or two packets per received ACK.

In Figure 16-10, the circled packets indicate where the sender's state changes from CWR back to normal, where the congestion avoidance algorithm takes over. Figure 16-11 shows this behavior in more detail.

The sender continues in congestion avoidance, achieving relatively stable throughput until time 17.232. At this point, severe network congestion begins to

Figure 16-11 TCP has completed its recovery and is back in the normal (congestion avoidance) state. It sends one or two packets for each ACK received.

form, contributing to a large increase in the RTT. In Figure 16-8, this happens at sequence number 720000, where the RTT grows to about 6.5s—a more than three fold increase from its previously stable value of about 2s. This effect is common with the onset of severe congestion. Eventually, the network congestion is sufficiently severe so as to cause a packet to be dropped. The sending TCP responds with its first retransmission.

16.5.4 Fast Retransmission and SACK Recovery (Event 2)

At time 21.209, after the dramatic increase in measured RTT, we observe the first retransmission. We can see this in more detail by zooming in as shown in Figure 16-12. The first retransmission (circled) is for the packet starting with sequence number 690201, matching the highest ACK received so far (also 690201). It is triggered by the receipt of a single duplicate ACK carrying the SACK block [698601,700001]. Recall that these numbers indicate the sequence number range already received at the receiver. In this case, it is a single packet.

At time 21.209, when the retransmission takes place, the largest sequence number sent so far is $761601 + 1400 - 1 = 763000$, and wnd is 52. In conjunction with this fast retransmit, ssthresh is reduced from 49 to 26, and TCP enters the Recovery state. This TCP remains in Recovery state until it receives a cumulative ACK for the recovery point: sequence number 763000 (or higher). In addition,

Figure 16-12 The first retransmission (circled) occurs at time 21.209. SACK blocks are used to guide the sender as to what packets to retransmit. Eight retransmissions in total occur between times 21.0 and 22.0.

cwnd is reduced to (flight size + 1) packets. However, because data has likely been lost, determining the flight size is not so straightforward. It is accomplished using the following relationship:

$$\text{flight size} = \text{packets_outstanding} + \text{packets_retransmitted} - \text{packets_removed}$$

The first term on the right-hand side represents all the packets sent once by the sender and not yet ACKed with the regular TCP cumulative ACK field. The second term represents any that have been resent (and not ACKed), and the final term represents any packets that are no longer in the network but also have not been ACKed by the basic TCP cumulative ACK. The value of packets_removed must be estimated because TCP has no reliable way to directly learn it. It represents the sum of any (out-of-order) packets cached at the receiver plus any packets that have been lost in the network. With SACK, it is possible to learn the number of packets cached at the receiver, but the number of lost packets must still be estimated.

The value of `packets_outstanding` here is $(763,001 - 690,201)/1400 = 72,800/1400 = 52$ and the number of packets cached in the receiver is $(700,001 - 698,601)/1400 = 1400/1400 = 1$ (derived from the sequence numbers in the SACK block). With FACK enabled, as it is here by default, holes in the receiver inferred by SACK information are considered to be lost. Thus, in this case, TCP estimates that $698,601 - 690,201 = 8400$ (6 packets) have been lost. The flight size is therefore $52 + 1 - (1 + 6) = 46$ packets, and `cwnd` is set to 47. While in the Recovery state, TCP reduces `ssthresh` by one packet for every two packets it receives, similar to the CWR state. After the first retransmission, another seven retransmissions take place, followed by transmission of new data, based on SACK option data carried in each of the arriving ACKs between times 21.2 and 21.7 (see Figure 16-13).

In this figure, much of the normal Wireshark information has been removed to more clearly see the SACK options on each ACK. By looking at the SACK sequence numbers (SLE and SRE), we can see that most of the time there are two active blocks at the receiver: [698601,700001], which holds one packet, and another [702801,763001] (at its largest), that grows to be 43 packets. During the recovery period, the general rate-halving algorithm applicable to the CWR and Recovery states reduces `ssthresh` by at least one packet for every pair of ACKs received. Because each received ACK effectively ACKs one packet in this case (through an increase in the SACK block size by one packet), flight size reduces by 1, which would permit another packet to be sent. However, because `cwnd` is also reduced by 1 for every other ACK, it takes two ACKs to liberate a new packet. Note how this differs from the CWR case. In that case, some ACKs provided acknowledgment for two packets, whereas here only one packet is ACKed (SACKed) per arriving ACK. Thus, for each of the transmissions and retransmissions shown in the plot, `cwnd` is reduced by 1 after each pair of ACKs has been received. During this recovery period, overall, `cwnd` shrinks from 47 to 20.

Most ACKs containing SACK options are duplicate ACKs for sequence number 690201 (44 of them), as Wireshark points out. There are five good ACKs that contain the SACK blocks [702801,763001] and [698601,700001]. Two more contain only the SACK block [702801,763001]. These good ACKs do not take the sender out of recovery, because their ACK numbers are all below the sequence number of the recovery point at 763000; they are partial ACKs, as discussed earlier.

TCP recovers from fast retransmit at time 23.301 with the arrival of a good ACK equal to a sequence number (765801) larger than the recovery point. At this point, `cwnd` is 20 and `ssthresh` is 26, meaning TCP is in slow start. By time 23.659, after several round trips, `cwnd` reaches the value 27, TCP is in the normal operating state, and the congestion avoidance algorithm takes over. This completes the sender's first fast retransmit recovery period.

16.5.5 Additional Local Congestion and Fast Retransmit Events

The next four events consist of local congestion, a fast retransmit, and two more local congestion episodes. They are very similar to the types of events we have seen already, so they are summarized here only briefly.

Figure 16-13 SACK recovery after fast retransmission. Packet 871 contains the first SACK option used on the connection. Subsequent ACKs contain SACK information until packet 950.

16.5.5.1 CWR Again (Event 3)

A CWR event due to local congestion occurs at time 30.745. At this point, $1,090,601 + 1400 - 1,051,401 = 40,600$ (29 packets) are outstanding, and $sssthresh$ is 31. This should allow two additional packets to be injected, but none are, because of local congestion. In this particular case, $cwnd$ is set to $flight\ size + 1 = 30$, and $sssthresh$ is reduced to 15. TCP exits the CWR state when $cwnd$ reaches $sssthresh$. This happens at time 34.759, after another significant increase in the connection's RTT.

16.5.5.2 Second Fast Retransmit (Event 4)

At time 36.914, there is another fast retransmit when $cwnd = 16$. Using the basic display from Wireshark, such retransmissions are easy to spot (see Figure 16-14).

Figure 16-14 A Linux TCP sender enters the Disorder state upon receiving a duplicate ACK or an ACK with SACK information. Packets arriving while in this state trigger transmissions of new data. Subsequent duplicate ACKs (or presence of SACK information) place the sender into the Recovery state where retransmissions take place.

Here, the ACK arriving at time 36.878 (packet 1366) carries the SACK block [1117201,1118601] and ACK number 1110201. This places Linux TCP in the Disorder state, where arriving packets liberate one packet each (similar to limited transmit) of new data. Packet 1367 is the packet liberated in this case.

With the arrival of the ACK at time 36.912 (packet 1368), containing SACK block [1117201,1120001] and a duplicate ACK, TCP enters the Recovery state and triggers the fast retransmit at time 36.914 (packet 1369). The highest sequence number sent so far is $1132601 + 1400 - 1 = 113400$. Recovery is eventually completed at time 37.455, with the arrival of the ACK containing sequence number 1134001 (packet 1391). Note that immediately following this ACK is a window update. For bulk data transfers such as the present example, where the receiver's window is large relative to the bandwidth-delay product of the network, such updates are not usually of much consequence. When we have interactive traffic, small windows, or servers that only occasionally read from the network, these updates can become quite important, as we saw in Chapter 15. When the first retransmission takes place, *sssthresh* is reduced from 16 to 8. Eventually, when recovery completes, *cwnd* = 4 and *sssthresh* = 8. This leaves the sender in slow start because *cwnd* is smaller than *sssthresh*.

16.5.5.3 CWR Again (Events 5 and 6)

After the arrival of the ACK for sequence number 1359401 at time 43.356, TCP once again enters the CWR state because of local congestion when it tries to send subsequent packets. This ultimately reduces *sssthresh* to 8 and *cwnd* becomes 15. A second transmission failure, while in the CWR state, brings *sssthresh* down to 12. The CWR state is exited with *cwnd* = 7 and *sssthresh* = 8.

Another round of local congestion at time 59.652 forces TCP into CWR when *cwnd* = 19 and *sssthresh* = 10. In this case, the CWR state is interrupted by a timeout that places TCP into the Loss state. This represents a new type of event for us to investigate.

16.5.6 Timeouts, Retransmissions, and Undoing *cwnd* Changes

Although TCP keeps a retransmission timer in case fast retransmit is unable to repair a loss, we have not yet seen it in operation. This is fortunate, because generally when a timeout occurs, the connection is experiencing significant congestion and performance problems. In the next portion of the trace, shown in Figure 16-15, we see how the sending TCP handles the situation when its retransmission timer expires.

16.5.6.1 First Timeout (Event 7)

A retransmission occurs at time 62.486 (packet 2157) for sequence number 1773801 (highlighted in Figure 16-15). Immediately prior to this, there is no evidence of duplicate ACKs or SACKs.

Figure 16-15 The sender experiences its first timeout when $RTO = 1.57s$. In this case, the sender declares the timeout to be spurious and undoes the modifications it made to its congestion control state.

In Figure 16-15, at time 62.486, about 1.58s have elapsed since the last ACK was received, but according to Figure 16-8, the estimated RTT at this point is only about 800ms. Thus, we may conclude this retransmission to be the result of a retransmission timer expiration. This places TCP into the Loss state, which ordinarily causes a drastic reduction of `cwnd` and effectively restarts the TCP in slow start. Here, TCP sets `cwnd = 1` and `ssthresh = 5`, placing TCP in slow start, as expected. The timeout also forces any stored SACK information to be discarded. However, the receiver continues to send SACK information, so the sender can still make use of new SACK information it receives.

Note

TCP is supposed to “forget” its knowledge about received SACK information when experiencing a timeout because of the possibility that a receiver may renege on SACK information it provided earlier. This is suggested by [RFC2018] because of the (obscure) possibility that a receiver may wish to adjust its buffering so as to delete out-of-order data it has accumulated. Although not common, such behavior is permitted. When a receiver reneges, it is required to include the most

recently received data blocks in the first SACK block of ACKs generated, even if it is discarded. Except for this block, additional blocks must cease to report data no longer being held at the receiver.

Most interestingly here, this congestion action is undone. As discussed earlier, the Eifel Response Algorithm can be invoked when TCP believes a retransmission timeout to be erroneous. In this case, it is declared erroneous because of evidence in the timestamp. The ACK received at time 62.757 for sequence number 1775201 (packet 2158) carries a TSOPT with TSV of 17152514. However, the retransmission has the TSV of 17155274. Because the TSER field in the ACK covering the retransmitted segment is earlier than the retransmission, the hole the retransmission was attempting to fill was not really a hole at all. Instead, the expiration of the retransmission timer must have been erroneous.

By declaring the retransmission timer expiration to be erroneous and invoking an Eifel-like response algorithm, TCP restores *cwnd* and *ssthresh* to their former values of 10 and immediately shifts to a normal operating state. This activates the congestion avoidance algorithm, and TCP continues without much fuss.

16.5.6.2 Fast Retransmit (Event 8)

The arrival of a duplicate ACK for sequence number 1789201 carrying SACK block [1792001,1793401] at time 67.510 (packet 2179) places TCP into the Disorder state once again. The largest sequence number sent so far when this state is entered is 1806000. Additional arriving SACKs trigger entry into the Recovery state and sending of another fast retransmit at time 67.550 for sequence number 1789201 (packet 2182). This reduces *ssthresh* to 5 and *cwnd* begins shrinking until it also reaches 5. Recovery is complete with the arrival of an ACK at time 67.916 containing sequence number 1806001 (packet 2197).

16.5.6.3 CWR Again (Event 9)

There is another local congestion event at time 77.121 when *cwnd* = 18. This sets *ssthresh* = 9 and places TCP into the CWR state once again. However, the reduction of *cwnd* in the CWR state this time is interrupted early by a timeout, when *cwnd* has been reduced by only 1, to 8.

16.5.6.4 Second Timeout (Event 10)

Another retransmission timeout triggers a retransmission at time 78.515 for sequence number 2175601 (not pictured). This sets *ssthresh* = 1; *ssthresh* is still 9 and the retransmitted segment carries the TSOPT TSV value of 17171306. As with timeout event 7, this congestion action is also undone, by the arrival of the ACK at time 80.093 for sequence number 2179801 (packet 2641) containing the TSOPT TSER value of 17169948. When this happens, the flight size estimate is $2,184,001 + 1400 - 2,179,801 = 5600$ bytes (four packets). *cwnd* were immediately restored to its pre-timeout condition (8), this would allow four packets to be immediately

injected into the network. Doing so is considered undesirable because it may lead to increased changes of dropped packets because of burstiness.

To prevent this bursty behavior, this Linux TCP implementation has a congestion window moderation procedure, which limits the maximum number of packets generated in response to a single ACK to `maxburst`, with a value of 3 packets in this example. In this case, `cwnd` is therefore set to $(\text{flight size} + \text{maxburst}) = 4 + 3 = 7$. This regulation is related to the parameter of the same name proposed for TCP and evaluated using the NS-2 network simulator. This simulator has been used extensively in the exploration and development of new TCP algorithms [NS2].

16.5.6.5 Timeout and Final Recovery (Event 11)

At time 88.929 a retransmission timer has expired and a retransmission for sequence number 2185401 occurs, as depicted in Figure 16-16.

Figure 16-16 A retransmission timer expires, initiating a timeout-based retransmission that cannot be undone. TCP continues in slow start.

The expiring timer places the sender into slow start with `ssthresh` = 5. This time, TCP is not able to undo the timeout, so `cwnd` is set to 1 and slow start progresses. This can be seen more clearly from the flow trace (see Figure 16-17).

Figure 16-17 In Wireshark, the slow start behavior is apparent after a retransmission timeout. Each arriving ACK liberates two or three packets.

The retransmission for sequence number 2185401 is highlighted. Following the retransmission, we see the typical slow start behavior we saw during the beginning of the connection, when each arriving ACK liberates two or three packets, depending on how many packets were covered by the ACK. By time 89.434, when `cwnd` has reached `ssthresh` at 5, TCP continues in congestion avoidance.

16.5.7 Connection Completion

The final exchange of packets commences with the sender's transmission of a FIN at time 99.757. Following this transmission, 13 ACKs arrive followed by the receiver's FIN. The very last packet (a final ACK) is sent at time 100.476. This exchange is depicted in Figure 16-18.

The largest sequence number sent is $2620801 + 640 - 1 = 2621440$, equivalent to the size of the overall transfer, 2.5MB. At time 99.757, $(2,619,401 + 1400 - 2,594,201)/1400 + 1 = 20$ packets are outstanding. The arrival of 13 ACKs (7 of which ACK two packets each) covers the whole window of $(2 \times 7) + (13 - 7) = 20$ packets. Note that the ACK arriving at time 100.474 ACKs the final two packets of sizes 1400 and 640 bytes, respectively: $2,621,442 - 2,619,401 = 1400 + 640$.

Figure 16-18 During the connection closing procedure, the receiver produces 13 pure ACKs to indicate that it has received all of the data the sender has produced. The final FIN-ACK exchange completes closure of the other half of the connection. Note that the FIN segments contain valid ACK numbers.

This extended example illustrates most of the algorithms described so far and includes aspects of the basic TCP algorithms (slow start, congestion avoidance), selective acknowledgment, rate halving, as well as some newer procedures such as spurious RTO detection. We now discuss some modifications and capabilities that are less widespread, more speculative, or more recent. The Linux TCP stack implements many of these procedures, but not all of them are enabled by default. Frequently, a small change using the `sysctl` program is sufficient to experiment with them. More recent versions of the Windows stack (i.e., Windows Vista and later) also implement improvements beyond the features discussed so far.

16.6 Sharing Congestion State

The discussion so far and the example we have just seen have focused on how a single TCP connection adapts to congestion along the path. If other connections between the same hosts are made later, these subsequent connections typically

have to establish their own values for `ssthresh` and `cwnd` over time as described previously. In many cases, subsequent connections could possibly learn of these values from earlier connections to the same hosts or from other currently active connections to the same hosts. This idea involves sharing the congestion state across multiple connections in the same machine. An early description in [RFC2140], entitled “TCP Control Block Interdependence,” describes how this might be accomplished. This work notes the difference between temporal sharing (new connections share information with others that are now CLOSED) and ensemble sharing (new connections share state with other active connections).

In an effort to generalize this idea and extend it to protocols and applications other than TCP, [RFC3124] describes the Congestion Manager, which provides a local operating system service available to protocol implementations to learn information such as path loss rate, estimated congestion, RTT, and so forth to destination hosts.

In Linux, this idea is made available in the same subsystem that contains routing information and is known as destination metrics, which we saw in Chapter 15. These metrics are enabled (but they were disabled for the extended example by setting the `sysctl` variable `net.ipv4.tcp_no_metrics_save` to 1). When a TCP connection goes to the CLOSED state, the following information is saved: RTT measurements (`rtt` and `rttvar`), an estimate of reordering, and the congestion control variables `cwnd` and `ssthresh`. These are used when new connections to the same destination start to help initialize the corresponding measurements.

16.7 TCP Friendliness

TCP being the dominant transport protocol on the Internet, it is commonplace for several TCP connections to be sharing one or more routers along their delivery paths. While they do not always share bandwidth equally in such circumstances, they do at least react to the dynamics of other TCP connections as they come and go over time. This is not guaranteed to be the case, however, when TCP competes for bandwidth with other (non-TCP) protocols, or when it competes with a TCP using some alternative set of controls on its congestion window.

To provide a guideline for protocol designers to avoid unfairly competing with TCP flows when operating cooperatively on the Internet, researchers have developed an equation-based rate control limit that provides a bound of the bandwidth used by a conventional TCP connection operating in a particular environment. This method is called TCP Friendly Rate Control (TFRC) [RFC5348][FHPW00]. It is designed to provide a sending rate limit based on a combination of connection parameters and with environmental factors such as RTT and packet drop rate. It also gives a more stable bandwidth utilization profile than conventional TCP, so it is expected to be appropriate for streaming applications that use moderately large packets (e.g., video transfer). TFRC uses the following equation to determine a sending rate:

$$X = s / \left(R \sqrt{2bp/3} \right) + 3pt_{\text{RTO}} (1 + 32p^2) \sqrt{3bp/8} \quad [2]$$

Here, X is the throughput rate limit (bytes/second), s is the packet size (bytes, excluding headers), R is the RTT (seconds), p is the number of loss events as a fraction of packets sent $[0,1.0]$, t_{RTO} is the retransmission timeout (seconds), and b is the maximum number of packets acknowledged by a single ACK. The value of t_{RTO} is recommended to be $4R$, and the recommended value of b is 1.

The TCP sending rate can be expressed another way, based on how it adjusts its window in response to receiving a good ACK during congestion avoidance. Recall from the earlier discussion that standard TCP, when using the congestion avoidance algorithm, increases cwnd by an additive amount of $1/\text{cwnd}$ for each arriving good ACK and decreases it by a multiplicative factor of one-half on a loss event. This is called additive increase/multiplicative decrease (AIMD) congestion control, and we can produce a generalized AIMD congestion avoidance equation by replacing the values of $1/\text{cwnd}$ and $1/2$ with variables a and b as follows:

$$\text{cwnd}_{t+1} = \text{cwnd}_t + a / \text{cwnd}_t$$

$$\text{cwnd}_{t+1} = \text{cwnd}_t \cdot b \cdot \text{cwnd}_t$$

Based on results from [FHPW00], this equation gives TCP the following sending rate, in packets per RTT:

$$T = \frac{\sqrt{\frac{a(2-b)}{2b}}}{\sqrt{p}} \quad [3]$$

For regular TCP, where $a = 1$ and $b = 0.5$, this simplifies to $T = 1.2/\sqrt{p}$, known as the simplified standard TCP response function. It relates the speed of TCP (regulation of cwnd) to the packet drop rate the TCP experiences, without accounting for retransmission timeouts. When TCP is not limited by other factors (sender's or receiver's buffers, window scaling, etc.), this relationship governs TCP's performance in benign operating environments.

Any alteration to TCP's response function obviously affects the way it (or another protocol implementing a similar congestion control scheme) competes with standard TCP. Therefore, new proposed congestion control schemes are typically analyzed using a measure of relative fairness. Relative fairness gives the ratio of the speed of the protocol using a modified congestion control scheme relative to standard TCP, as a function of the packet drop rate. This is a strong indicator of how fair any such modified schemes are with respect to sharing bandwidth across a common Internet path.

Note that understanding these equations is only the first step in creating a speed regulation regime that competes fairly with standard TCP. The details of implementing TFRC for any particular protocol can be subtle and include how to correctly measure the RTT, loss event rate, and packet size. These issues are discussed in some detail in [RFC5348].

16.8 TCP in High-Speed Environments

In high-speed networks with large BDPs (e.g., WANs of 1Gb/s or more), conventional TCP may not perform well because its window increase algorithm (the congestion avoidance algorithm, in particular) takes a long time to grow the window large enough to saturate the network path. Said another way, TCP can fail to take advantage of fast networks even when no congestion is present. This issue arises primarily from the fixed additive increase behavior of congestion avoidance. If we consider a TCP using 1500-byte packets operating over a 10Gb/s long-distance link, some 83,000 segments are required to be outstanding in order to fully utilize the available bandwidth, assuming no packet drops or errors in five billion packets. For an RTT of 100ms, this takes about 1.5 hours to achieve. In order to address this deficiency, a number of researchers and developers have explored ways to alter TCP in order for it to perform better in such networks, while retaining a degree of fairness to standard TCP, especially for more common lower-speed environments.

16.8.1 HighSpeed TCP (HSTCP) and Limited Slow Start

The experimental HighSpeed TCP (HSTCP) specifications [RFC3649][RFC3742] propose to alter the standard TCP behavior when the congestion window is larger than a base value `Low_Window`, suggested to be 38 MSS-size segments. This value corresponds to a packet drop rate of 10^3 based on the simplified TCP response function given previously. This function is linear on a log-log plot of sending rate versus packet loss rate, so it is really a power law function.

Note

Functions that form a line on a log-log plot are called *power law functions*. They have equations of the form $y = ax^k$, meaning $\log y = \log a + k \log x$ (a and k are constants). This equation forms a line with slope k on a log-log plot.

To construct the type of power law function required, we select two points and create the equation that describes the line passing between them. Consider two such points as (p_1, w_1) and (P_0, W_0) where $w_1 > W_0 > 0$ and $0 < p_1 < P_0$. On a linear plot, this would form a line with slope $(w_1 - W_0)/(p_1 - P_0)$, but on a log-log plot it forms a line with slope $S = (\log w_1 - \log W_0)/(\log p_1 - \log P_0)$. Then, based on the

equation in the Note, we have $w = Cp^S$, and we require some point, say (P_0, W_0) , to determine C . After some algebra, we find that $C = P_0^{-S} W_0$, meaning $w = p^S P_0^{-S} W_0$.

In Figure 16-19, we see a plot of both the conventional TCP response function and a proposed response function for HSTCP based on the point $(P_0, W_0) = (.0015, 31)$ and $S = -0.82$. Note that for larger packet drop rates (over about .001) the response functions are the same, so these equations apply only for a certain maximum value of p . Comparing the two lines, when the packet drop rate is small enough, HSTCP is allowed to send more aggressively.

Figure 16-19 With HighSpeed TCP, the TCP response function is altered to be more aggressive for low packet drop rates and large windows, leading to higher throughputs for high bandwidth-delay-product networks. Image from presentation by Sally Floyd to IETF TWVWG, Mar. 2003.

To have TCP achieve this response function, the congestion avoidance procedure is modified to take into account the current size of the window when making changes. This takes place, as with conventional TCP, upon the arrival of a good ACK. The response for a good arriving ACK is generalized as follows:

$$cwnd_{t+1} = cwnd_t + a(cwnd_t) / cwnd_t$$

When responding to a congestion event (e.g., packet loss, ECN indication), it responds as follows:

$$cwnd_{t+1} = cwnd_t - b(cwnd_t) * cwnd_t$$

Here, $a()$ is the additive increase function and $b()$ is the multiplicative decrease function. In this generalization of standard TCP, they are functions of the current window size. To achieve the desired response function, we start by generalizing from equation [3]:

$$W_0 = \frac{\sqrt{\frac{a(w)(2 - b(w))}{2b(w)}}}{\sqrt{P_0}}$$

This gives:

$$a(w) = 2P_0 W_0^2 b(w) / (2 - b(w))$$

This relationship does not have a unique solution—that is, there are many combinations of $a()$ and $b()$ that satisfy the relationship, even though some of them may not be practical or desirable for deployment.

Additional details of the changes proposed to the congestion avoidance procedure for TCP suggested by HSTCP are available in [RFC3649]. A companion document [RFC3742] describes how slow start can be modified to help TCP obtain a working congestion window in such environments. This is called limited slow start and is designed to slow down slow start, so that a TCP operating with large windows (thousands or tens of thousands of packets) does not double its window in one RTT.

With limited slow start, a new parameter called `max_ssthresh` is introduced. This value is not the maximum value of `ssthresh` but instead a threshold for `cwnd` that works as follows: If `cwnd` $\leq$ `max_ssthresh`, slow start proceeds as normal. If `max_ssthresh` $<$ `cwnd` $\leq$ `ssthresh`, then `cwnd` is increased by at most (`max_ssthresh`/2) SMSS per RTT. This is accomplished by modifying the management of `cwnd` during slow start as follows:

```
if (cwnd <= max_ssthresh) {
    cwnd = cwnd + SMSS           (regular slow start)
} else {
    K = int(cwnd / (0.5 * max_ssthresh))
    cwnd = cwnd + int((1/K)*SMSS) (limited slow start)
}
```

A suggested possible initial value for `max_ssthresh` is 100 packets, or 100*SMSS in bytes.

16.8.2 Binary Increase Congestion Control (BIC and CUBIC)

HSTCP is one of several proposals for modifying TCP to provide higher throughput for large BDP networks. While it considers throughput and fairness with respect to conventional TCPs in similar circumstances, and elects to be more

aggressive than standard TCP under certain circumstances, it does not attempt to directly control what happens when HSTCP connections with differing RTTs compete with each other (called “RTT fairness”). This was studied for standard TCP some years back, revealing that TCPs with shorter RTTs obtain a larger share of the bandwidth on shared links as compared to those having larger RTTs, when using the same packet size and ACK strategy [F91]. For TCPs that increase w_{nd} as a function of its size (so-called bandwidth-scalable TCPs), this unfairness can be even more severe. Whether RTT fairness should be considered desirable is subject to debate. Although RTT fairness would seem attractive from first principles, connections with larger RTTs are likely to be using more network resources (e.g., passing through more routers), so it may be reasonable for them to receive somewhat less throughput. In any case, knowing just how RTT (un)fairness behaves is a driving factor behind the popular TCP variants we explore next.

16.8.2.1 BIC-TCP

In an effort to create a scalable TCP and deal with the issue of RTT fairness, BIC-TCP (formerly called BI-TCP) [XHR04] was developed and deployed in Linux kernels starting with version 2.6.8. The main goal of BIC TCP is to provide linear RTT fairness even though congestion windows may be quite large (which is required to use high-bandwidth links). Linear RTT fairness means that connections receive a bandwidth share inversely proportional to their RTTs, rather than some more complicated or unknown function.

The approach modifies a standard TCP sender with two algorithms: binary search increase and additive increase. These algorithms are invoked after a congestion indication (e.g., packet loss), but only one of the algorithms is in operation at any given point in time. The binary search increase algorithm operates as follows: The current minimum window is the last point at which the connection experienced no packet loss during an entire RTT. The maximum window is the window size at which the connection last experienced loss, if known. The desired window lies somewhere between the two. Using a binary search technique, BIC-TCP selects a trial window in the midpoint of these two values and tries again recursively. If this point shows continued packet loss, it becomes the new maximum and the process repeats. If not, it becomes the new minimum and the process repeats. The process terminates when the difference between the minimum and maximum windows is less than a predefined threshold called the minimum increment, or S_{min} .

The algorithm tends to find the desirable window, also called the saturation point, in a logarithmic number of trials, whereas a standard TCP would require a linear number (half of the difference in window sizes, on average). Thus, this approach makes BIC-TCP more aggressive than standard TCP during certain periods of operation, but this is desired in order to take advantage of high-speed environments without unwanted delay. The protocol is unusual, relative to other proposals, because its increase function is concave at some points—that is, its increase gets smaller as it gets closer to the saturation point. Most other algorithms use large change increments nearest the saturation point.

The additive increase algorithm works as follows: When using binary search increase, the situation can arise where the distance from the current window size to the midpoint (in the sense of the binary search described previously) is large. Increasing the window to the midpoint in one RTT may be ill advised because of the potential for injecting large packet bursts into the network. This is prevented by the additive increase algorithm, which is invoked when the distance to the midpoint from the current window is more than some amount $S_{\max}$. When this happens, the increment is limited to $S_{\max}$ per RTT, called window clamping. Once the midpoint is closer than $S_{\max}$ to the trial window, binary search increase takes over. Overall, upon detection of a loss, the window is reduced by a multiplicative factor α , and its growth starts again with additive increase and switches to binary search once the desired increase amount is less than $S_{\max}$. The authors call the combined algorithms binary increase, or BI.

When the window grows beyond the current maximum, or no maximum is yet known because no loss event has occurred, it must be established. This is accomplished by a procedure known as max probing. The purpose of max probing is to use bandwidth when it becomes available. It proceeds in a way symmetric to the additive increase and binary increase algorithms. It starts in small initial increments, followed by larger increments if no congestion is indicated. The approach shows good stability because small changes are made near the saturation point, where the network is believed to be operating near its greatest capacity.

Linux (kernels 2.6.8 through 2.6.17) includes an implementation of BIC-TCP that is enabled by default. Four `sysctl` parameters control its operation: `net.ipv4.tcp_bic`, `net.ipv4.tcp_bic_beta`, `net.ipv4.tcp_bic_low_window`, and `net.ipv4.tcp_bic_fast_convergence`. The first Boolean variable controls whether BIC is used (as opposed to the conventional fast retransmit/recovery procedures). The next contains a scaling factor for `cwnd` to determine $S_{\max}$ (default 819). The next parameter controls the minimum size of the congestion window before the BIC-TCP control algorithms take over. Its default value is 14, meaning that for small window values standard TCP congestion control is used. The last parameter is a flag, enabled by default. When set, it affects the way the new maximum and target windows are selected when the binary increase algorithm is in a downward trend. During a window reduction, the new maximum and minimum windows are set to the current and scaled (down by a factor of `beta`) values of `cwnd`, respectively. If fast convergence is enabled and the value of the new maximum is less than its previous value before it was set to `cwnd`, the value of the maximum window is further reduced between the average of it and the minimum window. After this, whether or not fast convergence is enabled, the target window is the average of the maximum and minimum values. This helps to achieve even bandwidth sharing more quickly when multiple BIC-TCP flows are sharing the same router.

16.8.2.2 CUBIC

The authors of BIC-TCP revised their basic algorithms to form a new control algorithm called CUBIC [HRX08]. It has been the default congestion control algorithm used in Linux TCP since kernel version 2.6.18. It addresses concerns raised that BIC-TCP may be too aggressive under some circumstances. It also simplifies the window growth procedures. Instead of using a threshold ($s_{\max}$) to decide when to invoke the binary search increase versus additive increase, an odd-degree polynomial function, in particular a cubic function, is used instead to control the window increase function. Cubic functions can have both convex and concave portions, meaning that they can grow more slowly in some portions (concave) and more quickly in others (convex). Until BIC and CUBIC, virtually all of the TCP literature advocated convex window growth functions. The specific window growth function, used by CUBIC to set $cwnd$, is as follows:

$$W(t) = C(t - K)^3 + W_{\max}$$

In this equation, $W(t)$ is the window at time t . C is a constant parameter (default 0.4), t is the elapsed time in seconds since the last window reduction, and K is the time period the function takes to increase W to $W_{\max}$ when there is no further loss event. $W_{\max}$ is the last window size prior to the last window adjustment. K can be calculated as follows:

$$K = \sqrt[3]{\frac{W_{\max}}{C}}$$

where β is the multiplicative decrease constant (default 0.2). An illustration of the CUBIC window growth function for $\beta = 2.71$, $W_{\max} = 10$, and $C = 0.4$ on the interval $t = [0, 5]$ is shown in Figure 16-20.

This figure illustrates how the CUBIC window growth function contains both a concave portion and convex portion. When a fast retransmit occurs, $W_{\max}$ is set to $cwnd$, and new values of $cwnd$ and $ssthresh$ are set to $\beta * cwnd$. CUBIC uses a default value of 0.8 for β . The value $W(t + RTT)$ gives the next target congestion window value. When an additional ACK arrives during congestion avoidance, $cwnd$ is increased by $(W(t + RTT) - cwnd) / cwnd$.

It is worth noting that having t be the amount of elapsed time since the last window reduction event helps to ensure RTT fairness. Instead of changing the window by some fixed amount when ACKs arrive, the window change amount is a function of the elapsed time since the last window change. This decouples the window change operations from the particular pattern of ACK arrivals.

In addition to the cubic operating region, CUBIC also has a “TCP-friendly” region that operates when the window is small to ensure that CUBIC is not

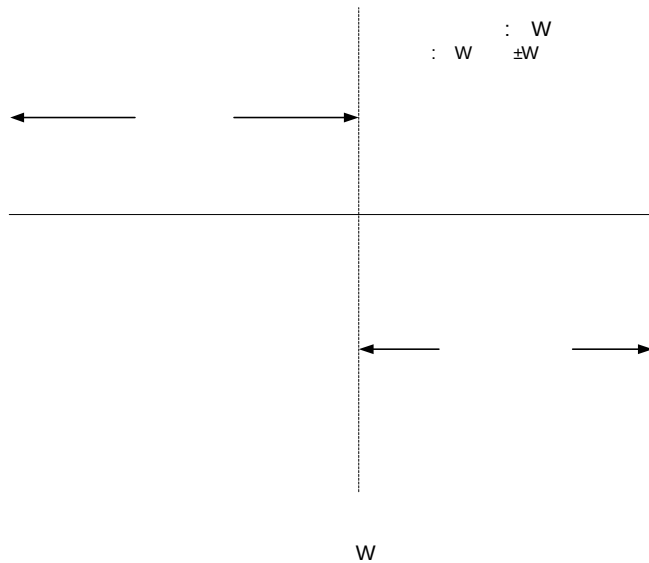


Figure 16-20 The CUBIC window growth function is a cubic function of t . It has a concave portion in the area where $W(t) < W_{\max}$. In this region, CUBIC searches for the saturation point by growing cwnd with decreasing aggressiveness. After $W_{\max}$ is reached, the growth function becomes convex, where it searches by growing cwnd with increasing aggressiveness.

penalized relative to regular TCP. More specifically, the window size of standard TCP in terms of the elapsed time t , $W_{\text{tcp}}(t)$, is given by

$$W_{\text{tcp}}(t) = 3 \frac{t}{RTT} \left(\frac{1}{1+} \right) + W_{\max}$$

So if cwnd is less than $W_{\text{tcp}}(t)$ when an ACK arrives during congestion avoidance, CUBIC sets $\text{cwnd} = W_{\text{tcp}}(t)$. This ensures TCP friendliness in common low- to moderate-speed networks, where CUBIC would otherwise be disadvantaged.

As mentioned earlier, CUBIC has been the default congestion control algorithm for Linux kernels since 2.6.18. Since kernel version 2.6.13, however, Linux supports pluggable congestion avoidance modules [P07], allowing the user to pick which algorithm to use. The variable `net.ipv4.tcp_congestion_control` contains the current default congestion control algorithm (default: `cubic`). The variable `net.ipv4.tcp_available_congestion_control` contains the congestion control algorithms loaded on the system (in general, additional ones can be loaded as kernel modules). The variable `net.ipv4.tcp_allowed_congestion_control` contains those algorithms permitted for use by applications (either selected specifically or by default). The default supports CUBIC and Reno.

16.9 Delay-Based Congestion Control

The approaches to congestion control we have seen so far are usually triggered by packet loss, detected using some combination of ACKs or SACKs, ECN (if available), and expiration of a retransmission timer. ECN (see Section 16.11) allows a sending TCP to be informed about congestion prior to the need for the network to drop packets, but this requires participation from routers within the network that may not be available. However, even without ECN it is still possible to try to determine from a host whether congestion is about to occur within the network. One clue that congestion may be forming is an increase in measured RTT as the sender injects more packets into the network. We saw this situation in Figure 16-8, where additional packets were being queued rather than delivered, contributing to a higher measured RTT (until packets were ultimately discarded). Several congestion control techniques depend on this observation. They are called delay-based congestion control algorithms, as opposed to the loss-based congestion control algorithms we have seen so far.

16.9.1 Vegas

In 1994, TCP Vegas was introduced [BP95]. It was the first delay-based congestion control approach for TCP published and tested by the community of TCP developers. Vegas operates by estimating the amount of data it expects to transfer in a certain amount of time and comparing this with the amount of data it is actually able to transfer. If the requisite amount of data is not transferred, it is likely to be held up in a router queue along the path. If this condition persists, the Vegas sender slows down. This is in contrast to the standard TCP approach, which forces a packet drop to occur in order to determine the point at which the network is congested.

While in its congestion avoidance phase, during each RTT, Vegas measures the amount of data transferred and divides this number by the minimum delay observed across the connection. It maintains two thresholds, α and β (where $\alpha < \beta$). When the difference in expected throughput (window size divided by the smallest RTT observed) versus achieved throughput is less than α , the congestion window is increased; when it is greater than β , the congestion window is decreased. Otherwise, it is left as is. All changes to the congestion window are linear, meaning the scheme is an additive increase/additive decrease (AIAD) congestion control scheme.

The authors describe α and β in terms of buffer utilization at a bottleneck link. The smallest values of interest are 1 for α and 3 for β . The reasoning behind these values is as follows: At least one packet buffer should be occupied in the network path (i.e., at the queue in the router incident with the minimum-bandwidth link on the path) to keep the network busy. If extra bandwidth becomes available, occupying two additional buffers (up to 3, the value for β) obviates the need to wait an extra RTT in order to inject more, which would be required if Vegas tried to

maintain only one buffer full. Furthermore, having the region () as the operating range leaves some room for minor changes in throughput without causing an immediate change in the window, a form of damping that aims to reduce rate oscillations.

With a slight modification, this approach can also be applied to the slow start period. Here, increasing `cwnd` by 1 for each good ACK is allowed only every other RTT. For those RTTs when it is not increased, a measurement is made to ensure that throughput is increasing. If not, the sender switches to the Vegas congestion avoidance scheme.

Under certain circumstances, Vegas can be “fooled” into believing that the forward-direction delay is higher than it really is. This happens when there is significant congestion in the reverse direction (recall that the paths in the two directions of a TCP connection may be different and have different states of congestion). In such cases, packets (ACKs) returning to the sending TCP are delayed, even though the sender is not really contributing to the (reverse-path) congestion. This causes Vegas to reduce the congestion window even though such an adjustment is not really necessary. This is a potential pitfall for most techniques based on measuring RTT as a basis for congestion control decisions. Indeed, significant traffic in the reverse direction can cause the ACK clock (Figure 16-1) to be significantly perturbed [M92].

Vegas is fair relative to other Vegas TCPs sharing the same path because each pushes the network to hold only a minimal amount of data. However, Vegas and standard TCP flows do not share paths equally. A standard TCP sender tends to fill queues in the network, whereas Vegas tends to keep them nearly empty. Consequently, as the standard sender injects more packets, the Vegas sender sees increased delay and slows down. Ultimately, this leads to an unfair bias in favor of the standard TCP. Vegas is supported by Linux but not enabled by default. For kernels prior to 2.6.13, the Boolean `sysctl` variable `net.ipv4.tcp_vegas_cong_avoid` determines whether it is used (default 0). The variables `net.ipv4.tcp_vegas_alpha` (default 2) and `net.ipv4.tcp_vegas_beta` (default 6) correspond to the alpha and beta described previously but are expressed in half-packet units (i.e., 6 corresponds to 3 packets). The variable `net.ipv4.tcp_vegas_gamma` (default 2) configures how many half-packets Vegas should attempt to keep outstanding during slow start. For kernels after 2.6.13, Vegas must be loaded as a separate kernel module and enabled by setting `net.ipv4.tcp_congestion_control` to `vegas`.

16.9.2 FAST

FAST TCP was developed with particular attention to operations in high-speed environments with large bandwidth-delay products [WJLH06]. Similar to Vegas in spirit, it adjusts the window based on the difference between an expected throughput rate and an experienced rate. It differs from Vegas by adjusting the window based not only on the window size, but also on the difference between

the current and expected performance. It updates the sending rate every other RTT using a rate-pacing technique. If the measured delay is significantly below a threshold, the window is updated aggressively followed by a period when the increase is less aggressive. When the delay increases, the reverse takes place. FAST differs from the other approaches we have discussed because it is the subject of several patents and is being commercialized independently. It has received somewhat less scrutiny from the research community, but an independent evaluation [S09] has shown it to have good stability and fairness properties.

16.9.3 TCP Westwood and Westwood+

TCP Westwood (TCPW) and TCP Westwood+ (TCPW+) aim at handling large bandwidth-delay-product paths by modifying a conventional TCP NewReno sender. TCPW+ is a correction to the original TCPW algorithm, so we will just refer to either as TCPW. In TCPW, the sender's eligible rate estimate (ERE) is an estimate of the bandwidth available on the connection. It is continuously computed in a fashion somewhat similar to Vegas (based upon the difference between an expected and an achieved rate), but with a variable measurement interval for the rates based on the dynamics of ACK arrivals. When congestion is low, the measurement interval is small, and vice versa. When a packet loss is detected, instead of reducing `cwnd` by half, TCPW computes an estimated BDP (ERE times the minimum RTT observed) and uses this as the new value for `ssthresh`. Agile probing [WYSG05] adaptively and repeatedly sets `ssthresh` when a connection would otherwise operate in slow start. This causes `cwnd` to grow exponentially in cases where `ssthresh` has been increased (by initiating slow start). Westwood can be enabled in Linux kernels after 2.6.13 by loading a TCPW module and setting `net.ipv4.tcp_congestion_control` to `westwood`.

16.9.4 Compound TCP

Starting with Windows Vista, it is possible to choose which congestion control procedure ("provider") TCP should use, in a way similar to Linux's pluggable congestion avoidance modules. One such option (but not the default, except for Windows Server 2008) is called Compound TCP (CTCP) [TSZS06]. CTCP makes window adjustments based upon packet loss, but also based on measured delays. In some sense it is a combination of standard TCP and Vegas, but with the scalability features of HSTCP.

The authors begin by recounting a number of results shown in the Vegas and FAST research that suggest that delay-based congestion control schemes tend to have better utilization, less self-induced packet loss, faster convergence (to the correct operating point), plus better RTT fairness and stabilization. However, as mentioned previously, delay-based approaches tend to lose bandwidth when competing with loss-based congestion control approaches. CTCP attempts to address this situation by combining a delay-based approach with a loss-based approach.

To do this, CTCP introduces a new window control variable called $dwnd$ (the “delay window”). The usable window W then becomes

$$W = \min(cwnd + dwnd, awnd)$$

The handling of $cwnd$ is similar to that of standard TCP, but the addition of $dwnd$ may allow additional packets to be sent if the delay conditions are appropriate. When ACKs arrive during congestion avoidance, $cwnd$ is updated as follows:

$$cwnd = cwnd + 1/(cwnd + dwnd)$$

The management of $dwnd$ is based on Vegas and is nonzero only during congestion avoidance (CTCP uses conventional slow start). As a connection operates, the minimum RTT measured is maintained in the variable $baseRTT$. Then, the difference in expected data outstanding versus the actual amount, $diff$, is computed as follows: $diff = W * (1 - (baseRTT / RTT))$, where RTT is the estimated (smoothed) RTT estimate. The value of $diff$ estimates the number of packets (or bytes) queued in the network. CTCP, like most delay-based schemes, attempts to keep $diff$ at a certain threshold, called θ , in order to ensure that the network remains utilized but not congested. Given this goal, the control process for $dwnd$ is then expressed as follows:

$$dwnd(t+1) = \begin{cases} dwnd(t) + (\alpha * win(t)^k \dots 1)^+, & \text{if } diff < \theta \\ (dwnd(t) \dots \beta * diff)^+, & \text{if } diff \geq \theta \\ (win(t) * (1 \dots \gamma) \dots cwnd(t))^+, & \text{if loss detected} \end{cases}$$

where $(x)^+$ means $\max(x, 0)$. Note that $dwnd$ can never be negative. Rather, it may be zero, in which case CTCP behaves like standard TCP.

In the first case, where the network may be underutilized, CTCP grows $dwnd$ according to the polynomial $\alpha * win(t)^k$. This is a form of binomial increase and accounts for the way CTCP can be made more aggressive (similar to HSTCP) when the buffer occupancy is estimated to be less than θ . In the second case, where the buffer occupancy appears to be growing beyond the desired threshold θ , the constant β dictates how quickly the delay-based component should be reduced (but recall that $dwnd$ is always added to $cwnd$). This is what contributes to CTCP's RTT and TCP fairness. When loss is detected, $dwnd$ has its own multiplicative decrease factor γ applied.

As can be seen, CTCP can be tuned using the parameters α , β , γ , and k . The value of k affects the level of aggressiveness. A value of about 0.8 was desired to be similar to HSTCP, but 0.75 was chosen for implementation reasons. The values of α and β affect smoothness and responsiveness. The default values are 0.125 and 0.5, respectively. For γ , the authors suggest a default value of 30 packets based on empirical evaluation. If this value is too small, there may not be enough packets

outstanding to obtain good delay measurements. Conversely, values that are too large could result in undesirable persistent congestion.

CTCP is relatively new, so further experimentation and evaluation will no doubt be performed to see how well and fairly it competes with standard TCP, and how well it is able to adapt to significant changes in available bandwidth. In a simulation study, the author of [W08] noted that CTCP can perform poorly when network buffers are small (i.e., smaller than 100KB). They also suggest that CTCP can fall victim to some of the problems with Vegas, including rerouting (adapting to new paths with different delays) and persistent congestion. Finally, they observe that if many CTCP flows, each trying to keep 100KB packets in flight, share the same bottleneck link, performance can be poor.

As mentioned previously, CTCP is not enabled by default on most versions of Windows. However, the following command can be used to select CTCP as the congestion provider:

```
C:\> netsh interface tcp set global congestionprovider=ctcp
```

It can be disabled by selecting a different provider (or **none**). CTCP has also been ported to Linux as a pluggable congestion avoidance module but is not included by default.

16.10 Buffer Bloat

Although memory has traditionally been expensive (and remains so for high-end routers), it is now commonplace to find commodity networking equipment that contains a significant amount of memory, potentially multiple megabytes of packet buffers. Perhaps ironically, this large amount of memory (as compared to traditional networking devices) can actually lead to degraded performance for protocols such as TCP. This problem has been termed **buffer bloat** [G11][DHGS07]. It relates to high amounts of latency introduced by queuing delay, primarily at the uplink side of residential gateways and access points in homes and small offices. The standard TCP congestion control algorithms, which tend to keep buffers full at bottleneck links, do not operate well when a large amount of buffering occurs between the sender and receiver because the congestion indicator (a packet drop) takes a long time to be delivered to a sender.

In [KWNP10], the authors find that upload bandwidth in the United States over cable and DSL ranges from about 256Kb/s to 4Mb/s . They also inferred buffer sizing on commodity routers in the range from 16KB to 256KB . Figure 16-21 shows how latency relates to data rate for several buffer sizes to help provide a perspective on these findings.

In this figure, the log-log graph displays the amount of latency experienced by data required to queue for various buffer sizes (1KB – 2MB). Residential Internet upload bandwidth rates (typically 250Kb/s to 10Mb/s) can lead to latencies in the

Figure 16-21 The log-log plot shows the latency due to queuing delay experienced by data in fully congested queues of various sizes. When large buffers remain full (“buffer bloat”), interactive applications can experience unacceptable latencies in the multiple-second range.

multiple-second range if buffers are sized to be a few hundred kilobytes or more. Interactive applications generally require one-way delays to be below 150ms to provide a good quality of experience to users [G114]. Thus, if buffers remain filled to capacity because of one or more large competing uploads (e.g., BitTorrent file sharing), interactive applications can be adversely affected.

Buffer bloat is not a problem in all networking equipment. Indeed, the primary concern appears to be in overbuffered end-user access devices. There are multiple potential ways to deal with the issue, including protocol modifications (e.g., delay-based congestion control such as Vegas, but it may be negatively affected by high jitter [DHGS07]), dynamic buffer sizing at the access devices (suggested in [KWNP10]), or a combination of the two. We next turn to a combination approach that may help the buffer bloat problem but also has a number of other benefits.

16.11 Active Queue Management and ECN

The discussion of TCP’s congestion response so far has assumed that the only way a TCP infers that congestion is happening is observation of packet drops. In particular, routers (the things that are mostly likely to become congested) do not ordinarily help inform the TCP at each host that congestion is imminent. Instead, they simply drop arriving packets when no more buffer space is available (called “drop tail”) and send packets that have already arrived in a first-in-first-out (FIFO) manner. When Internet routers are passive like this (that is, they simply discard packets

when overloaded and provide no feedback regarding their congestion state), there is little a TCP can do other than react after the fact. If, however, these routers had a way to more actively manage their queues (i.e., by using a more sophisticated scheduling and buffer management policy than FIFO/drop tail), perhaps the situation could be improved. If they could also signal their congestion state to TCP endpoints, so much the better.

Routers that apply scheduling and buffer management policies other than FIFO/drop tail are usually said to be *active* and the corresponding methods they use to manage their queues are called *active queue management (AQM)* mechanisms. The authors of [RFC2309] provide a discussion of the potential benefits of AQM. Although AQM can be useful independently, it becomes more useful when routers and switches implementing AQM have a common method for conveying their status to the end systems. For TCP, this is described in [RFC3168] and extended with additional security in an experimental specification [RFC3540]. These RFCs describe Explicit Congestion Notification (ECN), which is a way for routers to mark packets (by ensuring both of the ECN bits in the IP header are set) to indicate the onset of congestion.

Random Early Detection (RED) gateways [FJ93] are one mechanism suggested as being capable of detecting the onset of congestion and controlling the marking of packets. These gateways implement a queue management discipline that measures the average queue occupancy over time. If the occupancy exceeds the minimum (called *minthresh*) and is less than the maximum (called *maxthresh*), a packet is marked with an increasing probability. If the average queue occupancy exceeds *maxthresh*, packets are marked with a configurable maximum probability (called *MaxP*), which could be 1.0. RED can also be configured to drop packets instead of marking them.

Note

The RED algorithm is the basis for a number of variants (e.g., Cisco's WRED, which uses different RED instances based on IP DSCP or precedence values) that are supported on many routers and switches.

When received by a TCP, a congestion mark indicates that the packet has passed through a congested router. Of course, it is the sender (rather than the receiver) that really needs this information in order to react by slowing down. Thus, the receiver echoes this indication back to the sender in a series of ACK packets.

The ECN mechanism operates partially at the IP layer and so is potentially applicable to transport protocols other than TCP, although most of the work on ECN has been with TCP, and it is what we discuss here. When an ECN-capable router experiencing persistent congestion receives an IP packet, it looks in the IP header for an ECN-Capable Transport (ECT) indication (currently defined as either of the two ECN bits in the IP header being set). If set, the transport protocol responsible for sending the packet understands ECN. At this point, the router sets

a Congestion Experienced indication in the IP header (by setting both ECN bits to 1) and forwards the datagram. Routers are discouraged from setting a CE indication when congestion does not appear to be persistent (e.g., upon a single recent packet drop due to queue overrun) because the transport protocol is supposed to react given even a single CE indication.

The TCP receiver observing an incoming data packet with a CE set is obliged to return this indication to the sender (there is an experimental extension to add ECN to SYN + ACK segments as well [RFC5562]). Because the receiver normally returns information to the sender by using (unreliable) ACK packets, there is a significant chance that the congestion indicator could be lost. For this reason, TCP implements a small reliability-enhancing protocol for carrying the indication back to the sender. Upon receiving an incoming packet with CE set, the TCP receiver sets the ECN-Echo bit field in each ACK packet it sends until receiving a CWR bit field set to 1 from the TCP sender in a subsequent data packet. The CWR bit field being set indicates that the congestion window (i.e., sending rate) has been reduced.

Note

Although RED and ECN have been known for nearly two decades, they have not seen widespread Internet deployment. A variety of reasons have been asserted as to why (e.g., difficulty in setting RED parameters, a perception of limited benefits). In 2005, a “reexamination” of ECN [K05] pointed out that using ECN on only data packets limits its benefits substantially. An experimental extension [RFC5562] defines the use of ECN in SYN + ACK packets with the possibility of greatly increasing the utility of ECN for certain workloads (e.g., Web traffic).

A sending TCP receiving an ECN-Echo indicator in an ACK reacts the same way it would when detecting a single packet drop by adjusting `cwnd`, and it also arranges to set the CWR bit field in a subsequent data packet. The prescribed congestion response of the fast retransmit/recovery algorithms is invoked (of course, without the packet retransmission), causing the TCP to slow down prior to suffering packet drops. Note that the TCP should not overreach; in particular, it should not react more than once for the same window of data. Doing so would overly penalize an ECN TCP relative to others.

In Windows Vista and later, ECN needs to be enabled to be used:

```
C:\> netsh int tcp set global ecncapability=enabled
```

In Linux, ECN is enabled if the Boolean `sysctl` variable `net.ipv4.tcp_ecn` is nonzero. The default varies based on which Linux distribution is used, with off being most common. On Mac OS 10.5 and later, the variables `net.inet.tcp.ecn_initiate_out` and `net.inet.tcp.ecn_negotiate_in` control whether ECN is enabled for outgoing traffic and for incoming traffic with ECN flags set, respectively. Of course, without cooperation from routers or switches, the utility

of ECN is limited in any case. Only time will tell if the vision for AQM will ever be fully realized in the global Internet.

Note

RED and ECN have been used successfully in a radically different operating environment from that for which they were designed. Microsoft and Stanford have developed *Data Center TCP* (DCTCP) [A10], which uses RED implemented in layer 2 switches with simplified parameters to mark packets when instantaneous congestion is experienced. They also modify the TCP receiver behavior to set ECN-Echo in ACKs only when the last received packet contains a CE mark. They report a 90% reduction in buffer occupancy for comparable TCP throughput, allowing a tenfold increase in background traffic to be supported.

16.12 Attacks Involving TCP Congestion Control

We have seen already how TCP can be attacked by generating packets that cause TCP's connection state machine to terminate the connection. TCP can also be attacked (or at least induced to behave in peculiar ways) when operating in the ESTABLISHED state. Most attacks on TCP congestion control attempt to force a TCP to send faster or slower than it would under ordinary circumstances.

Perhaps the earliest attack involves the fabrication of ICMPv4 Source Quench messages. When these are delivered to a host running TCP, any connection to the IP address contained in the offending datagram inside the ICMP message slows down. While this may have been a vulnerability some years back, using Source Quench messages for congestion control has been deprecated for use by routers since about 1995 (via [RFC1812], Section 5.3.6). On the other hand, for end hosts, [RFC1122] stated that a TCP must react to a Source Quench by slowing down. Combining these two facts, the simplest solution is to block ICMP Source Quench traffic at the router or host, and this is now common.

A more sophisticated and more recent set of attacks have been considered by looking at misbehaving receive[SCWA99]. The authors describe three types of attacks that can cause a TCP sender to inject data at a rate faster than intended. Such attacks could be used, for example, to cause a Web client to have an unfair advantage over competing clients. The attacks are named ACK division, DupACK spoofing and Optimistic ACKing and are implemented in a TCP variant the authors (jokingly) call "TCP Daytona."

ACK division operates by producing more than one ACK for the range of bytes being acknowledged. Because the TCP congestion control typically operates based on the arrival of ACK packets (rather than the ACK field contained in the ACK itself), a sending TCP can be induced to increase `seqwnd` faster than it would otherwise. This problem can be mitigated by basing the congestion control computations on the amount of data acknowledged rather than the arrival of a packet, as is done with ABC.

DupACK spoofing causes a sender to increase its congestion window during fast recovery. Recall from the previous discussion that during standard fast recovery, `cwnd` is incremented for each duplicate ACK received. The attack involves creating extra duplicate ACKs that cause this to happen more quickly than intended. This attack is more difficult to defend against, because there is no clean way to map received duplicate ACKs to the segments they acknowledge (a nonce an associated value that changes with time, which we discuss in Chapter 18, would solve this problem). While the Timestamps option relates to this problem, it is an option and can be disabled on a per-connection basis. The best approach to addressing this problem appears to be modification of the sender side to limit the amount of outstanding data during recovery.

Optimistic ACKing involves producing ACKs for segments that have not yet arrived. Because TCP's congestion control computations are based on end-to-end RTTs, ACKing data that has not yet arrived causes the sender to react faster than it would because it is fooled into believing the actual RTT is smaller. Furthermore, there is little penalty for doing this, as a sender typically ignores ACKs for data it has not yet sent. While this approach does not preserve data reliability at the TCP layer as the other attacks do (i.e., ACKed data could still be lost), it is frequently the case (e.g., in HTTP/1.1) that missing data can be reconstructed by an application- or session-layer protocol. The authors describe a cumulative nonce that can address this problem and a way to alter the sizes of sent segments over time to better match up ACKs with sent segments. When the ACKs do not correspond, the sender can take action.

The problems described for misbehaving receivers have also received attention with respect to ECN by some of the same authors. Recall that with AQM using ECN, the TCP receiver returns the ECN indication to the sender in an ACK. The sender is then supposed to respond by slowing down. If the receiver fails to return the ECN indications to the sender (or routers in the network clear the indicators), the sender would never be informed of congestion and would not slow down. In [RFC3540], the authors describe an experimental way to use the ECT bit field of the ECN field (2 bits) of an IP packet as a form of nonce. The sender places a random binary value in the field, and the receiver returns a 1-bit sum (an XOR operation) of the values of this field over time. When generating an ACK, the receiver places the sum bit 7 of the TCP header (currently reserved as zero). A misbehaving receiver has a 50/50 chance of guessing the sum. Because each packet represents an independent trial and a successful misbehaving receiver must have every sum correct, its chance of doing so is $1/2^k$ for k packets (vanishingly small for a connection of any reasonable duration).

16.13 Summary

TCP was designed as the primary reliable transport protocol for the Internet. Although its initial design included a flow control capability, used to cause a

sender to slow down when a receiver could not keep up, no provision was made initially for preventing the sender from overwhelming the network in between. In the late 1980s, the slow start and congestion avoidance algorithms were developed to regulate a TCP sender's aggressiveness so as to avoid losing packets because of congestion in the network. These algorithms depend on using an implicit signal, packet loss, and an indicator of congestion. They are triggered when loss is detected, either by the fast retransmit algorithm or by retransmission timeouts.

Slow start and congestion avoidance regulate a sender's operation by introducing a congestion window at the sender. This is used in conjunction with the conventional window (based on window advertisements provided by the receiver). A standard TCP limits its window to the minimum of the two. Slow start grows the value of the congestion window exponentially with time, and congestion avoidance grows it about linearly with time. Only one of the two algorithms is in operation at any one time, and this decision is made by comparing the current value of the congestion window to the slow start threshold: if the congestion window exceeds the threshold, congestion avoidance takes control; otherwise, slow start is used. Slow start is used initially when TCP establishes its connection and after restart conditions due to timeouts. It can also be used when a connection has gone idle for a significant amount of time. The slow start threshold is adjusted dynamically during the course of the connection.

Congestion control has been a significant focus of the networking research community over the years. After more experience was gained with TCP and its slow start and congestion avoidance procedures, a number of improvements have been suggested, implemented, and standardized. By keeping track of when TCP is recovering from a collection of lost packets, the NewReno variant of TCP avoids some of the stalls that can occur with Reno variants when multiple packets are dropped in a single window of data. SACK TCP can improve upon NewReno's behavior by permitting the sender to intelligently repair more than one packet drop per RTT. With SACK TCP, careful accounting must be established to ensure that the sender is not overly aggressive with respect to other TCPs with which it may be sharing an Internet path.

Some of the more recent changes to TCP congestion management include rate halving, congestion window validation and moderation, and "undo" procedures. The rate-halving algorithm causes the congestion window to reduce gradually after detected loss events instead of reducing it immediately. Congestion window validation tries to ensure that the congestion window is not overly large if a sending application has been idle or unable to send for some time. Congestion window moderation limits the size of a burst in response to the receipt of a single ACK. The "undo" procedures, such as the Eifel Response Algorithm, undo congestion window modifications if the packet loss signal is deemed to be spurious, a condition detectable using a number of techniques. In such cases, the negative impact on performance by reducing the congestion window is minimized by restoring the congestion state to its condition prior to the reduction of the congestion window.

After significant experience with TCP, it was observed that the congestion avoidance procedure can take a long time to find and exploit additional bandwidth that becomes available. As a result, numerous proposals for “bandwidth-scalable” TCP variants have been made. One of the better-known versions (within the IETF) is HSTCP, which allows the congestion window to grow much more aggressively in operating regimes where few packets are dropped and windows are large, as compared with conventional TCP. Subsequent suggestions have included FAST and CTCP, which base their window growth procedures on packet loss and latency measures. Widely deployed in Linux, the BIC-TCP and CUBIC algorithms use growth functions that are convex in some portions and concave in others. This supports small window changes during the saturation point, leading to enhanced stability at the possible cost of somewhat sluggish response to new available bandwidth (but still faster than standard TCP).

A significant change to the operation of TCP and Internet routers has been proposed with the specification of Explicit Congestion Notification (ECN), which would allow TCP to detect the onset of congestion before a packet loss is experienced. Although simulations and research results have shown this to be desirable, it requires a moderate change to TCP implementations and a significant change to the way Internet routers operate. The extent to which this capability is deployed remains to be seen.

Although TCP provides the most widely used method for reliably moving data on the Internet, it does not implement much in the way of its own security. It is generally vulnerable to packet-forging attacks that can cause disruptions of connections; an attacker need only have a good guess at a viable (in-window) sequence number to launch such attacks. In addition, modification of the ACK stream (or ECN bits, if they are supported) can induce a sender to behave in ways that are unfair to other TCP connections. Furthermore, nothing physically prevents an overly aggressive sender from simply violating all congestion control rules.

Combining all of the various algorithms and techniques developed for TCP into a single TCP implementation is not an easy task (Linux 2.6.38 TCP/IPv4 is about 20,000 lines of C code), and analyzing traces of a real-world TCP in action can be time-consuming. Tools such as `astcpdump`, Wireshark, and `tcptrace` make this job considerably easier. Because of its dynamic adaptation to the performance of the network, understanding TCP's behavior is most easily accomplished with visualization techniques based on time-series plots, such as those used in this chapter.

16.14 References

[A10] M. Alizadeh et al., “Data Center TCP (DCTCP),” *Proc. ACM SIGCOMM*, Aug./Sept. 2010.

[ASA00] A. Aggarwal, S. Savage, and T. Anderson, “Understanding the Performance of TCP Pacing,” *Proc. INFOCOM*, Mar. 2004.

- [BP95] L. Brakmo and L. Peterson, "TCP Vegas: End to End Congestion Avoidance on a Global Internet," *IEEE JSAC* 13(8), Oct. 1995.
- [DHGS07] M. Dischinger, A. Haeberlen, K. Gummadi, and S. Saroiu, "Characterizing Residential Broadband Networks," *Proc. ACM IMC*, Oct. 2007.
- [F91] S. Floyd, "Connections with Multiple Congested Gateways in Packet-Switched Networks, Part 1: One-Way Traffic," *ACM Computer Communication Review* 21, 1991.
- [FF96] S. Floyd and K. Fall, "Simulation-Based Comparisons of Tahoe, Reno, and SACK TCP," *ACM Computer Communications Review* July 1996.
- [FHPW00] S. Floyd, M. Handley, J. Padhye, and J. Widmer, "Equation-Based Congestion Control for Unicast Applications," *Proc. ACM SIGCOMM*, Aug. 2000.
- [FJ93] S. Floyd and V. Jacobson, "Random Early Detection Gateways for Congestion Avoidance," *IEEE/ACM Transactions on Networking* 1(4), Aug. 1993.
- [G11] J. Gettys, "Bufferbloat: Dark Buffers in the Internet," *Internet Computing* May/June 2011.
- [G114] International Telecommunication Union Recommendation G.114, "One-Way Transmission Time," May 2003.
- [H96] J. Hoe, "Improving the Start-up Behavior of a Congestion Control Scheme for TCP," *Proc. ACM SIGCOMM*, Aug. 1996.
- [HRX08] S. Ha, I. Rhee, and L. Xu, "CUBIC: A New TCP-Friendly High-Speed TCP Variant," http://netsrv.csc.ncsu.edu/export/cubic_a_new_tcp_2008.pdf
- [J88] V. Jacobson, "Congestion Avoidance and Control," *Proc. ACM SIGCOMM*, Aug. 1988. This paper was later updated in 1992 to include M. Karels as coauthor. The update is available at <http://www-nrg.ee.lbl.gov/papers/congavoid.pdf>
- [J90] V. Jacobson, "Modified TCP Congestion Avoidance Algorithm," posting to the end2end-interest group mailing list, Apr. 1990, available at <ftp://ftp.ee.lbl.gov/email/vanj.90apr30.txt>
- [K05] A. Kuzmanovic, "The Power of Explicit Congestion Notification," *Proc. ACM SIGCOMM*, Aug. 2005.
- [KWNP10] C. Kreibich, N. Weaver, B. Nechaev, and V. Paxson, "Netalyzr: Illuminating Edge Network Neutrality, Security and Performance," *Proc. ACM IMC*, Nov. 2010.
- [LARTC] <http://lartc.org>
- [M92] J. Mogul, "Observing TCP Dynamics in Real Networks," *Proc. ACM SIGCOMM*, Aug. 1992.
- [MM05] M. Mathis, personal communication, Sept. 2005.

[MM96] M. Mathis and J. Mahdavi, "Forward Acknowledgment: Refining TCP Congestion Control," *Proc. ACM SIGCOMM*, Aug. 1996.

[NB08] J. Nievelt and V. Bhanu, "Developing TCP Chimney Drivers for Windows 7," presentation at Microsoft Windows Drivers Developer Conference, 2008.

[NS2] <http://www.isi.edu/nsnam/ns> (also see NS3 at <http://www.nsnam.org>)

[P07] <http://lwn.net/Articles/128681>

[PSCRH] M. Mathis, J. Mahdavi, and J. Semke, "TCP Rate Halving," <http://www.psc.edu/networking/projects/rate-halving>

[RFC1122] R. Braden, ed., "Requirements for Internet Hosts—Communication Layers," *Internet RFC 1122/STD 0003*, Oct. 1989.

[RFC1812] F. Baker, ed., "Requirements for IP Version 4 Routers," *Internet RFC 1812*, June 1995.

[RFC2018] M. Mathis, J. Mahdavi, S. Floyd, and A. Romanow, "TCP Selective Acknowledgment Options," *Internet RFC 2018*, Oct. 1996.

[RFC2140] J. Touch, "TCP Control Block Interdependence," *Internet RFC 2140*, Apr. 1997.

[RFC2309] B. Braden et al., "Recommendations on Queue Management and Congestion Avoidance in the Internet," *Internet RFC 2309* (informational), Apr. 1998.

[RFC2861] M. Handley, J. Padhye, and S. Floyd, "TCP Congestion Window Validation," *Internet RFC 2861* (experimental), June 2000.

[RFC3042] M. Allman, H. Balakrishnan, and S. Floyd, "Enhancing TCP's Loss Recovery Using Limited Transmit," *Internet RFC 3042*, Jan. 2001.

[RFC3124] H. Balakrishnan and S. Seshan, "The Congestion Manager," *Internet RFC 3124*, June 2001.

[RFC3168] K. Ramakrishnan, S. Floyd, and D. Black, "The Addition of Explicit Congestion Notification (ECN) to IP," *Internet RFC 3168*, Sept. 2001.

[RFC3465] M. Allman, "TCP Congestion Control with Appropriate Byte Count - ing (ABC)," *Internet RFC 3465* (experimental), Feb. 2003.

[RFC3517] E. Blanton, M. Allman, K. Fall, and L. Wang, "A Conservative Selective Acknowledgment (SACK)-Based Loss Recovery Algorithm for TCP," *Internet RFC 3517*, Apr. 2003.

[RFC3540] N. Spring, D. Wetherall, and D. Ely, "Robust Explicit Congestion Notification (ECN) Signaling with Nonces," *Internet RFC 3540* (experimental), June 2003.

- [RFC3649] S. Floyd, "HighSpeed TCP for Large Congestion Windows," Internet RFC 3649 (experimental), Dec. 2003.
- [RFC3742] S. Floyd, "Limited Slow-Start for TCP with Large Congestion Windows," Internet RFC 3742 (experimental), Mar. 2004.
- [RFC3782] S. Floyd, T. Henderson, and A. Gurtov, "The NewReno Modification to TCP's Fast Recovery Algorithm," Internet RFC 3782, Apr. 2004.
- [RFC4015] R. Ludwig and A. Gurtov, "The Eifel Response Algorithm for TCP," Internet RFC 4015, Feb. 2005.
- [RFC5348] S. Floyd, M. Handley, J. Padhye, and J. Widmer, "TCP Friendly Rate Control (TFRC): Protocol Specification," Internet RFC 5348, Sept. 2008.
- [RFC5562] A. Kuzmanovic, A. Mondal, S. Floyd, and K. Ramakrishnan, "Adding Explicit Congestion Notification (ECN) Capability to TCP's SYN/ACK Packets," Internet RFC 5562 (experimental), June 2009.
- [RFC5681] M. Allman, V. Paxson, and E. Blanton, "TCP Congestion Control," Internet RFC 5681, Sept. 2009.
- [RFC5690] S. Floyd, A. Arcia, D. Ros, and J. Iyengar, "Adding Acknowledgement Congestion Control to TCP," Internet RFC 5690 (informational), Feb. 2010.
- [RFC6077] D. Papadimitriou, ed., M. Welzl, M. Sharf, and B. Briscoe, "Open Research Issues in Internet Congestion Control," Internet RFC 6077 (informational), Feb. 2011.
- [S09] B. Sonkoly, Fairness and Stability Analysis of High Speed Transport Protocols Ph.D. Thesis, Budapest University of Technology and Economics, 2009.
- [SCWA99] S. Savage, N. Cardwell, D. Wetherall, and T. Anderson, "TCP Congestion Control with a Misbehaving Receiver," ACM Computer Communication Review Apr. 1999.
- [SK02] P. Sarolahti and A. Kuznetsov, "Congestion Control in Linux TCP," Proc. Usenix Freenix Track June 2002.
- [TCPTRACE] <http://jarok.cs.ohiou.edu/software/tcptrace/index.html>
- [TSZS06] K. Tan, J. Song, Q. Zhang, and M. Sridharan, "A Compound TCP Approach for High-Speed and Long-Distance Networks," Proc. INFOCOM, Apr. 2006.
- [W08] X. Wu, "A Simulation Study of Compound TCP," http://www.comp.nus.edu.sg/~wuxiucha/research/reactive/publication/ctcp_study.pdf
- [WJLH06] D. Wei, C. Jin, S. Low, and S. Hegde, "FAST TCP: Motivation, Architecture, Algorithms, Performance," IEEE/ACM Trans. on Networking Mar. 2006.
- [WQOS] <http://technet.microsoft.com/en-us/network/bb530836.aspx>

[WYSG05] R. Wang, K. Yamada, M. Sanadidi, and M. Gerla, "TCP with Sender-Side Intelligence to Handle Dynamic, Large, Leaky Pipes," IEEE JSAÇ 23(2), Feb. 2005.

[XHR04] L. Xu, K. Harfoush, and I. Rhee, "Binary Increase Congestion Control for Fast Long-Distance Networks," Proc. INFOCOM, Mar. 2004.

17

TCP Keepalive

17.1 Introduction

Many newcomers to TCP/IP are surprised to learn that no data whatsoever flows across an idle TCP connection. That is, if neither process at the ends of a TCP connection is sending data to the other, nothing is exchanged between the two TCP endpoints. There is no polling, for example, as you might find with other networking protocols. This means that we can start a client process that establishes a TCP connection with a server and walk away for hours, days, weeks, or months, and the connection should remain up. In theory, intermediate routers can crash and reboot, data lines may go down and back up, but as long as neither host at the ends of the connection reboots (or changes its IP address), the connection remains established. This is how TCP/IP was designed.

Note

The previous statement assumes that neither application—neither the client nor the server—has application-level timers to detect inactivity, causing either application to terminate. It also assumes that no intermediate router is keeping state about the connection (such as a NAT box) that is required for proper operation that it might delete because of inactivity or lose because of system failure. In today's Internet, these are big assumptions.

Under some circumstances, it is useful for a client or server to become aware of the termination or loss of connection with its peer. In other circumstances, it is desirable to keep a minimal amount of data flowing over a connection, even if the applications do not have any to exchange. TCPkeepalive provides a capability useful for both cases. Keepalive is a method for TCP to probe its peer without affecting the content of the data stream. It is driven by a keepalive timer. When the timer fires, a keepalive probe (keepalive or short) is sent, and the peer receiving the probe responds with an ACK.

Note

Keepalives are not part of the TCP specification. The Host Requirements RFC [RFC1122] says that this is because they could (1) cause perfectly good connections to break during transient Internet failures, (2) consume unnecessary bandwidth, and (3) cost money for an Internet path that charges for packets. Nevertheless, most implementations provide the keepalive capability.

TCP keepalive is a controversial feature. Many feel that polling of the other end has no place in TCP and should be done by the application, if desired. On the other hand, if many applications require such functionality, it is convenient to place it in TCP so that its implementation can be shared. The keepalive is an optionally enabled feature that can cause an otherwise good connection between two processes to be terminated because of a temporary loss of connectivity in the network joining the two end systems. For example, if the keepalive probes are sent during the time that an intermediate router has crashed and is rebooting, TCP incorrectly thinks its peer host has crashed.

The keepalive feature was originally intended for server applications that might tie up resources on behalf of a client and want to know if the client host crashes or goes away. Using TCP keepalive to detect dead clients is most useful for servers that expect to have a relatively short-duration dialogue with a noninteractive client (e.g., Web servers, POP and IMAP e-mail servers). Servers implementing more interactive-style services that last for a long time (e.g., remote login such as ssh and Windows Remote Desktop) might wish to avoid using keepalives.

A common example showing the utility of the keepalive feature nowadays is when a user uses the ssh (secure shell) remote login program to log in to a remote host through a NAT router. If the user were to establish the connection, do some work, then just power off the computer at the end of the day, without logging off, a half-open connection would be left. In Chapter 13 we showed that sending data across a half-open connection causes a reset to be returned, but that was from the server end, where the client was sending the data. If the client disappears, leaving the half-open connection on the server's end, and the server is waiting for some data from the client, the server will wait forever. The keepalive feature is intended to detect these half-open connections from the server side.

Another reason for using keepalives is somewhat the reverse. If the user does not power off the computer but instead leaves a connection open all night (and wishes to continue using it the next day), the connection goes idle for many hours. In Chapter 7 we discussed how most NAT routers include a timeout mechanism that flushes the state of a connection after some period of inactivity. If the NAT timeout is less than the several hours before the user returns to use the login session, and the NAT is not smart enough to probe the end station to make sure it is still active, or the NAT crashes, the connection is terminated. To avoid this common problem, ssh can be configured to use TCP keepalives. ssh also has the ability to use application-managed keepalives, and the two behave differently, especially with respect to their security properties. (Please see Section 17.3 for more on this.)

17.2 Description

Either end of a TCP connection may request keepalives, which are turned off by default, for their respective direction of the connection. A keepalive can be set for one side, both sides, or neither side. There are several configurable parameters that control the operation of keepalives. If there is no activity on the connection for some period of time (called the `keepalive time`), the side(s) with keepalive enabled sends a keepalive probe to its peer(s). If no response is received, the probe is repeated periodically with a period set by the `keepalive interval` until a number of probes equal to the number `keepalive probes` is reached. If this happens, the peer's system is determined to be unreachable and the connection is terminated.

A keepalive probe is an empty (or 1-byte) segment with sequence number equal to one less than the largest ACK number seen from the peer so far. Because this sequence number has already been ACKed by the receiving TCP, the arriving segment does no harm, but it elicits an ACK that is used to determine whether the connection is still operating. Neither the probe nor its ACK contains any new data (it is "garbage" data), and neither is retransmitted by TCP if lost. [RFC1122] dictates that because of this fact, the lack of response for a single keepalive probe should not be considered sufficient evidence that the connection has stopped operating. This is the reason for the `keepalive probe` parameter setting mentioned previously. Note that some (mostly older) TCP implementations do not respond to keepalives lacking the "garbage" byte of data.

Anytime it is operating, a TCP using keepalives may find its peer in one of four states:

1. The peer host is still up and running and reachable. The peer's TCP responds normally and the requestor knows that the other end is still up. The requestor's TCP resets the keepalive timer for later (equal to the value of the `keepalive time`). If there is application traffic across the connection before the next timer expires, the timer is reset back to the value of `keepalive time`.
2. The peer's host has crashed and is either down or in the process of rebooting. In either case, its TCP is not responding. The requestor does not receive a response to its probe, and it times out after a time specified by the `keepalive interval`. The requestor sends a total of `keepalive probes` of these probes, `keepalive interval` time apart, and if it does not receive a response, the requestor considers the peer's host as down and terminates the connection.
3. The client's host has crashed and rebooted. In this case, the server receives a response to its keepalive probe, but the response is a reset segment, causing the requestor to terminate the connection.
4. The peer's host is up and running but is unreachable from the requestor for some reason (e.g., the network cannot deliver traffic and may or may not

inform the peers of this fact using ICMP). This is effectively the same as state 2, because TCP cannot distinguish between the two. All TCP can tell is that no replies are received to its probes.

The requestor does not have to worry about the peer's host being shut down gracefully and then rebooting (as opposed to crashing). When the system is shut down by an operator, all application processes are terminated (i.e., the peer's process), which causes the peer's TCP to send a FIN on the connection. Receiving the FIN would cause the requestor's TCP to report an end-of-file to the requestor's process, allowing the requestor to detect this scenario and exit.

In the first state the requestor's application has no idea that keepalive probes are taking place (except that it chose to enable keepalives in the first place). Everything is handled at the TCP layer. It is transparent to the application until one of states 2, 3, or 4 is determined. In these three cases, an error is returned to the requestor's application by its TCP. (Normally the requestor has issued a read from the network, waiting for data from the peer. If the keepalive feature returns an error, it is returned to the requestor as the return value from the read.) In scenario 2 the error is something like "Connection timed out," and in scenario 3 we expect "Connection reset by peer." The fourth scenario may look as if the connection timed out, or may cause another error to be returned, depending on whether an ICMP error related to the connection is received and how it is processed (see Chapter 8). We look at all four scenarios in the next section.

The values of the variables `keepalive_time`, `keepalive_interval`, and `keepalive_probes` can usually be changed. Some systems allow these changes on a per-connection basis, while others allow them to be set only system-wide (or both in some cases). In Linux, these values are available as `sysctl` variables with the names `net.ipv4.tcp_keepalive_time`, `net.ipv4.tcp_keepalive_intvl`, and `net.ipv4.tcp_keepalive_probes`, respectively. The defaults are 7200 (seconds, or 2 hours), 75 (seconds), and 9 (probes).

In FreeBSD and Mac OS X, the first two values are also available as `sysctl` variables called `net.inet.tcp.keepidle` and `net.inet.tcp.keepintvl`, with default values 7,200,000 (milliseconds, or 2 hours) and 75,000 (milliseconds, or 75s), respectively. These systems also have a Boolean variable called `net.inet.tcp.always_keepalive`. If this value is enabled, all TCP connections have the keepalive function enabled, even if the application did not request it. In these systems, the number of probes is a fixed default value: 8 (FreeBSD) or 9 (Mac OS X).

In Windows, these values are available for modification via registry entries under the system key:

```
HKLM\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters
```

The value `KeepAliveTime` defaults to 7,200,000ms (2 hours). `KeepAliveInterval` defaults to 1000ms (1s). If there is no response to ten keepalive probes, Windows terminates the connection.

Note that [RFC1122] places certain restrictions on the use of keepalives. In particular, the `keepalive time` must be configurable and must not default to less than 2 hours. In addition, keepalives must not be enabled unless an application requests one (although this behavior is violated if the `net.inet.tcp.always_``keepalive` variable is set). Linux does not provide a native facility for adding keepalives to applications that do not request it, but a special library can be pre-loaded (i.e., loaded prior to ordinary shared libraries) to get this effect [LKA].

17.2.1 Keepalive Examples

We shall now go through states 2, 3, and 4 from the previous section, to see the packets exchanged using the keepalive mechanism. The operation in state 1 will be illustrated in the course of looking at the others.

17.2.1.1 Other End Crashes

Let us see what happens when the server host crashes and does not reboot. To simulate this we will do the following steps:

1. Using the `regedit` program on a Windows client, modify the registry key, and set `KeepAliveTime` to 7000ms (7s). This may require the system to be rebooted to accept the new value.
2. Establish an `ssh` connection between the Windows client and a Linux server using an option that enables TCP keepalives.
3. Verify that data can go across the connection.
4. Watch the client's TCP send keepalive packets every 7s, and see them acknowledged by the server's TCP.
5. Disconnect the network cable from the server, and leave it disconnected until the example is complete. This makes the client think the server host has crashed.
6. We expect the client to send ten keepalive probes, 1s apart, before declaring the connection dead.

Here is the interactive output on the client:

```
C:\> ssh -o TCPKeepAlive=yes 10.0.1.1
(password prompt and login continues)
Write failed: Connection reset by peer (about 15 seconds after disconnect)
```

Figure 17-1 shows the results using Wireshark. In this example, the connection has already been established. The Wireshark output begins with a keepalive (packet 1) that is not identified as such. At this point, Wireshark has not processed enough packets to determine that the one sequence number in packet 1 is below

Figure 17-1 TCP keepalives are generated every 7s after the connection becomes idle. Each contains a below-window sequence number that is ACKed by the peer. A cable disconnection after 1 minute causes subsequent keepalives to not be ACKed. The client tries ten times before giving up and terminating the connection. The termination is signaled to the server by the final reset segment (which the server cannot hear). This example also illustrates the use of DSACKs at the server and a spurious retransmission caused by the client delaying ACKs.

the receiver's left window edge and is therefore a keepalive. Packet 2 contains an ACK number that allows Wireshark to process the sequence numbers in subsequent packets appropriately.

Most of this connection consists of keepalives and corresponding ACKs. Packets 1, 3, 5, 7, 14, 16, 18, 20, and 22–31 are all keepalives. Packets 2, 4, 6, 8, 15, 17, 19, and 21 are the corresponding ACKs. Keepalives are sent periodically every 7s

provided they are ACKed. When no ACK is returned for a keepalive, the sender switches to a 1s interval for sending keepalives, according to the default value of `KeepAliveInterval`. This starts with packet 23 at time 62.120. The sender produces ten unacknowledged keepalives in total (packets 22–31). After that, it terminates the connection, which results in the final reset segment (packet 32) that is never received by the disconnected receiver. The user receives the following output when the connection terminates:

```
Write failed: Connection reset by peer
```

This is a clear indication that the connection has terminated, but it is not entirely accurate. It was really the sender that terminated the connection, but it did so based on the lack of response from the receiver.

Apart from the use of keepalive segments, there are some other interesting features of this connection we will mention briefly. First, the server uses DSACKs (see Chapter 14). Each ACK contains the sequence number range of the previously received in-window segment. Next, a small bit of data is exchanged at time 26.09. The data represents a single key press. It is sent to the server, ACKed by the server, and echoed back. The data is encrypted, causing the packets containing data to be 48 bytes in user data size (see Chapter 18).

Interestingly, the echoed character is sent twice. We can see that packet 11, which contains the echoed character is not ACKed immediately. Recall from Chapter 14 that Linux uses an RTO of at least 200ms. Here we see that the Linux server retransmits the echoed character 200ms later, which produces an immediate response from the client. Because this test was performed on an uncongested LAN, it is highly unlikely that segment 11 was dropped. Instead, it appears that Linux produced a spurious retransmission due to the client delaying ACKs. This is a similar sort of hazard we saw when exploring the poor interaction between the Nagle algorithm and delayed ACKs we discussed in Chapter 15. Here, the dynamic results in an unnecessary delay of about 200ms.

17.2.1.2 Other End Crashes and Reboots

In this example we will see what happens when the peer crashes and reboots. The initial scenario is the same as the previous one, except this time we set `KeepAliveTime` to 120,000 (2 minutes). We establish a connection and then wait just over 2 minutes to allow a keepalive message to be sent and ACKed. Then we disconnect the server from the network, reboot it, and then reconnect it. We expect the next keepalive probe to generate a reset from the server, because the server now knows nothing about this connection. Figure 17-2 presents the trace as displayed by Wireshark.

In this example, the connection has been established and small amounts of data are exchanged starting at seconds 0.00 and 3.46. Then the connection goes idle. After 2 minutes have elapsed (the keepalive time) the client sends the first keepalive probe at time 123.47, containing the “garbage” byte below the receiver’s

Figure 17-2 The server has rebooted between keepalives sent by the client. The last keepalive elicits a reset segment because the server no longer knows anything about the connection.

left window edge. It is acknowledged, and the server is disconnected, rebooted, and reconnected. At time 243.47, 120s later, the client sends its second keepalive probe. Although this reaches the server, the server no longer has any knowledge about the connection and responds with a reset segment (packet 18). This informs the client that the connection is no longer active, and the user is provided the same “Connection reset by peer” error message we saw before.

17.2.1.3 Other End Is Unreachable

In this case, the server has not crashed but becomes unreachable during the interval when the keepalive probes are sent. An intermediate router may have crashed, a phone line may be temporarily out of order, or something similar. To simulate this example we will use our `sock` program with the keepalive option set to

establish a connection to a Web server. We will use a Mac OS X client and an LDAP server (port 389) running on `ldap.mit.edu`. After shortening the client's keepalive time (for convenience) and opening the connection, we disconnect the network to see the effects. Here are the command lines and output at the client:

```
Mac# sysctl -w net.inet.tcp.keepidle=75000
Mac% sock -K ldap.mit.edu 389
recv error: Operation timed out      about 14 minutes later
```

The trace is displayed using Wireshark (see Figure 17-3).

Figure 17-3 The WAN connection is taken down after the first keepalive probe is acknowledged. Another probe is sent every 75s. After nine keepalives are sent without a response, the connection is terminated and the client sends a reset to its peer. For the client, the situation is very similar to when the server crashes, as illustrated in Figure 17-1.

In this figure we can see the entire connection. After the initial three-way handshake, the connection remains idle and a keepalive is sent and acknowledged at about time 75 (packet 4). This first keepalive is triggered by the value of the `net.inet.tcp.keepidle` variable. Shortly thereafter, the network is severed. Neither end of the connection produces data, so the next event is another keepalive sent by the client at time 150 (75s later, the value of the `net.inet.tcp.keepintvl` variable). This pattern repeats with packets 7–14, with no ACKs present, even though the server is up and running. Finally, the client gives up 75s after its ninth unacknowledged keepalive probe. The connection termination is indicated to the server by a reset segment at the end (packet 15). Of course, the server is unable to receive this packet because the network is not operating.

When a client TCP using keepalives is unable to communicate across the network with its peer, as this example shows, it retries some number of times before giving up. This is essentially the same behavior we saw when the other

end crashed. In most cases, the sending TCP cannot tell the difference. There are some exceptions, such as when ICMP indicates that the destination has become unreachable or otherwise unavailable because of problems in the network, but these conditions are relatively rare because ICMP is often blocked. As a result, mechanisms such as TCP keepalive (or similar mechanisms implemented by applications) are used to detect disconnection periods.

17.3 Attacks Involving TCP Keepalives

As we mentioned before, ssh (version 2) has an application-level form of keep-alive called server alive messages and client alive messages. These are different from TCP keepalive messages because they are sent over an encrypted channel at the application layer and contain data. TCP keepalives contain no user-level data, so the use of encryption is limited at best. The consequence is that TCP keepalives may be spoofed. When TCP keepalives are spoofed, the victim can be coerced into keeping resources allocated for a period longer than intended.

Although it may be a relatively minor concern, TCP keepalives are driven off a timer based on the various configuration parameters discussed earlier, and not off the dynamically adjusted retransmission timer used to retransmit segments with data. A passive observer could notice the existence of keepalives and their inter-arrival times to conceivably learn information about the configuration parameters (possibly identifying the type of sending system, called *fingerprinting*) or about the network topology (i.e., whether downstream routers are forwarding traffic or not). These issues could be of concern in some environments.

17.4 Summary

As we said earlier, the keepalive feature has been somewhat controversial. Protocol experts continue to debate whether it belongs in the transport layer or should be handled entirely by the application. All popular TCP implementations now include the keepalive feature, which applications may optionally use to establish a “heartbeat” of traffic moving across a connection. Doing so can help a server by allowing it to detect nonresponsive clients and can help clients by keeping connections active (e.g., to keep NAT state active) even if no application-layer data is flowing.

Keepalives operate by sending a probe packet (usually containing a “garbage” byte, although zero-length probes are also possible) across a connection after the connection has been idle for some relatively long period of time, often 2 hours. Four different scenarios can occur: the other end is still there, the other end has crashed, the other end has crashed and rebooted, or the other end is currently unreachable. We saw each of these scenarios with an example.

In the first two keepalive examples that we examined, had keepalives not been used, and without any application-level timer or activity, TCP would never have known that the other end had crashed (or crashed and rebooted). In the final example, however, nothing was wrong with the other end; the connection was temporarily down. We must be aware of this limitation when using keepalives and consider whether or not such behavior is desired.

Attacks against the keepalive mechanism include causing a system to keep resources allocated longer than intended and possibly learning some otherwise hidden information about the end systems (although such information may be of limited use to an attacker). In addition, by default TCP does not use its own encryption, so keepalives and keepalive ACKs can be spoofed, whereas application-level keepalives that employ encryption (e.g., ssh) cannot.

17.5 References

[LKA] <http://libkeepalive.sourceforge.net>

[RFC1122] R. Braden, ed., "Requirements for Internet Hosts," Internet RFC 1122, Oct. 1989.

This page intentionally left blank

18

Security: EAP, IPsec, TLS, DNSSEC, and DKIM

18.1 Introduction

In this chapter we will take a look at several forms of security used with TCP/IP. Security is a very broad and interesting topic, and covering it comprehensively is far beyond the scope of this book. Consequently, we will be interested to know about the various types of security threats on the Internet, and we will delve into some detail on those security mechanisms aimed at countering them that are applicable to the operation of various protocols such as IP, TCP, and the important e-mail and DNS application protocols.

Although our partitioning is not really formal, security threats can be broken down into attacks that target implementation problems by trying to subvert processes into running code that was not intended, trying to get users to run programs that do bad things, and using network protocols in compliant but unauthorized ways. We have already seen forms of these attacks in other chapters. For example, one of the earliest worms (self-propagating software) on the Internet used a buffer overflow that overwrites the server process's memory. Doing so allows a client program to inject software into a server that ultimately runs this injected code. The injected code then performs the same action, thereby causing the program to self-propagate. Naturally, such code could perform more malicious activities than simply self-propagation.

The various types of attacks and techniques can be combined, and complicated software and security analysis tools have been developed as the value of the information on the Internet has increased. A variety of texts, including [MSK09], discuss the tools and techniques in more detail. Today, essentially any software executed by a user or as a user against the user's intentions is known by the general term *malware* short for "malicious software." Entire industries have been

developed to both create and reduce the effects of malware. Malware can be delivered in e-mail messages or attachments (e.g., in spam), picked up while visiting a Web site (drive-byattacks), or acquired when using portable media such as portable USB drives.

In some cases, malware is used to take control of a large number of computers in the Internet (botnets). Botnets are controlled by individuals or organizations (bot herders) and can be used on a wide scale for a number of purposes such as sending spam, compromising other computers, exfiltrating information from the compromised system (e.g., credit card and bank account information, and the user's logged keystrokes), and launching DoS attacks by sending a large aggregate volume of Internet traffic to one or more victims. Botnets are now commonly offered as a service on a rental basis—a client can hire a bot herder to perform one or more nefarious tasks. One common task is to generate e-mails in hopes of inducing the recipient(s) to visit a particular Web site or purchase a particular product (phishing). When a specific victim is targeted in this way, the activity is usually called spear phishing.

Our interest is in understanding how secure communication protocols on the Internet work. Ironically, perhaps, many worms or viruses implement secure communication protocols. In most cases, we will see how the types of protocols we have already studied such as IP, TCP, e-mail, and DNS have been augmented with security extensions (sometimes in the form of additional protocols) to enhance security. We need to be somewhat precise in defining what “security” means in terms of a communication protocol, in order to understand if the techniques available to us are sufficient to provide our desired level of protection. Therefore, we shall begin by studying the properties of information protection considered desirable in the field of information security.

18.2 Basic Principles of Information Security

There are three primary properties of information, whether in a computer network or not, that may be desirable from an information security point of view: confidentiality, integrity, and availability (the CIA triad) [L01], summarized here:

- Confidentiality means that information is made known only to its intended users (which could include processing systems).
- Integrity means that information has not been modified in an unauthorized way before it is delivered.
- Availability means that information is available when needed.

These are core properties of information, yet there are other properties we may also desire, including authentication, nonrepudiation, and auditability. Authentication means that a particular identified party or principal is not impersonating

another principal. Nonrepudiation means that if some action is performed by a principal (e.g., agreeing to the terms of a contract), this fact can be proven later (i.e., cannot successfully be denied). Auditability means that some sort of trustworthy log or accounting describing how information has been used is available. Such logs are often important for forensic (i.e., legal and prosecutorial) purposes.

These principles are applicable to information in physical (e.g., printed) form, for which mechanisms such as safes, secured facilities, and guards have been used for thousands of years to enforce controlled sharing, storage, and dissemination. When information is to be moved through an unsecured environment, additional techniques are required. To see why, let us examine the types of threats to which information can be exposed when it travels through an unsecured communication channel.

18.3 Threats to Network Communication

When considering the design and operation of network protocols, ensuring that information has the desired properties of integrity, availability, and confidentiality can be quite a challenge because of the wide range of possible attacks that can be carried out in an otherwise uncontrolled network such as the Internet. Attacks can generally be categorized as either passive or active [VK83]. Identifying the category is useful because different techniques are required to provide security depending on the particular category. Passive attacks are mounted by monitoring or eavesdropping on the contents of network traffic, and if not handled they can lead to unauthorized release of information (loss of confidentiality). Active attacks can cause modification of information (with possible loss of integrity) or denial of service (loss of availability). Logically, such attacks are carried out by an “intruder” or adversary. This is often depicted using the scenario shown in Figure 18-1.

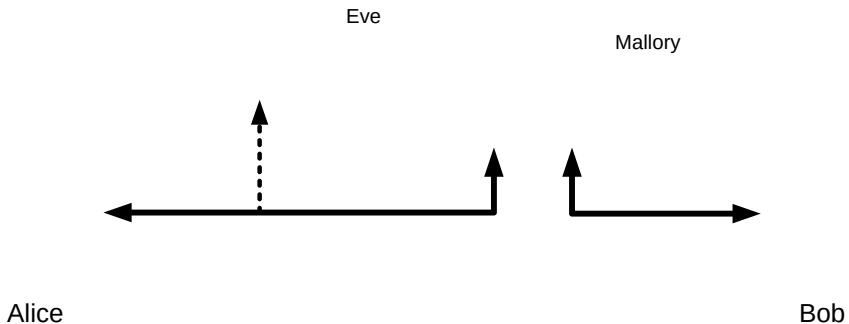


Figure 18-1 The principals, Alice and Bob, attempt to communicate securely, but Eve may eavesdrop and Mallory may modify messages in transit.

The figure depicts the principals, Alice and Bob, trying to communicate. However, there are two attackers, Eve and Mallory. Eve (eavesdropper) is only able to monitor the traffic exchanged between Alice and Bob and thus can carry out only passive attacks. Mallory (malicious attacker) can store, modify, and replay traffic passing between Alice and Bob, so she can carry out active and passive attacks. Table 18-1 summarizes the major categories of passive and active attacks that Alice and Bob may face.

Table 18-1 Attacks on communication are broadly classified as passive or active. Passive attacks are ordinarily more difficult to detect, and active attacks are ordinarily more difficult to prevent.

Passive		Active	
Type	Threats	Type	Threats
Eavesdropping	Confidentiality	Message stream modification	Authenticity, integrity
Traffic analysis	Confidentiality	Denial of service (DoS)	Availability
		Spurious association	Authenticity

From an attacker's perspective, Table 18-1 gives a quick summary of the passive attacks available to Eve and the active (and passive) attacks available to Mallory. Eve is able to eavesdrop (listen in on, also called capture or sniff) and perform traffic analysis on the traffic passing between Alice and Bob. Capturing the traffic could lead to compromise of confidentiality, as sensitive data may be available to Eve without Alice or Bob knowing. In addition, traffic analysis can determine the features of the traffic, such as its size and when it is sent, and possibly identify the parties to a communication. This information, although it does not reveal the exact contents of the communication, could also lead to disclosure of sensitive information and could be used to mount more powerful active attacks in the future.

While the passive attacks are essentially impossible for Alice or Bob to detect, Mallory is capable of performing more easily noticed active attacks. These include message stream modification (MSM), denial-of-service (DoS), and spurious association attacks. MSM attacks (including so-called man-in-the-middle or MITM attacks) are a broad category and include any way traffic is modified in transit, including deletion, reordering, and content modification. DoS might include deletion of traffic, or generation of such large volumes of traffic so as to overwhelm Alice, Bob, or the communication channel connecting them. Spurious associations include masquerading (Mallory pretends to be Bob or Alice) and replay whereby Alice or Bob's earlier (authentic) communications are replayed later, from Mallory's memory.

Two major methods are available to prevent the passive and active attacks we have just described. One method would be to ensure through physical security that only trusted parties have access to the communication infrastructure

connecting Alice and Bob. This approach is used in limited circumstances but is effectively impractical for any network spanning a large geographical distance. Of course, if the communication channel is wireless, securing it using only physical methods is effectively impossible. Given these considerations, some mechanism is needed to allow information to pass through unsecured communication channels in such a way that adversaries like Eve and Mallory are, for the most part at least, thwarted. This mechanism is cryptography. With effective and careful use of cryptography, passive attacks are rendered ineffective, and active attacks are made detectable (and to some degree preventable).

18.4 Basic Cryptography and Security Mechanisms

Cryptography evolved from the desire to protect the confidentiality, integrity, and authenticity of information carried through unsecured communication channels. Such a capability is clearly of significant importance in protecting confidential information such as military orders, intelligence, and recipes for creating especially dangerous or valuable materials. The use of cryptography, at least in a primitive form, dates back to at least 3500 BCE. The earliest systems were usually codes. Codes involve substitutions of groups of words, phrases, or sentences with groups of numbers or letters as given in a codebook. Codebooks needed to be kept secret in order to keep communications private, so distributing them required considerable care.

More advanced systems used ciphers in which both substitution and rearrangement are used. Several codes were used in the Middle Ages, and by the late 1800s large code and cipher systems were commonly used for diplomatic and military communications. By the early twentieth century, cryptography was well established but would not take its major leap forward until World War II. During this period, electromechanical cryptographic machines such as the German ENIGMA and Lorenz machines posed a challenge to Allied cryptanalysts (code breakers). One of the first digital computers, Colossus, was developed by the British to decipher Lorenz-enciphered messages. A functioning Colossus Mark 2 machine was created in 2007, after a 14-year effort, by Tony Sale of the National Museum of Computing at Bletchley Park, UK [TNMOC].

18.4.1 Cryptosystems

While the historical basis for cryptography is primarily for preserving confidentiality, other desirable properties such as integrity and authentication can also be achieved using cryptographic and related mathematical techniques. To help understand the basics, Figure 18-2 illustrates how the two most important types of cryptographic algorithms, called symmetric key and public (asymmetric) key ciphers, work.

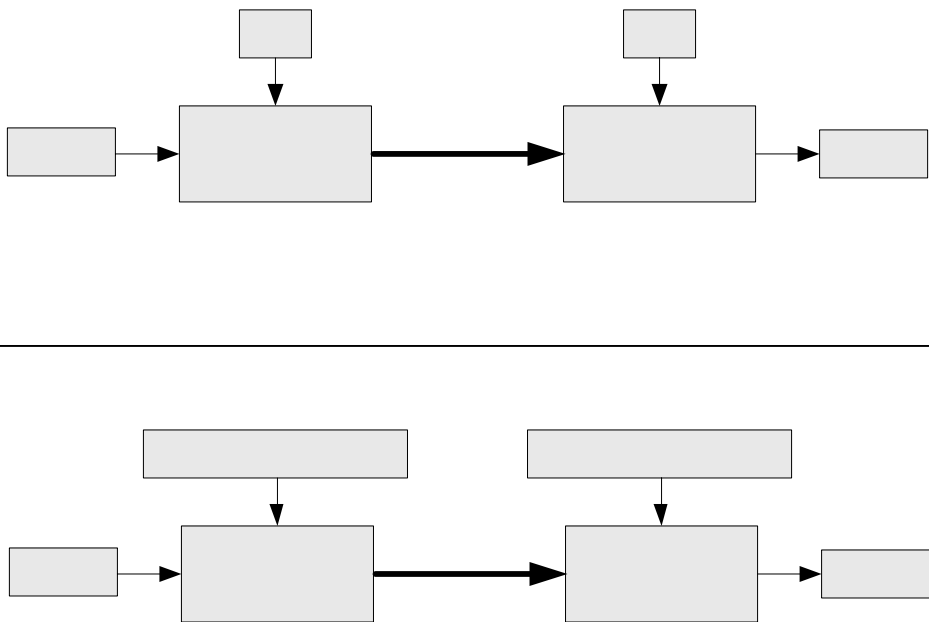


Figure 18-2 The unencrypted (cleartext) message is passed through an encryption algorithm to produce an encrypted (ciphertext) message. In a symmetric cryptosystem, the same (secret) key is used for encryption and decryption. In an asymmetric or public key cryptosystem, confidentiality is achieved by using the recipient's public key for encryption and private (secret) key for decryption.

This figure shows the high-level operation of symmetric and asymmetric key cryptography. In each case, a cleartext message is processed by an encryption algorithm to produce ciphertext (scrambled text). The key is a particular sequence of bits used to drive the encryption algorithm or cipher. With different keys, the same input produces different outputs. Combining the algorithms with supporting protocols and operating methods forms a cryptosystem. In a symmetric cryptosystem, the encryption and decryption keys are typically identical, as are the encryption and decryption algorithms. In an asymmetric cryptosystem, each principal is generally provided with a pair of keys consisting of one public and one private key. The public key is intended to be known to any party that might want to send a message to the key pair's owner. The public and private keys are mathematically related and are themselves outputs of a key generation algorithm. One of the major benefits of asymmetric key cryptosystems is that secret key material does not have to be securely distributed to every party that wishes to communicate.

Without knowing the symmetric key (in a symmetric cryptosystem) or the private key (in a public key cryptosystem), it is (believed to be) effectively impossible for any third party that intercepts the ciphertext to produce the corresponding cleartext. This provides the basis for confidentiality. For the symmetric key cryptosystem, it also provides a degree of authentication, because only a party holding the key is able to produce a useful ciphertext that can be decrypted to something sensible. A receiver can decrypt the ciphertext, look for a portion of the resulting cleartext to contain a particular agreed-upon value, and conclude that the sender holds the appropriate key and is therefore authentic. Furthermore, most encryption algorithms work in such a way that if messages are modified in transit, they are unable to produce useful cleartext upon decryption. Thus, symmetric cryptosystems provide a measure of both authentication and integrity protection for messages, but this approach alone is weak. Instead, special forms of checksums are usually coupled with symmetric cryptography to ensure integrity. We discuss these later, after the cryptographic preliminaries.

A symmetric encryption algorithm is usually classified as either a block cipher or a stream cipher. Block ciphers perform operations on a fixed number of bits (e.g., 64 or 128) at a time, and stream ciphers operate continuously on however many bits (or bytes) are provided as input. For years, the most popular symmetric encryption algorithm was the Data Encryption Standard (DES), a block cipher that uses 64-bit blocks and 56-bit keys. Eventually, the use of 56-bit keys was felt to be insecure, and many applications turned to triple-DES (also denoted 3DES or TDES—applying DES three times with two or three different keys to each block of data). Today, DES and 3DES have been largely phased out in favor of the Advanced Encryption Standard (AES) [FIPS197], also known occasionally by its original name the Rijndael algorithm (pronounced “rain-dahl”), in deference to its Belgian cryptographer inventors Vincent Rijmen and Joan Daemen. Different variants of AES provide key lengths of 128, 192, and 256 bits and are usually written with the corresponding extension (i.e., AES-128, AES-192, and AES-256).

Asymmetric cryptosystems have some additional interesting properties beyond those of symmetric key cryptosystems. Assuming we have Alice as sender and Bob as intended recipient, any third party is assumed to know Bob's public key and can therefore send him a secret message—only Bob is able to decrypt it because only Bob knows the private key corresponding to his public key. However, Bob has no real assurance that the message is authentic, because any party can create a message and send it to Bob, encrypted in Bob's public key. Fortunately, public key cryptosystems also provide another function when used in reverse: authentication of the sender. In this case, Alice can encrypt a message using her private key and send it to Bob (or anyone else). Using Alice's public key (known to all), anyone can verify that the message was authored by Alice and has not been modified. However, it is not confidential because everyone has access to Alice's public key. To achieve authenticity, integrity, and confidentiality, Alice can encrypt a message using her private key and encrypt the result using Bob's public key. The result is a message that is reliably authored by Alice and is also confidential to Bob. This process is illustrated in Figure 18-3.

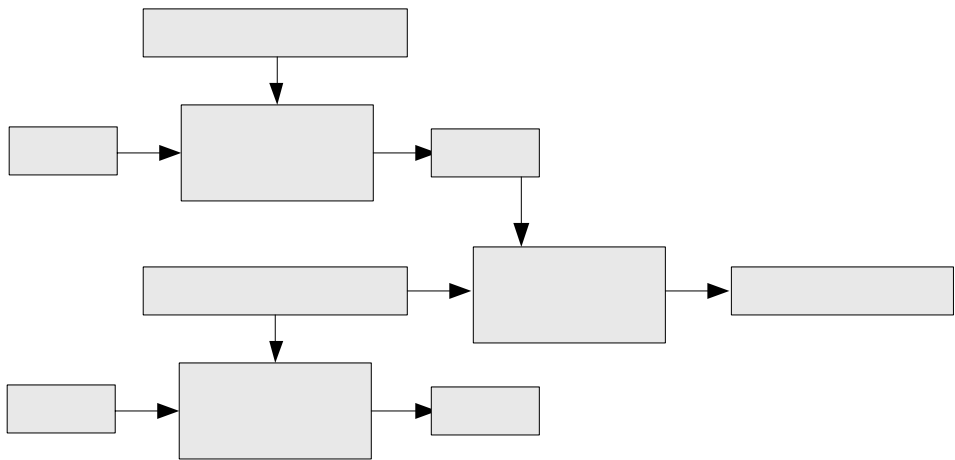


Figure 18-3 The asymmetric cryptosystem can be used for confidentiality (encryption), authentication (digital signatures or signing), or both. When used for both, it produces a signed output that is confidential to the sender and the receiver. Public keys, as their name suggests, are not kept secret.

When public key cryptography is used in “reverse” like this, it provides a digital signature. Digital signatures are important consequences of public key cryptography and can be used to help ensure authenticity and nonrepudiation. Only a party possessing Alice’s private key is able to author messages or carry out transactions as Alice.

In a hybrid cryptosystem, elements of both public key and symmetric key cryptography are used. Most often, public key operations are used to exchange a randomly generated confidential (symmetric) session key, which is used to encrypt traffic for a single transaction using a symmetric algorithm. The reason for doing so is performance—symmetric key operations are less computationally intensive than public key operations. Most systems today are of the hybrid type: public key cryptography is used to establish keys used for symmetric encryption of individual sessions.

18.4.2 Rivest, Shamir, and Adleman (RSA) Public Key Cryptography

We have seen how public key cryptography can be used for both digital signatures and confidentiality. The most common approach is called RSA in deference to its authors’ names, Rivest, Shamir, and Adleman [RSA78]. The security of this system hinges on the difficulty of factoring large numbers into constituent primes.

To initialize RSA, two large prime numbers p and q are generated, which usually involves checking a number of large odd numbers that are randomly generated until two primes are found. The product of these primes $n = pq$ is called the modulus. The length of n , p , and q is usually measured in bits, with n often being 1024 bits and the others being about 512, although larger sizes such as 2048 are now recommended. The value $\phi(n)$ is known in number theory as the Euler totient of the integer n . It gives the number of positive integers less than n that are also coprime to n (i.e., whose greatest common divisor is 1). Because of the way n is constructed for RSA, $\phi(n) = (q - 1)(p - 1)$.

Using the definition for $\phi(n)$, we can choose the RSA public exponent (called e for “encryption”) and derive a private exponent (called d for “decryption”) as multiplicative inverses using the relation $d = e^{-1} \pmod{\phi(n)}$. In practice, e is often some value with a fairly small population count (i.e., has a small number of 1 bits) such as 65,537 (10000000000000001 binary), for faster computations. To form an encrypted ciphertext c from a cleartext message m , the value $c = m^e \pmod{n}$ is computed. To form the value m from c , decryption is performed: $m = c^d \pmod{n}$. An RSA public key consists of the public exponent e and modulus n . The corresponding private key consists of the private exponent d and the modulus n .

As suggested earlier, public key algorithms such as RSA can also be used to produce digital signatures by essentially running RSA “in reverse.” To create an RSA signature of a message m , the value $s = m^d \pmod{n}$ can be produced as a signed version of m . Anyone receiving the value s can apply the public exponent e to produce $m = s^e \pmod{n}$, which provides the basis for verifying that whatever produced the value s was in possession of the private key d (otherwise the value m produced would not be sensible).

The security of RSA is based on the difficulty of factoring large numbers. In the context of RSA and our scenario of Figure 18-1, Eve is able to obtain m and e but does not know p , q , or $\phi(n)$. If she could determine any of these last three values, it would be trivial to determine d using the relation we have described. However, doing so appears to involve factoring n , and factoring numbers of 1000 or more bits is currently believed to be out of reach for even the best factorization algorithms. Indeed, factoring semiprimes (numbers that are a product of two primes) appears to represent the most difficult case for such algorithms.

18.4.3 Diffie-Hellman-Merkle Key Agreement (aka Diffie-Hellman or DH)

A common requirement in security protocols is to have two parties agree on a common set of secret bits that can be used as a symmetric key. Doing so in a network that may contain eavesdroppers (such as Eve) is a challenge, because it is not immediately obvious how to have two principals (such as Alice and Bob) agree on a common secret number without Eve knowing. The Diffie-Hellman-Merkle Key Agreement protocol (more commonly called simply Diffie-Hellman or DH) provides a method for accomplishing this task, based on the use of finite field arithmetic

[DH76].¹ DH techniques are used in many of the Internet-related security protocols [RFC2631] and are closely related to the RSA approach for public key cryptography. We shall have a brief look at how they work.

With the same cast of characters (Alice, Bob, etc.), let us assume that all parties are aware of two integers p and g . Let p be a (large) prime number and $g < p$ be a primitive root mod p . With these assumptions, every integer in the group $Z_p = \{1, \dots, p-1\}$ can be generated by raising g to some power. Said another way, for every n , there exists some k for which $g^k \equiv n \pmod{p}$. Finding the value (or values) of k given g , n , and p (called the discrete log problem) is considered to be difficult, resulting in the belief that DH is secure. Finding the value of n given g , k , and p is easy, resulting in the approach being practical.

For Alice and Bob to establish a shared secret key, they can use the following protocol: Alice chooses a secret random number a and computes $A = g^a \pmod{p}$, which she sends to Bob. Bob chooses a secret random number b and computes $B = g^b \pmod{p}$, which he sends to Alice. Alice and Bob arrive at the same shared secret $K = g^{ab} \pmod{p}$. Alice computes this value this way:

$$K = B^a \pmod{p} = g^{ba} \pmod{p}$$

and Bob computes it this way:

$$K = A^b \pmod{p} = g^{ab} \pmod{p}$$

Given that g^{ba} is equal to g^{ab} (because Z_p is so-called power associative and we assumed all parties are aware of the group Z_p being used), both Alice and Bob know K . Note that Eve has access only to g , p , A , and B so cannot determine K without solving the discrete log problem [MW99]. However, this basic protocol is vulnerable to an attack from Mallory. Mallory can pretend to be Bob when communicating with Alice and vice versa by supplying her own A and B values. However, the basic DH protocol can be extended to protect from this man-in-the-middle attack if the public values for A and B are authenticated [DOW92]. The classic approach, called the Station-to-Station protocol (STS), involves Alice and Bob signing their public values.

18.4.4 Signcryption and Elliptic Curve Cryptography (ECC)

When using RSA, additional security is provided with larger numbers. However, the basic mathematical operations required by RSA (e.g., exponentiation) can be computationally intensive and scale as the numbers grow. Reducing the effort of combining digital signatures and encryption for confidentiality, a class of signcryption schemes [Z97] (also called authenticated encryption) provides both features

1. The technique was described in a then-classified reference in 1973 by C. Cocks, "A Note on 'Non-Secret Encryption.'" See <http://www.cesg.gov.uk/publications/media/notense.pdf>.

at a cost less than the sum of the two if computed separately. However, even greater efficiency can sometimes be achieved by changing the mathematical basis for public key cryptography.

In a continuing search for security with greater efficiency and performance, researchers have explored other public key cryptosystems beyond RSA. An alternative based on the difficulty of finding the discrete logarithm of an elliptic curve element has emerged, known as elliptic curve cryptography (ECC, not to be confused with error-correcting code) [M85][K87][RFC5753]. For equivalent security, ECC offers the benefit of using keys that are considerably smaller than those of RSA (e.g., by about a factor of 6 for a 1024-bit RSA modulus). This leads to simpler and faster implementations, issues of considerable practical concern. ECC has been standardized for use in many of the applications where RSA still retains dominance, but adoption has remained somewhat sluggish because of patents on ECC technology held by the Certicom Corporation. (The RSA algorithm was also patented, but patent protection lapsed in the year 2000.)

18.4.5 Key Derivation and Perfect Forward Secrecy (PFS)

In communication scenarios where multiple messages are to be exchanged, it is common to establish a short-term session key to perform symmetric encryption. The session key is ordinarily a random number (see the following section) generated by a function called a key derivation function (KDF), based on some input such as a master key or a previous session key. If a session key is compromised, any of the data encrypted with the key is subject to compromise. However, it is common practice to change keys (ekey) multiple times during an extended communication session. A scheme in which the compromise of one session key keeps future communications secure is said to have perfect forward secrecy (PFS). Usually, schemes that provide PFS require additional key exchanges or verifications that introduce overhead. One example is the STS protocol for DH mentioned earlier.

18.4.6 Pseudorandom Numbers, Generators, and Function Families

In cryptography, random numbers are often used as initial input values to cryptographic functions, or for generating keys that are difficult to guess. Given that computers are not very random by nature, obtaining true random numbers is somewhat difficult. The numbers used in most computers for simulating randomness are called pseudorandom numbers. Such numbers are not usually truly random but instead exhibit a number of statistical properties that suggest that they are (e.g., when many of them are generated, they tend to be uniformly distributed across some range).

Pseudorandom numbers are produced by an algorithm or device known as a pseudorandom number generator (PRNG) or pseudorandom generator (PRG), depending on the author. Simple PRNGs are deterministic. That is, they have a small amount of internal state initialized by a seed value. Once the internal state is known, the

sequence of PNs can be determined. For example, the common Linear Congruential Generator (LCG) algorithm produces random-appearing values that are entirely predictable if the input parameters are known or guessed. Consequently, LCGs are perfectly fine for use in certain programs (e.g., games that simulate random events) but insufficient for cryptographic purposes.

A pseudorandom function family (PRF) is a family of functions that appear to be algorithmically indistinguishable (by polynomial time algorithms) from truly random functions [GGM86]. A PRF is a stronger concept than a PRG, as a PRG can be created from a PRF. PRFs are the basis for cryptographically strong (or secure) pseudorandom number generators, called CSPRNGs. CSPRNGs are necessary in cryptographic applications for several purposes, including session key generation, for which a sufficient amount of randomness must be guaranteed [RFC4086].

18.4.7 Nonces and Salt

A cryptographic nonce is a number that is used once (or for one transaction) in a cryptographic protocol. Most commonly, a nonce is a random or pseudorandom number that is used in authentication protocols to ensure freshness. Freshness is the (desirable) property that a message or operation has taken place in the very recent past. For example, in a challenge-response protocol, a server may provide a requesting client with a nonce, and the client may need to respond with authentication material as well as a copy of the nonce (or perhaps an encrypted copy of the nonce) within a certain period of time. This helps to avoid replay attacks, because old authentication exchanges that are replayed to the server would not contain the correct nonce value.

A salt or salt value, used in the cryptographic context, is a random or pseudorandom number used to frustrate brute-force attacks on secrets. Brute-force attacks usually involve repeatedly guessing a password, passphrase, key, or equivalent secret value and checking to see if the guess was correct. Salts work by frustrating the checking portion of a brute-force attack. The best-known example is the way passwords used to be handled in the UNIX system. Users' passwords were encrypted and stored in a password file that all users could read. When logging in, each user would provide a password that was used to double encrypt a fixed value. The result was then compared against the user's entry in the password file. A match indicated that a correct password was provided.

At the time, the encryption method (DES) was well known and there was concern that a hardware-based dictionary attack would be possible whereby many words from a dictionary were encrypted with DES ahead of time (forming a rainbow table) and compared against the password file. A pseudorandom 12-bit salt was added to perturb the DES algorithm in one of 4096 (nonstandard) ways for each password in an effort to thwart this attack. Ultimately, the 12-bit salt was determined to be insufficient with improved computers (that could guess more values) and was expanded.

18.4.8 Cryptographic Hash Functions and Message Digests

In most of the protocols we have studied, including Ethernet, IP, ICMP, UDP, and TCP, we have seen the use of a frame check sequence (FCS, either a checksum or a CRC) to determine whether a PDU has likely been delivered without bit errors. Such mathematical functions tend to trade off the likelihood of detecting random errors against the amount of overhead required to carry the FCS value. When considering security, however, we are interested in ensuring message integrity not only against random, infrequent errors, but also against intentional message stream modification attacks. We are worried about Mallory modifying messages as they travel through the network. Ordinary FCS functions are not sufficient for this purpose.

A checksum or FCS can be used to verify message integrity against an adversary like Mallory if properly constructed using special functions. Such functions are called cryptographic hash functions and often resemble portions of encryption algorithms. The output of a cryptographic hash function H , when provided a message M , is called the digest or fingerprint of the message, $H(M)$. A message digest is a type of strong FCS that is easy to compute and has the following important properties:

- Preimage resistance: Given $H(M)$, it should be difficult to determine M if not already known.
- Second preimage resistance: Given $H(M_1)$, it should be difficult to determine an $M_2 \neq M_1$ such that $H(M_1) = H(M_2)$.
- Collision resistance: It should be difficult to find any pair M_1, M_2 where $H(M_1) = H(M_2)$ when $M_2 \neq M_1$.

If a hash function has all of these properties, then if two messages have the same cryptographic hash value, they are, with negligible doubt, the same message. The two most common cryptographic hash algorithms are at present the Message Digest Algorithm 5 (MD5, [RFC1321]), which produces a 128-bit (16-byte) digest, and the Secure Hash Algorithm (SHA-1), which produces a 160-bit (20-byte) digest. More recently, a family of functions based on SHA called SHA-2 [RFC6234] produce digests with lengths of 224, 256, 384, or 512 bits (28, 32, 48, and 64 bytes, respectively). Others are under development.

Notes

Cryptographic hash functions are often based on a *compression function* f , which takes an input of length L and produces a collision-resistant but deterministic output of size less than L . The *Merkle-Damgård construction*, which essentially breaks an arbitrarily long input into blocks of length L , pads them, passes them to f , and combines the results, produces a cryptographic hash function capable of taking a long input and producing an output with collision resistance.

MD5 had been in widespread use with Internet protocols until it was reported broken in 2005 (i.e., two different 128-byte sequences were shown to have the same MD5 value) [WY05]. SHA-1 was used as an alternative, but it was also thought to possibly have weaknesses, so a SHA-2 family of algorithms was developed. Given SHA-2's similarity to SHA-1, there is concern that it, too, may have weaknesses. In December 2010, the National Institute of Standards and Technology (NIST) in the United States announced that five algorithms had been selected as final candidates for a new "SHA-3" cryptographic hash algorithm [CHP]. The selection of the final winning algorithm is scheduled for sometime after spring 2012.

18.4.9 Message Authentication Codes (MACs, HMAC, CMAC, and GMAC)

A message authentication code (unfortunately abbreviated MAC or sometimes MIC but unrelated to the link-layer MAC addresses we discussed in Chapter 3) can be used to ensure message integrity and authentication. MACs are usually based on keyed cryptographic hash functions. Such functions are like message digest algorithms (see Section 18.4.8) but require a private key to produce or verify the integrity of a message and may also be used to verify (authenticate) the message's sender.

MACs require resistance to various forms of forgery. For a given keyed hash function $H(M,K)$ taking input message M and key K , resistance to selective forgery means that it is difficult for an adversary not knowing K to form $H(M,K)$ given a specific M . $H(M,K)$ is resistant to existential forgery if it is difficult for an adversary lacking K to find any previously unknown valid combination of M and $H(M,K)$. Note that MACs do not provide exactly the same features as digital signatures. For example, they cannot be a solid basis for nonrepudiation because the secret key is known to more than one party.

A standard MAC that uses cryptographic hash functions in a particular way is called the keyed-hash message authentication code (HMAC) [FIPS198][RFC2104]. The HMAC "algorithm" uses a generic cryptographic hash algorithm, say $H(M)$. To form a t -byte HMAC on message M with key K using H (called HMAC- H), we use the following definition:

$$\text{HMAC-}H(K, M)^t = \text{ }_t(H((K \text{ opad}) \parallel H((K \text{ ipad}) \parallel M)))$$

In this definition, opad (outer pad) is an array containing the value 0x5C repeated $|K|$ times, and ipad (inner pad) is an array containing the value 0x36 repeated $|K|$ times. $\parallel$ is the vector XOR operator, and $\parallel$ is the concatenation operator. Normally the HMAC output is intended to be a certain number t of bytes in length, so the operator $\text{ }_t(M)$ takes the left-most t bytes of M .

The careful reader will observe that the definition of HMAC is a hash around another hash, of the form $H(K1 \parallel H(K2 \parallel M))$ using keys $K1$ and $K2$. This structure resists so-called extension attacks in which a selected pad value can be combined

(e.g., by Mallory) with an intercepted message and digest value to form a new, valid message and digest value (not sent by Alice). The values of $opad$ and $ipad$ are not critical but tend to produce K_1 and K_2 values with few bits in common (i.e., they have a large hamming distance). Certain extension attacks have been shown to be effective against naively constructed MACs such as those of the form $H(K \parallel M)$ or $H(M \parallel K)$ but ineffective against the HMAC construct (or NMAC construct [BCK96], of which HMAC is a derivative)[B06].

More recently, other forms of MACs have been standardized, called the cipher-based MAC (CMAC) [FIPS800-38B] and GMAC [NIST800-38D]. Instead of using a cryptographic hash function such as HMAC, these use a block cipher such as AES or 3DES. CMAC is envisioned for use in environments where it is more convenient or efficient to use a block cipher in place of a hash function. Details of CMAC using AES-128, called AES-CMAC, are provided in [RFC4493]. In essence, it works by encrypting a message block using AES-128 with a key K , taking the result and XORing it with the subsequent block, encrypting the result, and repeating the process until no more message blocks remain, with the output value being the result of the final encryption operation. If the final message block's length is an even multiple of the algorithm's block size, one subkey derived from K using a special subkey-generating algorithm [IK03], is used in performing the final encryption. If not, the final message block is first padded and a second subkey, also generated from K , is used to perform the final encryption. GMAC uses a special mode of AES called Galois/Counter Mode (GCM). It also uses a keyed hash function (called GHASH, which is not a cryptographic hash function). We will see more about cryptographic operating modes in the next section.

18.4.10 Cryptographic Suites and Cipher Suites

At this point we have seen mechanisms to ensure confidentiality, authenticity, and integrity of information sent across an unsecured communication network. There are other capabilities (e.g., nonrepudiation) that can also be achieved by selecting the appropriate mathematical or cryptographic techniques. The combination of techniques used in a particular system, especially those we see used with Internet protocols, are called a cryptographic suite or sometimes a cipher suite, although the first term is more accurate. A cryptographic suite defines not only an enciphering (encryption) algorithm but may also include a particular MAC algorithm, PRF, key agreement algorithm, signature algorithm, and associated key lengths and parameters.

Many cryptographic suites are defined for use with the security protocols we shall discuss. Usually, an encryption algorithm is specified by its name and description, how many bits are used for its keys (often a multiple of 128 bits), along with its operating mode. Encryption algorithms that have been standardized for use with Internet protocols include AES, 3DES, NULL [RFC2410], and CAMEL-LIA [RFC3713]. The NULL encryption algorithm does not modify the input and is used in certain circumstances where confidentiality is not required.

The operating mode of an encryption algorithm, especially a block cipher, describes how to use the encryption function for a single block repeatedly (e.g., in a cascade) to encrypt or decrypt an entire message with a single key. Common modes today include cipher block chaining (CBC) and counter (CTR) mode, although many others have been defined. When performing encryption using CBC mode, a cleartext block to be encrypted is first XORed with the previous ciphertext block (the first block is XORed with a random initialization vector or IV). Encrypting in CTR mode involves first creating a value combining a nonce (or IV) and a counter that increments with each successive block to be encrypted. The combination is then encrypted, the output is XORed with a cleartext block to produce a ciphertext block, and the process repeats for successive blocks. In effect, this approach uses a block cipher to produce a keystream. A keystream is a sequence of (random-appearing) bits that are combined (e.g., XORed) with cleartext bits to produce a ciphertext. Doing so essentially converts a block cipher into a stream cipher because no explicit padding of the input is required.

CBC requires a serial process for encryption and a partly serial process for decryption, whereas counter mode algorithms allow more efficient fully parallel encryption and decryption implementations. Consequently, counter mode is gaining popularity. In addition, variants of CTR mode (e.g., counter mode with CBC-MAC (CCM), Galois Counter Mode, or GCM) can be used for authenticated encryption [RFC4309], and possibly to authenticate (but not encrypt) additional data (called authenticated encryption with associated data (AEAD)) [RFC5116]. When authenticated encryption algorithms are used, separate MACs are generally not necessary. In the degenerate case of an AEAD algorithm operating on data that does not require confidentiality, a form of MAC is effectively produced (e.g., GMAC). When an encryption algorithm is specified as part of a cryptographic suite, its name usually includes the mode, and the key length is often implied. For example, ENCR_AES_CTR refers to AES-128 used in CTR mode.

When a PRF is included in the definition of a cryptographic suite, it is usually based on a cryptographic hash algorithm family such as SHA-2 [RFC6234] or a cryptographic MAC such as CMAC [RFC4434][RFC4615]. Constructions of this type generally include the name of the function serving as the basis. For example, the algorithm AES-CMAC-PRF-128 refers to a PRF constructed using a CMAC based on AES-128. It is also written as PRF_AES128_CMAC. The algorithm PRF_HMAC_SHA1 refers to a PRF based on HMAC-SHA1.

Key agreement parameters, when included with an Internet cryptographic suite definition, refer to DH group definitions, as no other key agreement protocol is in widespread use. When DH key agreement is used in generating keys for a particular encryption algorithm, care must be taken to ensure that the keys produced are of sufficient length (strength) to avoid compromising the security of the encryption algorithm. Consequently, more than 16 groups for use with DH in different contexts have been standardized [RFC5114]. The first 5 have become known as the "Oakley Groups" because they were specified by the Oakley protocol [RFC2409], an early component of IPsec that has since been deprecated. The

modular exponentiation. MODP groups are based on exponentiation and modular arithmetic. The elliptic curve groups modulo a prime p (ECP groups [RFC5903]) are based on curves over the Galois field $GF(p)$ for a prime p , and the elliptic curve groups modulo a power of two (EC2N) are based on curves over the field $GF(2^N)$ for some N .

A signature algorithm is sometimes included in the definition of a cryptographic suite. It may be used for signing a variety of values including data, MACs, and DH values. The most common is to use RSA to sign a hashed value for some block of data, although the digital signature standard (written as DSS or DSA to indicate the digital signature algorithm) [FIPS186-3] is also used in some circumstances. With the advent of ECC, signatures based on elliptic curves (e.g., ECDSA [X9.62-2005]) are also now supported in many systems.

The concept of a cryptographic suite evolved in the context of Internet security protocols because of a need for modularity and decoupled evolution. As computational power has improved, older cryptographic algorithms and smaller key lengths have fallen victim to various forms of brute-force attacks. In some cases, more sophisticated attacks have revealed flaws that necessitate the replacement of the underlying mathematical and cryptographic methods, but the basic protocol machinery is otherwise sound. As a result, the choice of a cryptographic suite can now be made separately from the communication protocol details and depends on factors such as convenience, performance, and security. Protocols tend to make use of the components of a cryptographic suite in a standard way, so an appropriate cryptographic suite can be “snapped in” when deemed appropriate. It is now common practice in protocol design to “outsource” the security processing to a separately defined set of cryptographic suites that have been analyzed by a large community with the necessary cryptographic and mathematical expertise. Although the ability to “snap in” a new cipher suite is appealing, it can still take years to standardize on acceptable suites and get them deployed. For interoperability, each participant in a communication exchange must usually employ the same suite. This can be a significant hurdle when cipher suites may be implemented in a wide range of software and hardware systems.

18.5 Certificates, Certificate Authorities (CAs), and PKIs

The tools provided by cryptography and related mathematics, including digital signatures and enciphering algorithms, provide a sound basis for constructing secure systems, but a great deal of additional work is required to create an entire system from these parts. Among the items of particular concern are the construction of secure protocols that use cryptographic methods in safe ways, and how keys are created, exchanged, and revoked (called key management). Key management remains one of the greatest challenges in deploying cryptographic systems on a widespread basis across multiple administrative domains.

One of the challenges with public key cryptosystems is to determine the correct public key for a principal or identity. In our running example, if Alice were to send her public key to Bob, Mallory could modify it in transit to be her own public key, and Bob (called the relying party here) might unknowingly be using Mallory's key, thinking it is Alice's. This would allow Mallory to effectively masquerade as Alice. To address this problem, a public key certificate is used to bind an identity to a particular public key using a digital signature. At first glance, this presents a certain "chicken-egg" problem: How can a public key become signed if the digital signature itself requires a reliable public key? There are two ways this is accomplished today.

One model, called a web of trust, involves having a certificate (identity/key binding) endorsed by a collection of existing users (called endorser). An endorser signs a certificate and distributes the signed certificate. The more endorser for a certificate over time, the more reliable it is likely to be. An entity checking a certificate might require some number of endorser or possibly some particular endorser to trust the certificate. The web of trust model is decentralized and "grassroots" in nature, with no central authority. This has mixed consequences. Having no central authority suggests that the scheme will not collapse because of a single point of failure, but it also means that a new entrant may experience some delay in getting its key endorsed to a degree sufficient to be trusted by a significant number of users. Some groups hold "key signing parties" to hasten this process. The web of trust model was first described as part of the Pretty Good Privacy (PGP) encryption system for electronic mail [NAZ00], which has evolved to support a standard encoding format called OpenPGP defined by [RFC4880].

A more formal approach, which has the added benefit of being provably secure under certain theoretical assumptions in exchange for more dependence on a centralized authority, involves the use of a public key infrastructure (PKI). A PKI is a service responsible for creating, revoking, distributing, and updating key pairs and certificates. It operates with a collection of certificate authorities (CAs). A CA is an entity and service set up to manage and attest to the bindings between identities and their corresponding public keys. There are several hundred commercial CAs. A CA usually employs a hierarchical signing scheme. This means that a public key may be signed using a parent key which is in turn signed by a grand-parent key, and so on. Ultimately a CA has one or more root certificates upon which many subordinate certificates depend for trust. An entity that is authoritative for certificates and keys (e.g., a CA) is called a trust anchor, although this term is also used to describe the certificates or other cryptographic material associated with such entities [RFC6024], which we discuss next.

18.5.1 Public Key Certificates, Certificate Authorities, and X.509

While several types of certificates have been used in the past, the one of most interest to us is based on an Internet profile of the ITU-T X.509 standard [RFC5280]. In addition, any particular certificate may be stored and exchanged in a number of

file or encoding formats. The most common ones include DER, PEM (a Base64 encoded version of DER), PKCS#7 (P7B), and PKCS#12 (PFX). We also saw the use of PKCS#1 [RFC3447] in Chapter 8. Today, Internet PKI-related standards tend to use the cryptographic message syntax [RFC5652], which is based on PKCS#7 version 1.5. In the following example, we use an X.509 certificate in PEM format, which is the default format for many Internet applications and has the added advantage of being easily displayed as ASCII.

Certificates are primarily used in identifying four types of entities on the Internet: individuals, servers, software publishers, and CAs. One popular commercial CA, Verisign, assigns a “class” to each certificate, in the range 1 through 5. Class 1 certificates are intended for individuals, class 2 for organizations, class 3 for servers and software signing, class 4 for online transactions between companies, and class 5 for private organizations and governments. Certificate classes are primarily a convenience for grouping and naming types of certificates and for defining different security policies associated with them. Generally speaking, a higher class number is supposed to indicate more rigorous controls on the process required to validate an identity (called identity proofing) prior to issuing the associated certificate.

This still does not totally solve the chicken-egg PKI bootstrapping problem mentioned before. In practice, systems requiring public key operations have root certificates for popular CAs installed at configuration time (e.g., Microsoft Internet Explorer, Mozilla’s Firefox, and Google’s Chrome are all capable of accessing a preconfigured database of root certificates). To see how this works, we can use a command that gives information about certificates. The `openssl` command, available for most common platforms including Linux and Windows, allows us to see the certificates for a Web site (some lines are wrapped for clarity):

```
Linux% CDIR=`openssl version -d | awk '{print $2}'`
Linux% openssl s_client -CApath $CDIR \
      -connect www.digicert.com:443 > digicert.out 2>1
^C (to interrupt)
```

The first command determines where the local system stores its preconfigured CA certificates. This is usually a directory that varies by system. In this case, the name of the directory is stored in the shell variable `CDIR`. We next make a connection to the HTTPS port (443) on the `www.digicert.com` server and redirect the output to the `digicert.out` file. The `openssl` command² takes care to print the entity identified by each of the certificates, and at what depth they are in the certificate hierarchy relative to the root (depth 0 is the server’s certificate, so the depth numbers are counted bottom to top). It also checks the certificates against the stored CA certificates to see if they verify properly. In this case, they do, as indicated by “verify return” having value `0` (ok).

2. Note that a similar command unique to Windows called `certutil` is available with Windows 2003 Server and the Windows Server 2003 Administration Tools Pack.

```
Linux% grep "return code" digicert.out
Verify return code: 0 (ok)
```

The file `digicert.out` contains not only a trace of the connection to the server but also a copy of the server's certificate. To get the certificate into a more usable form, we can extract the certificate data, convert it, and place the result into a PEM-encoded certificate file:

```
Linux% openssl x509 -in digicert.out -out digicert.pem
```

Given the certificate in PEM format, we can now use a variety of `openssl` functions to manipulate and inspect it. At the highest level, the certificate includes some data to be signed (called the "TBSCertificate") followed by a signature algorithm identifier and signature value. To see the server certificate, we can use the following command (some lines are wrapped or removed for clarity):

```
Linux% openssl x509 -in digicert.pem -text
Certificate:
  Data:
    Version: 3 (0x2)
    Serial Number:
      02:c7:1f:e0:1d:70:41:4b:8b:a7:e2:9e:5e:58:42:b9
    Signature Algorithm: sha1WithRSAEncryption
    Issuer: C=US, O=DigiCert Inc, OU=www.digicert.com,
      CN=DigiCert High Assurance EV CA-1
    Validity
      Not Before: Oct  6 00:00:00 2010 GMT
      Not After : Oct  9 23:59:59 2012 GMT
    Subject: 2.5.4.15=V1.0, Clause 5.(b)/
      1.3.6.1.4.1.311.60.2.1.3=us/
      1.3.6.1.4.1.311.60.2.1.2=Utah/
      serialNumber=5299537-0142,
      C=US, ST=Utah, L=Lindon, O=DigiCert, Inc.,
      CN=www.digicert.com
    Subject Public Key Info:
      Public Key Algorithm: rsaEncryption
      RSA Public Key: (2048 bit)
        Modulus (2048 bit):
          00:d1:76:0b:1e:4e:96:d2:08:c1:b8:75:bd:20:9c:
          66:7f:42:6b:54:8b:7f:7a:4a:f8:3e:df:70:68:1f:
          ...
          25:7b:40:e9:e3:cc:a2:0d:95:29:f4:08:ed:50:16:
          52:11:6f:de:a0:bb:34:bc:8b:b5:60:c1:ab:e4:78:
          75:9f
        Exponent: 65537 (0x10001)
    X509v3 extensions:
      X509v3 Authority Key Identifier:
        keyid:4C:58:CB:25:F0:41:4F:52:F4:
        28:C8:81:43:9B:A6:A8:A0:E6:92:E5
```

```

X509v3 Subject Key Identifier:
    4F:E0:97:FF:C1:AE:06:53:03:19:F7:
    0A:37:4B:9F:F0:13:E2:88:D8
X509v3 Subject Alternative Name:
    DNS:www.digicert.com, DNS:content.digicert.com
Authority Information Access:
    OCSP - URI:http://ocsp.digicert.com
    CA Issuers - URI:
        http://www.digicert.com/CACerts/
        DigiCertHighAssuranceEVCA-1.crt
Netscape Cert Type:
    SSL Client, SSL Server
X509v3 Key Usage: critical
    Digital Signature, Key Encipherment
X509v3 Basic Constraints: critical
    CA:FALSE
X509v3 CRL Distribution Points:
    URI:http://crl3.digicert.com/ev2009a.crl
    URI:http://crl4.digicert.com/ev2009a.crl
X509v3 Certificate Policies:
    Policy: 2.16.840.1.114412.2.1
        CPS: http://www.digicert.com/ssl-cps-repository.htm
    User Notice:
        Explicit Text:

X509v3 Extended Key Usage:
    TLS Web Server Authentication,
    TLS Web Client Authentication
Signature Algorithm: sha1WithRSAEncryption
    e1:e6:dd:0e:23:5f:08:9a:63:63:c7:a1:f3:95:f0:ca:7e:3c:
    57:81:2c:2a:19:2b:24:fe:e4:26:bd:91:27:7c:11:50:35:e7:
    ...
    fd:64:6f:97:8b:15:fb:d1:7a:f7:67:80:da:da:41:d8:e3:f9:
    e4:bd:92:97
-----BEGIN CERTIFICATE-----
MIIHLTCCBhwgAwIBAgIQAscF4B1wQUuLp+KeXlhCuTANBgkqhkiG9w0BAQUFADBp
MQswCQYDVQQGEwJVUzEVMBMGA1UEChMMRGlnaUNlcnQgSW5jMRkwFwYDVQQLExB3
...
8+qQ0wF/xY9rHM0+eIqy3da4AFhfW4sAmyafs7hcEMjUAkS6Yb0qIw8ud/1kb5eL
FfvRevdngNraQdjj+eS9kpc=
-----END CERTIFICATE-----

```

Looking at the command's output, we see a decoded version of the certificate followed by an ASCII (PEM) representation of the certificate (between the BEGIN CERTIFICATE and END CERTIFICATE indicators). The decoded certificate shows a data portion and a signature portion. Within the data portion is some metadata including a Version field, indicating the particular X.509 certificate type (3, the most recent, is encoded using hex value 0x02), a Serial Number of the particular certificate, a number assigned by the CA unique to each certificate, and a Validity field that gives the time during which the certificate should be treated as legitimate,

starting with the `Not Before` subfield and ending with the `Not After` subfield. The certificate metadata also indicates which signature algorithm is used to sign the data portion. In this case, it is signed by computing a hash using SHA-1 and signing the result using RSA. The signature itself appears at the end of the certificate.

The `Issuer` field indicates the distinguished name (jargon from the ITU-T X.500 standard) of the entity that issued the certificate and may have these special subfields (based on X.501): `C` (country), `L` (locale or city), `O` (organization), `OU` (organizational unit), `ST` (state or province), `CN` (common name). Other subfields have also been defined. In this case, we can see that an extended validation (EV) [CABF09] CA certificate has been used to sign the server's certificate.

EV certificates represent an industry response to certain phishing attacks involving malicious Web sites that were issued certificates without rigorous identity proofing. Issuing of an EV certificate takes place only under an agreed-upon set of stringent criteria, and a user visiting a Web site using EV certificates and a modern browser typically sees a green title bar and CA information to indicate the enhanced level of rigor. One of the requirements for EV certificates placed upon each CA is to provide a certification practice statement (CPS), which outlines the practices used in issuing certificates. Considerations for authors of CPSs (and certificate policies or CPs that apply on a per-certificate basis) are given in [RFC5280]. Note that although EV certificates may provide higher assurance (e.g., for some Web sites), most users do not pay careful attention to the cues provided by Web browsers that reveal this fact [BOPSW09].

The `Subject` field identifies the entity this certificate is about, and the owner of the public key contained in the subsequent `Subject Public Key Info` field. In this example, the `Subject` field is a somewhat complex structure like the `Issuer` field and contains multiple object IDs (OIDs) [ITU01D]. Most are decoded with names (e.g., `O`, `C`, `ST`, `L`, `CN`), but some are not because the particular version of `openssl` that printed the output did not understand them. The OID 1.3.6.1.4.1.311.60.2.1.3 is also called `jurisdictionOfIncorporationCountryName`, and 1.3.6.1.4.1.311.60.2.1.2 is called `jurisdictionOfIncorporationStateOrProvinceName`, both with obvious meanings. The OID 2.5.4.15 is `businessCategory` (see [CABF09] for details). Note that the `CN` subfield tends to be an important one when identifying subjects and issuers for certificates used on the Internet. For this certificate, it gives the correct matching name for the server (along with any names included in the `Subject Alternative Name` (SAN) extension). Nonmatching names or URLs (e.g., `https://digicert.com` instead of `https://www.digicert.com`) referring to the same server, when accessed, result in an error. Note that `CN` is not really the field for holding a DNS name; SANs are intended for this purpose.

When a certificate needs to be validated, a recursive process works up the certificate hierarchy to a root CA certificate by matching the issuer distinguished name in one certificate with the subject name in another. In this case, the certificate was issued by `DigiCert High Assurance EV CA-1` (the issuer's `CN` subfield). Assuming all certificates are current in their validity periods and are being used in appropriate ways, some parent certificate (immediate parent, grandparent, etc.,

but usually a root CA certificate) to the Subjectfield of the certificate we are evaluating must be trusted for validation to be successful.

The Subject Public Key Inf~~o~~field gives the algorithm and public key belonging to the entity specified in the Subjectfield. In this case, the public key is an RSA public key with a 2048-bit modulus and public exponent of 65537. The subject is in possession of the matching RSA private key (modulus plus private exponent) that is paired to the public key. If the private key is compromised, or if the public key needs to be changed for other reasons, the public and private keys must be regenerated and a new certificate issued. The old certificate is then revoked (see Section 18.5.2).

Version 3 X.509 certificates may include zero or more extensions. Extensions are either critical or noncritical, and some are required by the Internet profile in [RFC5280]. If critical, an extension must be processed and found acceptable by the relying party's (CPS jargon) policy. Noncritical extensions are processed if supported but do not otherwise cause errors. In the present example, there are ten X.509v3 extensions. Although many extensions have been defined, those we shall discuss tend to fall into two informal categories. The first category includes information about the subject and how the certificate in question can be used. The second category relates to items describing the issuer and may include key identification and URIs indicating locations of additional information related to the issuing CA that is not included elsewhere. The certificate in our example is an end entity(not CA) certificate. CA certificates often have somewhat different extensions or values for their extensions.

The Basic Constraints extension, a critical extension, indicates whether the certificate is a CA certificate. In this case it is not, so it cannot be used for signing other certificates. A certificate indicating that it is a CA certificate may be used in a certificate validation chain at a location other than a leaf. This is common for root CA certificates or for other certificate-signing certificates ("intermediate" certificates, such as the DigiCert High Assurance EV CA-1 certificate referenced in this example).

The Subject Key Identifie~~r~~ extension identifies the public key in the certificate. It allows different keys owned by the same subject to be differentiated. The Key Usage extension, a critical extension, determines the valid usage for the key. Possible usages include digital signature, nonrepudiation (content commitment), key encipherment, data encipherment, key agreement, certificate signing, CRL signing (see Section 18.5.2), encipher only, and decipher only. Because server certificates of this kind are primarily used for identifying the two endpoints of a connection and encrypting a session key (see Section 18.9), the possible usages may be somewhat limited, as in this case. The Extended Key Usage extension, which may be critical or noncritical, may provide further restrictions on the key use. Possible values of this extension when used in the Internet profile include the following: TLS client and server authentication, signing of downloadable code, e-mail protection (nonrepudiation and key agreement or encipherment), various IPsec operating modes (see Section 18.8), and timestamping. The SAN extension allows a single certificate to be

used for multiple purposes (e.g., for multiple Web sites with distinct DNS names). This alleviates the need to have a separate certificate for each Web site, which can significantly reduce cost and administrative burden. In this case, the certificate can be used for either of the DNS names `www.digicert.com` or `content.digicert.com` (but not `digicert.com`, as mentioned before). The Netscape Cert Type extension is now deprecated but was used to indicate key usage to Netscape software.

The remaining extensions in our example certificate relate to the management and status of the certificate and its issuing CA. The CRL Distribution Points (CDP) extension gives a list of URLs for finding the CA's certificate revocation list (CRL), a list of revoked certificates used to determine if a certificate in a validation chain has been revoked (see Section 18.5.2). The Certificate Policies (CP) extension includes certificate policies applicable to the certificate [RFC5280]. In this example, the CP extension contains a policy with two qualifiers. The Policy value of 2.16.840.1.114412.2.1 indicates that the certificate complies with an EV policy. The CP qualifier gives a pointer to the URI where the particular applicable CPS for the policy may be found. The User Notice qualifier may contain text intended to be displayed to a relying party. In this case it contains the following string:

Any use of this Certificate constitutes acceptance of the DigiCert EV CPS and the Relying Party Agreement which limit liability and are incorporated herein by reference.

The Authority Key Identifier identifies the public key corresponding to the private key used to sign the certificate. It is useful when an issuer has multiple private keys used for generating signatures. The Authority Information Access (AIA) extension indicates where information may be retrieved from the CA. In this case, it indicates a URI used to determine if the certificate has been revoked using an online query protocol (see Section 18.5.2). It also indicates the list of CA issuers, which includes a URL containing the CA certificate responsible for signing the example server certificate.

Following the extensions, the certificate contains the signature portion. It contains the identification of the signature algorithm (SHA-1 with RSA here), which must match the Signature Algorithm field we encountered earlier. In this case, the signature itself is a 256-byte value, corresponding to the 2048-bit modulus used for this use of RSA.

18.5.2 Validating and Revoking Certificates

We have already encountered the idea that a certificate may have to be revoked and possibly replaced with a freshly issued certificate. Within the IETF, [RFC5280] defines the use of X.509 version 3 certificates with X.509 version 2 CRLs for the Internet. This brings up the question of how a certificate is revoked and how this fact is made known to relying parties that need to know that the certificates on which they depend are no longer trustworthy.

To validate a certificate, a validation or certification path must be established that includes a set of validated certificates, usually up to some trust anchor (e.g., root certificate) that is already known to the relying party. One of the key steps involves determining if one or more of the certificates in a chain have been revoked. If so, the path validation fails. We saw some of this in Section 8.5.5.

There are several reasons why a certificate may need to be revoked, such as when a certificate's subject (or issuer) changes affiliations or name. When a certificate is revoked, it may no longer be used. The challenge is to ensure that entities that wish to use a certificate become aware if it has been revoked. In the Internet, there are two primary ways this is accomplished: CRLs and the Online Certificate Status Protocol (OCSP) [RFC2560]. When the CRL Distribution Point extension includes an HTTP or FTP URI scheme, as it does in the preceding example, the complete URL gives the name of a file encoded in DER format containing an X.509 CRL. In our example, we can retrieve the CRL corresponding to the certificate using the following command:

```
Linux% wget http://crl3.digicert.com/ev2009a.crl
```

and print it out as follows:

```
Linux% openssl crl -inform der -in ev2009a.crl -text
Certificate Revocation List (CRL):
    Version 2 (0x1)
    Signature Algorithm: sha1WithRSAEncryption
    Issuer: /C=US/O=DigiCert Inc/OU=www.digicert.com/
           CN=DigiCert High Assurance EV CA-1
    Last Update: Jan  2 06:20:13 2011 GMT
    Next Update: Jan  9 06:20:00 2011 GMT
    CRL extensions:
        X509v3 Authority Key Identifier:
            keyid:4C:58:CB:25:F0:41:4F:52:F4:
            28:C8:81:43:9B:A6:A8:A0:E6:92:E5

        X509v3 CRL Number:
            732
    Revoked Certificates:
    Serial Number: 0119BF8D1A24460EBE59355A11AD7B1C
    Revocation Date: Jul 29 19:25:40 2009 GMT
    CRL entry extensions:
        X509v3 CRL Reason Code:
            Unspecified
    ...
    Serial Number: 0D2ED685A9A828A21067D1826C5015A9
    Revocation Date: Dec 17 17:18:40 2010 GMT
    CRL entry extensions:
        X509v3 CRL Reason Code:
            Superseded
    Signature Algorithm: sha1WithRSAEncryption
    d4:a3:50:07:1b:b8:17:ff:e2:83:3d:b9:6a:3e:22:8d:e4:22:
    40:12:0b:cf:26:d9:16:99:b1:96:5a:86:ea:3e:8a:3f:f9:39:
    ...
```



```

c7:e0:92:f6:66:72:7e:a4:f0:fd:16:d4:ec:2f:10:35:ea:2d:
45:06:19:4b
-----BEGIN X509 CRL-----
MIIHeDCCBmACAQEWdQYJKoZIhvcNAQEFBQAwaTElMAkGA1UEBhMCVVMxFTATBgNV
BAoTDERpZ2lZdXJ0IEluYzEZMBcGA1UECzMqd3d3LmRpZ2ljZXJ0LmNvbTEoMCYG
...
hzcRf+ITVZ76LTHdzWDDPFujPyqPzMnkbGqGVsve9Gd4NcQiozOyoCDvaLezg069
EYmMayk9zXFSaBVdEZ5Tgkcrj0fFnsfgkvZmcn6k8P0W10wvEDXqLUUGGUs=
-----END X509 CRL-----

```

Here we can see the format of an X.509 v2 CRL. The format is very similar to that of a certificate, and the entire message is signed by a CA as certificates are. This is useful because CRLs can be distributed like certificates: using otherwise untrusted communication channels and servers. In comparison with a certificate, the validity period is replaced by a list of the previous and next CRL updates. There is no subject and no public key but instead a list of serial numbers for revoked certificates plus the time and reason for revocation. There may also be CRL extensions that are unique to CRLs. In this example, the Authority Key Identifier extension gives a number identifying the key used by the CA in signing the CRL. The CRL Number extension gives the sequence number of the CRL. Other values are given in [RFC5280].

The other primary method for determining if a certificate has been revoked is OCSP. OCSP is an application-level request/response protocol usually operated over HTTP (i.e., using the HTTP protocol with TCP/IP on TCP port 80). An OCSP request includes information identifying a particular certificate, plus some optional extensions. A response indicates whether the certificate is not revoked, unknown, or revoked. An error may be returned if the request cannot be parsed or otherwise acted upon. The key used for signing the OCSP response need not necessarily match the key used to sign the original certificate. This is possible if the issuer included a Key Usage extension indicating an alternate OCSP provider.

To see an OCSP request/response exchange, we can execute the following commands once we have obtained the appropriate Class 1 certificate in the file `DigiCertHighAssuranceEVCA-1.pem` (not shown). In the following example, some lines are wrapped for clarity:

```

Linux% CERT=DigiCertHighAssuranceEVCA-1.pem
Linux% openssl ocsp -issuer $CERT -cert digicert.pem \
-url http://ocsp.digicert.com -VAfile $CERT -no_nonce -text
OCSP Request Data:
  Version: 1 (0x0)
  Requestor List:
    Certificate ID:
      Hash Algorithm: sha1
      Issuer Name Hash: B8A299F09D061DD5C1588F76CC89FF57092B94DD
      Issuer Key Hash: 4C58CB25F0414F52F428C881439BA6A8A0E692E5
      Serial Number: 02C71FE01D70414B8BA7E29E5E5842B9

```

```

OCSP Response Data:
  OCSP Response Status: successful (0x0)
  Response Type: Basic OCSP Response
  Version: 1 (0x0)
  Responder Id: 4C58CB25F0414F52F428C881439BA6A8A0E692E5
  Produced At: Jan  2 08:03:24 2011 GMT
  Responses:
    Certificate ID:
      Hash Algorithm: sha1
      Issuer Name Hash: B8A299F09D061DD5C1588F76CC89FF57092B94DD
      Issuer Key Hash: 4C58CB25F0414F52F428C881439BA6A8A0E692E5
      Serial Number: 02C71FE01D70414B8BA7E29E5E5842B9
    Cert Status: good
    This Update: Jan  2 08:03:24 2011 GMT
    Next Update: Jan  9 08:18:24 2011 GMT

Response verify OK
digicert.pem: good
  This Update: Jan  2 08:03:24 2011 GMT
  Next Update: Jan  9 08:18:24 2011 GMT

```

As we can see, the OCSP transaction has indicated that the certificate is good. The request included the identification of a hash algorithm (SHA-1), a hash of the issuer name, a number identifying the issuer's key (the same as the Key ID extension in the certificate), plus the certificate's serial number. The responder, identified by the responder ID, identifies itself and signs the response. The response includes the hashes and numbers from the request, as well as the certificate status of "good" (i.e., not revoked). The OCSP protocol alleviates the client from having to download the latest CRL to check but still requires the client to form and verify the entire certification path. In some cases, this can be a considerable burden for the client.

To help address the burden of certificate chain formation and validation imposed on client systems, the Server-Based Certificate Validation Protocol (SCVP) has been defined in [RFC5055] but is not widely used. With SCVP, formulation of a certification path (called delegated path discovery or DPD) and, optionally, validation (called delegated path validation or DPV) of it can be offloaded to a server. Validation is offloaded only to a trusted server. Not only does this provide a method to reduce the load on clients, but it also offers a method for helping to ensure that a common validation policy is used consistently throughout an enterprise.

18.5.3 Attribute Certificates

In addition to public key certificates (PKCs) used to bind names to public keys, X.509 defines another type of certificate called an attribute certificate (AC). ACs are similar in structure to PKCs but lack a public key. They are used to indicate other information, including authorization information that may have a lifetime different from (e.g., shorter than) a corresponding PKC [RFC5755]. ACs contain other structures similar to PKCs, including extensions and AC policies.

18.6 TCP/IP Security Protocols and Layering

We have seen that cryptography provides a basis for building communication systems that have a number of desirable security properties. Protocols involving cryptography can (and do) exist at a number of different layers in the protocol stack. Consistent with our understanding of the OSI reference model we discussed in Chapter 1, we now see that encryption, and thus various forms of strong security, can be supported at essentially every layer.

As we might expect, security services at the link layer protect information only as it flows across a single communication hop, security at the network layer protects information flowing between hosts, security at the transport layer protects process-to-process communication, and security at the application layer protects information manipulated by applications. It is also possible to protect the data manipulated by applications independently of the communication layers (e.g., files can be encrypted and sent as e-mail attachments). Figure 18-4 illustrates the most common security protocols used in conjunction with TCP/IP.

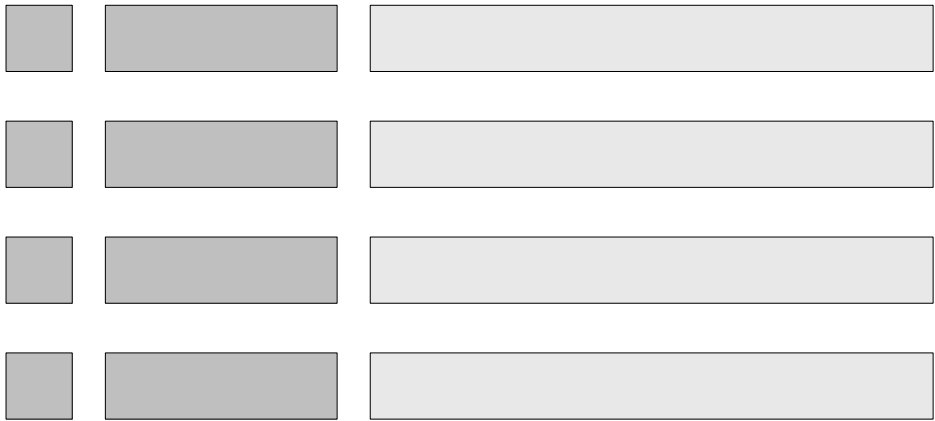


Figure 18-4 Security protocols exist at essentially every OSI stack layer, plus some “in-between” layers. Selecting the appropriate protocols for the threats to be addressed requires attention to detail.

In Figure 18-4, we can see that there are many security protocols, and the ones we care about at any given time depend on what scope of functionality we require. We shall discuss most of the protocols in Figure 18-4 in what follows, with particular emphasis on IPsec (machine-to-machine security at layer 3), TLS (Transport Layer Security designed for supporting applications), and DNSSEC. TLS and IPsec are the most prevalent, as TLS is used with all secure Web communications (HTTPS) and IPsec is used with most network-layer security, including VPNs.

DNSSEC, which secures the DNS (see Chapter 11), is being introduced slowly, but the perceived demand is significant. Security of the DNS will help to limit DNS hijacking attacks, in which client systems are redirected to bogus DNS servers that supply incorrect information. Two of the fairly popular protocols we do not discuss in detail are Kerberos [RFC4120]—a trusted third-party authentication system now used in Windows enterprise environments—and SSH [RFC4251]—the secure shell remote login and tunneling protocol used most often with UNIX-like systems. These protocols tend to be used among computers running particular operating systems, although this is by no means required. We have elected to use the detailed protocol descriptions in this chapter to cover the protocols that we believe will apply to an even broader Internet audience over time.

Although virtually every modern networking technology has some associated security approach, we shall move up the layers in the OSI stack from the bottom, starting with the link layer. We have already seen (see Chapter 3) that some of the link-layer protocols have their own security mechanisms (e.g., 802.11-2007 has WPA2 included in the specification, based on the earlier 802.11i specification). We shall be especially concerned with protocols that apply to more than one specific type of link layer network.

18.7 Network Access Control: 802.1X, 802.1AE, EAP, and PANA

Network Access Control (NAC) refers to methods used to authorize or deny network communications to particular systems or users. Defined by the IEEE, the 802.1X Port-Based Network Access Control (PNAC) standard is commonly used with TCP/IP networks to support LAN security in enterprises, for both wired and wireless networks. The purpose of PNAC is to provide access to a network (e.g., intranet or the Internet) only if a system and/or its user has been authenticated based on the system's network attachment point. Used in conjunction with the IETF standard Extensible Authentication Protocol (EAP) [RFC3748], 802.1X is sometimes called EAP over LAN (EAPoL), although the 802.1X standard covers more than just the EAPoL packet format.

The most common variant of 802.1X is based on the standard as published in 2004, however, [802.1X-2010] includes compatibility with 802.1AE (IEEE standard LAN encryption called MACSec) and 802.1AR (X.509 certificates for secure device identities). It also includes a somewhat complex MACSec key agreement protocol called MKA that we do not discuss further. In 802.1X, a system being authenticated implements a function known as a supplicant. The supplicant interacts with an authenticator and a backend authentication server to perform authentication and gain network access. VLANs (see Chapter 3) are often used in helping to enforce the access control decisions made by 802.1X.

EAP can be used with multiple link-layer technologies and supports multiple methods for implementing authentication, authorization and accounting (AAA). EAP does not perform encryption itself, so it must be used in conjunction with some other cryptographically strong protocol to be secure. When used with link-layer

encryption such as WPA2 on wireless networks or 802.1AE on wired networks, 802.1X is relatively secure. EAP uses the same concepts of supplicant and authentication server as does 802.1X, but with different terminology (EAP uses the terms peer, authenticator, and AAA server although even in EAP-related literature backend authentication servers is sometimes used). An example setup is shown in Figure 18-5.

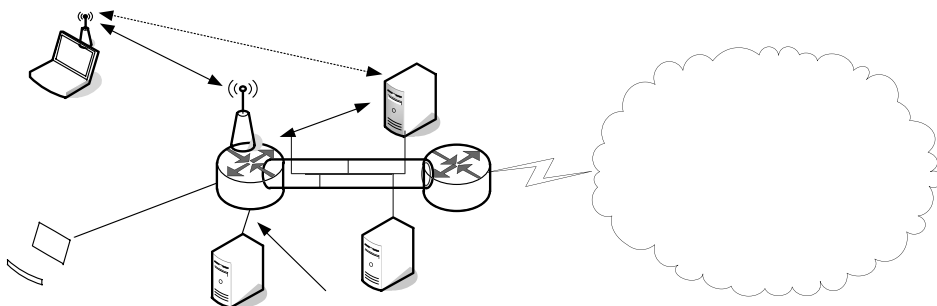


Figure 18-5 EAP, supported by 802.11i and 802.1X, allows for a peer (supplicant) to be authenticated by an authenticator that is separate from an AAA server. The authenticator can operate in “pass-through” mode in which it does little more than forward EAP packets. It can also participate more directly in the EAP protocol. The pass-through mode allows authenticators to avoid having to implement a large number of authentication methods.

In this figure we see a hypothetical enterprise network including wired and wireless peers, a protected network that includes the AAA server and another intranet server on a particular VLAN, and an unauthenticated or “remediation” VLAN. The authenticator’s job is to interact with unauthenticated peers and the AAA server (via AAA protocols such as RADIUS [RFC2865][RFC3162] or Diameter [RFC3588]) to determine if each peer should be granted access to the protected network. If so, this can be accomplished in several ways. The most common approach is to make a VLAN mapping adjustment so that the authenticated peer is assigned to the protected VLAN or to another VLAN that provides connectivity to the protected VLAN using a router (layer 3). An authenticator may use VLAN trunking (IEEE 802.1AX link aggregation; see Chapter 3) and may be capable of assigning VLAN tags based on port number or forwarding VLAN tagged frames sent by the peer.

Note

In some EAP deployments, the authenticator is used without an AAA server, and the authenticator must evaluate the peer’s credentials on its own. When referring to the location where authentication is determined, the term *EAP server* is used in the EAP literature. Generally, the EAP server is the AAA server (backend authentication server) when the authenticator acts in pass-through mode and is the authenticator otherwise.

In 802.1X, the protocol between the supplicant and the authenticator is divided into a lower and upper sublayer. The lower layer is called the port access control protocol (PACP). The higher layer is ordinarily some variant of EAP. For use with 802.1AR, the variant is called EAP-TLS [RFC5216]. PACP uses EAPoL frames for communication, even if EAP authentication is not used (e.g., when MKA is used). EAPoL frames use an EtherType field value of 0x888E (see Chapter 3).

Moving to IETF standards, EAP is not a single protocol but rather a framework for achieving authentication using a combination of other protocols, some of which we discuss throughout the chapter, including TLS and IKEv2. The baseline EAP packet format is shown in Figure 18-6.

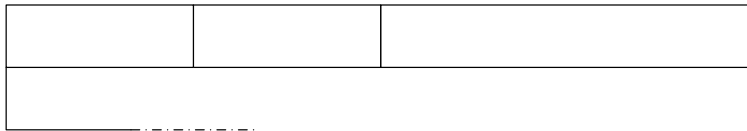


Figure 18-6 The EAP header includes a Code field for demultiplexing packet types (Request, Response, Success, Failure, Initiate, Finish). The identifier helps match requests to responses. For request and response messages, the first data byte is the Type field.

The EAP packet format is simple. In Figure 18-6, the Code field contains one of six EAP packet types: Request (1), Response (2), Success (3), Failure (4), Initiate (5), and Finish (6). The last two are defined by the EAP Re-authentication Protocol (see Section 18.7.2); the official field values are maintained by the IANA [IEAP]. The Identifier field contains a number chosen by the sender and is used to match requests with replies. The Length field gives the number of bytes in the EAP message, including the Code, Identifier, and Length fields. Requests and responses are used to perform identification and authentication with the peer, ultimately resulting in a Success or Failure indication. The protocol is capable of carrying an informative message so that human users can be given some instructions about what to do if their system is unable to authenticate. It is a reliable protocol that runs on a lower-layer protocol that is assumed to preserve order but is not assumed to be reliable. EAP itself does not implement other features such as congestion or flow control but may use protocols that do.

The typical EAP exchange starts with the authenticator sending a Request message to the peer. The peer responds with a Response message. Both messages use the same format, as shown in Figure 18-6. An overview of the exchange is shown in Figure 18-7.

The primary purpose of the Request and Response messages is to exchange whatever information is required to allow an authentication method to succeed. Numerous methods are defined within [RFC3748], and several are defined in other standards. The particular method being used is encoded in the Type field of

layer (for which there are many methods). The lower layer is responsible for transporting EAP frames in order. Perhaps ironically, some of the protocols used to transport EAP are actually higher-layer protocols, many of which we have discussed already. Examples of EAP “lower-layer” protocols include 802.1X, 802.11 (802.11i) (see Chapter 3), UDP with L2TP (see Chapter 3), UDP with IKEv2 (see Section 18.8.1), and TCP (see Chapters 12–17). Figure 18-8 shows how the layers are implemented in conjunction with a pass-through authenticator. A pass-through server would be the opposite but is not supported by RADIUS or Diameter.

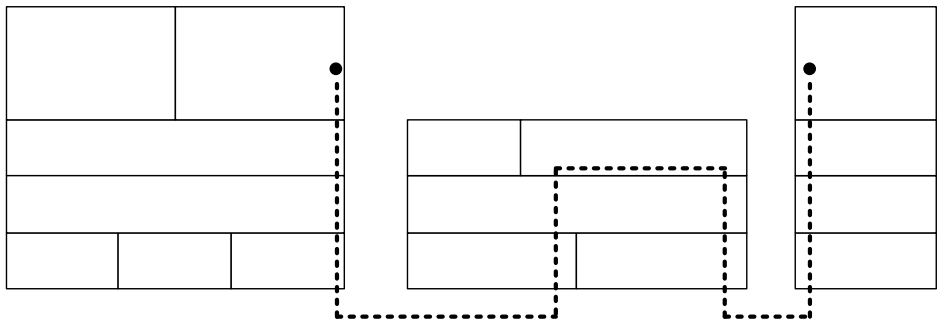


Figure 18.8 The EAP stack and implementation model. In the pass-through mode, the peer and AAA server are responsible for implementing the EAP authentication methods. The authenticator need only implement EAP message processing, the authenticator processing, and enough of an AAA protocol (e.g., RADIUS, Diameter) to exchange information with the AAA server.

In the “EAP stack” depicted in Figure 18-8, the EAP layer implements reliability and duplicate elimination. It also performs demultiplexing based on the code value in EAP packets. The peer/authenticator layer is responsible for implementing the peer and/or authenticator protocol messages, based on demultiplexing of the Codefield. The EAP methods layer consists of all the specific methods to be used for authentication, including any required protocol operations to handle large messages. This is necessary because the rest of the EAP protocol does not implement fragmentation and some methods may require large messages (e.g., containing certificates or certificate chains).

18.7.1 EAP Methods and Key Derivation

Given its architecture, many EAP authentication and encapsulation methods are available for use (more than 50). Some are specified by IETF standards, and others have evolved separately (e.g., from Cisco or Microsoft). Some of the more common

methods include TTLS [RFC5281], TLS [RFC5216], FAST [RFC4851], LEAP (Cisco proprietary), PEAP (EAP over TLS, Cisco proprietary), IKEv2 (experimental) [RFC5106], and MD5. Of these, only MD5 is specified in [RFC3748], but it is no longer recommended for use. Unfortunately, the complexity does not end when specifying one of these methods alone. Within each method there are sometimes different options for cryptographic suites or identity verification. With PEAP, for example, some versions of Microsoft Windows support MSCHAPv2 and TLS.

The reasons for having so many options are partly historical. As security and operational experience have evolved over time, some methods were found to be too insecure or insufficiently flexible. Some authentication methods require an operating PKI that can provide client certificates (e.g., EAP-TLS), while others (e.g., PEAP, TTLS) do not require such infrastructure. Older protocols (e.g., LEAP) were designed at a time when other standards such as 802.11 (incorporating 802.11i) were not yet mature. Consequently, depending on the particular environment, various combinations of smart cards or tokens, passwords, or certificates may be required to use EAP.

The purpose of the EAP methods is to establish authentication, and possibly authorization for network access. In some cases (e.g., EAP-TLS), the methods provide bidirectional authentication, whereby each end acts as both an authenticator and a peer. The type of authentication provided by a method is often a consequence of the cryptographic primitives it employs.

Some methods provide more than authentication. Those that provide key derivation are able to agree upon and export keys in a key hierarchy [RFC5247] and must provide for mutual authentication between the EAP peer and EAP server. The master session key (MSK, also called AAA-key) is used in deriving other keys using a KDF, either at an EAP peer or authenticator. MSKs are at least 64 bytes in length and are typically used to derive transient session keys (SSKs) that are used to enforce access control between a peer and an authenticator, often at lower layers. Extended MSKs (EMSKs) are also provided along with MSKs but are made available only to the EAP server or peer, not to pass-through authenticators, and are used in deriving root keys [RFC5295]. Root keys are keys associated with particular usages or domains. A usage-specific root key (USRK) is a key derived from an EMSK in the context with a particular usage. A domain-specific root key (DSRK) is a key derived from an EMSK for use in a particular domain (i.e., collection of systems). Child keys derived from a DSRK are known as domain-specific usage-specific root keys (DSUSRKs).

During an EAP exchange, multiple peer and server identities may be used, and a session identifier is allocated. On completion of an EAP-based authentication where key derivation is supported, the MSK, EMSK, peer identifier(s), server identifier(s), and a session ID are made available to lower layers. (A now-deprecated initialization vector might also be provided.) Keys generally have an associated lifetime (8 hours is recommended), after which EAP re-authentication is required. For an in-depth discussion of EAP's key management framework and an accompanying detailed security analysis, please see [RFC5247].

18.7.2 The EAP Re-authentication Protocol (ERP)

In cases where EAP authentication has completed successfully, it is often desirable to reduce latency if a subsequent authentication exchange is required (e.g., a mobile node moves from one access point to another). The EAP Re-authentication Protocol (ERP) [RFC5296] provides the ability to do this independent of any particular EAP method. EAP peers and servers that support ERP are called ER peers and servers, respectively. ERP uses a re-authentication root key (rRK) derived from a DSRK (or the EMSK, but [RFC5295] suggests avoiding this) along with a re-authentication integrity key (rIK) derived from the rRK used to prove knowledge of the rRK.

ERP operates in a single round-trip time, which is consistent with its goal of reducing re-authentication latency. ERP begins with a full conventional EAP exchange, assumed to be in the “home” domain. The MSK generated is distributed to the authenticator and peer as usual. However, the rIK and rRK values are also determined at this time and shared only between the peer and EAP server. These values can be used in the home domain, along with rMSKs generated for each authenticator. When the ER peer moves to a different domain, different values (DS-rIK and DS-rRK, which are DSUSRKs) are used. The domain of the ER server is contained in a TLV area in ERP messages, allowing peers to determine the domain of the server with which they are communicating. Details of the protocol are given in [RFC5296].

18.7.3 Protocol for Carrying Authentication for Network Access (PANA)

While combinations of EAP, 802.1X, and PPP have all been used to support authentication of the client (and network, in some cases), they are not entirely link-independent. EAP tends to be implemented for particular links, 802.1X applies to IEEE 802 networks, and PPP uses a point-to-point network model. To address this concern, the Protocol for Carrying Authentication for Network Access (PANA) has been defined in [RFC5191], [RFC5193], and [RFC6345] based on requirements set out in [RFC4058] and [RFC4016]. It acts as an EAP lower layer, meaning it acts as a “carrier” for EAP information. It uses UDP/IP (port 716) and is therefore applicable to more than a single type of link, and it is not limited to a point-to-point network model. In effect, PANA allows EAP authentication methods to be used on any link-layer technology for determining network access.

The PANA framework includes three main functional entities: the PANA Client (PaC), PANA Authentication Agent (PAA), and the PANA Relay Element (PRE). Normal usage also involves an Authentication Server (AS) and Enforcement Point (EP). The AS may be a conventional AAA server accessed using access protocols such as RADIUS or Diameter. The PAA is responsible for conveying authentication material from a PaC to the AS, and for configuration of the EP when network access is approved or revoked. Some of these entities may be colocated. The PaC and associated EAP peer are always colocated, as are the EAP authenticator and PAA. A PRE can be used to relay communications between a PaC and PAA when direct communication is not otherwise possible.

The PANA protocol consists of a set of request/response messages including an extensible set of attribute-value pairs managed by the IANA [IPANA]. The primary payloads are EAP messages, sent in UDP/IP datagrams as part of a PANA session. There are four phases in a PANA session: authentication/authorization, access, re-authentication, and termination. The re-authentication phase is really a portion of the access phase wherein the session lifetime is extended by re-executing EAP-based authentication. The termination phase is entered either explicitly or as the result of the session timing out (either because of lifetime exhaustion or failure of liveness detection). PANA sessions are identified by a 32-bit session identifier included in each PANA message.

PANA also provides a form of reliable transport protocol. Each message contains a 32-bit sequence number. The sender keeps track of the next sequence number to send, and receivers keep track of the next expected sequence number. Answers contain the same sequence number as the corresponding request. Initial sequence numbers are randomly selected by the sender of the message (i.e., PaC or PAA). PANA also implements time-based retransmission. PANA is a weak transport protocol—it operates in a stop-and-wait fashion, does not use an adaptive retransmission timer, and cannot perform repacketization. It does, however, perform exponential backoff on its retransmission timer when faced with multiple packet losses.

18.8 Layer 3 IP Security (IPsec)

IPsec is an architecture and collection of standards that provide data source authentication, integrity, confidentiality, and access control at the network layer for IPv4 and IPv6 [RFC4301], including Mobile IPv6 [RFC4877]. It also provides a way to exchange cryptographic keys between two communicating parties, a recommended set of cryptographic suites, and a method for signaling the use of compression. Each communicating party may be an individual host or a security gateway(SG) that provides a boundary between a protected and an unprotected portion of a network. Thus, IPsec can be used in applications such as remote access to a corporate LAN (forming a VPN), to interconnect different portions of an enterprise securely across the open Internet, or to secure the communications of hosts or routers acting as hosts when exchanging routing information. When choosing a security approach for newly developed protocols, IPsec is sometimes selected [RFC5406].

Figure 18-9 indicates the types of deployments that can be accomplished using IPsec. A host implementation of IPsec may be integrated within the IP stack itself or may act as a driver sitting “below” the rest of the network stack (called the “Bump in the Stack” or BITS implementation). Alternatively, it may reside inside an inline SG, which is sometimes called the “Bump in the Wire” or BITW implementation approach. For BITW implementations, both host and SG functionality is generally required, as the device typically needs to be managed remotely. This

is similar to the reasons we see applications and transport protocols implemented in routers that would otherwise be pure layer 3 devices (see Chapter 1). IPsec can support multicast communications, but we focus first on the simpler and more common unicast case.

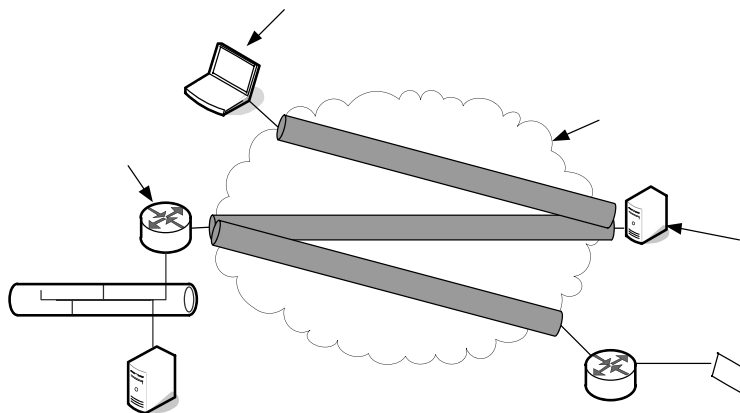


Figure 18-9 IPsec is applicable to securing host-to-host communications, host-to-gateway communications, and gateway-to-gateway communications. It also supports multicast distribution and mobility.

The operation of IPsec can be divided into the establishment phase, where key material is exchanged and a security association (SA) is built, followed by the data exchange phase, where different types of encapsulation schemes, called the Authentication Header (AH) and Encapsulating Security Payload (ESP), may be used in different modes such as tunnel mode or transport mode to protect the flow of IP datagrams. Each of these IPsec components uses a cryptographic suite, and IPsec is designed to support a wide range of suites. A complete IPsec implementation includes the SA establishment protocol, AH (optionally), ESP, and a collection of appropriate cryptographic suites, configuration information, and setup tools. An overview that summarizes the evolution and current specifications for all IPsec components is given in [RFC6071].

Although an IPsec implementation may be present in a system (it is required to be present for IPv6 implementations), IPsec operates only selectively on certain packets based on policies set by administrators. The policies are contained in a security policy database (SPD), logically resident with each IPsec implementation. IPsec also requires two additional databases called the security association database (SAD) and peer authorization database (PAD). These are consulted when determining how packets are to be handled, as illustrated in Figure 18-10.

Taking the (somewhat simplified) SG of Figure 18-10 as an example, particular fields of an arriving packet (traffic selector) are inspected to determine whether

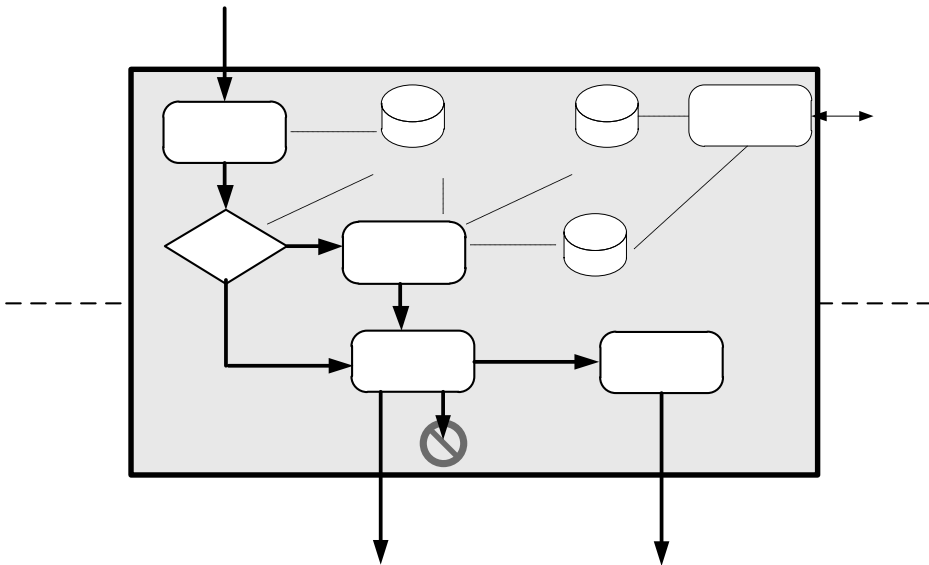


Figure 18-10 In a security gateway, IPsec packet processing takes place at layer 3 in a logical entity separating a protected and an unprotected network. The security policy database dictates the disposition of packets: bypass, discard, or protect. Protection generally involves applying or validating integrity protection or encryption. An administrator configures the SPD to achieve desired security goals.

the arriving packet is using IPsec and has a preexisting SA. If so, processing is relatively simple and usually involves applying either ESP or AH, as described in Sections 18.8.2 and 18.8.3. If not, the SPD is used to determine what type of SA should be established, if any, and the SAD is populated to contain information on the new SA. If a new SA needs to be established, the simplest way is using some automated key establishment protocol. Although IPsec mandates the support of manual keying, where keys are simply typed in by hand, this method does not scale well and is error-prone. Therefore, it is expected that normally a key establishment protocol is used in establishing SAs. For IPsec, the most recent version of this protocol is what we explore next.

18.8.1 Internet Key Exchange (IKEv2) Protocol

The first step in using IPsec is to establish an SA. An SA is a simplex (one-direction) authenticated association established between two communicating parties, or between a sender and multiple receivers if IPsec is supporting multicast. Most frequently, communication is bidirectional between two parties, so a pair of SAs is required to use IPsec effectively. A special protocol called the Internet Key Exchange

(IKE) is used to accomplish this task automatically. The current version of the protocol is called IKEv2 [RFC5996]. We will refer to it simply as IKE. Note that IKE is one of the more complicated pieces of IPsec, so once we understand it, the rest is comparatively straightforward. Note, however, that we will discuss only the major points of how IKE operates as a protocol. For particular details, such as the myriad cryptographic suites and configuration parameters supported, the reader should consult [RFC5996] directly.

To establish an SA, IKE begins with a simple request/response message pair that includes a request to establish the following parameters: an encryption algorithm, an integrity protection algorithm, a Diffie-Hellman group, and a PRF that gives a random-appearing output given any input bit string. In IKE, a PRF is used for generation of session keys. IKE first establishes an SA for itself (called an IKE_SA) and can subsequently establish SAs for either AH or ESP (called CHILD_SAs). IKE is also capable of negotiating the use of IP Payload Compression (IPComp) [RFC3173] with each CHILD_SA, because applying compression at other layers after performing encryption is ineffective. We discuss the details of AH and ESP in Sections 18.8.2 and 18.8.3.

IKE operates using pairs of messages called exchanges that are sent between an initiator and a responder. The first two exchanges, called IKE_SA_INIT and IKE_AUTH, establish an IKE_SA and a single CHILD_SA. Subsequently, CREATE_CHILD_SA exchanges, used to establish additional CHILD_SAs, and INFORMATIONAL exchanges, used to initiate changes in or gather status information about an SA, may occur. In most cases, a single IKE_SA_INIT and IKE_AUTH exchange (a total of four messages) is sufficient. Messages used in an exchange contain payloads identified by type numbers that identify the type of information carried in each payload. Multiple payloads per message are common, and some long messages may require IP fragmentation.

IKE messages are sent encapsulated in UDP using port number 500 or 4500. However, because IKE traffic may pass through a NAT where the port number is rewritten, an IKE receiver should be prepared to receive traffic originating from any port. Port 4500 is reserved for UDP-encapsulated ESP and IKE [RFC3948]. IKE messages appearing on port 4500 are required to have their initial 4 data bytes set to 0 (the “non-ESP marker”) to differentiate them from other (i.e., ESP or WESP) messages.

IKE initiators perform timer-based retransmissions when IKE messages appear to have been lost. Responders perform retransmissions only when triggered by an incoming request. An exponentially increasing retransmission timer is used for retransmissions, but the total number of retransmissions is left unspecified. Both initiators and responders keep track of their last transmitted messages and corresponding sequence numbers. Sequence numbers are used to match requests with responses, and to identify message retransmissions. This makes IKE a window-based protocol with a maximum window size given by a responder that is initialized when an SA is first set up but can be increased later. The maximum window size limits the total number of outstanding requests.

18.8.1.1 IKEv2 Message Formats

IKE messages contain a header followed by zero or more IKE payloads. The header structure is shown in Figure 18-11.

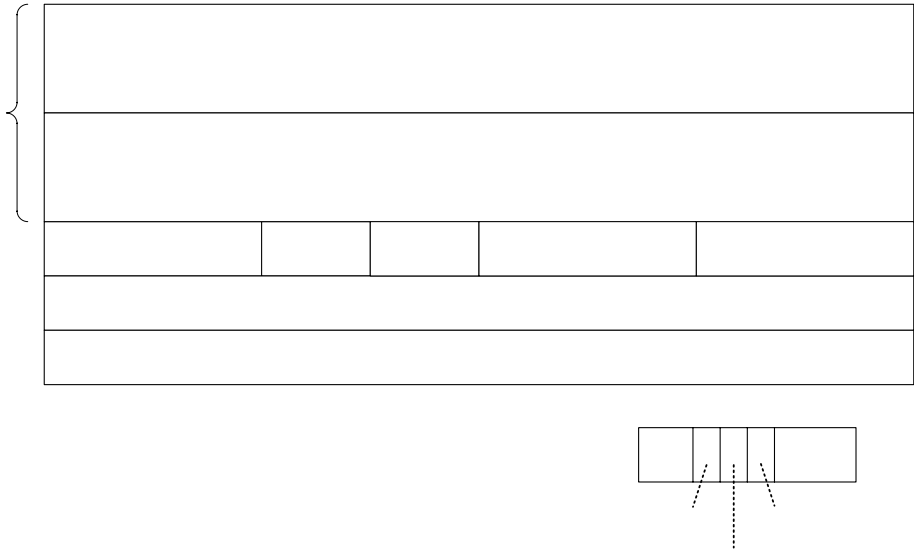


Figure 18-11 The IKE v2 header. All IKE messages contain a header followed by zero or more payloads. IKE uses 64-bit SPI values. The Exchange Type gives the purpose of the exchange and the payloads that may be expected in the message. The Flags field indicates whether the message was sent from an initiator or a responder. The Message ID associates requests with responses and is used for detecting replay attacks.

In the headers of IKE messages, as shown in Figure 18-11, the Security Parameter Index (SPI) is a 64-bit number that identifies a particular IKE_SA (other IPsec protocols use a 32-bit SPI value). Both the initiator and the responder have an SA for their peer, so each provides the SPI it is using, and this pair of values, combined with the IP addresses of the endpoints, can be used to form an effective connection identifier. The Next Payload field is discussed later in this section. The Major Version and Minor Version fields are set to 2 and 0, respectively, for this version of IKE. The major version number is changed when interoperability cannot be maintained between versions. The Exchange Type field gives the type of exchange of which the message is part: IKE_SA_INIT (34), IKE_AUTH (35), CREATE_CHILD_SA (36), INFORMATIONAL (37), and IKE_SESSION_RESUME (38; see [RFC5723]). Other values are reserved; the range 240–255 is reserved for private use. Three bit fields are defined for the Flags field (bits are labeled right

to left, starting from 0): I (Initiator, bit 3), V (Version, bit 4), and R (Response, bit 5). The I bit field is set by the original initiator and cleared by the recipient for return messages. The V bit field indicates that the sender supports a higher major version number of the protocol than is currently being used. The R bit field indicates that the message is a response to a previous message using the same message ID.

The Message ID field in IKE acts somewhat like the Sequence Number field in TCP (see Figure 12-3 in Chapter 12), except the message ID starts with 0 for the initiator and 1 for the responder. The field is incremented by 1 for each subsequent transmission, and responses use the same message ID as the requests. The I and R bit fields differentiate requests from responses. Message IDs are remembered when sent or received. Doing so allows each end to perform replay detection. Old message IDs are not processed. Wrapping of the Message ID field (possible, but not likely with 4 billion IKE messages) is handled by reinitiating the IKE_SA_INIT exchange.

The other fields (Next Payload and Length) help describe what the IKE message contains. Each message contains zero or more payloads, and each payload has its own particular structure. The Length field gives the size (in bytes) of the header plus all payloads in the message. The Next Payload field gives the type of the following payload. At present, 16 nontrivial types are defined (value 0 indicates no next payload), as shown in Table 18-2. The official current list can be found in [IKEPARAMS], which contains all standardized field values for IKEv2.

Table 18-2 IKEv2 payload types. A value of 0 indicates no next payload.

Value	Notation	Purpose	Value	Notation	Purpose
33	SA	Security association	41	N	Notify
34	KE	Key exchange	42	D	Delete
35	IDi	Identification (initiator)	43	V	Vendor ID
36	IDr	Identification (responder)	44	TSi	Traffic selector (initiator)
37	CERT	Certificate	45	TSr	Traffic selector (responder)
38	CERTREQ	Certificate request (indicates trust anchors)	46	SK { }	Encrypted and authenticated (contains other payloads)
39	AUTH	Authentication	47	CP	Configuration
40	Ni, Nr	Nonces (initiator, responder)	48	EAP	Extensible authentication (EAP)

The ranges 1–32 and 49–255 are reserved; the range 128–255 is reserved for private use. Each IKE payload begins with an IKE generic payload header shown in Figure 18-12.

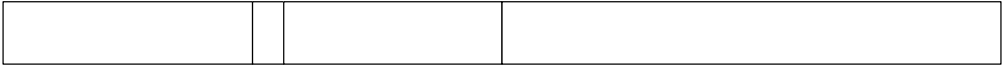


Figure 18-12 A “generic” IKEv2 payload header. Each payload begins with a header of this form.

The generic payload header is fixed at 32 bits, and the Next Payload and Payload Length fields provide for a “chain” of variable-size payloads (up to 65,535 bytes each, including the 4-byte payload header) to be present in a single IKE message. Each payload type has its own set of special headers. The C (critical) bit field indicates that the current payload (not the one identified by the Next Payload field) is deemed “critical” for a successful IKE exchange. Receivers of critical payloads that do not understand the type code (provided in the previous payload’s Next Payload field or in the IKE header’s Next Payload field) must abort the IKE exchange. Note that this capability provides the ability to create new payload types that may not be understood by all implementations.

18.8.1.2 The IKE_SA_INIT Exchange

To get a better idea of how IKE operates, we will start by describing the IKE_SA_INIT exchange. It is the first of two exchanges, IKE_SA_INIT and IKE_AUTH, constituting the “initial exchanges” of IKE shown in Figure 18-13. The initial exchanges were formerly known as Phase 1 in earlier versions of IKE. Other exchanges (CREATE_CHILD_SA and INFORMATIONAL) may be initiated by either party only after the initial exchanges have completed, and they are always secured (encrypted and integrity-protected) based on the parameters established using the first two exchanges.

As shown in Figure 18-13, IKE_SA_INIT negotiates the choice of cryptographic suite, exchanges nonces, and performs a DH key agreement. It may also include additional information, depending on the particular implementation and deployment scenario. It begins when the initiator sends an IKE message containing its set of supported cryptographic suites, DH information, and nonce using three payloads (SA, KE, and Ni). Details of each payload type are given in Section 3 of [RFC5996], and we discuss some of them in Section 18.8.1.3; note that in some implementations additional payloads are also included. A lack of response to this message triggers retransmissions at the initiator.

Upon receiving the first message, the responder becomes aware that an IKE transaction is requested by the initiator, the initiator’s supported cryptographic suites, and configuration parameters. The responder selects an acceptable cryptographic suite and expresses this in the SAR1 payload (see Section 18.8.1.3). It also provides its portion of the DH key agreement parameters in KER, its nonce in Nr, and an optional request for the initiator’s certificate in the CERTREQ payload. CERTREQ payloads include an indication of CAs the responder finds acceptable for validating certificates that may be used in subsequent exchanges (i.e., it indicates the responder’s trust anchors). A message containing the responder’s IKE

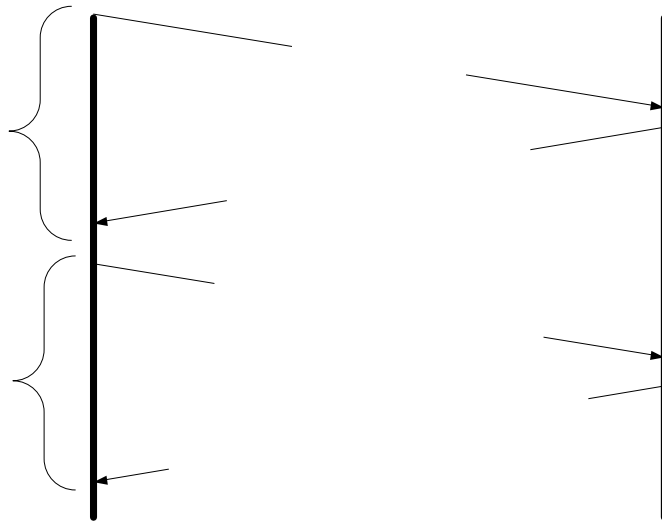


Figure 18-13 The IKE_SA_INIT and IKE_AUTH exchange involves payloads used to establish the first two security associations (IKE_SA and one CHILD_SA). Certificates and certificate request payloads (with trust anchors) may also be included, as may Notification and Configuration payloads (not shown).

header and all of these payloads is then sent in response to the initiator, completing the IKE_SA_INIT exchange. In some implementations, extra payloads (e.g., Notify and Configuration payloads; see Section 18.8.1.5) are also included. To better understand how IKE_SA_INIT operates, we shall begin by discussing its most important payloads: SA, KE, Ni, and Nr.

18.8.1.3 Security Association (SA) Payloads and Proposals

SA payloads contain an SPI value and a set of proposals (often one). Proposals are built using proposal structures that are somewhat complex. Each proposal structure is numbered and contains an IPsec protocol ID. A protocol ID indicates one of the following IPsec protocols: IKE, AH, or ESP (see Sections 18.8.2 and 18.8.3). Multiple proposal structures using the same proposal number are considered to be part of the same proposal (an “AND” of the specified protocols). Proposal structures with different proposal numbers are considered different proposals (an “OR” of the specified protocols).

Each proposal/protocol structure contains one or more transform structures that describe algorithms to be used with the specified protocols. Typically, AH has a single transform (integrity check algorithm), ESP has two (integrity check and encryption algorithms), and IKE has four (DH group number, PRF, integrity

check, and encryption algorithms). Combined encryption/integrity algorithms (e.g., authenticated encryption algorithms) are expressed solely as encryption algorithms with no separate integrity protection specification. A special extended sequence number “transform,” which is really just a Boolean value, indicates whether sequence numbers used with the SA (i.e., for AH or ESP) should be computed using 32 or 64 bits.

If there are multiple transforms of the same type, the proposal is the union of the transforms (i.e., any are acceptable). If there are multiple transforms with different types, the proposal is the intersection. An individual transform may have zero or more attributes. These are necessary when a transform can be used in more than one way (e.g., a transform capable of processing keys of differing lengths would have an associated attribute with the particular key length to be used for the proposal). Most transforms do not require attributes, but the relatively common AES encryption transform does.

18.8.1.4 Key Exchange (KE) and Nonce (Ni, Nr) Payloads

In addition to SA payloads, IKE_SA_INIT messages include a KE (Key Exchange) and Nonce payload (written as Ni, Nr, or sometimes No). The KE payload contains the DH group number and key exchange data representing the public numbers used in forming an ephemeral Diffie-Hellman key (initial shared secret). The DH group number gives the group in which the public value was computed. The Nonce payload contains a recently generated nonce between 16 and 256 bytes in length. It is used in generating key material to ensure freshness and protect against replay attacks.

Once the DH exchange completes, each side can compute its SKEYSEED value, which is used for all subsequent key generation associated with the IKE_SA (unless a key-generating EAP method is used for this purpose; see Section 18.8.1.9), a total of seven secret values: SK_d, SK_ai, SK_ar, SK_ei, SK_er, SK_pi, and SK_pr. These values are computed as follows:

$$\begin{aligned} \text{SKEYSEED} &= \text{prf}(\text{Ni} \parallel \text{Nr}, g^{\text{ir}}) \\ \{\text{SK}_d \parallel \text{SK}_{ai} \parallel \text{SK}_{ar} \parallel \text{SK}_{ei} \parallel \text{SK}_{er} \parallel \text{SK}_{pi} \parallel \text{SK}_{pr}\} &= \\ &\text{prf}^+ (\text{SKEYSEED}, \text{Ni} \parallel \text{Nr} \parallel \text{SPI} \parallel \text{SPIr}) \end{aligned}$$

Here, $\parallel$ is the concatenation operator. The cascading PRF function $\text{prf}^+(K, S) = T1 \parallel T2 \parallel \dots$, where $T1 = \text{prf}(K, S \parallel 0x01)$, $T2 = \text{prf}(K, T1 \parallel S \parallel 0x03)$, $T3 = \text{prf}(K, T2 \parallel S \parallel 0x05)$, $T4 = \text{prf}(K, T3 \parallel S \parallel 0x07)$, The value g^{ir} is the shared secret established during the DH exchange. Ni and Nr are nonces (stripped of any payload headers). Note that each direction of each SA uses different keys, which explains why so many keys are required. The SK_d key is used for deriving keys for CHILD_SAs. The SK_a and SK_e keys are for authentication and encryption, respectively. The SK_p keys are used in generating AUTH payloads during the IKE_AUTH exchange.

18.8.1.5 Notification (N) and Configuration (CP) Payloads

The N payload is a Notification or Notify payload. Although this type of payload is not shown in Figure 18-13, we shall see it used in the examples later. It can be used for conveying error messages and indications of various processing capabilities with most of the IKE exchange types. It contains a variable-length SPI field and a 16-bit field to indicate the notification type [IKEPARAMS]. Values below 8192 are used for standard errors, and values above 16383 are used for status indicators. For example, when requesting the creation of a transport mode SA instead of the default tunnel mode, a Notify payload containing the `USE_TRANSPORT_MODE` value (16391) is used. If IP compression [RFC3173] is supported, this fact can be indicated by the `IPCOMP_SUPPORTED` value (16387). If Robust Header Compression (ROHC) [RFC5857] is supported, this can be indicated using the `ROHC_SUPPORTED` value (16416), which also includes ROHC parameters used to establish a so-called ROHCIPsec SA. A desire to use the “wrapped ESP” mode (see Section 18.8.3.2) is indicated using the `USE_WESP_MODE` value (16415). Notify payloads may contain a variable-length data portion whose content depends on the notification type.

A CP or Configuration payload also contains additional information like a Notify payload but is used primarily for initial system configuration. For example, obtaining information that might ordinarily be conveyed using DHCP (see Chapter 6) can be carried over IKE using a CP. Configuration payloads are of the following major types: `CFG_REQUEST`, `CFG_REPLY`, `CFG_SET`, and `CFG_ACK`. CPs use attribute-value (ATV) pairs that contain a variable-length associated data area. Some 20 ATV pairs are defined [IKEPARAMS]. Most involve methods to learn about IPv4 or IPv6 addresses, subnet masks, or DNS server addresses. IPv6 configuration requires special attention because of the way IPv6 ordinarily employs ICMPv6 for stateless autoconfiguration and Neighbor Discovery (see Chapter 8). An experimental specification [RFC5739] explores how IKEv2 can be used in configuring an IPv6 node across an IPsec association in a VPN configuration.

18.8.1.6 Algorithm Selection and Application

IKE divides the set of transforms forming a cryptographic suite into four types: encryption (type 1, used with IKE and ESP), PRF (type 2, used with IKE), integrity protection (type 3, used with IKE and AH and optional in ESP), and DH group (type 4, used with IKE and optional in AH and ESP). Although IKE is capable of negotiating which particular cryptographic suite is to be used for each direction of an SA, support for a baseline set of algorithms (transforms) is deemed mandatory for any implementation. In addition, several algorithms have been chosen as recommended, with the strong possibility that they will be mandatory in the future. These algorithms are provided in [RFC4307] (see Table 18-3).

The IANA also keeps an official registry of values [IKEPARAMS], and although the list here includes the mandatory algorithms at the time of writing, many other algorithms, groups, and techniques have been proposed and published, including options for ECC-based digital signatures (see [RFC4754]).

Table 18-3 Mandatory-to-implement algorithms for use with IKEv2, grouped by type number

Purpose	Name	Number	Status	Original Defining RFC/ Reference
IKE Transform Type 1 (encryption)	ENCR_3DES	3	Required	[RFC2451]
	ENCR_NULL	11	Optional	[RFC2410]
	ENCR_AES_CBC	12	Recommended	[RFC3602]
	ENCR_AES_CTR	13	Recommended	[RFC3686]
IKE Transform Type 2 (for PRFs)	PRF_HMAC_MD5	1	Optional	[RFC2104]
	PRF_HMAC_SHA1	2	Required	[RFC2104]
	PRF_AES128_CBC	4	Recommended	[RFC4434]
IKE Transform Type 3 (integrity)	AUTH_HMAC_MD5_96	1	Optional	[RFC2403]
	AUTH_HMAC_SHA1_96	2	Required	[RFC2404]
	AUTH_AES_XCBC_96	5	Recommended	[RFC3566]
IKE Transform Type 4 (DH groups)	1024 MODP (Group 2)	2	Required	[RFC2409]
	2048 MODP (Group 14)	14	Recommended	[RFC3526]

18.8.1.7 The IKE_AUTH Exchange

As mentioned earlier, the SKEYSEED value is used to derive encryption and authentication keys that are in turn used to secure payloads during the IKE_AUTH exchange. These keys are called SK_e and SK_a respectively. The notation SK{P1, P2, ..., PN} indicates that payloads P1, ..., PN are encrypted and integrity-protected using these keys. The primary purpose of the IKE_AUTH exchange is to provide identity validation for each peer. It also exchanges sufficient information to establish the first CHILD_SA.

To begin the IKE_AUTH exchange, the initiator sends the payload SK{IDi, AUTH, SAI2, TSi, TSr}. Given the proper decryption key, it provides the initiator's identity, authentication information validating the initiator's identity, another SA payload for the first CHILD_SA called SAI2, and a pair of traffic selectors (payloads TSi and TSr, discussed in Section 18.8.1.8). The initiator may also include its certificate in a CERT payload, a certificate request in a CERTREQ payload that identifies its trust anchors, and identification of the responder in the IDr payload. Sending the responder's identity is useful in the case where the responder has multiple identities associated with the same IP address and needs to ensure that the proper SA is set up. Several different identity types are supported for ID payloads, including IP address, FQDN, e-mail address, and distinguished name (to be used with X.509 certificates). The various types are maintained in the IKEv2 Identification Payload ID Types registry [IKEPARAMS].

The final message of the exchange includes the responder's identity (IDr), authentication material to prove the responder's identity (AUTH), the other SA

constituting the CHILD_SA (SAr2), and a set of traffic selectors (TSi and TSr), which may be subsets of the original TSi and TSr values. All payloads in the IKE_AUTH exchange are encrypted and integrity-protected. A certificate payload (CERT) containing one or more certificates may also be sent at this point. If so, any public key required to validate the AUTH payload appears first in the certificate list. The specific contents vary depending on the cryptographic suite selected. During the exchanges, both sides must check all applicable signatures in order to be safe from compromise, including MITM attacks.

18.8.1.8 Traffic Selectors and TS Payloads

Traffic selectors indicate the fields and corresponding values of an IP datagram that cause it to be “selected” for IPsec processing. They are used in combination with an IPsec SPD to determine whether the containing datagram should be protected using IPsec. As mentioned previously, datagrams that are not protected are either bypassed or dropped by IPsec processing.

The contents of a TS payload may include IPv4 or IPv6 address ranges, port number ranges, and an IPv4 protocol ID or IPv6 header value. Ranges are sometimes denoted with wildcard notation. For example, the notation 192.0.2.* or 192.0.2.0/24 would represent the range 192.0.2.0–192.0.2.255. Traffic selectors can be used to help implement policies such as which cryptographic suite is required to establish an SA to a particular host or port range. Most of these details are handled in the management interface to the SPD. During an IKE_AUTH exchange, each party specifies a TSi and TSr payload containing TS values. When one range is smaller than another, the smaller range is selected for use in a process called “narrowing.”

18.8.1.9 EAP and IKE

Although IKE includes its own authentication methods (see Section 2.15 of [RFC5996]), it can also make use of EAP (see Sections 2.16 and 3.16 of [RFC5996]). With EAP, a wide array of authentication methods can be used beyond the relatively limited set of pre-shared keys or public key certificates otherwise required by IKE. Indeed, these limited sets of options for keying are one reason for the relatively limited success of IPsec more generally.

A desire to use EAP is indicated by omitting the first AUTH payload from the IKE_AUTH exchange in message 3 (Figure 18-1). By including the IDi payload but no AUTH payload, the initiator asserts an identity but does not prove it. If EAP is acceptable, the responder returns an EAP payload and defers sending the SAr2, TSi, and TSr payloads until the EAP-based authentication is complete. This happens once the initiator has finally sent an EAP-acceptable AUTH payload that can be verified by the responder after one or more EAP payloads have been exchanged.

One issue regarding EAP with IKE involves a possible inefficiency due to double authentication. In particular, older EAP methods provided only one direction of authentication (peer to authenticator), so IKE requires certificate-based

authentication to perform authentication in the other direction. Recognizing that deploying the necessary key infrastructure is sometimes difficult, and that newer EAP methods support mutual authentication and key derivation, [RFC5998] provides a way to use only EAP for authentication. Using an EAP_ONLY_AUTHENTICATION Notification payload sent by the initiator, the responder is able to suppress sending the AUTH and CERT payloads carried in message 4 (in Figure 18-1). In this case, subsequent AUTH payloads use the key generated by EAP instead of SK_pi and SK_pr.

Performing EAP-only authentication relies on EAP methods that are sufficiently secure so as to obviate the need for IKE authentication. These are called safeEAP methods. To be safe, an EAP method must provide mutual authentication, be capable of generating keys, and be resistant to dictionary attacks. Some 13 methods are given in [RFC5998], including EAP-TLS, EAP-FAST, and EAP-TTLS, that are believed to be safe.

18.8.1.10 Better-than-Nothing Security (BTNS)

A relatively recent development with IKE and IPsec is called better-than-nothing security (BTNS, pronounced “buttons”). BTNS aims to address some of the usability and ease of deployment issues with IPsec, especially the need to establish a PKI or other deployed authentication system [RFC5387] to use certificates. Technically, BTNS is essentially unauthenticated IPsec [RFC5386], and it can be supported when IKE is used to establish an SA. With BTNS, public keys are used, but their containing certificates are not checked against a chain or root certificate. Consequently, an SA can ensure that the same entity is communicating over time but cannot ensure that any particular, validated entity established the SA. This form of authentication is called continuity of association and is weaker than the data origin authentication present in ordinary IPsec. BTNS makes no other substantive changes to IPsec; the formats of IKE, AH, and ESP messages remain the same.

18.8.1.11 The CREATE_CHILD_SA Exchange

The CREATE_CHILD_SA exchange is used to create CHILD_SAs for ESP or AH, or to rekey existing SAs (either IKE_SAs or CHILD_SAs) once the initial exchanges have completed. It uses a single exchange of packets and may be initiated by either side of the IKE_SA established during the initial exchanges. There are two variants, depending on whether a CHILD_SA or IKE_SA is being modified. Figure 18-14 shows the variants, where the initiator is the entity initiating the CREATE_CHILD_SA exchange and not necessarily the original initiator of the IKE_SA.

In Figure 18-14, the first exchange depicts a CREATE_CHILD_SA used to create a new CHILD_SA or rekey an existing one. Rekeying is indicated by the presence of an N(REKEY_SA) Notification payload sent by the initiator. To complete the rekey operation, a new SA is first created, and the old one is subsequently deleted (see the next section). The new SA and traffic selector (TS) information allows most of the connection parameters to be altered. If desired, new DH values can also be exchanged at this point using KE payloads. This provides better

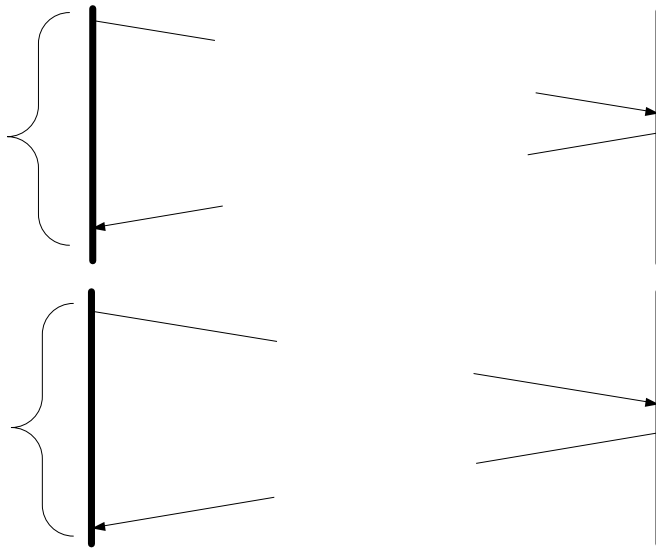


Figure 18-14 The CREATE_CHILD_SA exchange can be used to create or rekey a CHILD_SA, or to rekey an IKE_SA. A Notification payload is used when modifying a CHILD_SA to indicate the SPI of the SA to modify.

forward secrecy for the new SA. Rekeying an IKE_SA uses a similar exchange, except the KE payloads are required and the TS payloads are not used, as shown in the second part of Figure 18-14.

18.8.1.12 The INFORMATIONAL Exchange

The INFORMATIONAL exchange is used for conveying status and error information, usually using Notify (N) payloads. It is also used for deleting SAs using a Delete (D) payload and therefore constitutes one portion of the SA rekeying procedure. The exchange is shown in Figure 18-15.

An INFORMATIONAL exchange can take place only after successful completion of the initial exchanges. It includes an optional set of notifications, Delete (D) payloads that specify SAs to delete by SPI value, and Configuration (CP) payloads. Some response is always required for any message received from an initiator, even if it is an empty IKE message (i.e., contains only a header). Otherwise, the initiator would retransmit its message unnecessarily. In unusual cases, INFORMATIONAL messages may be sent outside the context of an INFORMATIONAL exchange, usually to signal the receipt of an IPsec message containing an unrecognized SPI value or unsupported IKE major version number.

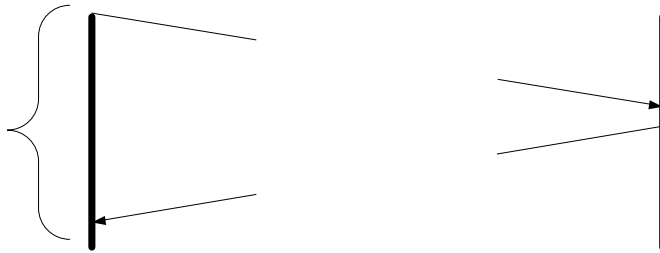


Figure 18-15 The INFORMATIONAL exchange is used to convey status information and delete SAs. It makes use of Notification (N), Delete (D), and Configuration (CP) payloads.

18.8.1.13 Mobile IKE (MOBIKE)

Once the IKE_SA has been established, it is ordinarily used until no longer required. However, when IPsec operates in an environment where IP addresses may change because of mobility or interface failure, a variant of IKE has been specified in [RFC4555] called MOBIKE. MOBIKE augments the basic IKEv2 protocol to include additional “address change” options available in INFORMATIONAL exchanges. MOBIKE specifies what to do when the changed addresses are known. It does not address the discovery problem of how to determine these addresses.

18.8.2 Authentication Header (AH)

Defined in [RFC4302], the IP Authentication Header (AH), one of the three major components of IPsec, is an optional portion of the IPsec protocol suite that provides a method for achieving origin authentication and integrity (but not confidentiality) of IP datagrams. By providing only integrity and not confidentiality (and not working with NAT; see the remainder of this section), AH is the (far) less popular of the two primary IPsec data-securing protocols. In transport mode, AH uses a header placed between the layer 3 (IPv4, IPv6 base, or IPv6 extension) header and the following protocol header (e.g., UDP, TCP, ICMP). With IPv6, AH may appear immediately before a Destination Options extension header, if present. In tunnel mode, the “inner” IP header carries the original IP datagram, containing the ultimate IP source and destination information, and a newly created “outer” IP header contains information describing the IPsec peers. In this mode, AH protects the entire inner IP datagram. Generally speaking, transport mode is used between end hosts that are directly connected, and tunnel mode is used between SGs or between a single host and an SG (e.g., for supporting a VPN). The IPv4 and IPv6 encapsulations for transport-mode AH, using TCP as an example, are shown in Figure 18-16.

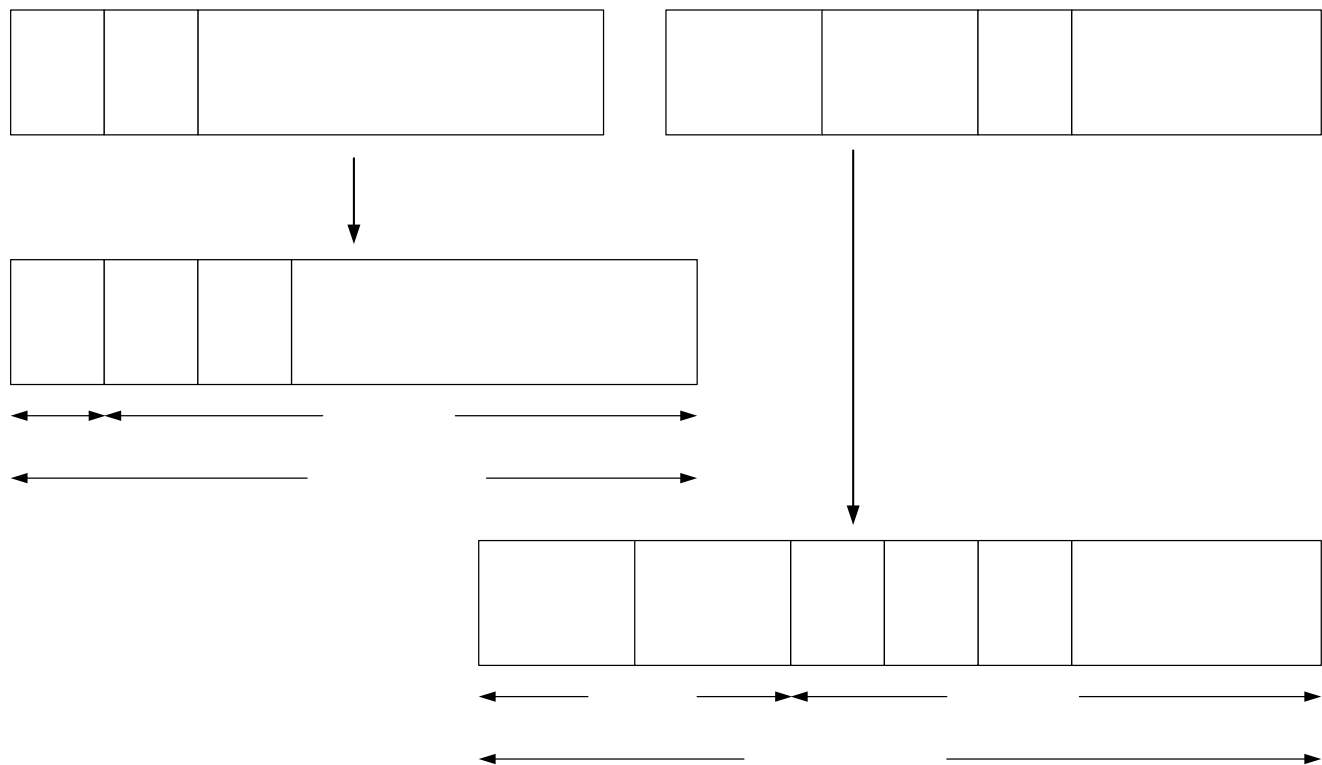


Figure 18-16 The IPsec Authentication Header is used to provide authentication and integrity protection for IPv4 and IPv6 datagrams. In transport mode (depicted here with TCP), a conventional IP datagram is modified to include the AH.

In the figure, the IPv4 encapsulation uses a special IPv4 protocol number (51). For IPv6, the AH is placed between the destination and other options. In either case, the resulting datagram has a mutable portion of its header and an immutable portion of its header. The mutable portion is changed as the datagram moves through the network. Modifications include changing the IPv4 TTL or IPv6 Hop Limit field, IPv6 Flow Label field, DS Field and ECN bits. The immutable portion, containing the source and destination IP addresses, is not changed by the network and is integrity-protected using fields in the AH. This prevents transport mode AH datagrams from being rewritten by NATs, a potential problem for many deployments. Transport mode cannot be used with fragments (IPv4 or IPv6).

An alternative to transport mode is AH tunnel mode, shown in Figure 18-17. In this mode, the original datagram is untouched and instead is inserted inside an integrity-protecting new IP datagram.

In tunnel mode, the entire original IP datagram is encapsulated and protected with the AH. The “inner” header is unmodified, and the “outer” header is created using the source and destination IP addresses associated with an SG or host. In such cases, AH protects all of the original datagram, plus some portions of the new header (which prevents it being modified by a NAT).

Both modes of AH use the same AH shown in Figure 18-18. It identifies the datagram length and associated SA and includes integrity check information. The Payload Length specifies the length of the AH in 32-bit-word units minus 2. The Security Parameters Index (SPI) field contains a 32-bit identifier of an SA at the receiver that contains SA-derived information relating to the association. For multicast SAs, the SPI value is handled in a special way (see Section 18.8.4). The Sequence Number is a 32-bit field that increments by 1 for each packet sent on the SA. This field is used for replay protection if enabled by the receiver (but it is always included by the sender, even if not checked by the receiver). An extended sequence number (ESN) operating mode is also defined and recommended and is negotiated during the IKE_SA_INIT exchange. If enabled, the sequence number is calculated using 64 bits, but only the lower-order 32 bits are included in the Sequence Number field. The length of the Integrity Check Value (ICV) field is variable and depends on the cryptographic suite used. This field is always an integral multiple of 32 bits in length.

The algorithm used for integrity protection is specified in the corresponding SA as a type 3 transform and can be established manually or by using some automatic method such as IKE. The optional, recommended, and mandatory algorithms for AH (and ESP, later) are provided in [RFC4835] and include HMAC-MD5-96 (optional), AES-XCBC-MAC-96 (recommended), and HMAC-SHA1-96 (mandatory). The integrity check is computed over the following portions of the datagram: header fields before the AH that are either immutable in transit or predictable in value when arriving at the destination AH SA endpoint, the AH, everything after the AH, high-order bits of the ESN (if employed, even though they are not sent), plus any padding.

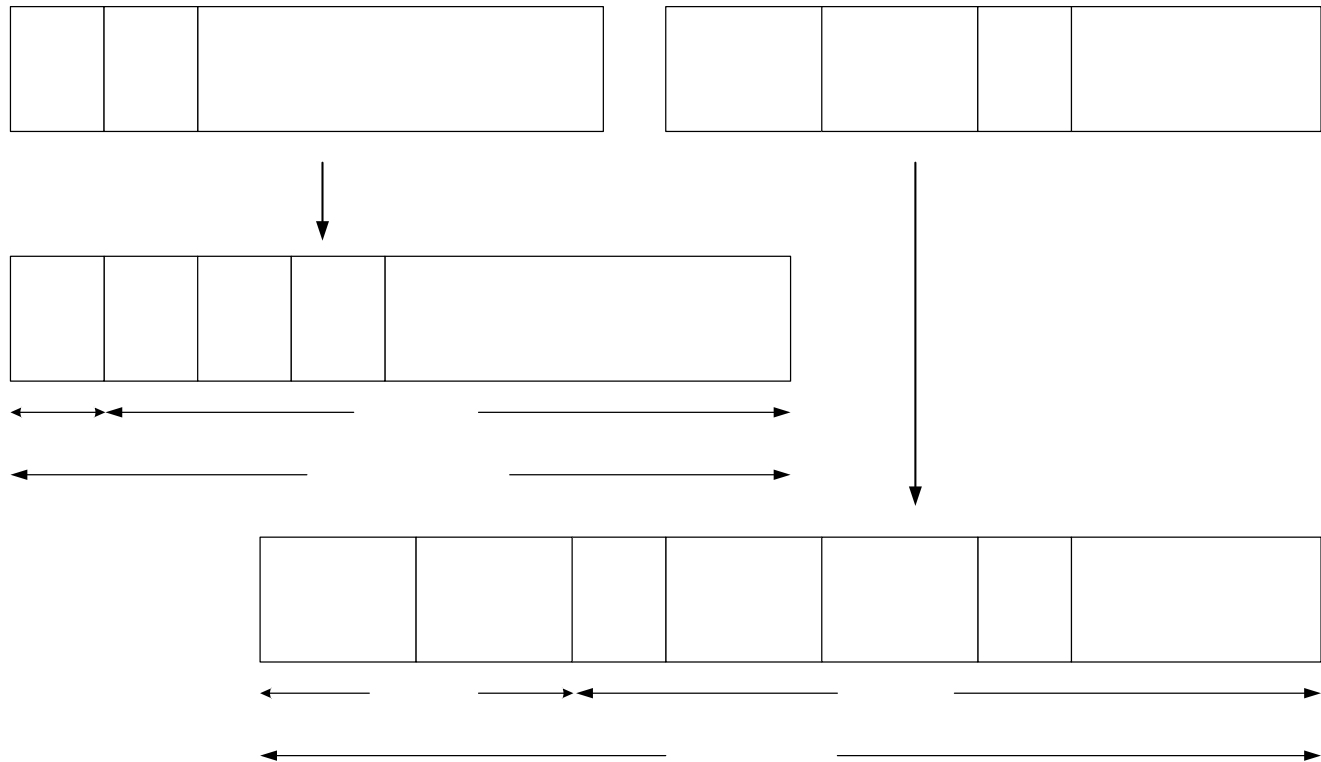


Figure 18-17 The IPsec tunnel mode AH encapsulations provide authentication and integrity protection for IPv4 and IPv6 datagrams. In tunnel mode (depicted here carrying TCP), a conventional IP datagram is encapsulated inside a new “outside” IP datagram that carries the original datagram.

Figure 18-18 The IPsec AH is used to provide authentication and integrity protection for IPv4 and IPv6 datagrams in either transport or tunnel mode. The SPI value indicates which SA the AH belongs to. The Sequence Number field is used for countering replay attacks. The ICV provides a form of MAC over the immutable portions of the payload.

Some controversy has arisen over the disposition of mutable fields such as the ECN bits used to signal incipient congestion (see Chapters 5 and 16) when tunnel modes are used. In [RFC4301], such fields are simply copied to the corresponding fields present in the newly created “outer” IP header. In [RFC6040], however, normal mode and compatibility mode for tunnel encapsulation are defined. In normal mode, the CE and ECT bit fields are copied to the new header on encapsulation. In compatibility mode, the bits are cleared, producing an “outer” packet indicating a non-ECN-capable transport. During decapsulation, if the outer or inner header contains a CE indication, the indication is copied to the packet produced after decapsulation unless the original packet did not indicate ECT (in which case the packet is dropped). In addition, if ECT is indicated by either the outer or inner headers, ECT is set to true in the decapsulated packet.

18.8.3 Encapsulating Security Payload (ESP)

The ESP protocol of IPsec, defined in [RFC4303] (where it is called ESP (v3) even though ESP provides no formal version numbers), provides a selectable combination of confidentiality, integrity, origin authentication, and anti-replay protection for IP datagrams. It can employ a NULL encryption method [RFC2410], which is mandatory to support, if only integrity is to be used. Conversely, encryption can be used for confidentiality without integrity protection, although this combination is effective only against passive attacks and is highly discouraged. In the context of ESP, integrity includes data origin authentication. Given its flexibility and feature set, ESP is (far) more popular than AH.

18.8.3.1 Transport and Tunnel Modes

Like AH, ESP has transport and tunnel modes. In tunnel mode, an “outer” IP packet includes an “inner” IP packet that may be entirely encrypted. This provides for a limited form of traffic flow confidentiality(TFC) because the “inner”

datagram's size and contents can be hidden using encryption. ESP may be used in combination with AH, if desired, and supports both IPv4 and IPv6. Using ESP in "integrity-only" mode may be preferable to AH in some cases for performance reasons (ESP may be more amenable to pipelining) and is a required configuration option for IPsec implementations. The encapsulations for ESP transport mode are shown in Figure 18-19.

The transport mode structure is similar to AH transport mode, except ESP trailer structures are used in support of ESP's encryption and integrity protection methods (see Section 18.8.3). As with AH, ESP transport mode cannot be used with fragments. The tunnel mode encapsulations for ESP, similar to those for AH, are shown in Figure 18-20.

ESP does not use a strict header in the same way AH does. Instead, there is an overall ESP structure that includes a header and trailer portion. There is an optional (second) trailer structure if ESP is used with an integrity protection mechanism that requires space for additional check bits (labeled ESP ICV). The ESP structure is shown in Figure 18-21.

ESP-encapsulated IP datagrams use the value 50 in the Protocol (IPv4) or Next Header (IPv6) header fields. The ESP payload structure, shown in Figure 18-21, includes the SPI and sequence numbers, used in the same way as with AH. The primary difference is in the payload area. This area may be confidentiality-protected (encrypted) and can include a variable-length pad portion required by some encryption algorithms.

The payload is required to end on a 32-bit boundary (64 for IPv6) and have the last two 8-bit fields identify the Pad Length and Next Header (Protocol) field values. The Pad Length and Next Header fields constitute the ESP trailer shown in Figures 18-19 and 18-20. Certain cryptographic algorithms may employ an IV. If present, the IV appears at the beginning of the payload area (not shown). Additional padding for TFC purposes (called TFC padding) is permitted to appear within the payload area in front of the ESP trailer (see Figure 2 of [RFC4303] for details). It is used to disguise the length of the datagram to help resist traffic analysis attacks, although this feature does not appear to be widely used. The Next Header field contains values chosen from the same space used in the IPv4 Protocol field or IPv6 Next Header field (e.g., 4 for IPv4, 41 for IPv6). It may contain the value 59, indicating "no next header," when carrying a dummy packet that is to be discarded. Dummy packets are another method sometimes used for resisting traffic analysis attacks.

The ESP ICV is a variable-length trailer used if integrity support is enabled and required by the integrity-checking algorithm. It is computed over the ESP header, payload, and ESP trailer. Implicit values (e.g., high-order ESN bits) are also included. The length of the ICV is known as a consequence of selecting the particular integrity-checking method. It is therefore established at the time the corresponding SA is set up and not changed as long as the SA exists.

Anti-replay is supported provided integrity protection is enabled. This is accomplished using a sequence number derived from a running counter. The

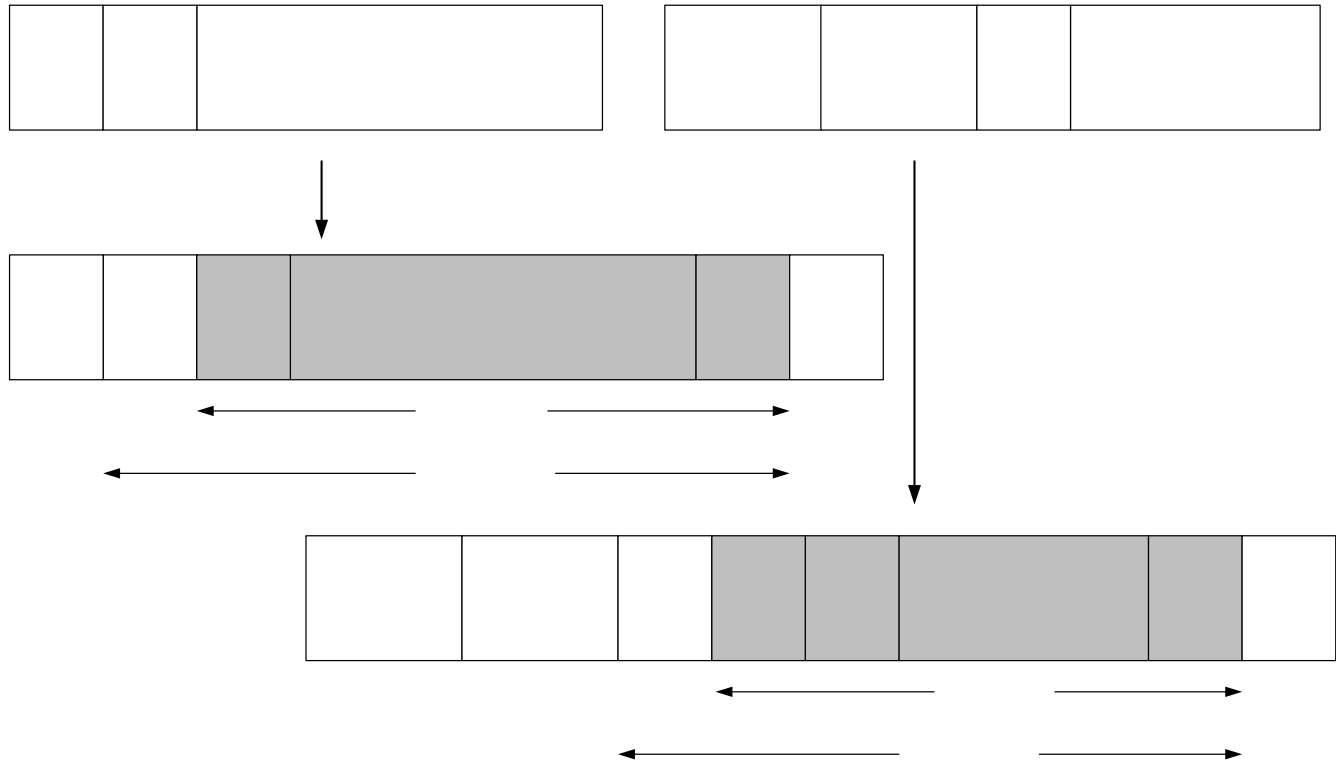


Figure 18-19 The IPsec ESP is used to provide confidentiality (encryption), authentication, and integrity protection for IPv4 and IPv6 datagrams. In transport mode (depicted here with TCP), a conventional IP datagram is modified to include the ESP header. ESP in transport mode allows the transport payload to be encrypted, authenticated, and integrity-protected.

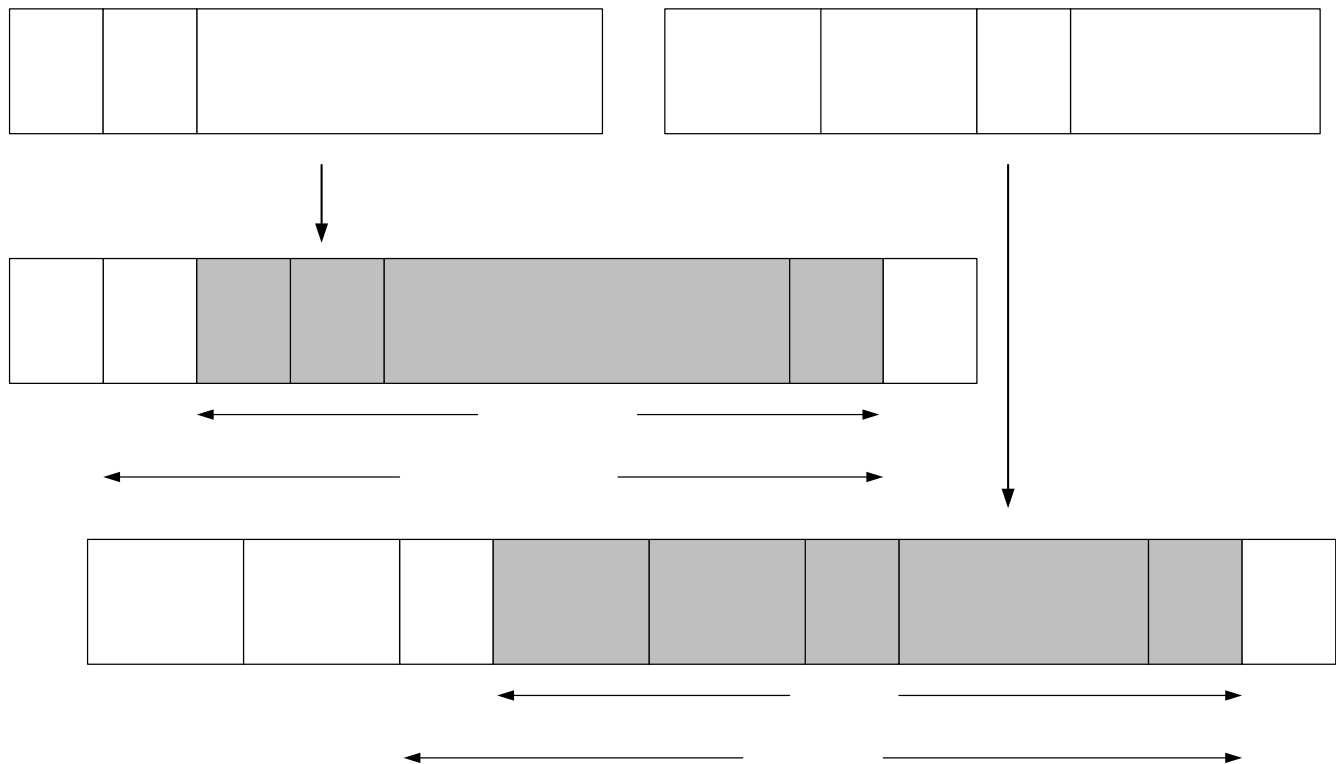


Figure 18-20 In tunnel mode (depicted here with TCP), ESP encapsulates a conventional IP datagram inside a new “outside” IP datagram that carries the original datagram. ESP allows the outer datagram to be modified (e.g., for NAT traversal) while the inner datagram remains intact. ESP is more popular than AH for most applications.

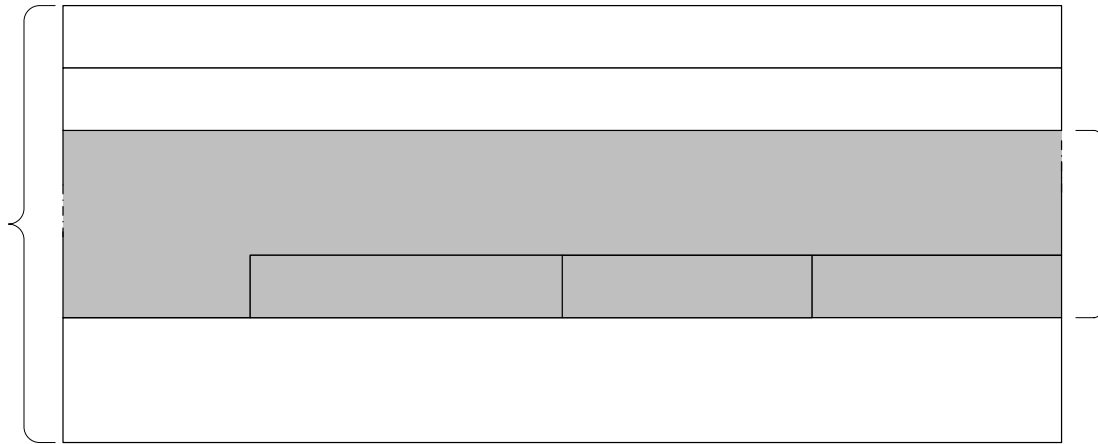


Figure 18-21 The ESP message structure includes the encrypted payload in the middle. The SPI and Sequence Number constitute the ESP header, and the combination of the Pad, Pad Length, and Next Header fields constitutes the ESP trailer. An optional ESP ICV trailer is also used when integrity protection is employed.

counter is initialized to 0 when an SA is first set up and incremented before being copied into each datagram sent on the SA. When anti-replay is enabled (the normal default), the sender checks to see that the counter has not wrapped and creates a new SA if wrapping is about to occur. The receiver implementing anti-replay keeps a valid window of sequence numbers (similar in some ways to the TCP receiver's window). Datagrams containing out-of-window sequence numbers are dropped.

For systems that implement auditing, ESP processing can result in one or more auditable events. These events include the following: no valid SA exists for a session, the datagram given to ESP for processing is a fragment, the anti-replay counter is about to wrap, a received packet was out of the valid anti-replay window, the integrity check failed. Auditable events are recorded in a logging system. These events include metadata such as the SPI value, current date and time, source and destination IP addresses, sequence number, and IPv6 flow ID (if present).

18.8.3.2 ESP-NUL, Wrapped ESP (WESP), and Traffic Visibility

As mentioned previously, ESP ordinarily provides privacy using encryption, but it can also operate in an integrity-only mode using the NULL encryption algorithm. Integrity-only mode (also called ESP-NUL) may be desirable in some circumstances, especially in enterprise environments where sophisticated packet inspection takes place within the network and confidentiality may be addressed in other ways. For example, some network infrastructure devices inspect packets for unwanted content (e.g., malware signatures) and are capable of providing alerts or shutting down network access when policy is violated. Such devices are essentially disabled if ESP is used with encryption in an end-to-end fashion (i.e., the way it was designed). Said another way, unless they have traffic visibility, they cannot do their jobs.

When a packet inspection device is faced with ESP traffic, it needs to make a decision about whether the traffic is encrypted (i.e., whether NULL encryption is being used or not). Given that the negotiation of an IPsec cryptographic suite is handled outside ESP (e.g., manually or using a protocol such as IKE), there are two current methods for doing so. The first is simply to use a set of nonstandard heuristics to make a guess [RFC5879]. Use of these has the benefit of not requiring any modification to ESP for supporting traffic visibility. The other method is to add a special description to ESP to indicate whether encryption is used. Wrapped ESP (WESP) [RFC5840], a standards-track RFC, defines a header that is placed ahead of the ESP packet structure. WESP uses a different protocol number (141) from ESP and can be negotiated with IKE using the USE_WESP_MODE (value 16415) Notify payload. The variable-length WESP header includes fields to indicate the location of payload information, along with a Flags field (maintained by the IANA [IWESP]) containing a bit indicating whether ESP-NUL is being used. Although WESP makes the job of determining whether ESP-NUL is being used or not easier for network infrastructure, its utility also depends on end hosts using the WESP header appropriately. Given that WESP is relatively new, this is not yet

the case today. On the other hand, the WESP format is extensible, so once implemented it could be adapted for other purposes in the future.

18.8.4 Multicast

IPsec optionally supports multicast operations [RFC5374], although this capability is not often used. The most basic form involves using manual key configuration, but there are also multicast group key establishment methods called group key management (GKM) protocols managed by group controller/key server (GCKSs). These are used to produce group security associations (GSAs), which include one or more IPsec SAs plus one or more GKM SAs used to provide parameters for establishing the IPsec SAs [RFC3740]. Given that members may dynamically join or leave a group, GKM protocols must deal with rekeying more frequently and carefully than regular two-party key establishment protocols, and such protocols have been a favorite topic for security researchers [AKNT04]. We shall not explore the details of how GKMs operate (such an explanation would be lengthy), but the interested reader may consult documentation for GDOI [RFC3547] or GSAKMP [RFC4535].

At present, multicast IPsec operation requires all members of a group to be homogeneous in their algorithmic and protocol processing capabilities. Both any-source and single-source multicast (ASM and SSM) operations are supported (see Chapter 9), and the same procedures are used for IPv4 local broadcast addresses and for IPv6 anycast addresses. Host IPsec implementations may use any combination of tunnel and transport mode, but SGs must use tunnel mode where the tunnel destination addresses are multicast addresses.

Multicast IP datagrams present a challenge for IPsec when a tunnel mode is used because the outer IP datagram's addressing needs to be a multicast destination address in order to be routed efficiently using a multicast-capable infrastructure. This requires a special procedure, known as tunnel mode with address preservation to be applied when placing datagrams into AH or ESP tunnels. In short, this procedure involves choosing the outer IP source and destination addresses to match the inner addresses (assuming the same version of IP is being used). The purposes of doing so are (1) to ensure that multicast routing is invoked on the datagram and (2) to ensure that the reverse path forwarding (RPF) check used in computing multicast routes works properly (see Chapter 9).

Introduction of multicast requires modification of some of the low-level IPsec machinery we saw in Figure 18-10. For example, the SPD and SAD are modified to include an “address preservation” flag used in implementing the address-preserving tunnel modes. In addition, a directionality flag in the SPD is used to determine under what circumstances SAs should be automatically created. This ensures that no SAs are created that would use prohibited multicast source addresses as a consequence of simply reversing source and destination IP addresses (as with unicast SAs). The SPD may need to include state as to when a GKM protocol needs to be invoked (e.g., for obtaining a needed group key), and a group PAD (GPAD) holds

the information specific to each GCKS, including for which traffic selectors each GCKS is able to produce SAs and authentication information that may be required to engage in a particular GKM protocol with a particular GCKS. GPAD material is not consulted by non-GKM protocols such as IKE, but the PAD and GPAD structures might be implemented together.

18.8.5 L2TP/IPsec

The Layer 2 Tunneling Protocol (L2TP) (see Chapter 3) supports tunneling of layer 2 traffic such as PPP through IP and non-IP networks. It relies on authentication methods that provide some authentication during connection initiation, but no subsequent per-packet authentication, integrity protection, or confidentiality. To address this concern, L2TP can be combined with IPsec [RFC3193]. The combination, called L2TP/IPsec, provides a recommended method to establish remote layer 2 VPN access to enterprise (or home) networks. L2TP can be secured with IPsec using either a direct L2TP-over-IP encapsulation (protocol number 115) or a UDP/IP encapsulation that eases NAT traversal.

L2TP/IPsec uses IKE by default, although other keying methods are possible. It uses an ESP SA in either transport mode (support required) or tunnel mode (support optional). The SA is used to secure the L2TP traffic, which is then responsible for establishing the layer 2 tunnel. Because it is really a combination of two protocols, both of which involve authentication, L2TP/IPsec often requires two distinct authentication procedures: one for the machine (using IPsec with pre-shared keys or certificates) and another for the user (e.g., using a name and password or access token).

L2TP/IPsec is supported on most modern platforms. On Windows, creating a new connection with the “Connect to a workplace” option can be used to enable L2TP and L2TP/IPsec. Some smartphones (e.g., Android, iPhone) support L2TP in their networking configuration setup screens. Mac OS X includes an L2TP/IPsec network adapter type that can be added using the system preferences. On Linux, it may be necessary to configure both IPsec and L2TP for them to work together. If L2TP is not required on such systems, direct IPsec may be preferable.

18.8.6 IPsec NAT Traversal

Using NATs with IPsec can present something of a challenge, primarily because IP addresses have traditionally been used in identifying communication endpoints and are assumed to not change. These assumptions were not entirely avoided (or obviated) when IPsec was first designed, so NAT has posed a problem. This is one factor contributing to the relatively slow deployment of IPsec. However, today IPsec supports both changing addresses (with MOBIKE) and NAT traversal.

To have a complete NAT traversal solution, we must take into account IKE, AH, and ESP in both transport and tunnel modes. As we shall see, when NATs must be accommodated, not all combinations of IPsec may be usable with all applications.

Guidance for what a solution requires is given in [RFC3715]. We shall first discuss a variety of issues that highlight fundamental incompatibilities between NATs and IPsec and then describe the methods that have been adopted to handle the problems.

One fundamental problem arises with AH and how NATs update the addresses in datagrams. Because the AH includes a MAC computation covering the datagram's IP addresses, a NAT is unable to rewrite addresses without invalidating the AH. Note that ESP does not share this issue, as its integrity protection mechanism does not include the IP addresses in its MAC.

Another problem arises with the UDP and TCP transport protocols because of the pseudo-header checksum, which incorporates IP addresses in its computation. When the transport-layer checksum is integrity-protected or encrypted, the NAT is unable to update the checksum without forming an invalid packet. A similar situation can arise for NAT when changing port numbers, or for other protocols that perform layering violations.

A third major problem relates to the ID payloads in IKE. There are several ways to identify an IKE peer, one of which is to use IP addresses. As these addresses are embedded within an encrypted IKE payload, they are not able to be modified by a conventional NAT, leading to failure. Alternative methods for identifying peers may be available, however (e.g., FQDN or the distinguished name from an X.509 certificate).

A fourth significant concern is how a NAT or NAT demultiplexes incoming traffic to the proper host. In protocols such as TCP and UDP, the port number is used for this purpose. However, IPsec AH and ESP act like transport protocols that carry no port numbers but instead use an SPI value. While some NATs can make use of the SPI value for demultiplexing, these values are chosen by an IPsec responder as a local matter and multiple independent hosts may choose the same value. Because a NAT cannot easily modify these values, it is possible for a NAT to improperly demultiplex incoming (returning) traffic, with a potential for erroneous delivery.

There are other potential problems for NATs that become more acute when IPsec is employed. For example, application protocols that carry IP addresses (e.g., SIP), if integrity-protected or encrypted, cannot be modified by a conventional NAT. In addition, configuration and analysis are more difficult because traffic that could otherwise be decoded for analysis is now obscured because of encryption. Fortunately, some network analysis tools (e.g., Wireshark) can process encrypted traffic if provided the necessary key material.

The primary approach to dealing with most of the NAT traversal concerns is to encapsulate IPsec ESP and IKE traffic using UDP/IP, which can be modified by conventional NATs when necessary. (There is no supported solution for NAT traversal of AH.) An IKE initiator can use UDP port 500 or 4500 for sending IKE and then transition to using port 4500 for UDP-encapsulated ESP and IKE, whether or not a NAT is present. UDP ESP encapsulation is prohibited on port 500 according

to [RFC5996]. The purpose of using port 4500 is to avoid some NATs that improperly process IPsec traffic on port 500.

NAT traversal for IKE is an optional feature of an IKE implementation. If supported, the following two Notification payloads can be included with the IKE_SA_INIT exchange: NAT_DETECTION_DESTINATION_IP and NAT_DETECTION_SOURCE_IP. If present, these appear after the Ni and Nr payloads and before CERTREQ payloads. The data associated with these payloads includes a SHA-1 hash of the SPIs for the SA, the source or destination IP address, and the source or destination port number. Such information is preserved as the IKE messages are passed through NATs. When receiving IKE messages that suggest a NAT is present, IKE processing continues using a UDP/IP encapsulation on port 4500, which tends to pass through NATs unimpeded.

After having traversed one or more NATs, arriving IKE traffic being used to set up a transport-mode SA may contain traffic selectors (TS payloads) with IP addresses or ranges that are not meaningful (i.e., they are private IP addresses “behind” a NAT) and that do not match the IP addresses contained in the addressing fields of the IKE datagram arriving at the responder. This is handled by first storing the addresses in TSi and TSr IKE payloads for later use and later replacing them with the source and destination IP addresses present in the received datagram. In essence, this is a form of “delayed NAT” on TS payloads performed by the recipient. The resulting datagram and TS payloads are used to query the SPD in order to determine the security policy for the requested SA. If transport mode is used, the responder completes the exchange and the initiator performs similar TS payload substitution processing (see Section 2.23.1 of [RFC5996] for more details).

18.8.7 Example

There are several open-source and proprietary IPsec implementations. Windows 7 supports IKEv2 and MOBIKE in Microsoft's Agile VPN subsystem. Linux includes kernel-level IPsec support in kernel version 2.6 and later, and the OpenSwan and StrongSwan packages can be used to implement complete VPN solutions. In the following example, we use a Linux server running StrongSwan (IPv4 address 10.0.0.3) with a Windows 7 client (IPv4 address 10.0.1.48) using RSA-based machine certificates we have created for authentication to demonstrate IKE. The IKE initial exchanges are shown in Figure 18-22.

Looking at this figure, we can see that Wireshark decodes the IKE exchange using ISAKMP as the protocol name. This is the now-deprecated Internet Security Association and Key Management Protocol and is the historical name of what ultimately became IKE. The IKE header contains the initiator's SPI (labeled “Initiator cookie”) and the responder's SPI, which has not yet been established. The version number is 2, indicating that this packet contains IKEv2, and the exchange type is IKE_SA_INIT.

Looking closer, we can see this is an IKE_SA_INIT message containing five payloads: one SA, one KE, one Nonce, and two of type Notify. The SA payload includes six proposals, each of which contains a list of transforms. The proposals

Figure 18-22 A trace of the initial IKE exchanges, highlighting the first packet. The IKE_SA_INIT exchange is carried on UDP port 500 and includes the initiator's SPI, proposals for cryptographic suite algorithms, DH key exchange material, a nonce, and Notify payloads used to indicate addresses for NAT traversal. Each proposal in the SA payload requests the establishment of an IKE_SA using a set of transforms for encryption, integrity protection, a PRF used for generation of random numbers, and DH group parameters used in key agreement.

represent sets of algorithms the initiator is willing to use. Proposal 6 (the last one) has been expanded to show more detail. It suggests AES in CBC mode with a 256-bit key length for encryption, HMAC with SHA-256 for integrity protection, a PRF based on SHA-384, and the alternate 1024-bit MODP group for DH key agreement. The other proposals (not detailed) include suggestions for 3DES encryption, AES encryption with different key lengths, SHA-1 for integrity protection, and other SHA variants for the PRF. Following the SA payload, the Key Exchange payload contains the public information required to perform a DH exchange using the “alternate 1024-bit MODP group.” In the other payloads, we find a nonce containing a 48-byte random bit string and two Notify payloads used for NAT traversal. The first Notify payload is of type NAT_DETECTION_SOURCE_IP, and the second contains NAT_DETECTION_DESTINATION_IP. The value in the first contains a 20-byte SHA-1 hash over these values: 8 bytes of the initiator’s SPI, 8 bytes of the responder’s SPI (0 here), 4 bytes of source IPv4 address, and 2 bytes of UDP source port number. The value in the second covers the same as the first, except the destination port is used in place of the source port. Figure 18-23 illustrates the response to the first IKE_SA_INIT message.

In this figure, the IKE_SA_INIT message contains the following payloads: SA, KE, Nonce, three of type Notify, and a Certificate Request. The SA payload contains only one proposal, comprising the following transforms: 3DES for encryption, HMAC_SHA1_96 for integrity, HMAC_SHA1 for the PRF, and group 2 for the DH exchange. The KE payload contains a 128-byte value from the 1024-bit MODP group. The Nonce payload contains a 32-byte random value for freshness. The next two Notify payloads contain NAT_DETECTION_SOURCE_IP and NAT_DETECTION_DESTINATION_IP, as described earlier. Following these are new payloads we have not yet encountered: CERTREQ and MULTIPLE_AUTH_SUPPORTED.

The Certificate Request (CERTREQ) payload indicates the responder’s preferred certificates. In this case, the responder indicates that any certificates later supplied by the initiator should be associated with a particular certificate authority. The encoding used to express the CA is one of several defined in Section 3.6 of [RFC5996], but only the values 4, 12, and 13 are currently standardized. Here, the payload contains the value 4, meaning the Certificate Authority Data subfield contains a concatenation of SHA-1 hashes of the public keys (X.509 Subject Public Key Info element) of trusted CAs. Given that the length of this subfield is only 20 bytes in this example, we can see that only a single CA is listed. It happens to be the SHA-1 hash of the DER encoding of the public key of the sample root certificate for the “Test CA” we created for this example.

Note

The binary *Distinguished Encoding Rules* (DER) format is a subset of the ASN.1 standard *Basic Encoding Rules* (BER). DER permits values to be encoded in only a single, unambiguous way. DER is one of the two most popular ways to encode X.509 certificates. The other is PEM, an ASCII format, which we showed earlier. Various utilities, including `openssl`, may be used to convert between the two formats.

Figure 18-23 The completion of the IKE_SA_INIT exchange includes the responder's SPI (labeled "cookie"), a single proposal with transforms, DH parameters, a nonce value, and NAT traversal address parameters. This message also includes a CERTREQ payload to indicate and request acceptable certificates, and a notification indicating that multiple authentication methods (in series) are supported.

The final payload in Figure 18-23 is a Notify payload containing the `MULTIPLE_AUTH_SUPPORTED` indication and no associated data. Defined as an experimental extension to IKE in [RFC4739], it indicates the ability to use more than one authentication method. Such a situation may arise, for example, when using an `IKE_AUTH` exchange based on certificates to establish IKE SAs to a service provider, followed by some form of EAP-based authentication for the individual user.

The remaining packets shown in Figure 18-23 contain `IKE_AUTH` messages that are encrypted. They are carried using source and destination port number 4500 instead of 500, and the encapsulation uses the special “non-ESP marker” containing 4 bytes of 0 [RFC3947], indicating that the traffic is IKE and not ESP. The marker and port numbers are also used for the `INFORMATIONAL` exchanges we discussed previously.

Wireshark has the capability to decrypt encrypted IKE traffic if provided with the proper keys and SPI values. By providing a copy of the log trace file from the IKE server to Wireshark (located under Edit | Preferences | Protocols | ISAKMP), we can see the decrypted IKE payload information. (The Wireshark developers tend to prefer the original names of protocols such as ISAKMP and SSL instead of IKE and TLS, so that is what we see when looking at Wireshark output.)

The third packet in Figure 18-22 is the first fragment of a UDP/IP datagram that Wireshark reassembles when it receives the second fragment (packet 4). The decrypted and reassembled result is shown in Figure 18-24.

Here we can see the contents of the reassembled and decrypted UDP/IPv4 fragments constituting the first packet of the `IKE_AUTH` exchange. The client provides the following IKE payloads: `IDi`, `CERT`, `CERTREQ`, `AUTH`, `N(MOBIKE_SUPPORTED)`, `CP`, `SA`, `TSi`, and `TSr`. The `IDi` payload contains the name of the initiator, test client. The `CERT` payload contains a client certificate for test client signed by the Test CA certificate authority that we know the corresponding server should accept (because it was configured to). The `CERTREQ` payload contains requests for Test CA as well as 21 other CAs (not shown) known by this Windows 7 client. The `AUTH` payload contains a data block signed using the RSA private key of the initiator (see Section 2.15 of [RFC5996]), which provides origin authentication. The `N(MOBIKE_SUPPORTED)` indicates the client's willingness to follow the MOBIKE protocol. The `CP(CFG_REQUEST)` payload (not detailed) contains the following attributes: `INTERNAL_IP4_ADDRESS`, `INTERNAL_IP4_DNS`, `INTERNAL_IP4_NBNS`, and a `PRIVATE_USE` type (23456). These are used to help in configuring VPN access and serve a similar purpose to the configuration information typically provided locally by DHCP (see Chapter 6). `NBNS` refers to a NetBIOS name server. NetBIOS is an API that can be implemented on a number of networking protocols and is common in Microsoft Windows environments.

The `SA` payload in Figure 18-24 represents the information required to form a `CHILD_SA`. There are two proposals (not detailed), each for ESP using 32-bit SPI values (note that IKE uses 64-bit SPI values) with `AUTH_HMAC_SHA1_96` as the integrity algorithm and not using extended sequence numbers (indicated using a proposal transform). The first proposal suggests the use of `ENCR_AES_CBC`

Figure 18-24 The IKE_AUTH exchange contains encrypted information and operates on UDP port 4500. The reassembly of two fragments produces an IKE message with an Encrypted/Authenticated data payload containing the following payloads: Identification initiator (IDi), Certificate (CERT), Certificate Request (CERTREQ), Authentication (AUTH), Notify (N), Configuration (CP), Security Association (SA), Traffic Selector initiator (TSi), and Traffic Selector responder (TSr).

(256-bit keys) for encryption, and the second suggests ENCR_3DES. Because there is no N(USE_TRANSPORT_MODE) payload present, we conclude that each of the proposals involves using ESP in the default tunnel mode.

The Traffic Selector (TSi and TSr) payloads in Figure 18-24 indicate the IPv4 and IPv6 address ranges that are permitted to be associated with the forming SA. The TSi has both a TS_IPv6_ADDR_RANGE and TS_IPv4_ADDR_RANGE that contain their entire address and port number ranges. TSr (not detailed) contains the same values.

The first IKE_AUTH message we just discussed is fairly complicated and requires more than a single 1500-byte UDP/IPv4 datagram to hold it. After processing by the responder, the final message in the exchange is produced. It is shown in Figure 18-25.

In this figure, the server sends a response with the following payloads: IDr, CERT, AUTH, CP(CFG_REPLY), SA, TSi, TSr, N(AUTH_LIFETIME), N(MOBIKE_SUPPORTED), and N(NO_ADDITIONAL_ADDRESSES). The IDr payload contains a DER-encoded name of the server. The CERT payload contains the matching (server) certificate, and the AUTH payload indicates knowledge of the corresponding private key. The CP(CFG_REPLY) payload includes an INTERNAL_IP4_ADDRESS attribute, which is useful for VPN configuration. The SA payload is similar to the client's SA payload from Figure 18-24 and includes a single proposal with transforms ENCR_AES_CBC (256-bit keys), AUTH_HMAC_SHA1_96, and no ESNs.

The TSi and TSr values in this packet have been “narrowed” to be much smaller ranges than in the client's IKE_AUTH message. In this case, the TSi is narrowed to the single IPv4 address 10.100.0.1. The TSr has been narrowed to 10.0.0.0/16. Each uses the full port range 0–65535. This is a relatively simple case of narrowing. In cases where more than one discontinuous subset of the range specified by the initiator is acceptable, an N(ADDITIONAL_TS_POSSIBLE) payload may be generated. Narrowing is used to achieve mutually agreeable address ranges for an SA.

The N(AUTH_LIFETIME) payload indicates that the authentication is going to last at most only 2.8 hours (10,154s, expressed as 000027aa in the trace). The N(MOBIKE_SUPPORTED) payload indicates the responder's support for MOBIKE. The N(NO_ADDITIONAL_ADDRESSES) payload (not detailed) is used with MOBIKE to indicate that no additional IP addresses other than those used in the exchange are being used.

At this point, a tunnel mode ESP CHILD_SA has been set up and traffic can flow. We do not detail the traffic flow containing ESP packets (they are comparatively straightforward) but instead jump to the point where the SAs are to be torn down. This is accomplished using two sets of INFORMATIONAL exchanges containing Delete payloads—one for the ESP SA and one for the IKE SA. Figure 18-26 shows the request to close the ESP SA.

We can see in this figure the SA being deleted based on a close request at the client. Like other IKE traffic, it includes an encrypted and authenticated payload. The encrypted payload in turn includes a single Delete payload. The Delete payload can indicate that more than one SPI is to be deleted, but in this case it

Figure 18-25 Completing the IKE_AUTH exchange, the responder produces an Encrypted/Authenticated data payload containing the following payloads: Identification responder (IDr), CERT, AUTH, CP(CFG_REPLY), SA, narrowed TSi and TSR, along with N(AUTH_LIFETIME), N(MOBIKE_SUPPORTED), and N(NO_ADDITIONAL_ADDRESSES). The first CHILD_SA can now commence.

Figure 18-26 A request to delete the child ESP SA with SPI 6cfca5ef is carried on the IKE SA. The Delete payload shows Port: 1, which is mislabeled by Wireshark. (It should be Number of SPIs: 1.)

indicates only the one with SPI value 0x6cfca5ef. Packet 7 from the responder is essentially the same but contains a different setting in the Flags field (responder instead of initiator and response instead of request), a different encryption IV and integrity checksum data, and specification of a different SPI (c348faf2) in the Delete payload.

To close the IKE_SA, another exchange of INFORMATIONAL messages is required. The initiator begins with the packet shown in Figure 18-27. We can see here a request to close the IKE SA. Encrypted like other traffic, the Delete payload does not need to include an SPI value because it is implied to be the IKE SA carrying the deletion request. To complete the IKE SA deletion, the responder replies with an IKE message containing only an empty encrypted/authenticated payload type in packet 9. Its Next Payload type field is NONE (zero). This indicates the completion of the IKE SA deletion.

Figure 18-27 A request to delete the IKE SA. SPI values are not required because the entire message is carried on the IKE SA and there is no ambiguity.

18.9 Transport Layer Security (TLS and DTLS)

So far we have discussed security protocols at layers 2 and 3. The most widely used protocol for security operates just above the transport layer and is called Transport Layer Security (TLS). TLS is used for securing Web communications and for several other popular protocols, including POP and IMAP (which are called POP3S and IMAPS, respectively, when protected with TLS). One reason for TLS's popularity is that it can be implemented within or underneath applications that ride on top of the lower layers, whereas protocols such as EAP and IPsec usually require capabilities within the operating systems and protocol implementations of hosts and embedded devices.

There are several versions of TLS and its predecessor, the Secure Sockets Layer (SSL) [RFC6101]. We shall focus on TLS version 1.2 [RFC5246], which is the most

recent at the time of writing. TLS 1.2 can support backward compatibility with most older versions of TLS and SSL (e.g., TLS 1.0, 1.1, and SSL 3.0). However, SSL 2.0 is weaker, and while interoperability with it is possible, it is now prohibited [RFC6176]. After discussing TLS 1.2, which operates over a stream-oriented protocol (usually TCP), we will look at the datagram-oriented variant called the Datagram Transport Layer Security (DTLS) [RFC4347]. DTLS is slowly gaining popularity for some applications such as VPN implementations that do not use IPsec. Its current specification is based on TLS 1.1 [RFC4346], but updates are under way [IETF DTLS].

18.9.1 TLS 1.2

The security goals of TLS are not unlike those for IPsec, but TLS operates at a higher layer. Confidentiality and data integrity are provided based on a variety of cryptographic suites that use certificates that can be provided by a PKI. TLS can also establish secure connections between two anonymous parties (without using certificates), but this application is vulnerable to a MITM attack (not surprising, given that each end is not even strongly identified). The TLS protocol has two layers of its own, called the record layer and the upper layer. The Record protocol implements the record (lower) layer and is assumed to be layered on a reliable underlying protocol (e.g., TCP). Figure 18-28 shows the basic organization.

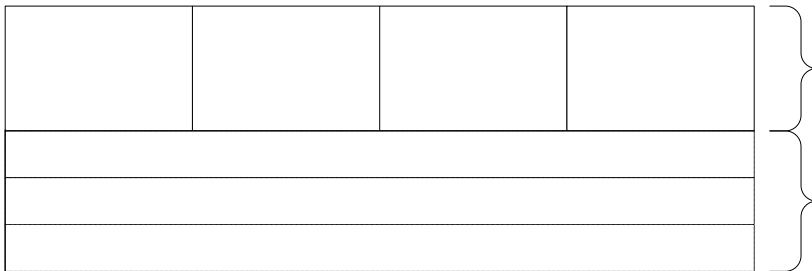


Figure 18-28 The TLS protocol “stack” has a lower record layer and three of its own upper-layer protocols called handshaking protocols. A fourth upper-layer protocol is the application protocol using TLS. The record layer provides fragmentation, compression, integrity protection, and encryption. The handshaking protocols perform many of the same tasks for TLS that IKE does for IPsec.

TLS is a client/server protocol, designed to support security for a connection between two applications. The Record protocol provides fragmentation, compression, integrity protection, and encryption for data objects exchanged between clients and servers, and the handshake protocols establish identities, perform authentication, indicate alerts, and provide unique key material for the Record protocol to use on each connection. The handshaking protocols comprise four specific protocols: the Handshake protocol, the Alert protocol, the Change Cipher Spec protocol, and the application data protocol. Like IPsec, TLS is extensible and

can accommodate existing or future cryptographic suites, which TLS calls cipher suites (CS). Many such combinations have been defined, and the IANA maintains a registry of the current set [TLSPARAMS]. Modern variants of TLS are based on SSL 3.0, originally developed by Netscape. TLS and SSL do not directly interoperate, but there are negotiation mechanisms that allow clients and servers to dynamically discover which protocol to use when a connection is first established.

The Change Cipher Spec protocol is used to change the current operating parameters. This is accomplished by first using the Handshake protocol to set up a “pending” state, followed by an indication to switch from the current state to the pending state (which then becomes the current state). Such switching is allowed only after the pending state has been readied. TLS depends on five cryptographic operations: digital signing, stream cipher encryption, block cipher encryption, AEAD, and public key encryption. For integrity protection, the TLS record layer uses HMAC. For key generation, TLS 1.2 uses a PRF based on HMAC with SHA-256. TLS also integrates an optional compression algorithm that is negotiated when a connection is first established.

18.9.1.1 TLS Record Protocol

The Record protocol uses an extensible set of record content type values to identify which message type (i.e., which of the higher-layer protocols) is being multiplexed. At any given point in time, the Record protocol has an active current connection state and another set of state parameters called the pending connection state. Each connection state is further divided into a read state and a write state. Each of these states specifies a compression algorithm, encryption algorithm, and MAC algorithm to be used for communication, along with any necessary keys and parameters. When a key is changed, the pending state is first set up using the Handshake protocol, and then a synchronization operation (usually accomplished using the Cipher Change protocol) sets the current state equal to the pending state. When first initialized, all states are set up with NULL encryption, no compression, and no MAC processing.

The Record protocol's processing flow is shown in Figure 18-29. It divides (fragments) higher-layer information blocks into records called TLSPlaintext records, which can be at most 2^4 bytes in length (but are usually much less). The choice of record size resides within TLS; higher-layer message boundaries are not preserved. Once formed, TLSPlaintext records are compressed using a compression algorithm [RFC3749] identified in the current connection state. There is always one compression protocol active, although it may be (and usually is) the NULL compression protocol (which, not surprisingly, provides no compression gain). The compression algorithm converts a TLSPlaintext record into a TLSCompressed structure. Compression algorithms are required to be lossless and may not produce an output that is larger than the input by more than 1KB. To protect the payload from disclosure and modification, encryption and integrity protection algorithms convert a TLSCompressed structure into a TLSCiphertext structure, which is then sent on the underlying transport connection.

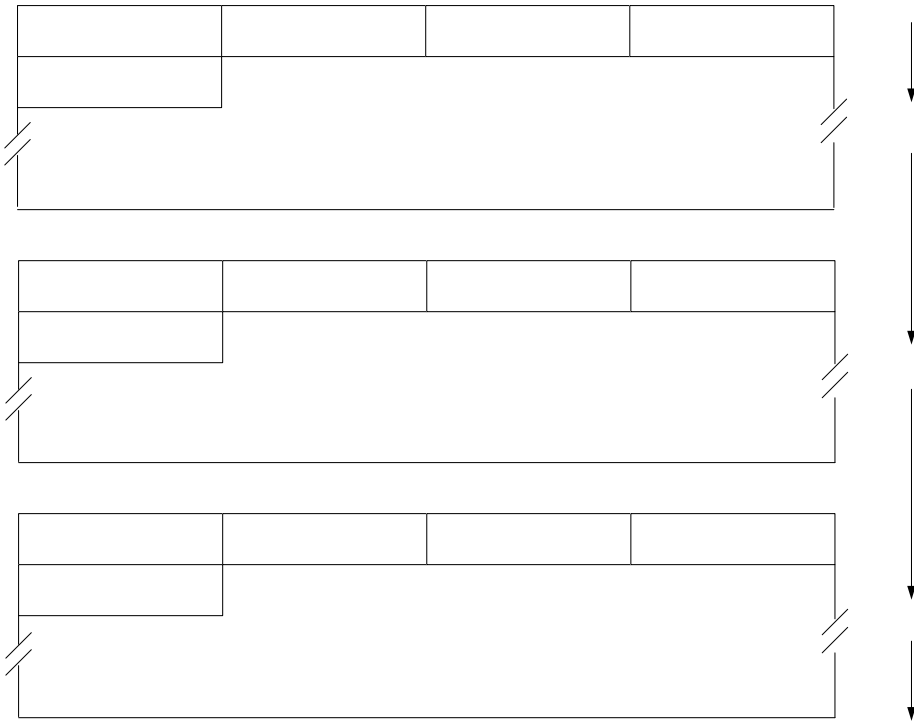


Figure 18-29 The TLS record layer starts with a `TLSP Plaintext` record, which is compressed by a lossless compression algorithm to form a `TLSCompressed` record. The `TLSCompressed` record is encrypted (and has a MAC applied) to form a `TLSCiphertext` record, which is sent for transmission. Conventional stream and block ciphers require a MAC, and block ciphers may include padding. When using AEAD ciphers, a nonce is included with the encrypted and integrity-protected content, but no separate MAC is used.

Referring to Figure 18-29, when producing a `TLSCiphertext` structure, a sequence number is first computed (but not placed in the message), then a MAC is computed if necessary, and finally symmetric encryption is performed. Prior to encryption, the message may be padded (up to 255 bytes) to meet any block length requirements imposed by the encryption algorithm (e.g., for block ciphers). A MAC is not required for AEAD algorithms that provide both integrity and encryption (e.g., CCM, GCM), but a nonce is used in such cases.

Keys for the Record protocol are derived from a master secret provided by some method outside the Record protocol, most often by the Handshake protocol. Using the master secret, along with random values provided by the client and server applications at the beginning of the connection, the following keys are generated:

$$M_c \parallel M_s \parallel D_c \parallel D_s \parallel IV_c \parallel IV_s = \text{PRF}(\text{master_secret}, \text{"Key expansion", server_random} + \text{client_random})$$

In this assignment, $|$ is the splitting operator and $+$ is the concatenation operator. M_c denotes the MAC write key for the client, M_s denotes the MAC write key for the server, D_c denotes the client's data write key, D_s denotes the server's data write key, IV_c denotes the client's IV, and IV_s denotes the server's IV. With the $|$ operator, each key uses however many bytes from the PRF function are required. MAC, encryption, and IV keys, if used, have a fixed length based on the cipher suite selected. The last two values are used only in cases where implicit nonce generation takes place with AEAD ciphers (see Section 3.2.1 of [RFC5116]). According to [RFC5246], the cipher suite requiring the most material is AES_256_CBC_SHA256. It requires four 32-byte keys, for a total of 128 bytes.

18.9.1.2 TLS Handshaking Protocols

There are three subprotocols to TLS, which perform tasks roughly equivalent to those performed by IKE in IPsec. More specifically, these other protocols are identified by numbers used for multiplexing and demultiplexing by the record layer and are called the Handshake protocol (22), Alert protocol (21), and Cipher Change protocol (20). The Cipher Change protocol is very simple. It consists of one message containing a single byte that has the value 1. The purpose of the message is to indicate to the peer a desire to change from the current to the pending state. Receiving such a message moves the read pending state to the current state and causes an indication to the record layer to transition to the pending write state as soon as possible. This message is used by both client and server.

The Alert protocol is used to deliver status information from one end of a TLS connection to another. This can include terminating conditions (either fatal errors or controlled shutdowns) or nonfatal error conditions. As of the publication of [RFC5246], 24 alert messages were defined in standards. More than half of them are always fatal (e.g., bad MACs, missing or unknown messages, algorithm failures).

The Handshake protocol sets up the relevant connection operating parameters. It allows the TLS endpoints to achieve six major objectives: agree on algorithms and exchange random values used in forming symmetric encryption keys, establish algorithm operating parameters, exchange certificates and perform mutual authentication, generate a session-specific secret, provide security parameters to the record layer, and verify that all of these operations have executed properly. Figure 18-30 shows the messages required.

The handshake shown in Figure 18-30 begins with Hello messages. The ClientHello message is usually the first message sent from client to server. It contains a session ID, proposals for the cryptographic suite number (CS in Figure 18-30), and a set of acceptable compression algorithms (which are usually just NULL, although [RFC3749] also defines DEFLATE). TLS supports in excess of 250 cipher suite options [TLSPARAMS].

The ClientHello message also contains the TLS version number and a random number called ClientHello.random. Upon receiving the ClientHello message, the server checks to see if the session ID is present in its cache. If so, the server

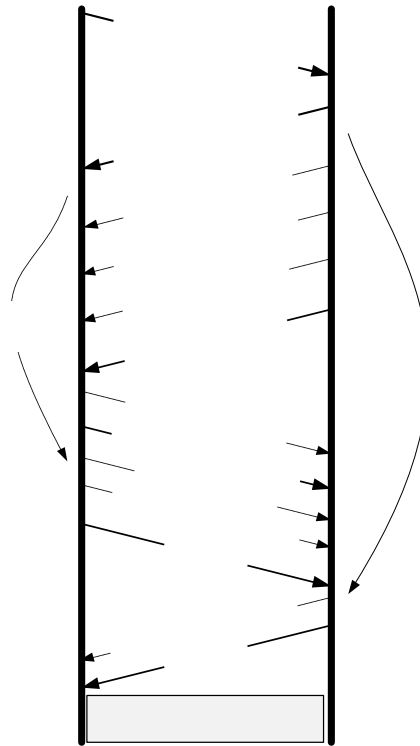


Figure 18-30 The normal TLS connection initiation exchange consists of several messages that may be pipelined. Required messages have solid arrows and are shown in boldface type. An abbreviated exchange takes place if a previously existing connection can be restarted. This avoids endpoint authentication, which can be costly for systems with limited processing capabilities.

may agree to continue a previously existing connection (called a “resume”) by performing an abbreviated handshake. The abbreviated handshake is key to TLS performance and avoids having to repeatedly verify the authenticity of each endpoint, but it does require synchronization with respect to the cipher specification. The ServerHello message completes the first part of the exchange by carrying the server’s random number (`ServerHello.random`) to the client. This message also contains a session ID value. If the value is the same as that provided by the client, it indicates the server’s willingness to resume. If not, it has the value 0 and a full handshake is required.

If a full (nonabbreviated) handshake is executed, the exchange of Hello messages results in each end becoming aware of the cipher suites, compression algorithms, and random values of its peer. The server selects among the cryptographic suites specified by the client and may be required to provide its certificate chain in

a Certificate message if it is to be authenticated (which is the typical case for secure Web traffic or HTTPS). The server may also send a ServerKeyExchange message if its certificate is not valid for signing, it has no certificate, or a temporary or ephemeral key is to be used to generate session keys.

Note

The ServerKeyExchange message is used only in cases where the Certificate (server) message does not contain enough information to establish a premaster secret. Such cases include anonymous or ephemeral DH key agreement (i.e., cipher suites starting with TLS_DHE_anon, TLS_DHE_DSS, TLS_DHE_RSA). The ServerKeyExchange message is *not* used for other suites, including those starting with TLS_RSA, TLS_DH_DSS, or TLS_DH_RSA.

At this point, the server may require client authentication. If so, it generates a CertificateRequest message. Once this message is sent, the server completes the second portion of the exchange by sending the mandatory ServerHelloDone message. Upon receiving this (possibly pipelined) message from the server, the client may be required to prove its identity (i.e., knowledge of an appropriate private key corresponding to a certificate). If so, it first sends its certificate using a Certificate message in the same format used by the server. It then sends the mandatory ClientKeyExchange message. The contents of this message depend on the cryptographic suite used, but it generally contains either an RSA-encrypted key or Diffie-Hellman parameters that may be used to create a type of seed for creating new keys (called the premaster secret). Finally, it sends a CertificateVerify message to demonstrate that it possesses the private key corresponding to the previously provided certificate, if the server requested client authentication. This message contains a signature on the hash of all of the handshake messages the client has received and sent up to this point.

The final portion of the exchange includes a ChangeCipherSpec message, which is an independent TLS protocol content type (i.e., technically not a Handshake protocol message). However, the mandatory Handshake protocol Finished messages can be exchanged only after a successful exchange of ChangeCipherSpec messages. The Finished messages are the first ones to be protected using the parameters exchanged up to this point. The Finished message themselves contain “verify data,” which consists of the following value:

$$\text{verify_data} = \text{PRF}(\text{master_secret}, \text{finished_label}, \text{Hash}(\text{handshake_messages}))$$

where finished_label has the value “client finished” for the client and “server finished” for the server. The particular hash function Hash is associated with the selection of the PRF made during the initial Hello exchange. TLS 1.2 provides the ability to have variable-length verify data, but all previous versions and current cipher suites produce 12 bytes of verify data. The 48-byte master_secret value is computed as follows:

```

master_secret = PRF(pre_master_secret, master_secret,
ClientHello.random + ServerHello.random)

```

where + is the concatenation operator. The Finished message is important because it can be used to know with a high degree of certainty that the Handshake protocol has completed successfully and subsequent data exchange can take place.

18.9.1.3 TLS Extensions

If we compare the capabilities of IKE and TLS we have discussed so far, we can see that IKE includes the ability to carry information beyond that required for basic SA establishment. This is accomplished using IKE Notify and Configuration payloads. To provide a similar extensible mechanism for TLS, various extensions can be included with TLS 1.2 messages in a standard way. The baseline specification for TLS 1.2 [RFC5246] includes a “signature algorithms” extension that a client uses to specify to a server what types of hash and signature algorithms it supports (MD5, SHA-1, SHA-224, SHA-256, SHA-384, SHA-512 for hashes and RSA, DSA, ECDSA for digital signatures are defined). They are indicated in descending order of preference by pairs, as some systems allow only certain combinations. The current list of extensions is given in [TLSEXT].

Previous versions of TLS had about a half-dozen extensions, and [RFC6066] updates these extensions for TLS 1.2. It defines the following extensions: server_name (DNS-style name of the server being contacted), max_fragment_length (maximum length of a message as 2 bytes for n having values 9–12), client_certificate_url (indicates support for the CertificateURL handshake message used to send the URL of a certificate instead of a complete certificate), trusted_ca_keys (hashes or the names of trusted CA public keys and/or certificates), truncated_hmac (use the first 80 bits of HMAC calculations only), and status_request (requests that a server invoke OCSP and provide the DER-encoded response in a CertificateStatus handshake message to check a certificate). Each of these extensions may be present in an (extended) ClientHello message and in some circumstances may appear in the ServerHello message to indicate agreement. Aside from these extensions and the two handshake messages already mentioned, [RFC6066] also defines four alert messages: certificate_unobtainable, unrecognized_name, bad_certificate_status_response, and bad_certificate_hash_value. These are self-explanatory and are not sent unless the peer has demonstrated understanding of the extended ClientHello type message.

Several other extensions have been defined or are reserved. The user_mapping extension [RFC4681] provides a method for providing context for the user identifier (e.g., Windows domain). Another expands the cert_type extension to include not only X.509 certificates but also OpenPGP certificates [RFC6091]. Elliptic curve cipher suites are described by the informational document [RFC4492]. The Secure Remote Password protocol (SRP) can be integrated with TLS according to the methods defined in the informational document [RFC5054]. A use_srtp extension designed to produce a version of the Secure Real-Time protocol (SRTP) based on DTLS (see Section 18.9.2) is given in [RFC5764]. A method to eliminate the state a

server must store to perform session resumption is given by the SessionTicket TLS extension [RFC5077]. It involves placing the necessary state in an encrypted form in the client. Finally, an important renegotiation_info extension is used to combat a renegotiation vulnerability. We shall describe it in more detail next.

18.9.1.4 Renegotiation

TLS supports the ability to renegotiate cryptographic connection parameters while maintaining the same connection. This can be initiated by either the server or the client. If the server wishes to renegotiate the connection parameters, it generates a HelloRequest message, and the client responds with a new ClientHello message, which begins the renegotiation procedure. The client is also able to generate such a ClientHello message spontaneously, without prompting from the server.

Support for renegotiation is optional but “highly recommended” and is used, for example, when sequence numbers are about to wrap. Renegotiation can be refused by generating a “no_renegotiation” (type 100) warning alert. Although this type of alert is not required to be terminal, receiving such an alert may, by local policy, result in connection termination.

In 2009, a successful attack on TLS was demonstrated using the renegotiation capability. We describe it in more detail in Section 18.12. The vulnerability allows an attacker to establish a malicious TLS session with a server that can later be spliced into a subsequent legitimate session by a client using a MITM attack. The server believes that only a standards-compliant renegotiation has taken place. A solution to the problem, given in [RFC5746], involves binding any renegotiation more closely with the existing session using a TLS extension called renegotiation_info (type 0xff01). When creating a new connection, renegotiation_info is empty. When client renegotiation takes place, it contains “client_verify_data,” and when server renegotiation takes place it contains a concatenation of “client_verify_data” and “server_verify_data.” The client_verify_data is defined to be the same verify_data used with the Finished message sent by the client on the completion of the last handshake. This is a 12-byte value in TLS (36 for SSLv3). The server_verify_data is defined to be the verify_data used with the Finished message sent by the server on completion of the last handshake.

Some deployed TLS (and SSL) servers abort a connection when unknown extensions are present. To handle this issue when deploying the (relatively new) renegotiation_info extension, an alternative is available. The TLS cipher suite TLS_EMPTY_RENEGOTIATION_INFO_SCSV can be used during connection establishment to indicate the equivalent of an empty renegotiation_info extension. This is using a signaling cipher suite value (SCSV) not to encode a real cipher suite, but instead to indicate a certain set of functions. (A similar trick is used in DNSSEC for NSEC3 records; see Section 18.10.1.3.)

18.9.1.5 Example

In the example shown in Figure 18-31, we see the messages exchanged during a connection setup with TLS 1.2 using TCP/IP on the local loopback interface. The client and server have RSA certificates, which each provides to its peer. The initial

TCP handshake and window update, as well as the 127.0.0.1 source and destination IPv4 addresses, are not shown. The trace has been annotated with right and left arrows for additional clarity. The arrows pointing to the right indicate TCP segments containing at least one TLS message sent by the client headed for the server. Left-pointing arrows indicate messages from the server to the client. To see this output, Wireshark was told to decode the trace by first choosing SSL under the Analyze | Decode As ... menu.

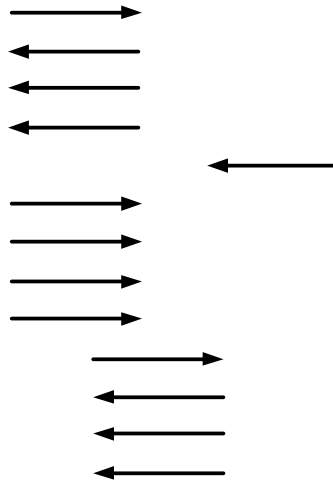


Figure 18-31 A normal TLS 1.2 connection establishment as shown by Wireshark. The server runs on port 5556. Client messages sent to the server are highlighted by arrows pointing to the right. Server messages sent to the client are shown with left-pointing arrows. TCP ACKs are interspersed with the TLS messages. After the Change Cipher Spec message (segment 21), other messages are encrypted and authenticated. Segment 13 also includes the ServerHelloDone message.

In Figure 18-31, after the initial TCP-level handshake, the TLS exchange begins with a ClientHello message. TCP pure ACKs are seen interspersed with the TLS messages. After the ChangeCipherSpec message has been processed, the subsequent information is encrypted. To see what is happening in more detail, we shall expand the first few TLS messages. Figure 18-32 shows the detailed contents of the ClientHello message.

Figure 18-32 A ClientHello message in TLS 1.2 contains version information, supported cipher suites and compression algorithms, random data, and a number of extensions. Here, the client supports Diffie-Hellman key agreement as well as key exchange using RSA. It uses AES-256 in CBC mode for encryption and SHA-256 for integrity protection.

The ClientHello message detailed in Figure 18-32 is a Record protocol message carrying the ClientHello handshake message. It contains a 32-bit UNIX timestamp counting seconds since midnight, January 1, 1970, plus a random 28-byte value (ClientHello.random) used in forming keys. As this is a brand-new connection, its session ID is 0. Six bytes are devoted to carrying the client's three supported cipher suites in preference order (most preferred first). Each suite is encoded using a 16-bit value specified by the TLS Cipher Suite Registry in [TLSPARAMS]. Only a single compression method is supported—the NULL method, which achieves no compression gain and is typical. Also, 50 bytes are included for extensions. The cert_type extension indicates that either X.509 or OpenPGP certificates are understood. The server_name extension contains 127.0.0.1, which was the name of the server provided to the client application. The renegotiation_info is empty, as this is the first handshake, as is the SessionTicket TLS extension. The signature_algorithms extension indicates that the following combinations can be processed by the client: sha1-rsa, sha1-dsa, sha256-rsa, sha384-rsa, and sha512-rsa.

In this sample exchange, the server has been configured with only one cipher suite, TLS_DHE_RSA_WITH_AES_256_CBC_SHA256 (0x006b). The server indicates this fact when responding to the ClientHello by using the ServerHello message shown in Figure 18-33.

In this figure, the server responds with a ServerHello message to the client's ClientHello. The server provides its copy of the current time and its 28-byte random value. It also includes a random 32-byte session ID. The server supports only a single cipher suite (DH key agreement using RSA certificates with AES-256 encryption in CBC mode for encryption and SHA-256 for integrity protection). Like the client, it does not support any compression methods. It includes an empty renegotiation_info extension and an empty SessionTicket TLS extension. Following this first message, the server continues with a Certificate message, as shown in Figure 18-34.

The message in Figure 18-34 carries the server's 841-byte X.509v3 certificate to the client, which has been signed by a sample certificate authority called Test CA shown in the Issuer field. The field called SubjectPublicKeyInfo contains the server's 270-byte public RSA key, which the client will use in authenticating the server. There are six extensions in the certificate: basicConstraints (critical), subjectAltName (contains a DNS name for the server using the certificate), extKeyUsage (extended key usage, indicating that the purpose of the key is for authenticating a server), keyUsage (critical; indicates that the enclosed key may be used for key encipherment or for generating digital signatures), subjectKeyIdentifier (a 20-byte number identifying the signed public key), and the authorityKeyIdentifier (a 20-byte number identifying the key used by the certificate authority to produce this certificate).

The ClientKeyExchange message is not detailed as it mostly includes binary information used in forming the DH exchange. The next message of interest is segment 13, which is a single TCP segment containing both a CertificateRequest message and a ServerHelloDone message. Figure 18-35 shows the contents.

Figure 18-33 A ServerHello message in TLS 1.2 contains version information, supported cipher suites and compression algorithms, and a number of extensions. Here, the client supports Diffie-Hellman key agreement. It uses AES-256 for encryption and SHA-256 for integrity protection.

Figure 18-35 shows a TCP segment containing both a CertificateRequest message and a ServerHelloDone message. The CertificateRequest is requesting the client to provide its certificate and to verify its authenticity using a subsequent CertificateVerify message. The type of certificate requested should be signed using either RSA or DSS from the Test CA certificate authority. The signature algorithms listed are sha1-rsa, sha1-dsa, sha256-rsa, sha384-rsa, and sha512-rsa.

Packet 15 (not detailed) contains the Certificate message that has the certificate chain for the client and its public key. In this case, the subject field contains “test client” and the issuer is Test CA. Thus, the client’s and server’s certificates were signed by the same CA and the chain is a single certificate. For the client to prove that it possesses the corresponding private key, it generates the CertificateVerify

Figure 18-34 Following the ServerHello, the server generates a Certificate message to carry its certificate. The client can use the certificate to authenticate the server. The same message format is used when the server authenticates the client.

Figure 18-35 The server's CertificateRequest and ServerHelloDone messages are contained in the same TCP segment. The client can use the certificate to authenticate the server. The same message format is used when the server authenticates the client.

message (packet 19). The CertificateVerify message contains a signature on a hash of all the session's handshake messages sent or received so far, signed using the private key of the client. This proves not only that the client is authentic, but that it has participated appropriately in the TLS exchanges up to this point and not lost or reordered any messages. After the CertificateVerify message, the Change Cipher message begins the subsequent (encrypted) communication.

18.9.2 TLS with Datagrams (DTLS)

The TLS protocol assumes a stream-based underlying transport protocol for delivering its messages. A datagram version (DTLS) relaxes this assumption but aims to otherwise achieve the same security goals as TLS using essentially all the same message formats. It was originally motivated by protocols such as SIP that run on UDP but do not care to use IPsec [RFC5406]. DTLS has also been adapted for use with DCCP [RFC5238] and SCTP [RFC6083]. The current version at the time of writing is DTLS 1.0 [RFC4347], based on TLS 1.1. An update, based on TLS1.2, is in the works [IDDTLS]. It uses the same protocol layering shown in Figure 18-28 and most of the same message exchanges.

The main challenge of providing TLS-like service without a reliable transport is that datagrams may get lost, reordered, or duplicated. These problems can affect encryption and the Handshake protocol, both of which have ordering dependencies in TLS. To handle them, DTLS adds an explicit sequence number to each record carried by the record layer (they were implicit with regular TLS) and a timeout-based retransmission scheme with (different) sequence numbers from those used by the Handshake protocol.

18.9.2.1 DTLS Record Layer

In TLS, the ordering of records is important because the MAC computation of one record depends on its predecessor. More specifically, the MAC computation depends on an implicit 64-bit sequence number for each record that is incorrect in the presence of datagram reordering or loss. To remedy this problem, DTLS uses explicit sequence numbers at the record layer. These sequence numbers are reset to the value 0 after each ChangeCipherSpec message is sent. They are used in combination with an additional 16-bit epoch number incorporated into each record's header. The epoch number is incremented by 1 for each change of cipher state. This handles the situation where multiple messages containing the same sequence number, generated as a result of multiple proximate handshakes, might be in flight simultaneously.

The MAC computation in DTLS is modified from its TLS counterpart to include the 64-bit concatenation of the two new fields (epoch first, followed by sequence number). This allows each record to be handled independently. Note that with TLS, a bad MAC results in connection termination. With DTLS, a full connection abort is not necessary, and a receiver may choose to simply discard the

record containing the invalid MAC or send an alert message (which, if generated, must be terminal).

Duplicates are simply dropped or are optionally considered as a replay and possible attack. Replay detection, if supported, is based on keeping a window of current sequence numbers at the receiver. The window is required to be at least 32 messages but is suggested to be at least 64. The scheme is similar to that used in IPsec for AH and ESP. Records arriving with sequence numbers less than the left window edge are silently discarded as old or duplicative. Those within the window are checked as possible duplicates. A message within the window carrying a valid MAC is kept, even if out of order. Those with invalid MACs are discarded. Those with valid MACs that exceed the right window edge cause the right window edge to be advanced. Thus, the right window edge represents the validated message with the highest sequence number.

A single datagram may contain multiple DTLS records, but no single record may span multiple datagrams. The record layer allows applications to implement a PMTUD process similar to TCP's (see Chapter 15) and avoids sending datagrams it believes are likely to be fragmented. Indeed, applications are supposed to receive an error indication if they attempt to send application messages that exceed the PMTU or maximum application datagram size (PMTU minus DTLS overhead). An exception to this rule is how DTLS handles the Handshake protocol, which can involve relatively large messages.

18.9.2.2 DTLS Handshake Protocol

Handshake protocol messages can be as large as 2^{16} bytes but in practice are several kilobytes. This can exceed a typical maximum UDP datagram size of 1.5KB. To handle this situation, a Handshake protocol message may span multiple DTLS records using a fragmentation procedure. Each fragment is contained in a record, which is contained in an underlying datagram. To implement fragmentation, each Handshake message contains a 16-bit Sequence Number field, a 24-bit Fragment Offset field, and a 24-bit Fragment Length field.

To perform fragmentation, the original message's content is divided into multiple contiguous data ranges. Each range is required to be less than the maximum fragment size. Each range is placed in a message fragment. Each fragment contains the same sequence number as the original message. The Fragment Offset and Fragment Length fields are expressed in bytes. Senders avoid overlapping data ranges, but receivers are required to handle this possibility because senders may be required to adjust their record size over time and retransmissions may be necessary.

To handle message loss, DTLS implements a simple timeout and retransmission capability that operates on groups of messages called flights. Figure 18-36 shows both the full (left) and abbreviated (right) establishment exchanges, along with the DTLS Handshake protocol state machine.

In Figure 18-36, flight numbers are given in the area between the full and abbreviated exchanges. The full exchange is very similar to the full TLS exchange

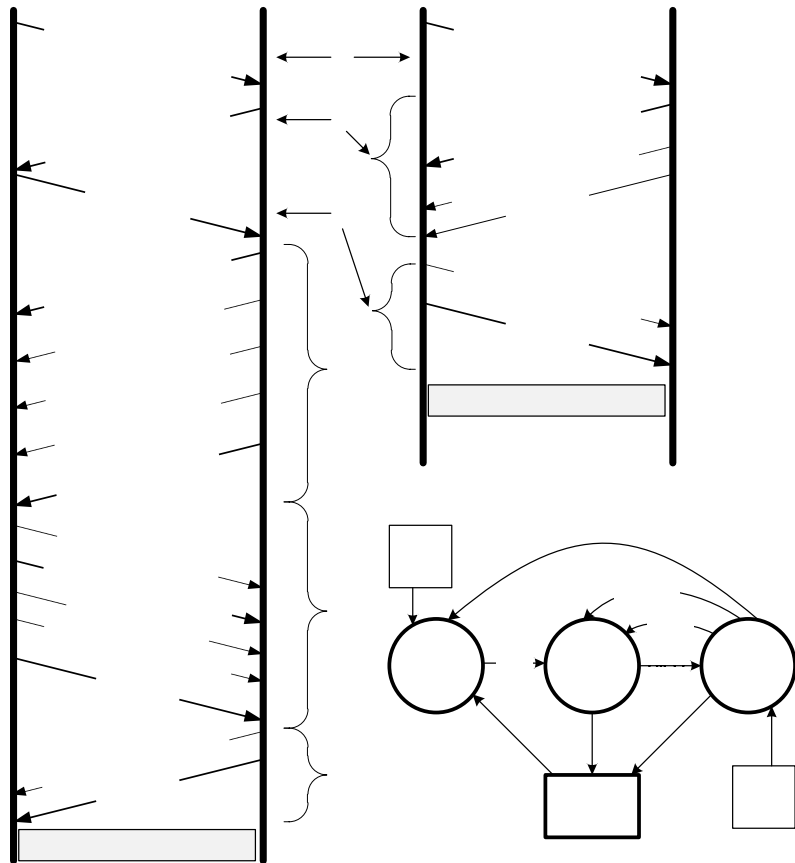


Figure 18-36 In DTLS, the possibility of lost datagrams must be handled. The initial full exchange (left) comprises six “flights” of information, each of which can be retransmitted. The DTLS abbreviated exchange (top right) uses only three and differs slightly from TLS. DTLS maintains a three-state finite state machine (bottom right) when processing the protocol.

shown in Figure 18-30, except for the additional `HelloVerifyRequest` and second `ClientHello` messages (which now contain cookies). The abbreviated exchange is different, however. In DTLS the server sends the first `Finished` message, whereas in TLS the client sends the first `Finished` message.

The lower right portion of Figure 18-36 depicts the state machine used by DTLS implementations when performing the Handshake protocol. There are three primary states: `Preparing`, `Sending`, and `Waiting`. The client starts in the `Preparing` state as it creates its `ClientHello` message. The server begins in the `Waiting`

state with no buffered messages or active retransmission timer. When sending, a retransmission timer is set and the Waiting state is entered upon completion of the transmission. Expiration of the retransmission (RTX) timer brings the protocol back to the Sending state to perform a retransmission, as does the receipt of a retransmitted flight from the peer. In this latter case the local system performs a retransmission of its own flight with the rationale that its previous transmission must have been partially or completely lost, as indicated by the presence of peer retransmission. If everything goes well, a flight is received, and the local system either finishes or returns to the Preparing state to form its next flight for transmission.

The state machine is driven by a retransmission timer with a recommended default value of 1s. If no response for a flight has been received within the timeout duration, the flight is retransmitted using the same Handshake protocol sequence numbers; record-layer sequence numbers still advance. Subsequent retransmissions without a response result in doubling of the RTX timeout value, up to a value of at least 60s. This value may be reset after a successful transmission or a long idle period (ten times the current timer value or more).

18.9.2.3 DTLS DoS Protection

When datagrams are used instead of a reliable byte stream protocol, some additional security considerations come into play. Of special concern are two potential DoS attacks. It is relatively simple for an attacker to forge a source IP address when sending a ClientHello message. Many such messages could cause a DoS attack at the DTLS server because of exhaustion of processing resources when forming responses. A variant of this attack involves having multiple attacking machines include the same forged source (victim) IP address. The responding server(s) then send(s) responses to the victim's IP address, causing the victim machine to undergo a DoS attack.

A stateless cookie validation procedure incorporated into the Hello exchange helps resist both DoS attacks. When a server receives a ClientHello message, it generates a new HelloVerifyRequest message containing a 32-bit cookie (which may be a function of a secret, the client's IP addresses, and the connection parameters). A subsequent ClientHello message must contain a copy of the appropriate cookie. Otherwise, the server refuses the exchange. This allows the server to quickly dispense with requests that do not provide valid cookies. It does not protect against coordinated attacks from multiple legitimate IP addresses that can complete the cookie exchange.

18.10 DNS Security (DNSSEC)

Now that we have discussed popular security protocols at the link, network, and transport layers, we move to the application layer. Although it is not yet widely deployed at the time of writing, we shall focus on how to provide enhanced

security for the Domain Name System (DNS). Security for DNS covers both data within the DNS (resource records or RRs) as well as security of transactions that synchronize or update contents of DNS servers. Given its important role in the operation of the Internet, a major effort has been undertaken to deploy these security mechanisms. The mechanisms are called the Domain Name System Security Extensions (DNSSEC) and are discussed in a family of RFCs [RFC4033][RFC4034][RFC4035]. These RFCs are sometimes referred to as DNSSECbis because they replace an earlier set of specifications for DNSSEC. As we explore DNSSEC in further detail, it may be worthwhile to review the description of basic DNS (see Chapter 11).

The extensions provide origin authentication and integrity assurance for DNS data, along with a (limited) key distribution facility. That is, the extensions provide a cryptographically secure way to determine what entity has authored a block of DNS information and that the information has been received unaltered. DNSSEC also provides authenticated nonexistence DNS responses indicating the nonexistence of a particular domain name include protection similar to that of responses for existing domain names. DNSSEC does not provide privacy (confidentiality) of DNS information, DoS protection, or access control. Transaction security, used with DNSSEC, is defined separately, and we will mention it briefly after discussing the core DNSSEC data security capabilities.

DNSSEC accommodates resolvers with varying levels of security “awareness.” A validating security-aware resolver (also called validating resolver) checks cryptographic signatures to ensure that the DNS data it handles is secure. Other resolvers, including stub resolvers on hosts and the “resolver side” of recursive name servers, may be security-aware but may not perform cryptographic validation. Instead, such resolvers should establish secure associations with validating resolvers. We shall focus on the validating resolvers, as they are the most sophisticated and interesting. When operating, they are able to ascertain whether DNS information is secure (valid with all signatures checked), insecure (valid signatures indicate that something should not be present but is), bogus (proper data appears to be present but cannot be validated for some reason), or indeterminate (veracity cannot be determined, usually because of lack of signatures). The indeterminate case is the default case when no other information is available.

DNSSEC works securely only when a zone is signed by a domain administrator, there is some basis for trust, and both server and resolver software participate. Validating resolvers check signatures to ensure that DNS information is secure, and they must be configured with one or more initial trust anchors that are similar to root certificates in a PKI. Note, however, that DNSSEC is not a PKI; in particular, it provides only limited signing and key revocation. It does not implement an analog to certificate revocation lists [RFC5011].

When performing a DNS query with DNSSEC, a security-aware resolver uses EDNS0 and enables the DO (DNSSEC OK) bit in an OPT meta-RR present in the request. This bit indicates the client’s interest in and ability to process DNSSEC-related information along with its support for EDNS0. The DO bit is

the first (high-order) bit of the second 16-bit field in the “extended RCODE and flags” portion of the EDNS0 meta-RR (see Section 3 of [RFC3225] and Section 4 of [RFC2671]). Servers that receive requests in which the DO bit is not set (or present) are prohibited from returning most of the RRs discussed in Section 18.10.1 unless such records are explicitly asked for in the request. This helps to improve DNS performance because it avoids having to carry security-related RRs that are never processed by security-unaware resolvers. This can be especially beneficial because DNS typically uses relatively small UDP packets and falls back to using TCP, which increases latency due to its three-way handshake, for large responses.

When a server processes a request from a DNSSEC-enabled resolver, it checks the CD (checking disabled) bit in the DNS request (see Chapter 11). If set, this indicates that the client is willing to accept nonvalidated data in a response. When preparing a response, a server ordinarily validates the data it is returning cryptographically. Successful validation results in the AD (authentic data) bit being set in the response [RFC4035]. A security-aware but nonvalidating resolver can in principle trust this information if it has a secure path to the server. However, the arguably best case is to use validating stub resolvers that perform cryptographic validation and consequently set the CD bit on queries. This provides end-to-end security of the DNS (i.e., an intermediate resolver need not be trusted), and it reduces the computational load on the intermediate servers that would otherwise have to perform cryptographic validation.

18.10.1 DNSSEC Resource Records

As specified in [RFC4034], DNSSEC uses four new resource records (RRs) and two message header bits (CD and AD). It also requires EDNS0 support and uses the DO bit field we mentioned previously. Two of the four RRs are used to contain signatures for portions of the DNS name space, and the other two are used in helping to distribute and validate keys. A change in [RFC5155] created two additional new RRs, intended to replace one of the original four.

18.10.1.1 DNS Security (DNSKEY) Resource Records

We begin by looking at how DNSSEC stores and distributes keys. DNSSEC uses the DNSKEY resource record to hold public keys. The keys are intended for use with DNSSEC only; other RRs (e.g., the CERT RR [RFC4398]) may be useful for holding keys or certificates for other purposes. The format of the RDATA portion of a DNSKEY RR is shown in Figure 18-37.

The Flagsfield in Figure 18-37 has 3 bits currently defined. Bit 7 is the Zone Key bit field. If set, the DNSKEY RR owner's name must be the name of a zone and the included key is called either a Zone Signing Key (ZSK) or a Key Signing Key (KSK). If not set, the record holds some other kind of DNS key that cannot be used for validating signatures for zones. Bit 15 is called the Secure Entry Point (SEP) bit. It is a hint that can be used by debugging or signing software to make an informed guess as to the purpose of the key. Signature validation does not interpret the SEP

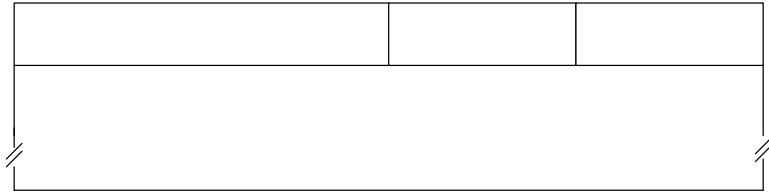


Figure 18-37 The RDATA portion of the DNSKEY RR contains a public key used only for DNSSEC. The Flags field includes a Zone Key indicator (bit 7), a Secure Entry Point indicator (bit 15), and Revoked indicator (bit 8). Generally, the zone key is set for all DNSSEC keys. If the advisory SEP bit is also set, the key is typically called a key signing key and is used for validating delegations to child zones. If not, the key is usually a zone signing key, has a shorter validity period, and is typically used to sign zone contents and not delegations. The included key is to be used with the algorithm specified in the Algorithm field.

bit, but keys with this bit set are usually KSKs and are used to secure the DNS hierarchy by validating keys in child zones (via DS records; see Section 18.10.1.2). Bit 8 is the Revoked bit [RFC5011] if set the key cannot be used for validation. The Protocol field holds the value 3 for this version of DNSSEC. The Algorithm field indicates the signing algorithm [DNSSECALG]. Only DSA and RSA with SHA-1 (values 3 and 5, respectively) are defined for use with DNSKEY RRs according to [RFC4034], but additional specifications support other algorithms (e.g., see [RFC5933] for ECC-GOST (value 12), [RFC5702] for SHA-256 (value 8)). These values are also used with several of the other DNSSEC RRs. The Public Key field holds a public key whose format depends on the Algorithm field.

18.10.1.2 Delegation Signer (DS) Resource Records

A delegation signer (DS) resource record is used to refer to a DNSKEY RR, usually from a parent zone to a descendant zone. These records are used during the authentication process to verify a public key (see Section 18.10.2). The DS RR format is shown in Figure 18-38.

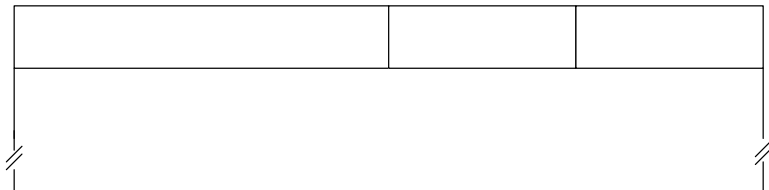


Figure 18-38 The RDATA portion of the DS RR contains a nonunique reference to a DNSKEY RR in the Key Tag field. It also contains a message digest of the DNSKEY RR and its owner, plus indications of the type of digest and algorithm.

The Key Tagfield in Figure 18-38 is a reference to a DNSKEY RR. However, it is not unique. Multiple DNSKEY RRs may have the same tag value, so the field is used only as a search hint (confirming that validation is still necessary). The value for this field is computed as the 16-bit unsigned sum of data comprising the referenced DNSKEY RR RDATA area (carries are ignored) as shown in Figure 18-37. The Algorithm field uses the same values as the DNSKEY RR Algorithm field. The Digest Typefield indicates the type of signature used. Only value 1 (SHA-1) is defined by [RFC4034], but SHA-256 (value 2) is specified for use by [RFC4509]. The current list is contained in the DS RR Type Digest Algorithms registry [DSRRYPES]. The Digestfield contains the digest of the DNSKEY RR being referenced. More specifically, the digest is computed as follows:

$$\text{digest} = \text{digest_algorithm}(\text{DNSKEY owner name} \mid \text{DNSKEY RDATA})$$

where $\mid$ is the concatenation operator and the DNSKEY RDATA value is computed from the referenced DNSKEY RR as follows:

$$\text{DNSKEY RDATA} = \text{Flags} \mid \text{Protocol} \mid \text{Algorithm} \mid \text{PublicKey}$$

For the case of SHA-1, the digest is 20 bytes in length. For SHA-256 it is 32 bytes. The DS RR is used to provide a downward link in the authentication chain across zone boundaries, so the referenced DNSKEY RR must be a zone key (i.e., bit 7 of the Flagsfield in the DNSKEY RR must be set).

Note

At the time of writing, a variant of the DS RR called DS2 is under consideration [IDDS2]. It introduces a Canonical Signer Name to the DS RR so that multiple zones with identical content can be named differently and signed by multiple (different) signers. In addition, there is a DLV RR [RFC4431] that has been used to provide delegations in cases where a parent zone is not signed or has not published DS RRs. The format of a DLV RR is identical to that of a DS RR; only the interpretation differs.

18.10.1.3 NextSECure (NSEC and NSEC3) Resource Records

Now that we have seen the RRs needed to hold and securely refer to keys, we move on to the records used to validate the structure of a zone and the resource records it contains. The NextSECure (NSEC) RR is used to hold the “next” RRset owner’s domain name in the canonical ordering of names (see Section 18.10.2.1) or a delegation point NS type RRset. (Recall, an RRset is a set of RRs with the same owner, class, TTL, and type but with different data.) It also holds a list of RR types present at the NSEC RR owner’s name. This provides authentication and integrity verification for the zone structure. The format of an NSEC RR is shown in Figure 18-39.



Figure 18-39 The RDATA portion of the NSEC RR contains the name of the next RRset owner for the zone in canonical order. It also contains an indication of which RR types were present at the NSEC RR owner's domain name.

The NSEC RR is used to form a chain of names corresponding to RRsets within a zone. Consequently, an RRset not present in the chain can be shown to not exist. This provides the authenticated denial of existence feature mentioned previously. The Next Domain Name field holds the next entry in the canonically ordered domain name chain for the zone without using the domain name compression technique described in Chapter 11. The value of this field for the last NSEC record of the chain is the zone apex (the owner name of the zone's SOA RR).

The Type Bit Map field of the NSEC RR holds a bitmap of RR types present at the NSEC RR owner's domain name. There is a maximum of 64K possible types, about 100 of which have been defined to date [DNSPARAMS]. Only a fraction of these are in widespread use. For example, the Internet's root zone (domain name "."), which became operational with DNSSEC on July 15, 2010, contains a Next Domain Name field of `ac` (a `ccTLD`) and a bitmap indicating the presence of records of the following types: NS, SOA, RRSIG, NSEC, and DNSKEY.

To encode the presence of a type, the whole space of RR types is divided into 256 "window blocks," numbered 0 through 255. For each block number, the presence of up to 256 RR types can be encoded using a bit mask. Given a block number N and bit position P , the corresponding RR type number is $(N * 256 + P)$. For example, in block 1, bit position 2 corresponds to RR type 258 (a type not currently defined). The field is encoded as follows:

Type Bit Map = (window block number | bitmap length | bitmap) *

where `|` is the concatenation operator and `*` represents Kleene closure (i.e., zero or more). Each instance of the window block number contains a value in the range 0–255, and the bitmap length contains the length of the corresponding bitmap in bytes (maximum value 32). The window block number and bitmap length are each single bytes, and the bitmap can be as long as 32 bytes (256 bits, one for each possible RR type in the window). Blocks in which no RR type is present are not included. The encoding is optimized for a sparse presence of types across blocks. For example, if only RR types 1 (A) and 15 (MX) were present, the encoding for the field would be as follows: `0x00024001 = (0x00 | 0x02 | 0x4001)`.

The original structure of NSEC records defined in [RFC4034] creates a situation in which anyone is able to enumerate the authority records in a zone by walking the NSEC chain, called zone enumeration. This is an unwanted opportunity for “leakage” of information for many deployments. As a result, a pair of RRs, intended to replace NSEC, is defined in [RFC5155]. The first is called NSEC3. It uses cryptographic hashes of RR owner domain names rather than unencoded domain names. The format is shown in Figure 18-40.

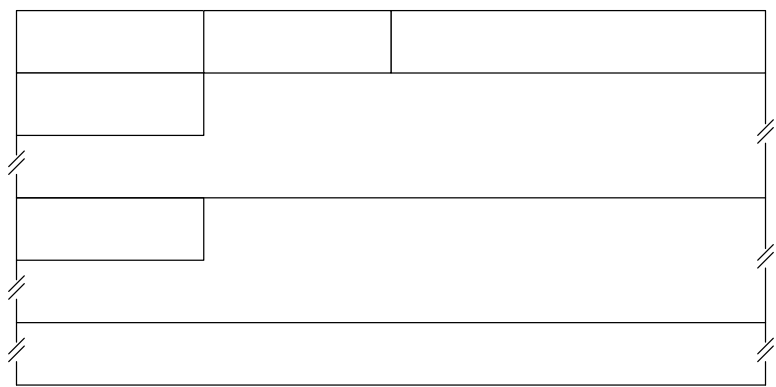


Figure 18-40 The RDATA portion of the NSEC3 RR contains a hash of the name of the next RRset owner for the zone in canonical order. The hash function has been applied the number of times specified in the Iterations field. The variable-length Salt value is appended to the name prior to applying the hash function to provide dictionary attack resistance. The Type Bit Maps field uses the same structure as NSEC RRs. NSEC3PARAM records are similar but contain only the hash parameters (not the Next Hashed Owner or Type Bit Maps fields).

In the NSEC3 record, the Hash Algorithm field identifies the hash function applied to the next owner name to produce the Next Hashed Owner field. Only SHA-1 (value 1) is defined to date [NSEC3PARAMS]. The low-order bit of the Flags field contains an opt-out flag. If set, it indicates that the NSEC3 record may cover unsigned delegations. This is used in cases where a delegation (NS RRset) refers to a child zone that is not required to be or is not desired to be signed. The Iterations field indicates how many times the hash function has been applied. A larger number of iterations may help to protect against finding the owner names corresponding to hash values found in NSEC3 records (dictionary attacks). The Salt Length field gives the length of the Salt field in bytes. The Salt field contains a value appended to the original owner name prior to computing the hash function. Its purpose is to help thwart dictionary attacks.

The second RR specified by [RFC5155] is called the NSEC3PARAM RR (not shown separately). It uses the same format as the NSEC3 RR, except the hash

Length Next Hashed Owner and Type Bit Maps fields are not present. It is used by an authoritative name server when choosing NSEC3 records to use in a negative response. The NSEC3PARAM RR provides the parameters needed for computing a hashed owner name.

To obtain the hash value for the Next Hashed Owner field, the following computation is performed:

$$\begin{aligned} \text{IH}(0) &= \text{H}(\text{owner name} \parallel \text{Salt}) \\ \text{IH}(k) &= \text{H}(\text{IH}(k-1) \parallel \text{Salt}) \text{ if } k > 0 \\ \text{Next Hashed Owner} &= \text{H}(\text{IH}(\text{Iterations}) \parallel \text{Salt}) \end{aligned}$$

where H is the hash function specified in the Hash Algorithm field and the owner name is in canonical form. The iterations and salt values are taken from the corresponding fields of the NSEC3 RR.

To avoid confusion between NSEC and NSEC3 RR types, [RFC5155] allocates and requires the use of special security algorithm numbers 6 and 7 as aliases for identifiers 3 (DSA) and 5 (SHA-1) in zones employing NSEC3 RRs. Resolvers unaware of the NSEC3 record type receiving these values treat the resulting records as insecure. This provides a certain limited form of backward compatibility (i.e., failing, but doing so without incorrectly interpreting RR data).

18.10.1.4 Resource Record Signature (RRSIG) Resource Records

Moving from the DNS structure to its contents, we require a way to provide origin authentication and integrity protection for RRs. DNSSEC signs and validates signatures on RRsets using the Resource Record Signature (RRSIG) RR, and every authoritative RR in a zone must be signed (glue records and delegation NS records present in parent zones aren't). An RRSIG RR contains a digital signature for a particular RRset, along with information to identify which public key can be used to validate the signature, as shown in Figure 18-41.

The Type Covered field indicates the type of the RRset to which the signature applies. The value is taken from the standard set of RR types in [DNSPARAMS]. The Algorithm field indicates the signing algorithm. Only DSA and RSA with SHA-1 (values 3 and 5, respectively) are defined for use with RRSIG RRs according to [RFC4034], but [RFC5702] covers SHA-2 algorithms and [RFC5933] covers GOST algorithms (from the Russian Federation). The Labels field gives the number of labels in the original owner's name of the RRSIG RR. The Original TTL field holds a copy of the TTL from the RRset as it appears in the authoritative zone (caching name servers may reduce the TTL). The Signature Expiration and Signature Inception fields indicate the starting and ending validity times for the signature, expressed in seconds since January 1, 1970, 00:00:00 UTC. The Key Tag field helps to identify the DNSKEY RR that can be used to obtain the public key necessary to validate the signature contained in the Signature field, using the format described previously for the DS RR.

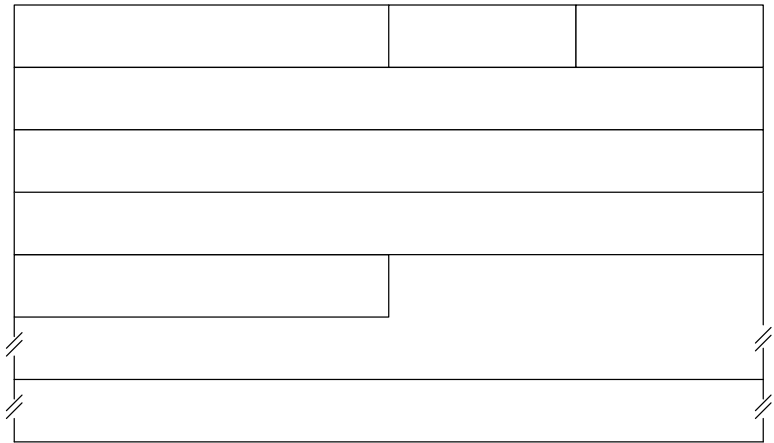


Figure 18-41 The RDATA portion of the RRSIG RR contains a signature for an RRset. The TTL of the RRset as it appears in the authoritative zone is also included, along with indicators of the algorithm and signature validity period. The Key Tag field refers to a DNSKEY RR containing a public key that can be used to validate the signature. The Labels field indicates how many labels constitute the original owning name of the RR.

18.10.2 DNSSEC Operation

Now that we have covered all the RRs required by DNSSEC, we can see how to use DNSSEC to secure zones. We shall first require the definition of a canonical ordering mentioned earlier when defining the NSEC and NSEC3 record types. The purpose of a defined canonical ordering for a zone is to be able to enumerate a zone's contents in a reproducible way that can be signed (different orders of the same contents would produce different values for any good hash function). Once we are familiar with the ordering, we look at how a zone is signed and how signed records describing a zone are validated.

18.10.2.1 Canonical Orderings and Forms

There are three canonical orderings of interest to us: the canonical name order within a zone, the canonical form for a single RR, and the canonical ordering of an RRset [RFC4034]. Recall from Chapter 11 that each RR has an owner name (owner's domain name) consisting of labels. By treating each label in a name as a left-justified string of bytes and treating uppercase US-ASCII letters as lowercase, we can form a list of names. We first sort the names by their most significant (right-most) label, then by the next most significant label, and so on. The absence of a byte sorts before a zero-value byte. A valid canonical ordering would be `com`, `company.com`, `*.company.com`, `UK.company.COM`, `usa.company.com`. Wild-cards can be used.

For a particular RR, there is a well-defined canonical form. This form requires the RR to adhere to the following rules:

1. Every domain name is an FQDN and fully expanded (no compression labels).
2. All uppercase US-ASCII letters in the owner name are replaced by the corresponding lowercase versions.
3. All uppercase US-ASCII letters are replaced by their lowercase versions for any domain names present in the RDATA portion of records with type numbers 2–9, 12, 14, 15, 17, 18, 21, 24, 26, 33, 35, 36, 39, and 38.
4. Any wildcards (*) are not substituted.
5. The TTL is set to its original value as it appeared in the originating authoritative zone or the Original TTL field of the covering RRSIG RR.

Note

A number of clarifications and important changes are being applied to the baseline DNSSECbis family of documents. The reader is encouraged to consult the most recent version of [IDDCIN] for further details.

The canonical order of the RRs within an RRset follows essentially the same rule as for owner names but applies to an RR's RDATA contents in canonical form treated as a left-justified byte string.

18.10.2.2 Signed Zones and Zone Cuts

DNSSEC depends on signed zones. Such zones include RRSIG, DNSKEY, and NSEC (or NSEC3) RRs and may contain DS RRs if there is a signed delegation point. Signing makes use of public key cryptography where the public keys are stored in and distributed by the DNS. Figure 18-42 shows an abstract delegation point between a parent and child zone.

In the figure, the parent zone contains its own DNSKEY RR, which provides the public key corresponding to the private key used to sign all authoritative RRsets in the zone using RRSIG RRs (multiple DNSKEYs are possible). A DS RR in the parent provides a hash of one of the DNSKEY RRs in the child's apex. This establishes a chain of trust from the parent to the child. A validating resolver that trusts the parent's DS RR can validate the child's DNSKEY RR and ultimately the RRSIGs and signed RRsets within the child zone. This happens only if the validator has a root of trust that can be connected to the parent's DNSKEY RR.

18.10.2.3 Resolver Operation Example

Given a chain of signed zones and a security-aware validating resolver, we can see how the contents of a DNS response can be validated. In the best case, a zone

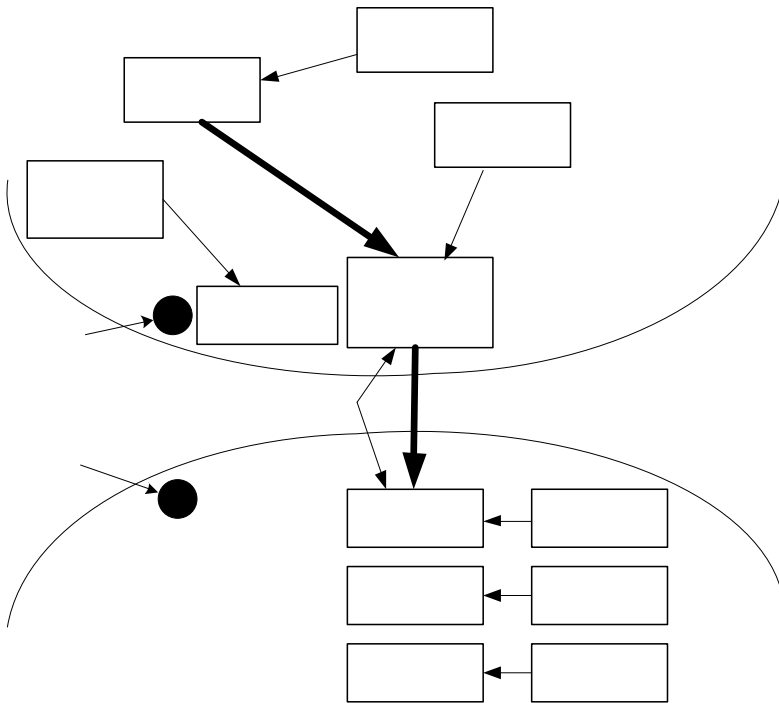


Figure 18-42 A zone cut for an authenticated delegated zone includes a DS RR in the parent containing a hash of the DNSKEY RR(s) in the child. All RRsets are signed with corresponding RRSIG RRs except the delegation NS RRs (and glue records) in the parent. NSEC RRs can be used to verify the types present in the zone and include an SOA RR type indication at the apex in the child zone.

can be reached through a chain of trust from the root zone. ICANN keeps a list of which zones have been enabled for DNSSEC by having DS records present in the root zone and signed DNSKEY RRs [TLD-REPORT].

Assume that we wish to resolve and verify an A RR type for the domain name `www.icann.org`. Proceeding from the root downward, we shall at first require the root's trust anchor (i.e., DNSKEY RRs), DS records for `org.` contained in one of the root name servers, and perhaps RRSIG and NSEC (NSEC3) records. We then repeat the process using `org.` and `icann.org.` domain names and corresponding DNS servers. We begin with the root zone:

```
Linux% dig @a.root-servers.net. . dnskey +noquestion +nocomments \
+nostats +multiline
;; Truncated, retrying in TCP mode.
; <<>> DiG 9.7.2-P3 <<>> @a.root-servers.net. . dnskey
+noquestion +nocomments +nostats +multiline
; (1 server found)
;; global options: +cmd
. 86400 IN      DNSKEY      257 3 8 ( AwEAAgAIKl ... ) ; key id = 19036
```

```
. 86400 IN      DNSKEY      256 3 8 ( AwEAAb5gVAz ... ) ; key_id = 21639
. 86400 IN      DNSKEY      256 3 8 ( AwEAAcAPhPM ... ) ; key_id = 40288
```

Here we can see the trust anchor for the root zone, which constitutes the root of trust for all DNSSEC in the Internet. The first key is a KSK, indicated by the value 257 (SEPbit is 1), which is the preferred one used in forming trust chains. The others are marked as ZSKs. Next, we would like to ensure that all the records we have just seen are supposed to be present and have appropriate signatures. The root's RRSIG records of interest can be seen as follows:

```
Linux% dig @a.root-servers.net. . rrsig +noquestion +nocomments \
+nostats +noauthority +noadditional
;; Truncated, retrying in TCP mode.

; <<>> DiG 9.7.2-P3 <<>> @a.root-servers.net. . rrsig +noquestion
+nocomments +nostats +noauthority +noadditional
; (1 server found)
;; global options: +cmd
. 86400 IN      RRSIG NSEC 8 0 86400 20101228000000 20101220230000
40288 . RyoGB1dxxX...
. 86400 IN      RRSIG DNSKEY 8 0 86400 20110105235959 20101221000000
19036 . f8bZNvPmHR...

...
```

The RRSIG covering the DNSKEY record uses key tag 19036, which matches the KSK contained in the root zone's DNSKEY RR. The root contains other RRSIG records (for its SOA and NS records), but we are more concerned with the RRSIGs for the DNSKEY and NSEC RRs. Just to be extra-sure that the DNSKEY RR should be present, we can inspect the root's NSEC RR to verify that its type is present:

```
Linux% dig @a.root-servers.net. . nsec +noquestion +nocomments \
+nostats +noauthority +noadditional
; <<>> DiG 9.7.2-P3 <<>> @a.root-servers.net. . nsec +noquestion
+nocomments +nostats +noauthority +noadditional
; (1 server found)
;; global options: +cmd
. 86400 IN      NSEC ac. NS SOA RRSIG NSEC DNSKEY
```

This confirms that the root zone officially contains RRset types NS, SOA, RRSIG, NSEC, and DNSKEY, so we are in good shape so far. (Note also that `ac.` is the first TLD in the canonical ordering of the root zone.) Next we need to check out the signatures on the delegation from the root to `org.`. This can be done as follows:

```
Linux% dig @a.root-servers.net. org. rrsig +noquestion +nocomments \
+nostats +noadditional +dnssec
; <<>> DiG 9.7.2-P3 <<>> @a.root-servers.net. org. rrsig +noquestion
+nocomments +nostats +noadditional +dnssec
```

```

; (1 server found)
;; global options: +cmd
org. 172800      IN      NS      d0.org.afiliias-nst.org.
org. 172800      IN      NS      b2.org.afiliias-nst.org.
org. 172800      IN      NS      a0.org.afiliias-nst.info.
org. 172800      IN      NS      b0.org.afiliias-nst.org.
org. 172800      IN      NS      a2.org.afiliias-nst.info.
org. 172800      IN      NS      c0.org.afiliias-nst.info.
org. 86400       IN      DS      21366 7 2 96EEB2FFD9 ...
org. 86400       IN      DS      21366 7 1 E6C1716CFB ...
org. 86400 IN     RRSIG DS 8 1 86400 20101228000000 20101220230000
                        40288 . jpcJ0Gclvvlx9Kvz5 ...

```

The presence of the DS RRset and its associated RRSIG suggests that indeed there is a DNSSEC secured delegation. The RRSIG RR contains the key tag 40288, which refers to the third DNSKEY RR we saw earlier for the root zone (the ZSK). The NS records provide us with the names of the next servers to use in the next steps for our query. We can proceed by repeating the queries we made for the root, but this time using `org.` We direct such queries at one of the servers specified in the NS RR for `org.` in the root:

```

Linux% dig @d0.org.afiliias-nst.org. org. dnskey +dnssec +nostats \
+noquestion +multiline
; <<>> DiG 9.7.2-P3 <<>> @d0.org.afiliias-nst.org. org. dnskey +dnssec
      +nostats +noquestion +multiline
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 8061
;; flags: qr aa rd; QUERY: 1, ANSWER: 6, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags: do; udp: 4096
;; ANSWER SECTION:
org. 900 IN  DNSKEY   256 3 7 ( AwEAAZTERUF ... ) ; key id = 1743
org. 900 IN  DNSKEY   256 3 7 ( AwEAAazTpnm ... ) ; key id = 43172
org. 900 IN  DNSKEY   257 3 7 ( AwEAAyPyfj3 ... ) ; key id = 21366
org. 900 IN  DNSKEY   257 3 7 ( AwEAAZTjbIO ... ) ; key id = 9795
org. 900 IN  RRSIG DNSKEY 7 1 900 20101231154644
                        20101217144644 21366 org.
                        aIZgEsoJ0+Q8ZXM ...
org. 900 IN  RRSIG DNSKEY 7 1 900 20101231154644
                        20101217144644 43172 org. MwwosWBdEmM8CiM ...

```

Here we can see that four DNSKEY RRs exist, two of which are KSKs (value 257) and two of which are ZSKs (value 256). The third one listed (21366) corresponds to the DS RR we found located in the root zone. The RRSIG RRs use this key, plus the ZSK with ID 43172. To verify their presence as legitimate, we can look for NSEC or NSEC3 records that may be present for `org.`:

```
Linux% dig @d0.org.afiliast-nst.org. org. nsec +dnssec +nostats \
+noquestion
; <<>> DiG 9.7.2-P3 <<>> @d0.org.afiliast-nst.org. nsec org. +dnssec
      +nostats +noquestion
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 61632
;; flags: qr aa rd; QUERY: 1, ANSWER: 0, AUTHORITY: 4, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags: do; udp: 4096
;; AUTHORITY SECTION:
h9p7u7tr2u91d0v0ljs9l1gidnp90u3h.org. 86400 IN NSEC3 1 1 1
      D399EAAB
      H9RSFB7FPF2L8HG35CMPC765TDK23RP6
      NS SOA RRSIG DNSKEY NSEC3PARAM
h9p7u7tr2u91d0v0ljs9l1gidnp90u3h.org. 86400 IN RRSIG NSEC3 7 2
      86400 20110105003654
      20101221233654
      43172 org. eBtNa4fok ...
```

Here we see an NSEC3 record with owner name equal to the hashed version of org.. It indicates the presence of a DNSKEY and RRSIG record, as well as NS and NSEC3PARAM records. Following the last type, we can determine the NSEC3 information:

```
Linux% ./dig @a0.org.afiliast-nst.info. org. nsec3param +dnssec \
+nostats +noadditional +noauthority +noquestion
; <<>> DiG 9.7.2-P3 <<>> @a0.org.afiliast-nst.info. org. nsec3param
      +dnssec +nostats +noadditional +noauthority +noquestion
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 38602
;; flags: qr aa rd; QUERY: 1, ANSWER: 2, AUTHORITY: 7, ADDITIONAL: 13
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags: do; udp: 4096
;; ANSWER SECTION:
org.          900    IN      NSEC3PARAM 1 0 1 D399EAAB
org.          900    IN      RRSIG NSEC3PARAM 7 1 900 20101231154644
      20101217144644 43172 org. fS2kFw53e1Y ...
```

We can see that this NSEC3PARAM RR matches the NSEC3 RR because of the match of the value D399EAAB (signature). We can also see that the signature in the RRSIG RR came from the private key associated with DNSKEY having ID 43172. If all signatures match, so far we have a valid chain of trust. To complete the chain, we need information about icann.org.:

```
Linux% dig @a0.org.afiliass-nst.info. icann.org. any +dnssec +nostats \
+noadditional
; <<>> DiG 9.7.2-P3 <<>> @a0.org.afiliass-nst.info. icann.org. any
+dnssec +nostats +noadditional
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 61234
;; flags: qr rd; QUERY: 1, ANSWER: 0, AUTHORITY: 8, ADDITIONAL: 3
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
;icann.org.                IN      ANY

;; AUTHORITY SECTION:
icann.org.                86400 IN     NS      a.iana-servers.net.
icann.org.                86400 IN     NS      b.iana-servers.org.
icann.org.                86400 IN     NS      c.iana-servers.net.
icann.org.                86400 IN     NS      d.iana-servers.net.
icann.org.                86400 IN     NS      ns.icann.org.
icann.org.                86400 IN     DS      41643 7 1 93358DB ...
icann.org.                86400 IN     DS      41643 7 2 B8AB67D ...
icann.org.                86400 IN     RRSIG   DS 7 2 86400 20101231154644
                        20101217144644 43172 org. cZ1Z30w// ...
```

We can see the DS RR indicating the signed delegation for icann.org. from org.. The RRSIG for the DS RRset is signed based on the ZSK with ID 43172. Using one of the servers present in the NS records, we can look at the final server:

```
Linux% dig @a.iana-servers.net. icann.org. dnskey +dnssec +nostats \
+noquestion +multiline
; <<>> DiG 9.7.2-P3 <<>> @a.iana-servers.net. icann.org. dnskey +dnssec
+nostats +noquestion +multiline
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 22065
;; flags: qr aa rd; QUERY: 1, ANSWER: 5, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
;; ANSWER SECTION:
icann.org. 3600 IN   DNSKEY 256 3 7 ( AwEAAbDmrVc ... ) ; key id = 41295
icann.org. 3600 IN   DNSKEY 256 3 7 ( AwEAAbgrYZd ... ) ; key id = 55469
icann.org. 3600 IN   DNSKEY 257 3 7 ( AwEAAZuSdr4 ... ) ; key id = 7455
icann.org. 3600 IN   DNSKEY 257 3 7 ( AwEAAcyguBH ... ) ; key id = 41643
icann.org. 3600 IN   RRSIG   DNSKEY 7 2 3600 20101229153632
                        20101222042536 41643 icann.org.
                        UxR/5vy0IS ...
```

Here we can see that four DNSKEY RRs exist—two KSKs and two ZSKs. The fourth one listed (41643) corresponds to the DS RR we found located in the `org.` zone. The RRSIG RR uses this key. To find the answer to our ultimate query, we request the A record:

```
Linux% dig @a.iana-servers.net. www.icann.org. a +dnssec +nostats \
+noquestion +noauthority +noadditional
; <<>> DiG 9.7.2-P3 <<>> @a.iana-servers.net. www.icann.org. a +dnssec
      +nostats +noquestion +noauthority +noadditional
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->HEADER<<- opcode: QUERY, status: NOERROR, id: 56258
;; flags: qr aa rd; QUERY: 1, ANSWER: 2, AUTHORITY: 6, ADDITIONAL: 3
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
;; ANSWER SECTION:
www.icann.org.      600    IN      A        192.0.32.7
www.icann.org.      600    IN      RRSIG A 7 3 600 20101229143630
                    20101222042536 55469 icann.org.
                    YRhLL/RA ...
```

We have finally reached the end of the chase for the A RR for `www.icann.org.` It contains the IP address `192.0.32.7`, signed by an RRSIG RR using key ID 55469. This is the key from the fourth DNSKEY RR we saw at the apex of the `icann.org.` zone. So at this point it would appear that all is order. However, we have not demonstrated that all the signature values are actually correct. To do this validation, the following command may be executed:

```
Linux% dig @a.root-servers.net. www.icann.org. a +sigchase +topdown \
+trusted-key=trusted-keys
```

This command works if the `dig` program has been compiled with the `-DDIG_SIGCHASE=1` compile-time option and the file `trusted-keys` contains the root's DNSKEY RRset. After many lines of output, we find that it does indicate success. A simpler method for checking the validity can be achieved using a DNS/DNSSEC-checking Web site such as <http://dnsviz.net>. Output from such a query is shown in Figure 18-43.

Here we can see a successful validation for the A and AAAA RR types for the domain name `www.icann.org.` Each rectangle represents a zone and contains its name and the time it was analyzed. Within each zone are ovals representing elements in the chain of trust, either DNSKEY or DS RRs. Dashed ovals indicate that the keys are not being used for signatures of interest. Arrows between ovals indicate RRSIG or DS digests. Two types of algorithms are represented. In the root

Figure 18-43 A visualization of a DNSSEC chain of trust. Rectangles represent zones. Ovals represent chain nodes, and shaded ovals have the SEP bit set. Arrows indicate valid RRSIG records or DS digests. The double-circle oval indicates a trust anchor.

zone, “alg = 8” indicates that RSA/SHA-256 [RFC5702] signatures are in use. In other zones, “alg = 7” indicates RSA/SHA-1 that permits the use of NSEC3 records [RFC5155]. For the DS RR in the root zone, “digest algs = 1,2” indicates that SHA-1 [RFC4034] and SHA-256 [RFC4509] are supported.

18.10.3 Transaction Authentication (TSIG, TKEY, and SIG(0))

Some transactions in DNS, such as zone transfers and dynamic updates, could compromise the DNS structure or contents if improperly used. Consequently, they require some form of authentication. Even conventional DNS resolution may require authentication if a resolver expects to depend on validated DNS resolutions but does not implement full DNSSEC processing. With transaction authentication, the exchange between a particular resolver and server (or between servers) is protected. Note, however, that transactional security does not directly protect the contents of the DNS, as does DNSSEC. As a result, DNSSEC and transaction authentication are complementary and can be deployed together. DNSSEC provides data origin authentication and integrity of zone data, while transaction authentication provides integrity and authentication for a particular transaction between a client and a server without checking the correctness of the content being exchanged.

There are two primary methods for authenticating DNS transactions: TSIG and SIG(0). TSIG uses shared keys and SIG(0) uses public/private key pairs. To help ease the burden of deployment, a TKEY RR type can be used to help form keys (e.g., by holding public DH values) for either TSIG or SIG(0). We will begin by discussing TSIG, the more common of the transaction security mechanisms.

18.10.3.1 TSIG

Secret Key Transaction Authentication for DNS (TSIG) [RFC2845] adds transactional authentication for DNS exchanges using signatures based on shared secret keys. TSIG makes use of a TSIG pseudo-RR that is computed on demand and is used only to secure a single transaction. The format of the RDATA portion of a TSIG pseudo-RR is shown in Figure 18-44.

The figure shows the format of a TSIG pseudo-RR. Such RRs are sent in the additional data section of a DNS request or response. The original MAC algorithm specified in [RFC2845] was based on HMAC-MD5, but newer GSS-API (Kerberos) [RFC3645] and SHA-1- and SHA-256-based algorithms have since been specified in [RFC4635]; the current list is available at [TSIGALG]. The algorithm names were envisioned to be encoded as domain names (e.g., HMAC-MD5.SIG-ALG.REG.INT), but now most use descriptive strings (e.g., hmac-sha1, hmac-sha256). The 48-bit Signed Timefield is in UNIX time format (seconds since January 1, 1970, UTC) and gives the time the message contents were signed. This field is covered in the digital signature and is designed to detect and prevent replay attacks. The consequence of using an absolute time here is that peers using TSIG must agree on the time to within the number of seconds specified by the Fudgefield. The MAC

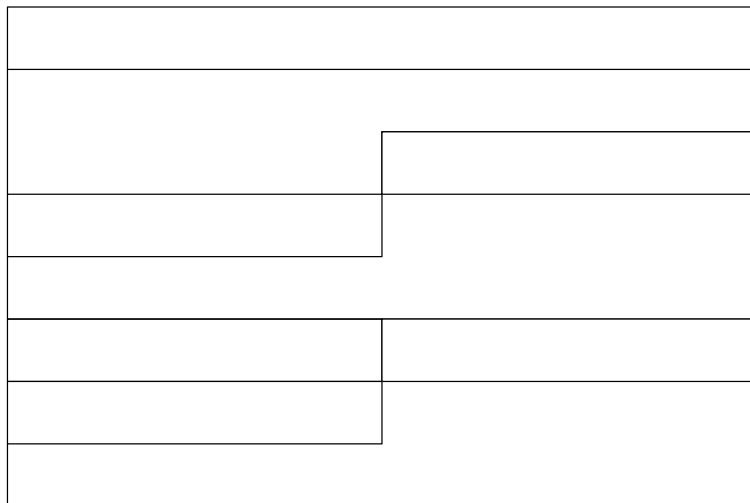


Figure 18-44 The TSIG pseudo-RR RDATA area contains a signature algorithm ID, signature time and time fudge factor, and a MAC. Originally, only an MD5-based signature was used, but now SHA-1- and SHA-2-based signatures have been standardized. TSIG peers must be time-synchronized to within the number of seconds in the Fudgefield. TSIG RRs are carried in the additional data section of a DNS message.

Sizefield gives the number of bytes required to contain the MAC in the MAC field and depends on the particular MAC algorithm. The Other Lengthfield gives the size of the Other Datafield in bytes, which is used only in carrying error messages.

To see TSIG in action, we can construct a sample zone called `dynzone.` and perform a signed dynamic update. We use the `nsupdate` program supplied with BIND9 to perform the update:

```
Linux% nsupdate
> zone dynzone.
> server 127.0.0.1
> key tsigkey.dynzone. 1234567890abcdef
> update delete two.dynzone.
> send
```

This series of instructions forms a DNS update message signed using TSIG that is sent to the server once the `send` instruction is issued. The request is shown in Figure 18-45.

In this figure, a dynamic DNS update request has been signed using the HMAC-MD5 signature algorithm. The signing key's name is `tsigkey.dynzone.`. The request is to update the zone `dynzone.` by removing the entry `two.dynzone.`. The name of the signature algorithm is `HMAC-MD5.SIG-ALG.REG.INT`, which is

Figure 18-45 A DNS dynamic update signed using TSIG. The request is to delete the RR `fortwo.dynzone..` The request is signed using the key with name `tsigkey.dynzone..` The signature algorithm is HMAC-MD5, which produces a 128-bit (16-byte) signature.

the only signature algorithm supported by this particular software package. Note that the Original ID field (15746 decimal) matches the value of the Transaction ID field (0x3d82). The response confirms that the update was successful, as shown in Figure 18-46.

Figure 18-46 shows a successful response to a DNS dynamic update request signed using TSIG. The Flags field indicates that a dynamic update response contains no errors. Once again, the TSIG pseudo-RR is contained in the additional information area.

Figure 18-46 A DNS dynamic update response signed using TSIG. The RRsettwo.dynzone. has been successfully removed using dynamic update.

18.10.3.2 SIG(0)

Early versions of DNSSEC included signature (SIG) resource records that correspond to the modern RRSIG RRs discussed previously. However, a particular kind of SIG RR called SIG(0) [RFC2931] does not cover static records in the DNS but instead is generated dynamically for transactions. The 0 part of SIG(0) refers to the length of data within an RR covered by the signature. As a result, SIG(0) records

can in principle be used instead of TSIG RRs to achieve the same result. However, they are implemented in different ways. Most importantly, SIG(0) places its basis of trust in public keys instead of shared keys. SIG(0) appears to be shrinking in popularity in favor of TSIG, so we do not discuss it further.

18.10.3.3 TKEY

The TKEY meta-RR type is intended to simplify the deployment of DNS transaction security such as TSIG and SIG(0) [RFC2930]. To do this, TKEY RRs are dynamically created and sent in the additional information section of DNS requests and responses. They can contain either keys or material used to form keys such as DH public values. It may be useful in local deployments but is not in widespread use.

18.10.4 DNSSEC with DNS64

In Chapter 11 we described DNS64, which translates IPv6 DNS requests into IPv4 DNS requests and can synthesize AAAA records based on A records found in the IPv4 DNS. The scheme is useful for allowing IPv6-only hosts to access IPv4 servers and services. DNS64 works by synthesizing AAAA records. With DNSSEC, however, DNS RRs need to be signed by the signing authority (typically the domain name owner or zone administrator). This presents a challenge: How can DNS64 synthesize RRs if it lacks the keys to produce DNSSEC-compatible signatures? The answer is, essentially, that it does not (see Sections 5.5 and 6.2 in [RFC6147]).

To operate DNS64 in conjunction with DNSSEC, the validation function is performed either in the host (where DNS64 could be implemented) or by the DNS64 device, assuming there exists a secure channel between a stub resolver and the DNS64 acting as a recursive name server. A validating DNS64 is known as vDNS64. A vDNS64 interprets the CD and DO bits in an incoming query. If neither is set, the vDNS64 performs synthesis and validation but does not set the AD bit in the (validated) response. If the DO bit is set and the CD bit is not, the vDNS64 performs validation and synthesis and returns a validated response with the AD bit set (which the client presumably interprets as meaning that the returned RRs are authentic). Note that the DNS64 first requests AAAA records on the IPv4 side and synthesizes A records only when it can validate that no AAAA records with the same owner exist. If both the DO and CD bits are set, the DNS64 may perform validation but not synthesis. In this case, it is presumed that the client will perform validation. This case represents a potential problem because if the client is security-aware but translation-oblivious, the returned RRs will probably not be usable in the IPv6 addressing realm.

18.11 DomainKeys Identified Mail (DKIM)

DomainKeys Identified Mail (DKIM) [RFC5585] is intended to provide an association between an entity and a domain name that can be used to help determine the party responsible for originating a message, especially in the e-mail context.

It provides a method to help authenticate the signer of a message, which is not necessarily the sender, and this can be used in helping to fight spam at the e-mail distribution level (i.e., between mail agents). This is accomplished by adding a DKIM-Signature field to the basic Internet message format [RFC5322]. This field contains a digital signature of the header and body of the message. DKIM replaces an earlier standard called DomainKeys, which uses the DomainKey-Signature field.

18.11.1 DKIM Signatures

To produce a digital signature for a message, a Signing Domain Identifier (SDID) uses RSA/SHA-1 or RSA/SHA-256 and an associated private key. SDIDs are domain names from the DNS and are used to retrieve public keys stored as TXT RRs. A DKIM signature is encoded as a message header field using Base64 (such as PEM) that signs an explicitly listed set of message fields and the message body. When receiving an e-mail, for example, a mail transfer agent uses the SDID to perform a DNS query to find the corresponding public key, which it then uses to verify the signature. This avoids requiring a PKI. The owning domain name is constructed from the domain itself along with the selector (public key selector). For example, the public key for the selector key35 in domain example.com would be a TXT RR owned by key35._domainkey.example.com.

The DKIM-Signature field [RFC6376] is added to a message header and may contain several subfields (see [DKPARAMS] for the complete list). The operation of DKIM is conceptually similar to the DNS Sender Policy Framework (SPF; see Chapter 11) but is stronger because of the cryptographic digital signature. DKIM and SPF can be used together.

DKIM-enabled domains may elect to participate in Author Domain Signing Practices (ADSP) [RFC5617]. ADSP involves the creation of a machine-readable signing practices statement for a domain. Such records are placed in the DNS using TXT RRs with owner name equal to _adsp._domainkey.domain.. At present ADSP records are simple and indicate only how the authoring domain uses DKIM signatures. The values may be unknown, all, or discardable. These are really hints as to what a receiving agent might do with a received message. The value unknown indicates no particular statement, all indicates that the author signs all messages but unsigned ones may still be worthwhile, and discardable indicates that unsigned messages should be considered subject to discarding. discardable is the most stringent level.

18.11.2 Example

To get an idea of how a DKIM signature appears in an e-mail, we can simply extract the DKIM-Signature field from an e-mail message generated from a large e-mail provider such as Google's Gmail:

```
DKIM-Signature: v=1; a=rsa-sha256; c=relaxed/relaxed;
                d=gmail.com; s=gamma;
```

```

h=domainkey-signature:mime-version:received:
  sender:received:date
  :x-google-sender-auth:message-id:subject:from:to:content-type;
bh=PU2XIErWsXvhvt1W96ntPWZ2VImjVZ3vBY2T/A+wA3A=;
b=WneQe6kpeu/BfMfa2RSlAl1TvYKfIKmoQRXNc
  IQJDIVoE38+fGDaj0uhNm8vXp/8kJ
  I8HqtkV4/P6/QVPMN+/5bS5dsnlhz0S/YoP
  bZx0Lt2bD67G4HPsvm6eLsaIC9rQECUSL
  MdaTBK3BgFhYo3nenq3+8GxTe9I+ZBcqWAVPU=

```

This indicates a version 1 signature and digest algorithm of SHA-256 signed using RSA. The header and body canonicalization algorithms are both “relaxed,” as shown by the `c=` field. Canonicalization algorithms are used to rewrite messages in a consistent form. The current options are “simple” (the default), which does not alter the text, and “relaxed,” which can rewrite the input in common ways such as altering whitespace and wrapping long header lines. The selector (`s=`) is called gamma and the domain (`d=`) is gmail.com. We shall use these later to retrieve the appropriate public key. The header fields used in computing the signature (indicated by `h=`) include domainkey-signature (predecessor to DKIM), version of MIME, received, sender date, x-google-sender-auth, message-id, subject, from, and content-type. The `bh=` subfield indicates the hash value on the message body expressed in Base64. The `b=` value contains the RSA signature on the hash of the headers listed in the `h=` subfield.

To retrieve the public key to validate the signature, we can form the following query:

```

Linux% dig gamma._domainkey.gmail.com. txt +nostats +noquestion
; <<>> DiG 9.7.2-P3 <<>> gamma._domainkey.gmail.com. txt
      +nostats +noquestion
;; global options: +cmd
;; Got answer:
;; ->HEADER<<- opcode: QUERY, status: NOERROR, id: 17372
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0

;; ANSWER SECTION:
gamma._domainkey.gmail.com. 296      IN      TXT      "k=rsa\; t=y\; p=MIGfMA0GCS
qGSIb3DQEBAQUAA4GNADCBiQKBgQDIhyR3oIt0y22Z0aBrIve9m/iME3Rq0JreasANSpg2YHTYV
+Xtp4xwf5gTjCmHQEMOs0qYu0FYiNQpQogJ2t0Mfx9zNu06rFRBDjiIU9tpx2T+NGlWZ8qhbilo
5By8apJavLyqTLavyPSrvsX0B3YzC63T4Age2CDqZYA+0wSMWQIDAQAB"

```

This result indicates that the key is an RSA public key. The `t=y` entry denotes that the domain is testing DKIM, meaning that the results of any DKIM validation should not ultimately affect the message delivery process. To see an example of an ADSP, we can execute the following command:

```

Linux% host -t txt _adsp._domainkey.paypal.com.
_adsp._domainkey.paypal.com descriptive text "dkim=discardable"

```


Here we can see that Paypal has elected to use the most stringent DKIM signing policy, suggesting that messages failing DKIM validation should be subject to being discarded. The use of ADSP statements at present is fairly rare because of the wide variety of e-mail systems and the ways that various mail agents rewrite messages.

18.12 Attacks on Security Protocols

Attacks on security protocols are somewhat different from the attacks on protocols we have seen in other chapters. Attacks discussed in other chapters tend to compromise some protocol that was never really designed with security in mind by taking advantage of some design or implementation flaw. Attacks against security protocols not only take these forms but may also involve cryptographic attacks that somehow subvert the mathematical basis upon which the security depends. Attacks can be successful against poor algorithms, weak or too-short keys, or poor combinations of various components that render an otherwise secure system much weaker. (A classic and fascinating example can be seen in the cryptanalysis of the VENONA system [VENONA].)

To understand some of the types of attacks targeting security protocols, we will begin from the lowest layer and work our way up. A number of attacks have been waged against 802.11 and EAP. Early security in 802.11 (e.g., WEP and WPA-TKIP) has been shown to be easily compromised cryptographically [TWP07] [OM09], and WPA2-AES is believed to be substantially more resilient, although use of poorly selected pre-shared keys (PSKs) can represent a significant vulnerability to dictionary attacks.

EAP does not have its own authentication method but can inherit vulnerabilities of the authentication methods on which it depends. Once again, systems based on EAP using keys derived from user passwords (e.g., EAP-GSS, EAP-LEAP, EAP-SIM) are often vulnerable to dictionary attacks. 802.1X/EAP is vulnerable to MITM attacks involving tunneled authentication protocols as discussed in [ANN02]. The problem relates to deriving a session key after only one side of a two-party connection has been authenticated. For example, if a server authenticates to a client and this exchange is used as the basis to form a tunnel secured by a derived session key where another protocol that authenticates in the reverse direction operates inside, a MITM attack involving impersonation of the legitimate client becomes possible.

A number of attacks have been published against IPsec, including a class of attacks that exploit the use of encryption without integrity protection [PY06], a configuration option supported but discouraged by the IPsec documentation. In essence, the ability to modify the ciphertext undetected using a bit flipping attack can cause encrypted datagrams to be decrypted into datagrams that have been corrupted in predictable ways. For example, a tunnel mode ESP datagram with its bits flipped appropriately may decrypt to a datagram with an artificially

increased Internet Header Length (IHL) field that causes the payload to be processed as (invalid) IP options, ultimately generating an ICMP message that may be of use to an attacker.

At the transport layer, SSL 2.0 was shown to be vulnerable to a cipher suite rollback attack in which a MITM could cause each end of an SSL connection to conclude that the peer is capable of only weak encryption. Doing so causes the peers to adopt an insecure cipher suite, which the attacker can exploit. A more sophisticated attack on SSL/TLS took advantage of the order of operations performed at a receiver: decrypt, remove padding, and check MAC. If the padding length or MAC is incorrect, an SSL error message is generated. By observing the timing of these error messages, it was possible to create a padding oracle [CHVV03] to recover plaintext from OpenSSH. A padding oracle tells whether the plaintext used to create a ciphertext had a valid amount of padding. As mentioned previously, a more recent attack (on TLS 1.2) involves a MITM attack whereby a prefix of arbitrary length is injected into a TLS association, which is then renegotiated (but continued) when a legitimate client arrives [RD09]. The solution involves binding the previous channel parameters to the subsequent channel parameters using a TLS extension. The issue of channel binding and security is covered more broadly in [RFC5056].

Securing the DNS has been a long time coming, but the importance was underscored by the Kaminsky cache poisoning attack we described in Chapter 11. One of the original problems was the enumeration attack made available (actually required) by the use of NSEC records and countered by the use of NSEC3 records, if used properly [BM09]. At the end of 2009, Dan Bernstein mentioned a number of problems with DNSSEC in his keynote talk at a workshop [B09]: it can be used as a basis for amplification of DoS attacks, it leaks zone data even with NSEC3, its implementations contain exploitable bugs, signatures cannot be revoked, the cryptography may be subject to cryptanalysis, and some NS and A records pose vulnerabilities. At the time of writing, the root zone has been signed only recently, and few organizations have fully adopted DNSSEC. It is therefore likely that a variety of improvements and modifications will be implemented in the years to come.

18.13 Summary

The subject of security is broad and interesting, and we have only scraped the surface in this chapter. We desire several important properties of communication security, and typically these consist of some combination of confidentiality, authentication, integrity, and nonrepudiation. Cryptography is our most important tool for achieving these information security properties. It involves a set of algorithms and keys. The two most important forms are symmetric or “secret key” cryptography, which has good computational performance but requires keys to be kept secret, and public key (asymmetric key) cryptography whereby each

principal has a key pair and one key is made public. Public key cryptography supports both authentication and confidentiality and can be combined with symmetric key cryptography for better performance. Other algorithms that involve mathematics closely related to cryptography include Diffie-Hellman key agreement used to establish symmetric keys, pseudorandom functions for selecting random components to form keys, and MACs used to check message integrity. Protocols that use random nonces attempt to ensure freshness and resist replay attacks by requiring queries and responses to hold a common recently generated value. Salt (in the cryptographic sense) is used to perturb algorithms or input to algorithms in order to make dictionary attacks more difficult to mount.

When relying on a public key, we ordinarily want the public key to be signed or authenticated by some entity or group that we trust. A public key infrastructure or PKI that involves one or more certificate authorities is commonly used for this purpose, but web of trust models are also available. The most common format for holding PKI public keys (and other material) is based on the ITU-T X.509 standard for PKI and certificates. Certificates are usually signed recursively forming a tree, culminating at some top-level root of trust or trust anchor. To ensure that the trust chain is in place, certificates must be validated to ensure that the trust chain is unbroken and each chain element has not been revoked. Certificate status can be evaluated using widely distributed certificate revocation lists (CRLs) or using an online protocol such as OCSP. The entire certificate validation process can also be delegated to another party using SCVP, a protocol developed for this specific purpose.

There are a variety of file formats for holding certificates and keys. The DER or CER format is a binary encoding based on ASN.1. The PEM format expresses the DER encoding in ASCII, so such files are easily edited and inspected. The PKCS#12 (successor to Microsoft's PFX) format can hold both certificates and private keys and is ordinarily encrypted for protection of the private key material. A variety of programs such as `openssl` are capable of converting between formats.

There are security protocols at every protocol layer, and some between layers. Working from layer 2 up, some link technologies include their own encryption and authentication protocols, although these are not ordinarily considered TCP/IP protocols. In TCP/IP, EAP is used to establish authentication with a wide variety of mechanisms such as machine certificates, user certificates, smart cards, passwords, and so on. EAP is most often used in enterprise settings that have a backend authorization or AAA server. EAP can also be used for authentication in other protocols such as IPsec.

IPsec is a collection of protocols that provide security at layer 3: IKE, AH, and ESP. IKE establishes and manages security association between two parties. Security associations can involve authentication (AH) or encryption (ESP) and can operate in either transport or tunnel mode. In transport mode, the IP header is modified for authentication or encryption, while in tunnel mode an IP datagram in its entirety is placed inside a new IP datagram. ESP is the most popular. All IPsec

protocols can use different algorithms and parameters (cryptographic suites) for encryption, integrity protection, DH key agreement, and authentication.

Moving up the stack, transport-layer security (current version TLS 1.2) protects information moved between applications. It has its own internal layering consisting of a record-layer protocol and three handshaking protocols called the Cipher Change protocol, Alert protocol, and Handshake protocol. In addition, the Record protocol supports application data. The record layer is responsible for encrypting and integrity-protecting data based on parameters supplied by the Handshake protocol. The Cipher Change protocol is invoked to change from a previously set-up pending protocol state to an active protocol state. The Alert protocol indicates errors or connection problems. TLS with TCP/IP is the most widely used security protocol and supports encrypted Web browser connections (HTTPS). A variant of TLS called DTLS adapts TLS for use with datagrams and protocols such as UDP and DCCP.

To help secure host names and the Web better, DNSSEC is targeted at providing security for the DNS. On July 15, 2010, the Internet's signed root zone was put into operation, satisfying a prerequisite for worldwide deployment. DNSSEC works by employing several new resource records in the DNS: DNSKEY, DS, NSEC/NSEC3/NSEC3PARAM, and RRSIG. The first two hold and refer to public keys used for signing the structure and contents of a zone. The NSEC or NSEC3/NSEC3PARAM records help provide a canonical ordering of names and list of types present for a domain name. This allows a query to reliably determine the nonexistence of a domain name or presence of a particular type for a particular domain name. RRSIG records hold signatures on other records, and for a zone to be signed, all authoritative RRs within the zone must have associated RRSIG RRs. Once set up, security of DNS queries is checked by a validating resolver or name server that requires a trust anchor. Such systems check to ensure that digital signatures match the public keys supplied by the DNS. This allows for errors to be generated when some record is found to be inconsistent, and it is hoped it can thwart domain name hijacking attacks in which attackers masquerade as legitimate hosts. In some cases, DNS transactions are also secured. The TSIG and SIG(0) protocols provide a form of channel authentication, but only in the scope of DNS transactions. These protocols are used for transactions such as DNS dynamic updates and zone transfers.

Attacks on security protocols include not only the common exploitation of implementation bugs and insecure designs but also mathematical compromises and "side channel" attacks that are used to discover secret information (e.g., bits of keys). Over the years it has become clear that flexibility is needed in the strength of the cryptography used to secure communications, so most of the protocols we have discussed provide for cryptographic suites that can evolve as computational power improves and additional experience is gained. Many seemingly secure protocols, even those that have received extensive scrutiny by experts, have fallen prey to an energetic set of analysts who seek exploitable flaws, especially when MITM and other active attacks are possible. Extreme care is required in designing new security protocols and operating existing protocols in a secure fashion.

18.14 References

- [802.1X-2010] “IEEE Standard for Port-Based Network Access Control,” IEEE Std 802.1X-2010, Feb. 2010.
- [AKNT04] Y. Amir, Y. Kim, C. Nita-Rotaru, and G. Tsudik, “On the Performance of Group Key Agreement Protocols,” *ACM Transactions on Information and System Security* 7(3), Aug. 2004.
- [ANN02] N. Asokan, V. Niemi, and K. Nyberg, “Man-in-the-Middle in Tunneled Authentication Protocols (Extended Abstract),” *Proc. 11th Security Protocols Workshop/LNCS 3364* (Springer, 2003).
- [B06] M. Bellare, “New Proofs for NMAC and HMAC: Security without Collision-Resistance” (preliminary version in *CRYPTO 06*, June 2006).
- [B09] D. Bernstein, “Breaking DNSSEC,” keynote talk at Workshop on Offensive Technologies (WOOT), Aug. 2009.
- [BCK96] M. Bellare, R. Canetti, and H. Krawczyk, “Keying Hash Functions for Message Authentication” (abridged version in *CRYPTO 96/LNCS 1109*, June 1996).
- [BM09] J. Bau and J. Mitchell, “A Security Evaluation of DNSSEC with NSEC3,” *Network and Distributed System Security Symposium (NDSS)*, Feb.–Mar. 2010.
- [BOPSW09] R. Biddle et al., “Browser Interfaces and Extended Validation SSL Certificates: An Empirical Study,” *Proc. ACM Cloud Security Workshop*, Nov. 2009.
- [CABF09] CA/Browser Forum, “Guidelines for the Issuance and Management of Extended Validation Certificates (v1.2),” 2009, http://www.cabforum.org/Guidelines_v1_2.pdf
- [CHP] National Institute of Standards and Technology, Cryptographic Hash Project, Computer Security Division—Computer Security Resource Center <http://csrc.nist.gov/groups/ST/hash>
- [CHVV03] B. Canvel, A. Hiltgen, S. Vaudenay, and M. Vuagnoux, “Password Interception in a SSL/TLS Channel,” *CRYPTO 2003/LNCS 2729*
- [DH76] W. Diffie and M. Hellman, “New Directions in Cryptography,” *IEEE Transactions on Information Theory* 22, Nov. 1976.
- [DKPARAMS] <http://www.iana.org/assignments/dkim-parameters>
- [DNSSECALG] <http://www.iana.org/assignments/dns-sec-alg-numbers>
- [DOW92] W. Diffie, P. Oorschot, and M. Wiener, “Authentication and authenticated Key Exchanges,” *Designs, Codes and Cryptography* 2, June 1992.
- [DSRRTYPES] <http://www.iana.org/assignments/ds-rr-types>

- [FIPS186-3] National Institute for Standards and Technology, "Digital Signature Standard (DSS)," FIPS PUB 186-3, June 2009.
- [FIPS197] National Institute for Standards and Technology, "Advanced Encryption Standard (AES)," FIPS PUB 197, Nov. 2001.
- [FIPS198] National Institute for Standards and Technology, "The Keyed-Hash Message Authentication Code (HMAC)," FIPS PUB 198, Mar. 2002.
- [FIPS800-38B] National Institute for Standards and Technology, "Recommendation for Block Cipher Modes of Operation: The CMAC Mode for Authentication," NIST Special Publication 800-38B, May 2005.
- [GGM86] O. Goldreich, S. Goldwasser, and S. Micali, "How to Construct Random Functions," *Journal of the ACM* 33(4), Oct. 1986.
- [IDDCIN] S. Weiler and D. Blacka, "Clarifications and Implementation Notes for DNSSECbis," Internet draft-ietf-dnsext-dnssec-bis-updates, work in progress, July 2011.
- [IDDS2] B. Dickson, "DNSSEC Delegation Signature with Canonical Signer Name," Internet draft-dickson-dnsext-ds2 (expired), work in progress, Nov. 2010.
- [IDDTLS] E. Rescorla and N. Modadugu, "Datagram Transport Layer Security Version 1.2," Internet draft-ietf-tls-rfc4347-bis, work in progress, July 2011.
- [IEAP] <http://www.iana.org/assignments/eap-numbers>
- [IK03] T. Iwata and K. Kurosawa, "OMAC: One-Key CBC MAC," *Proc. Fast Software Encryption*, Mar. 2003.
- [IKEPARAMS] <http://www.iana.org/assignments/ikev2-parameters>
- [IPANA] <http://www.iana.org/assignments/pana-parameters>
- [ITUOID] <http://www.itu.int/ITU-T/asn1>
- [IWESP] <http://www.iana.org/assignments/wesp-flags>
- [K87] N. Koblitz, "Elliptic Curve Cryptosystems," *Mathematics of Computer* 48, 1987.
- [L01] C. Landwehr, "Computer Security," Springer-Verlag Online, July 2001.
- [M85] V. Miller, "Uses of Elliptic Curves in Cryptography," *Advances in Cryptology: CRYPTO '85* Lecture Notes in Computer Science, Volume 218 (Springer-Verlag, 1986).
- [MSK09] S. McClure, J. Scambray, and G. Kurtz, *Hacking Exposed*, Sixth Edition (McGraw-Hill, 2009).

[MW99] U. Maurer and S. Wolf, "The Relationship between Breaking the Diffie-Hellman Protocol and Computing Discrete Logarithms," *Siam Journal on Computing*, 28(5), 1999.

[NAZ00] Network Associates and P. Zimmermann, Introduction to Cryptography, Part of PGP 7.0 Documentation, available from <http://www.pgpi.org/doc/guide/7.0/en>

[NIST800-38B] National Institute for Standards and Technology, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," NIST Special Publication 800-38D, Nov. 2005.

[NSEC3PARAMS] <http://www.iana.org/assignments/dnssec-nsec3-parameters>

[OM09] T. Ohigashi and M. Morii, "A Practical Message Falsification Attack on WPA," Joint Workshop on Information Security, Aug. 2009.

[PY06] K. Paterson and A. Yau, "Cryptography in Theory and Practice: The Case of Encryption in IPsec," EUROCRYPT 2006/LNCS 4004

[RD09] M. Ray and S. Dispensa, "Renegotiating TLS," PhoneFactor Technical Report, Nov. 2009.

[RFC1321] R. Rivest, "The MD5 Message-Digest Algorithm," Internet RFC 1321 (informational), Apr. 1992.

[RFC2104] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-Hashing for Message Authentication," Internet RFC 2104 (informational), Feb. 1997.

[RFC2403] C. Madson and R. Glenn, "The Use of HMAC-MD5-96 within ESP and AH," Internet RFC 2403, Nov. 1998.

[RFC2404] C. Madson and R. Glenn, "The Use of HMAC-SHA-1-96 within ESP and AH," Internet RFC 2404, Nov. 1998.

[RFC2409] D. Harkins and D. Carrel, "The Internet Key Exchange (IKE)," Internet RFC 2409 (obsolete), Nov. 1998.

[RFC2410] R. Glenn and S. Kent, "The NULL Encryption Algorithm and Its Use with IPsec," Internet RFC 2410, Nov. 1998.

[RFC2451] R. Pereira and R. Adams, "The ESP CBC-Mode Cipher Algorithms," Internet RFC 2451, Nov. 1998.

[RFC2560] M. Myers, R. Ankney, A. Malpani, S. Galperin, and C. Adams, "X.509 Internet Public Key Infrastructure Online Certificate Status Protocol—OCSP," Internet RFC 2560, June 1999.

[RFC2631] E. Rescorla, "Diffie-Hellman Key Agreement Method," Internet RFC 2631, June 1999.

- [RFC2671] P. Vixie, "Extension Mechanisms for DNS (EDNS0)," Internet RFC 2671, Aug. 1999.
- [RFC2845] P. Vixie, O. Gudmundsson, D. Eastlake 3rd, and B. Wellington, "Secret Key Transaction Authentication for DNS (TSIG)," Internet RFC 2845, May 2000.
- [RFC2865] C. Rigney, S. Willens, A. Rubens, and W. Simpson, "Remote Authentication Dial In User Service (RADIUS)," Internet RFC 2865, June 2000.
- [RFC2930] D. Eastlake 3rd, "Secret Key Establishment for DNS (TKEY RR)," Internet RFC 2930, Sept. 2000.
- [RFC2931] D. Eastlake 3rd, "DNS Request and Transaction Signatures (SIG(0)s)," Internet RFC 2931, Sept. 2000.
- [RFC3162] B. Aboba, G. Zorn, and D. Mitton, "RADIUS and IPv6," Internet RFC 3162, Aug. 2001.
- [RFC3173] A. Shacham, B. Monsour, R. Pereira, and M. Thomas, "IP Payload Compression Protocol (IPComp)," Internet RFC 3173, Sept. 2001.
- [RFC3193] B. Patel, B. Aboba, W. Dixon, G. Zorn, and S. Booth, "Securing L2TP Using IPsec," Internet RFC 3193, Nov. 2001.
- [RFC3225] D. Conrad, "Indicating Resolver Support of DNSSEC," Internet RFC 3225, Dec. 2001.
- [RFC3447] J. Jonsson and B. Kaliski, "Public-Key Cryptography Standards (PKCS) #1: RSA Cryptography Specifications Version 2.1," Internet RFC 3447 (informational), Feb. 2003.
- [RFC3526] T. Kivinen and M. Kojo, "More Modular Exponential (MODP) Diffie-Hellman Groups for Internet Key Exchange (IKE)," Internet RFC 3526, May 2003.
- [RFC3547] M. Baugher, B. Weis, T. Hardjono, and H. Harney, "The Group Domain of Interpretation," Internet RFC 3547, July 2003.
- [RFC3566] S. Frankel and H. Herbert, "The AES-XCBC-MAC-96 Algorithm and Its Use with IPsec," Internet RFC 3566, Sept. 2003.
- [RFC3588] P. Calhoun, J. Loughney, E. Guttman, G. Zorn, and J. Arkko, "Diameter Base Protocol," Internet RFC 3588, Sept. 2003.
- [RFC3602] S. Frankel, R. Glenn, and S. Kelly, "The AES-CBC Cipher Algorithm and Its Use with IPsec," Internet RFC 3602, Sept. 2003.
- [RFC3645] S. Kwan, P. Garg, J. Gilroy, L. Esibov, J. Westhead, and R. Hall, "Generic Security Service Algorithm for Secret Key Transaction Authentication for DNS (GSS-TSIG)," Internet RFC 3645, Oct. 2003.

- [RFC3686] R. Housley, "Using Advanced Encryption Standard (AES) Counter Mode with IPsec Encapsulating Security Payload (ESP)," Internet RFC 3686, Jan. 2004.
- [RFC3713] M. Matsui, J. Nakajima, and S. Moriai, "A Description of the Camellia Encryption Algorithm," Internet RFC 3713 (informational), Apr. 2004.
- [RFC3715] B. Aboba and W. Dixon, "IPsec-Network Address Translation (NAT) Compatibility Requirements," Internet RFC 3715 (informational), Mar. 2001.
- [RFC3740] T. Hardjono and B. Weis, "The Multicast Group Security Architecture," Internet RFC 3740 (informational), Mar. 2004.
- [RFC3748] B. Aboba, L. Blunk, J. Vollbrecht, J. Carlson, and H. Levkowitz, ed., "Extensible Authentication Protocol (EAP)," June 2004.
- [RFC3749] S. Hollenbeck, "Transport Layer Security Protocol Compression Methods," Internet RFC 3749, May 2004.
- [RFC3947] T. Kivinen, B. Swander, A. Huttunen, and V. Volpe, "Negotiation of NAT-Traversal in the IKE," Internet RFC 3947, Jan. 2005.
- [RFC3948] A. Huttunen, B. Swander, V. Volpe, L. DiBurro, and M. Stenberg, "UDP Encapsulation of IPsec ESP Packets," Internet RFC 3948, Jan. 2005.
- [RFC4016] M. Parthasarathy, "Protocol for Carrying Authentication and Network Access (PANA) Threat Analysis and Security Requirements," Internet RFC 4016 (informational), Mar. 2005.
- [RFC4033] R. Arends, R. Austein, M. Larson, D. Massey, and S. Rose, "DNS Security Introduction and Requirements," Internet RFC 4033, Mar. 2005.
- [RFC4034] R. Arends, R. Austein, M. Larson, D. Massey, and S. Rose, "Resource Records for the DNS Security Extensions," Internet RFC 4034, Mar. 2005.
- [RFC4035] R. Arends, R. Austein, M. Larson, D. Massey, and S. Rose, "Protocol Modifications for the DNS Security Extensions," Internet RFC 4035, Mar. 2005.
- [RFC4058] A. Yegin, ed., Y. Ohba, R. Penno, G. Tsirtsis, and C. Wang, "Protocol for Carrying Authentication for Network Access (PANA) Requirements," Internet RFC 4058 (informational), May 2005.
- [RFC4086] D. Eastlake 3rd, J. Schiller, and S. Crocker, "Randomness Requirements for Security," Internet RFC 4086/BCP 0106, June 2005.
- [RFC4120] C. Neuman, T. Yu, S. Hartman, and K. Raeburn, "The Kerberos Network Authentication Service (V5)," Internet RFC 4120, July 2005.
- [RFC4251] T. Ylonen and C. Lonvick, ed., "The Secure Shell (SSH) Protocol Architecture," Internet RFC 4251, Jan. 2006.

- [RFC4301] S. Kent and K. Seo, "Security Architecture for the Internet Protocol," Internet RFC 4301, Dec. 2005.
- [RFC4302] S. Kent, "IP Authentication Header," Internet RFC 4302, Dec. 2005.
- [RFC4303] S. Kent, "IP Encapsulating Security Payload (ESP)," Internet RFC 4303, Dec. 2005.
- [RFC4307] J. Schiller, "Cryptographic Algorithms for Use in the Internet Key Exchange Version 2 (IKEv2)," Internet RFC 4307, Dec. 2005.
- [RFC4309] R. Housley, "Using Advanced Encryption Standard (AES) CCM Mode with IPsec Encapsulating Security Payload (ESP)," Internet RFC 4309, Dec. 2005.
- [RFC4346] T. Dierks and E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.1," Internet RFC 4346 (obsolete), Apr. 2006.
- [RFC4347] E. Rescorla and N. Modadugu, "Datagram Transport Layer Security," Internet RFC 4347, Apr. 2006.
- [RFC4398] S. Josefsson, "Storing Certificates in the Domain Name System (DNS)," Internet RFC 4398, Mar. 2006.
- [RFC4431] M. Andrews and S. Weiler, "The DNSSEC Lookaside Validation (DLV) DNS Resource Record," Internet RFC 4431 (informational), Feb. 2006.
- [RFC4434] P. Hoffman, "The AES-XCBC-PRF-128 Algorithm for the Internet Key Exchange Protocol (IKE)," Internet RFC 4434, Feb. 2006.
- [RFC4492] S. Blake-Wilson, N. Bolyard, V. Gupta, C. Hawk, and B. Moeller, "Elliptic Curve Cryptography (ECC) Cipher Suites for Transport Layer Security (TLS)," Internet RFC 4492 (informational), May 2006.
- [RFC4493] JH. Song, R. Poovendran, J. Lee, and T. Iwata, "The AES-CMAC Algorithm," Internet RFC 4493 (informational), June 2006.
- [RFC4509] W. Hardaker, "Use of SHA-256 in DNSSEC Delegation Signer (DS) Resource Records (RRs)," Internet RFC 4509, May 2006.
- [RFC4535] H. Harney, U. Meth, A. Colegrove, and G. Gross, "GSAKMP: Group Secure Association Key Management Protocol," Internet RFC 4535, June 2006.
- [RFC4555] P. Eronen, "IKEv2 Mobility and Multihoming Protocol (MOBIKE)," Internet RFC 4555, June 2006.
- [RFC4615] J. Song, R. Poovendran, J. Lee, and T. Iwata, "The Advanced Encryption Standard-Cipher-Based Message Authentication Code-Pseudo-Random Function-128 (AES-CMAC-PRF-128) Algorithm for the Internet Key Exchange Protocol (IKE)," Internet RFC 4615, Aug. 2006.

[RFC4635] D. Eastlake 3rd, "HMAC SHA (Hashed Message Authentication Code, Secure Hash Algorithm) TSIG Algorithm Identifiers," Internet RFC 4635, Aug. 2006.

[RFC4681] S. Santesson, A. Medvinsky, and J. Ball, "TLS User Mapping Extension," Internet RFC 4681, Oct. 2006.

[RFC4739] P. Eronen and J. Korhonen, "Multiple Authentication Exchanges in the Internet Key Exchange (IKEv2) Protocol," Internet RFC 4739 (experimental), Nov. 2006.

[RFC4754] D. Fu and J. Solinas, "IKE and IKEv2 Authentication Using the Elliptic Curve Digital Signature Algorithm (ECDSA)," Internet RFC 4754, Jan. 2007.

[RFC4835] V. Manral, "Cryptographic Algorithm Implementation Requirements for Encapsulating Security Payload (ESP) and Authentication Header (AH)," Internet RFC 4835, Apr. 2007.

[RFC4851] N. Cam-Winget, D. McGrew, J. Salowey, and H. Zhou, "The Flexible Authentication via Secure Tunneling Extensible Authentication Protocol Method (EAP-FAST)," Internet RFC 4851 (informational), May 2007.

[RFC4877] V. Devarapalli and F. Dupont, "Mobile IPv6 Operation with IKEv2 and the Revised IPsec Architecture," Internet RFC 4877, Apr. 2007.

[RFC4880] J. Callas, L. Donnerhackle, H. Finney, D. Shaw, and R. Thayer, "OpenPGP Message Format," Internet RFC 4880, Nov. 2007.

[RFC5011] M. StJohns, "Automated Updates of DNS Security (DNSSEC) Trust Anchors," Internet RFC 5011, Sep. 2007.

[RFC5054] D. Taylor, T. Wu, N. Mavrogiannopoulos, and T. Perrin, "Using the Secure Remote Password (SRP) Protocol for TLS Authentication," Internet RFC 5054 (informational), Nov. 2007.

[RFC5055] T. Freeman, R. Housley, A. Malpani, D. Cooper, and W. Polk, "Server-Based Certificate Validation Protocol (SCVP)," Internet RFC 5055, Dec. 2007.

[RFC5056] N. Williams, "On the Use of Channel Bindings to Secure Channels," Internet RFC 5056, Nov. 2007.

[RFC5077] J. Salowey, H. Zhou, P. Eronen, and H. Tschofenig, "Transport Layer Security (TLS) Session Resumption without Server-Side State," Internet RFC 5077, Jan. 2008.

[RFC5106] H. Tschofenig, D. Kroesenberg, A. Pashalidis, Y. Ohba, and F. Bersani, "The Extensible Authentication Protocol-Internet Key Exchange Protocol Version 2 (EAP-IKEv2) Method," Internet RFC 5106 (experimental), Feb. 2008.

[RFC5114] M. Lepinski and S. Kent, "Additional Diffie-Hellman Groups for Use with IETF Standards," Internet RFC 5114 (informational), Jan. 2008.

- [RFC5116] D. McGrew, "An Interface and Algorithms for Authenticated Encryption," Internet RFC 5116, Jan. 2008.
- [RFC5155] B. Laurie, G. Sisson, R. Arends, and D. Blacka, "DNS Security (DNS-SEC) Hashed Authenticated Denial of Existence," Internet RFC 5155, Mar. 2008.
- [RFC5191] D. Forsberg, Y. Ohba, ed., B. Patil, H. Tschofenig, and A. Yegin, "Protocol for Carrying Authentication for Network Access (PANA)," Internet RFC 5191, May 2008.
- [RFC5193] P. Jayaraman, R. Lopez, Y. Ohba, ed., M. Parthasarathy, and A. Yegin, "Protocol for Carrying Authentication for Network Access (PANA) Framework," Internet RFC 5193 (informational), May 2008.
- [RFC5216] D. Simon, B. Aboba, and R. Hurst, "The EAP-TLS Authentication Protocol," Internet RFC 5216, Mar. 2008.
- [RFC5238] T. Phelan, "Datagram Transport Layer Security (DTLS) over the Datagram Congestion Control Protocol (DCCP)," Internet RFC 5238, May 2008.
- [RFC5246] T. Dierks and E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.2," Internet RFC 5246, Aug. 2008.
- [RFC5247] B. Aboba, D. Simon, and P. Eronen, "Extensible Authentication Protocol (EAP) Key Management Framework," Internet RFC 5247, Aug. 2008.
- [RFC5280] D. Cooper, S. Santesson, S. Farrell, S. Boeyen, R. Housley, and W. Polk, "Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile," Internet RFC 5280, May 2008.
- [RFC5281] P. Funk and S. Blake-Wilson, "Extensible Authentication Protocol Tunneled Transport Layer Security Authenticated Protocol Version 0 (EAP-TTLSv0)," Internet RFC 5281 (informational), Aug. 2008.
- [RFC5295] J. Salowey, L. Dondeti, V. Narayanan, and M. Nakhjiri, "Specification for the Derivation of Root Keys from an Extended Master Session Key (EMSK)," Internet RFC 5295, Aug. 2008.
- [RFC5296] V. Narayanan and L. Dondeti, "EAP Extensions for EAP Re-authentication Protocol (ERP)," Internet RFC 5296, Aug. 2008.
- [RFC5322] P. Resnick, ed., "Internet Message Format," Internet RFC 5322, Oct. 2008.
- [RFC5374] B. Weis, G. Gross, and D. Ignjatovic, "Multicast Extensions to the Security Architecture for the Internet Protocol," Internet RFC 5374, Nov. 2008.
- [RFC5386] N. Williams and M. Richardson, "Better-than-Nothing Security: An Unauthenticated Mode of IPsec," Internet RFC 5386, Nov. 2008.

[RFC5387] J. Touch, D. Black, and Y. Wang, "Problem and Applicability Statement for Better-than-Nothing Security (BTNS)," Internet RFC 5387 (informational), Nov. 2008.

[RFC5406] S. Bellovin, "Guidelines for Specifying the Use of IPsec Version 2," Internet RFC 5406/BCP 0146, Feb. 2009.

[RFC5585] T. Hansen, D. Crocker, and P. Hallam-Baker, "DomainKeys Identified Mail (DKIM) Service Overview," Internet RFC 5585 (informational), July 2009.

[RFC5617] E. Allman, J. Fenton, M. Delany, and J. Levine, "DomainKeys Identified Mail (DKIM) Author Domain Signing Practices (ADSP)," Internet RFC 5617, Aug. 2009.

[RFC5652] R. Housley, "Cryptographic Message Syntax (CMS)," Internet RFC 5652/STD 0070, Sept. 2009.

[RFC5702] J. Jansen, "Use of SHA-2 Algorithms with RSA in DNSKEY and RRSIG Resource Records for DNSSEC," Internet RFC 5702, Oct. 2009.

[RFC5723] Y. Sheffer and H. Tschofenig, "Internet Key Exchange Protocol Version 2 (IKEv2) Session Resumption," Internet RFC 5723, Jan. 2010.

[RFC5739] P. Eronen, J. Laganier, and C. Madson, "IPv6 Configuration in Internet Key Exchange Protocol Version 2 (IKEv2)," Internet RFC 5739 (experimental), Feb. 2010.

[RFC5746] E. Rescorla, M. Ray, S. Dispensa, and N. Oskov, "Transport Layer Security (TLS) Renegotiation Indication Extension," Internet RFC 5746, Feb. 2010.

[RFC5753] S. Turner and D. Brown, "Use of Elliptic Curve Cryptography (ECC) Algorithms in Cryptographic Message Syntax (CMS)," Internet RFC 5753 (informational), Jan. 2010.

[RFC5755] S. Farrell, R. Housley, and S. Turner, "An Internet Attribute Certificate Profile for Authorization," Internet RFC 5755, Jan. 2010.

[RFC5764] D. McGrew and E. Rescorla, "Datagram Transport Layer Security (DTLS) Extension to Establish Keys for the Secure Real-Time Transport Protocol (SRTP)," Internet RFC 5764, May 2010.

[RFC5840] K. Grewal, G. Montenegro, and M. Bhatia, "Wrapped Encapsulating Security Payload (ESP) for Traffic Visibility," Internet RFC 5840, Apr. 2010.

[RFC5857] E. Ertekin, C. Christou, R. Jasani, T. Kivinen, and C. Bormann, "IKEv2 Extensions to Support Robust Header Compression over IPsec," Internet RFC 5857, May 2010.

[RFC5879] T. Kivinen and D. McDonald, "Heuristics for Detecting ESP-NULL Packets," Internet RFC 5879 (informational), May 2010.

- [RFC5903] D. Fu and J. Solinas, "Elliptic Curve Groups Modulo a Prime (ECP Groups) for IKE and IKEv2," Internet RFC 5903 (informational), June 2010.
- [RFC5933] V. Dolmatov, ed., A. Chuprina, and I. Ustinov, "Use of GOST Signature Algorithms in DNSKEY and RRSIG Resource Records for DNSSEC," Internet RFC 5933, July 2010.
- [RFC5996] C. Kaufman, P. Hoffman, Y. Nir, and P. Eronen, "Internet Key Exchange Protocol Version 2 (IKEv2)," Internet RFC 5996, Sept. 2010.
- [RFC5998] P. Eronen, H. Tschofenig, and Y. Sheffer, "An Extension for EAP-Only Authentication in IKEv2," Sept. 2010.
- [RFC6024] R. Reddy and C. Wallace, "Trust Anchor Management Requirements," Internet RFC 6024 (informational), Oct. 2010.
- [RFC6040] B. Briscoe, "Tunnelling of Explicit Congestion Notification," Internet RFC 6040, Nov. 2010.
- [RFC6066] D. Eastlake 3rd, "Transport Layer Security (TLS) Extensions: Extension Definitions," Internet RFC 6066, Jan. 2011.
- [RFC6071] S. Frankel and S. Krishnan, "IP Security (IPsec) and Internet Key Exchange (IKE) Document Roadmap," Internet RFC 6071 (informational), Feb. 2011.
- [RFC6083] M. Tuexen, R. Seggelmann, and E. Rescorla, "Datagram Transport Layer Security (DTLS) for Stream Control Transmission Protocol (SCTP)," Internet RFC 6083, Jan. 2011.
- [RFC6091] N. Mavrogiannopoulos and D. Gillmor, "Using OpenPGP Keys for Transport Layer Security (TLS) Authentication," Internet RFC 6091 (informational), Feb. 2011.
- [RFC6101] A. Freier, P. Karlton, and P. Kocher, "The Secure Socket Layer (SSL) Protocol Version 3.0," Internet RFC 6101, Aug. 2011.
- [RFC6147] M. Bagnulo, A. Sullivan, P. Matthews, and I. van Beijnum, "DN64: DNS Extensions for Network Address Translation from IPv6 Clients to IPv4 Servers," Internet RFC 6147, Apr. 2011.
- [RFC6176] S. Turner and S. Polk, "Prohibiting Secure Sockets Layer (SSL) Version 2.0," Internet RFC 6176, Mar. 2011.
- [RFC6234] D. Eastlake 3rd and T. Hansen, "US Secure Hash Algorithms (SHA and SHA-based HMAC and HKDF)," Internet RFC 6234 (informational), May 2011.
- [RFC6345] P. Duffy, S. Chakrabarti, R. Cragie, Y. Ohba, ed., and A. Yegin, "Protocol for Carrying authentication for Network Access (PANA) Relay Element," Internet RFC 6345, Aug. 2011.

[RFC6376] D. Crocker, ed., T. Hansen, ed., M. Kucherawy, ed., "DomainKeys Identified Mail (DKIM) Signatures," Internet RFC 6376, Sep. 2011.

[RSA78] R. Rivest, A. Shamir, and L. Adleman, "A Method for Obtaining Digital Signatures and Public Key Cryptosystems," *Communications of the ACM* 21(2), Feb. 1978.

[TLD-REPORT] http://stats.research.icann.org/dns/tld_report

[TLSEXT] <http://www.iana.org/assignments/tls-extensiontype-values>

[TLSPARAMS] <http://www.iana.org/assignments/tls-parameters>

[TNMOC] The National Museum of Computing, <http://www.tnmoc.org>

[TSIGALG] <http://www.iana.org/assignments/tsig-algorithm-names>

[TWP07] E. Tews, R. Weinmann, and A. Pyshkin, "Breaking 104 Bit WEP in Less than 60 Seconds," *Proc. 8th International Workshop on Information Security Applications* (Springer, 2007).

[VENONA] R. L. Benson, National Security Agency Center for Cryptologic History, "The VENONA Story," http://www.nsa.gov/public_info/declass/venona

[VK83] V. Voydock and S. Kent, "Security Mechanisms in High-Level Network Protocols," *ACM Computing Surveys* 15, June 1983.

[WY05] X. Wang and H. Yu, "How to Break MD5 and Other Hash Functions," *EUROCRYPT*, May 2005.

[X9.62-2005] American National Standards Institute, "Public Key Cryptography for the Financial Services Industry: The Elliptic Curve Digital Signature Standard (ECDSA)," ANSI X9.62, 2005.

[Z97] Y. Zheng, "Digital Signcryption or How to Achieve $\text{Cost}(\text{Signature} \ \& \ \text{Encryption}) \ll \text{Cost}(\text{Signature}) + \text{Cost}(\text{Encryption})$," *Proc. CRYPTO Lecture Notes in Computer Science, Volume 1294* (Springer-Verlag, 1997).

Glossary of Acronyms

- 3GPP 3rd Generation Partnership Project (cellular SDO responsible for GSM, W-CDMA, LTE, etc.)
- 3GPP2 3rd Generation Partnership Project 2 (cellular SDO responsible for CDMA2000, EV-DO, etc.)
- 6rd IPv6 Rapid Deployment (an IPv6 transition mechanism in which IPv6 traffic is carried over IPv4 networks, similar to 6to4 but using IPv6 prefix assignments based on unicast address assignments)
- 6to4 Six to Four (carrying IPv6 traffic in IPv4 tunnels, some operational challenges have occurred)
- A Address (IPv4) (DNS RR carrying an IPv4 address)
- AAA Authentication, Authorization and Accounting (management capabilities associated with certain access protocols such as RADIUS and Diameter)
- AAAA Address (IPv6) (DNS RR carrying an IPv6 address)
- ABC Appropriate Byte Counting (in TCP congestion control, a method to account for the number of bytes ACKed instead of a constant factor when performing CWND computations; can mitigate the slow window growth associated with delayed ACKs)
- AC Attribute Certificate (a type of certificate used to carry attributes such as authorizations, but does not include a public key and therefore differs from a PKC)
- ACCM Asynchronous Control Character Map (in PPP, indicates which bytes need to be escaped to avoid having unwanted effects)
- ACD Automatic Collision Detection (procedure to detect and avoid IP address assignment collisions)
- ACFC Address and Control Field Compression (in PPP, eliminating the address and control fields to reduce overhead)
- ACK Acknowledgment (an indication that data has arrived at a receiver successfully; applicable to multiple layers of the protocol stack)
- ACL Access Control List (list of filtering rules determining which traffic is permitted, e.g., through a firewall)
- ADSP Author Domain Signing Practices (with DKIM, a policy statement pertaining to how DKIM is used or deployed within a particular domain)

- AEAD** Authenticated Encryption with Associated Data (algorithms that perform encryption and authentication on one portion of their input and authentication on another portion)
- AES** Advanced Encryption Standard (current-generation U.S. encryption standard)
- AF** Assured Forwarding (a PHB offering priority classes and prioritization within classes)
- AFTR** Address Family Transition Router element (in DS-Lite, a SPNAT used to share a small number of IPv4 addresses with multiple customers)
- AH** Authentication Header (optional IPsec protocol providing for authentication of IP traffic, including header information, which is incompatible with NATs)
- AIA** Authority Information Access (an X.509 certificate extension indicating resources useful in validating a certificate)
- AIAD** Additive Increase Additive Decrease (in TCP, methods that moderate CWND by adding to its value when congestion appears to be low and subtracting from it when congestion appears to be increasing; not the standard TCP algorithm)
- AIMD** Additive Increase Multiplicative Decrease (in TCP, methods that moderate CWND by adding to its value when congestion appears to be low and multiplying it by a fraction less than one when congestion appears to be increasing)
- ALG** Application Layer Gateway (an agent, usually software, that converts protocols at the application layer)
- A-MPDU** Aggregated MPDU (frame containing multiple MPDUs, part of IEEE 802.11n)
- A-MSDU** Aggregated MSDU (frame containing multiple MSDUs, part of IEEE 802.11n)
- ANDSF** Access Network Discovery and Selection Function (a portion of MoS indicating information about networks that may be used to influence handoff and network selection)
- AODV** Ad-hoc On-Demand Distance Vector routing protocol (early ad-hoc on-demand routing protocol using distance vectors)
- AP** Access Point (802.11 STA usually used to interconnect wireless and wired network segments)
- API** Application Programming Interface (functions invoked by applications to obtain effects such as sending and receiving network traffic)
- APIPA** Automatic Private IP Addressing (a mechanism whereby a node self-configures its own IP address from a particular range; usually applies to IPv4 nodes)

- APSD Automatic Power Save Delivery (periodic batch processing of 802.11 frames in support of PSM)
- AQM Active Queue Management (queue management methods that react to the traffic dynamics, not including "drop-tail" typical of FCFS/FIFO queue management)
- ARP Address Resolution Protocol (a protocol above the link layer that resolves IPv4 addresses to MAC layer addresses, uses link layer broadcast addressing)
- ARQ Automatic Repeat Request (the retransmission of information; usually after inferred loss)
- AS Authentication Server (with PANA, server where authentication checks are performed)
- AS Autonomous System (a 16- or 32-bit number used in connection with inter-ISP routing to identify a collection of network prefixes and their owner)
- ASM All-Source Multicast (multicast wherein any party can source traffic)
- ASN.1 Abstract Syntax Notation One (an ISO standard defining the abstract syntax for information but not the corresponding encoding format; BER and DER are encodings for ASN.1 information)
- AUS Application Unique String (input string to the DDDS algorithm)
- AUTH Authentication (with IKE, payload containing information required to perform authentication of the sender)
- AXFR Zone Transfer (full exchange of DNS zone information; uses TCP)
- B4 Bridging Broadband element (in DS-Lite, a router which encapsulates IPv4 traffic in IPv6 tunnels terminated at an AFTR, a B4 does not perform NAT functions)
- BACP Bandwidth Allocation Control Protocol (with PPP, a protocol for configuring BoD)
- BAP Bandwidth Allocation Protocol (a protocol used to configure links in a bundle for MPPP)
- BCMCS Broadcast and Multicast Service Controller (in cellular networks, manages multicast)
- BER Basic Encoding Rules (an ITU standard encoding syntax; a subset of ASN.1)
- BER Bit Error Rate (number of bit errors expected per number of bits in transit)
- BGP Border Gateway Protocol (inter-domain routing protocol with policy support)
- BIND9 Berkeley Internet Name Domain (version 9) (a name server software implementation popular on UNIX-like systems)
- BITS Bump In the Stack (option for implementing IPsec in the host)

- BITW** Bump In the Wire (option for implementing IPsec in the network)
- BL** Bulk Leasequery (in DHCP, a request/response protocol to convey current lease information)
- BoD** Bandwidth on Demand (ability to dynamically adjust available link bandwidth)
- BOOTP** Bootstrap Protocol (precursor to DHCP; used to configure hosts)
- BPDU** Bridge PDU (PDUs used by STP; exchanged by switches and bridges)
- BPSK** Binary Phase Shift Keying (modulating binary using two signal phases)
- BSD** Berkeley Software Distribution (UC Berkeley's version of UNIX, included the first widely used implementation of TCP/IP)
- BDSP** Boot Server Discovery Protocol (an extension to DHCP developed by Apple to discover a boot image server)
- BSS** Basic Service Set (IEEE 802.11 terminology for an access point and associated stations)
- BTNS** Better Than Nothing Security (with IPsec, an option for using certificates without a full PKI but which is vulnerable to MITM attacks)
- BU** Binding Update (in MIP, establishes the mapping between a MN's CoA and HoA)
- CA** Certificate Authority (organization responsible for generating and issuing public/private key pairs and signing and distributing signed public keys and CRLs)
- CALIPSO** Common Architecture Label IPv6 Security Option (security labels for IP packets; not widely used)
- CBC** Cipher Block Chaining (an encryption mode that uses the XOR operation to chain encrypted blocks together in an effort to resist re-arrangement attacks)
- CCBP** Callback Control Protocol (in PPP, establishes a callback number)
- CCA** Clear Channel Assessment (802.11 PHY-layer mechanism that detects channel usage)
- CCITT** Comité Consultatif International Téléphonique et Télégraphique (now ITU-T)
- CCM** Counter mode with CBC Message Authentication Code (an authenticated encryption mode combining CTR mode encryption with CBC-MAC)
- CCMP** Counter Mode with CBC-MAC Protocol (encryption used with WPA2; from IEEE 802.11i; successor to WPA)
- CCP** Compression Control Protocol (in PPP, established the compression methods to use)

-
- ccTLD Country Code TLD (a TLD based on the ISO3661-2 country code list)
- CDP CRL Distribution Point (a location where a CA's current CRL may be obtained)
- CERT Certificate (with IKE, payload containing a certificate)
- CERT Computer Emergency Response Team (groups that handle computer security incidents, including the first CERT at Carnegie Mellon University and U.S. Government's US-CERT)
- CERTREQ Certificate Request (with IKE, payload indicating trust anchor as an indication of acceptable certificates)
- CGA Cryptographically Generated Address (address generated based on a hash on a public key)
- CHAP Challenge-Handshake Authentication Protocol (protocol requiring a challenge to match a response; vulnerable to MITM attacks)
- CIA confidentiality, integrity, and availability (principles of information security; the "CIA triad")
- CIDR Classless Inter-Domain Routing (a move to address the ROAD problem by removing the IP address class boundaries but requiring an associated CIDR mask to be used with inter-domain routing)
- CMAC Cipher-based Message Authentication Code (a particular way of using encryption algorithms as a MAC)
- CN Correspondent Node (an MN's conversation peer in MIP scenario)
- CNAME Canonical Name (DNS RR providing an alias for another domain name)
- CoA Care-of Address (MN's address assigned while visiting non-home network)
- CoS Class of Service (general term referring to differentiated services based on different classes of traffic; a concept supported by the Diff Serv architecture)
- CoT Care-of Test (in a RR check, message sent to MN via its CoA resulting in MN obtaining a portion of a key used to secure BUs)
- CoTI Care-of Test Init (in a RR check, triggers receiver to send a CoT message)
- CP Configuration Payload (with IKE, extensible structure for conveying configuration parameters)
- CPS Certification Practice Statement (a CA's policy statement about how certificates are issued or managed)
- CRC Cyclic Redundancy Check (mathematical functions used to check for bit errors)
- CRL Certificate Revocation List (a list of invalid certificates issued by a CA)

- CS** Cipher Suite (in TLS, the choice of cryptographic algorithm suite)
- CS** Class Selector (in IP, a DSCP value designed to be compatible with the bit values associated with the now-deprecated “Type of Service” and “Traffic Class” IP header fields)
- CSMA/CA** Carrier-Sense Multiple Access/Collision Avoidance (WiFi’s MAC protocol, which involves sending when a link is idle and backing off if it is not)
- CSMA/CD** Carrier-Sense Multiple Access/Collision Detection (Ethernet’s classic MAC protocol, which involves sending when a link is idle and backing off if collisions are detected)
- CSPRNG** Cryptographically Secure Pseudo-Random Number Generator (a PRNG suitable for cryptographic use)
- CSRG** Computer Systems Research Group (developers of BSD UNIX at UC Berkeley)
- CTCP** Compound TCP (a “scalable” TCP variant implemented in modern Windows systems that combines both delay-based and packet-loss based window adjustments)
- CTR** Counter (an encryption mode that uses a counter value to impose a required order on encrypted blocks while permitting parallel execution of encryption or decryption on multiple blocks)
- CTS** Clear To Send (message authorizing sender of RTS to send)
- CW** Contention Window (range of time an 802.11 station will wait before sending under DCF)
- CWND** Congestion Window (in TCP, a limit placed on the sender’s window size to avoid or reduce congestion)
- CWR** Congestion Window Reducing (or Reduced) (in TCP, reduction of the sender’s usable window size)
- CWV** Congestion Window Verification (in TCP, a method to check and update the current value of CWND when deemed necessary)
- DAD** Duplicate Address Detection (with IPv6 ND and SLAAC, DAD helps determine whether a candidate IPv6 address is already in use by sending an NS message for the proposed address)
- DCCP** Datagram Congestion Control Protocol (a protocol that provides best-effort datagram service to applications and also controls congestion)
- DCF** Distributed Coordination Function (CSMA/CA MAC for 802.11 networks)
- DDDS** Dynamic Delegation Discovery System (methods to support lazy binding of strings to data; usually used with DNS for discovery of servers for various application protocols)

-
- DDoS** Distributed DoS (a network-based attack often launched by botnets)
- DER** Distinguished Encoding Rules (an ITU standard encoding syntax; a subset of BER for ASN.1 that requires a unique representation to be used for each value)
- DES** Data Encryption Standard (an older U.S. standard for symmetric data encryption using 56-bit keys)
- DF** Don't Fragment (an IPv4 header bit indicating no fragmentation should be performed; important for PMTUD)
- DH** Diffie-Hellman (mathematical protocol to establish a secret value between two parties even in the presence of an eavesdropper)
- DHCP** Dynamic Host Configuration Protocol (evolved from BOOTP; sets up systems with configuration information such as leased IP addresses, default router, and DNS server IP address)
- DIFS** DCF Inter-Frame Space (time between frames under 802.11 DCF)
- DIX** Digital, Intel, Xerox (creators and name of early Ethernet standard)
- DKIM** Domain Keys Identified Mail (a protocol for cryptographically binding the sending domain of e-mail with the associated originating mail servers)
- DLNA** Digital Living Network Alliance (an industry group focused on interoperability and protocols for consumer media devices such as TVs, DVD players, DVRs, etc.)
- DMZ** De-Militarized Zone (a network segment outside an organization's inside firewall, usually used for hosts providing services to customers or the public)
- DNA** Detecting Network Attachment (procedures to detect a change in connection state)
- DNAME** Non-Terminal Name Redirection (DNS RR supporting generation of multiple CNAME records using a DNS subtree aliasing mechanism)
- DNS** Domain Name System (maps names to IP addresses and more)
- DNS64** DNS IPv4/IPv6 translation (a mechanism for IPv4/IPv6 coexistence to translate IPv4 DNS information for IPv6 DNS use)
- DNSKEY** Key for DNS (DNS RR used with DNSSEC to hold a public key)
- DNSSEC** DNS Security (original authentication and integrity assurance for DNS data)
- DNSSL** DNS Search List (used with RAs, indicates list of default domain extensions)
- DOI** Digital Object Identifier (a method for naming content objects and associating them with information records)
- DoS** Denial of Service (a type of resource exhaustion attack)

- DPD Delegated Path Discovery (method for delegating the collection of all information required to validate a certificate path)
- DPV Delegated Path Validation (method for delegating the entire validation procedure for a certificate)
- DS Delegation Signer (in DNS, an RR used with DNSSEC to secure a delegation)
- DS Differentiated Services (in IP traffic management, methods to provide performance differentiation for traffic delivery)
- DS Distribution Service (in 802.11 LANs, the network or service used to interconnect APs, which is most often a wired 802.3/Ethernet network)
- DSA Digital Signature Algorithm (an algorithm for generating digital signatures based on the discrete logarithm problem)
- DSACK Duplicate SACK (in TCP, a SACK variant that includes description of received duplicated segments)
- DSCP DS Code Point (field value in packet indicating a particular forwarding behavior is desired)
- DSL Digital Subscriber Line (dedicated broadband data link over POTS line)
- DS-Lite Dual Stack Lite (a framework for IPv6-based service providers to provide access to dual stack or single stack clients using a combination of IPv4-in-IPv6 tunneling and NAT)
- DSRK Domain-Specific Root Key (key derived from an EMSK intended for use by systems under a single administrative authority)
- DSS Digital Signature Standard (a U.S. standard for digital signatures based on DSA)
- DSUSRK Domain-Specific USRK (a key combining the usage policies of a USRK and DSRK)
- DTLS Datagram TLS (variant of TLS used with datagram protocols such as UDP)
- DUID DHCP Unique Identifier (value placed in DHCP request to match responses)
- DUP Duplicate (used in multiple context—e.g., DUP ACKs)
- EAP Extensible Authentication Protocol (framework supporting various authentication methods)
- EAP-FAST EAP-Flexible Authentication via Security Tunneling (Cisco's EAP method using TLS that replaces its earlier LEAP EAP method)
- EAPOL EAP over LAN (e.g., EAP over Ethernet as used in IEEE 802.1X)
- EAP-TTLS EAP-Tunneled Transport Layer Security (an EAP method based on earlier TLS EAP method, but requires only server side to obtain certificate)

- EC2N Elliptic Curve groups modulo a power of 2 (groups based on elliptic curves, in the abstract algebra sense, over the Galois Field $GF(2^n)$)
- ECC Error Correcting Code (redundant bits added to information bits usable to correct errors)
- ECDSA Elliptic Curve Digital Signature Algorithm (a variant of DSA using ECC)
- ECE ECN Echo (in TCP with ECN, the reflection of ECN information to a TCP sender)
- ECN Explicit Congestion Notification (direct method of indicating congestion—e.g., by routers to hosts)
- ECP Elliptic Curve groups modulo a Prime (groups based on elliptic curves, in the abstract algebra sense, over the Galois Field $G(P)$ for a prime P)
- ECT ECN-Capable Transport (a transport protocol capable of interpreting ECN indicators)
- EDCA Enhanced Distributed Channel Access (802.11 coordinating function supporting QoS, from 802.11e)
- EDNS0 Extension mechanisms for DNS (version 0) (a method to extend DNS RRs, version 0, needed by DNSSEC)
- EF Expedited Forwarding (a PHB offering a service class as if no congestion were present, generally implying it is the highest priority and requiring admission control to avoid oversubscription)
- EFO Expanded Flags Option (used with DHCP, indicates presence of additional options)
- EIFS Extended IFS (extended IFS used when receiving unrecognized frame under 802.11 DCF)
- EMSK Extended MSK (a secondary key generated in addition to the MSK by EAP after key derivation)
- ENUM E.164 to URI DDDS Application (a particular DDDS used to map E.164 telephony-style addresses to URIs)
- EP Enforcement Point (with PANA, point where access control policies are enforced)
- EQM Equal Modulation (using the same modulation scheme on different data streams simultaneously)
- ERE Eligible Rate Estimate (part of TCP Westwood+; estimate of the amount of bandwidth that could be used by a connection)
- ERP EAP Re-authentication Protocol (an EAP extension to reduce the latency when re-establishing authentication)

- ESN Extended Sequence Number (in IPsec, an extended sequence number of 64 bits used to combat replay attacks; normal sequence numbers are 32 bits)
- ESP Encapsulating Security Payload (required IPsec protocol providing for authentication and/or confidentiality of traffic)
- ESSID Extended Service Set Identifier (IEEE 802.11 network name)
- EUI Extended Unique Identifier (MAC-layer address prefix format defined by IEEE, extended from OUI)
- EV Extended Validation (a form of certificate with enhanced identity validation performed prior to issuance)
- EV-DO Evolution, Data Optimized (or Only) (3GPP2 wireless broadband standard; an evolution of CDMA2000)
- ACK Forward Acknowledgment (in TCP, one more than the highest sequence number known to have reached the receiver; determined using SACK)
- FCFS First Come, First Served (scheduling discipline with in-order service; no priority)
- FCS Frame Check Sequence (general term for bits used to check for bit errors)
- FEC Forward Error Correction (using redundant bits to correct errors in data bits)
- FIFO First In, First Out (queue management discipline with in-order service; no re-arrangements)
- FIN Finish (a TCP header bit and last segment type sent on a TCP connection)
- FMIP Mobile IP with Fast Handovers (modification to MIPv6 with early handovers)
- FQDN Fully Qualifies Domain Name (a domain name with full domain extension included)
- F-RTO Forward RTO (in TCP, a method to infer whether a retransmission was spurious and if so facilitate the avoidance of unnecessary retransmissions)
- FTP File Transfer Protocol (a TCP-based file transfer protocol using separate control and data connections)
- GCKS Group Controller/Key Server (in IPsec, used with GKM; holds and issues keys for GSAs)
- GCM Galois/Counter Mode (an authenticated encryption mode combining CTR mode encryption with Galois mode authentication)
- GDOI Group Domain of Interpretation (in IPsec, a group key management protocol based on ISAKMP and IKE)
- GENA General Event Notification Architecture (an XML-based notification framework using HTTP over multicast UDP; used with UPnP)
- GI Guard Interval (in communications engineering, minimum time between transmissions used to avoid inter-symbol interference)

- GKM** Group Key Management (in IPsec, methods to distribute key material to a group in order to support group SA formation)
- GMAC** Galois Message Authentication Code (an authentication-only variant of GCM)
- GMI** Group Membership Interval (in IGMP and MLD, the amount of time a multicast router waits before deciding there is no particular source or no more group members; set to $Q_{RV} * Q_I + Q_{RI}$)
- GMRP** Generic Multicast Registration Protocol (replaced by MMRP)
- GPAD** Group PAD (with IPsec, abstraction of a database containing authentication data for all GCKS entities)
- GRE** Generic Routing Encapsulation (generic encapsulation within IP datagrams)
- GSA** Group Security Association (in IPsec, an SA established among group members using a multicast protocol)
- GSAKMP** Group Secure Association Key Management Protocol (a framework for creating groups with common cryptographic information, distributing policy, performing access control, generating group keys, and recovering from group dynamic changes)
- GSPD** Group SPD (in IPsec, an SPD capable of holding information for both SAs and GSAs)
- GSS-API** Generic Security Services API (an API to access myriad security services such as authentication, confidentiality, etc.; typically used with the Kerberos authentication system)
- gTLD** Generic TLD (a TLD—such as COM, EDU, MIL—not based on country code)
- GVRP** Generic Attribute Registration Protocol (replaced by MRP)
- HA** Home Agent (system offering MIP helper service to an MN)
- HAIO** Home Agent Information Option (in ICMPv6, an option supporting MIPv6 to indicate address of an HA)
- HCF** Hybrid Coordination Function (coordinating function supporting both priority and contention-based 802.11 channel access)
- HDLC** High-level Data Link Control (a popular ISO standard data link protocol, the basis for the most popular variant of PPP)
- HELD** HTTP-Enabled Location Delivery (a protocol for delivering LCI using HTTP/TCP/IP)
- HIP** Host Identity Protocol (a research protocol architecture focusing on mobility and security)
- HMAC** Hash-based Message Authentication Code (a particular way of using hashing algorithms as a MAC)

- HoA Home Address (in MIP, a MN's address from its home network)
- HOPOPT IPv6 Hop-by-Hop Option (an IPv6 option type applicable to each hop in a path)
- HoT Home Test (in an RR check, message sent to MN via HA resulting in MN obtaining a portion of a key used to secure BUs)
- HoTI Home Test Init (in an RR check, triggers receiver to send a HoT message)
- HSPA High-Speed Packet Access (3GPP wireless broadband standard; an evolution of WCDMA)
- HSTCP Highspeed TCP (a "scalable" TCP variant in which CWND is adjusted based in part on its current value; designed to operate more effectively in high capacity environments)
- HT High Throughput (higher speeds associated with the IEEE 802.11n standard)
- HTML Hyper-Text Markup Language (the basic language of the WWW)
- HTTP Hyper-Text Transfer Protocol (primary protocol of the WWW; often carries HTML)
- HTTPMU HTTP using UDP (a method for carrying HTTP traffic on UDP using multicast addressing; used to carry SSDP messages in UPnP)
- HTTPS HTTP over SSL/TLS (standard for secure WWW exchange)
- HWRP Hybrid Wireless Routing Protocol (routing protocol proposed for IEEE 802.11s)
- IA Identity Association (in DHCP, a collection of addresses)
- IAB Internet Architecture Board (one of IETF's governing bodies; responsible for architectural oversight and appointment of liaisons to other SDOs)
- IAID IA Identifier (in DHCP, an ID referring to a particular IA)
- IANA Internet Assigned Numbers Authority (maintains protocol numbers and field values)
- IBSS Independent Basic Service Set (802.11 ad-hoc network)
- ICANN Internet Corporation for Assigned Names and Numbers (non-profit governing body for domain names and related policy)
- ICE Interactive Connectivity Establishment (a framework for performing NAT traversal, which entails trying direct connections, STUN, and finally TURN to enable communication in the presence of NATs)
- ICMP Internet Control Message Protocol (an information and error reporting protocol considered part of IP)
- ICS Internet Connection Sharing (alternative name for NAT; used with Microsoft Windows)

-
- ICV Integrity Check Value (a value used to check the integrity of a message—e.g., cryptographic hash)
- ID Identification (in IKE, payload indicating identity of sender)
- IDN Internationalized Domain Name (domain name encoding non-ASCII characters)
- IEEE Institute of Electrical and Electronics Engineers (SDO for link-layer protocols and more)
- IESG Internet Engineering Steering Group (IETF's governing body with RFC approval authority)
- IETF Internet Engineering Task Force (SDO for Internet standards)
- IGD, IGDDC Internet Gateway Device/Discovery and Control (a UPnP protocol for discovering and configuring gateway devices such as home NATs)
- IGMP Internet Group Message Protocol (a protocol to manage IPv4 multicast groups; used by routers and end hosts)
- IHL Internet Header Length (IPv4 header field indicating the header length in 32-bit words)
- IID Interface Identifier (numeric identifier usually based on MAC address; used when choosing IPv6 addresses, but not used for this purpose when privacy extensions are enabled)
- IKE Internet Key Exchange (part of IPsec; a protocol to dynamically establish security associations including keys and operating parameters)
- IMAP Internet Message Access Protocol (used to retrieve e-mail headers and messages from servers)
- IMAPS IMAP over SSL/TLS (a secure protocol for fetching e-mail, supported by most e-mail programs)
- IN Internet (in DNS, the class name indicating Internet information)
- IND Inverse Neighbor Discovery (provides RARP-like function for IPv6)
- IP Internet Protocol (standard best-effort Internet packet protocol implementing a common abstract datagram on any link layer network)
- IPCP IP Control Protocol (in PPP, an NCP used to configure an IPv4 network link)
- IPG Inter-Packet Gap (minimum spacing between frames in a MAC protocol)
- IPsec IP Security (a framework for securing IP traffic, including the IKE, AH, and ESP protocols)
- IPv6CP IPv6 Control Protocol (in PPP, an NCP used to configure an IPv6 network link)

- IRIS** Internet Registry Information Service (database containing information relating address ranges, associated AS numbers, contact information, and name servers)
- IRTF** Internet Research Task Force (research groups affiliated with IETF via the IAB)
- ISAKMP** Internet Security Association and Key Management Protocol (in IPsec, SA establishment protocol pre-dating IKE)
- ISATAP** Intra-Site Automatic Tunnel Addressing Protocol (an automatic IPv6-to-IPv4 tunneling technology supported by Microsoft)
- ISDN** Integrated Services Digital Network (combination circuit/packet switched data service)
- IS-IS** Intermediate System to Intermediate System (ISO link-state routing protocol)
- ISL** Cisco's Inter-Switch Protocol (Cisco's protocol for maintaining VLAN information among switches)
- ISM** Industrial, Scientific, and Medical (licence-free frequency bands in much of the world, used by Wi-Fi)
- ISN** Initial Sequence Number (in TCP, the first sequence number for a connection; assigned to the SYN)
- ISO** International Organization for Standardization (SDO responsible for defining various protocols and encodings once considered for replacing TCP/IP)
- ISOC** Internet Society (Internet standards leadership nonprofit corporation)
- ISP** Internet Service Provider (an entity, often a business, that allocates addresses, provides DNS and routing, and works with other ISPs)
- ITU** International Telecommunications Union (SDO for radio and telephony standards)
- ITU-T** ITU Telecommunication Standardization Sector (formerly CCITT; one of the three "sectors" of ITU responsible for standards or "recommendations" such as ASN.1, X.25, DSL)
- IW** Initial Window (in TCP, the initial value of CWND)
- IXFR** Incremental Zone Transfer (incremental exchange of DNS zone information, uses TCP)
- KE** Key Exchange (with IKE, payload used for establishing keys; generally uses DH)
- KSK** Key Signing Key (a key used with DNSSEC for signing other keys; typically has the SEP bit set)
- L2TP** Layer 2 Tunneling Protocol (IETF standard link layer tunneling protocol)

-
- LACP Link Aggregation Control Protocol (part of IEEE 802.1AX for managing link aggregates)
- LAG Link Aggregation Group (set of links acting together as one virtual higher-performance link)
- LAN Local Area Network (a network within a small geographic area such as a single site, office, or home)
- LCG Linear Congruential Generator (a deterministic type of popular PRNG, which is not a CSPRNG)
- LCI Location Configuration Information (data representing the location—geographical or civic—of a system)
- LCI Logical Channel Identifier (in circuit switching, identifier for a virtual channel)
- LCN Logical Channel Number (in circuit switching, number of a virtual channel)
- LCP Link Control Protocol (in PPP, used to establish a link)
- LDAP Lightweight Directory Access Protocol (a lookup protocol based on the ISO X.500 DAP protocol)
- LDRA Lightweight DHCP Relay Agent (mechanisms to allow layer 2 devices to act as DHCP relay agents)
- LEAP Lightweight Extensible Authentication Protocol (Cisco's EAP method using WEP or TKIP keys; now known to have vulnerabilities)
- LLA Link Layer Address (in FMIPv6, a mobility header option to indicate link layer address)
- LLC Logical Link Control (sublayer of the MAC layer related to link control)
- LLMNR Link Local Multicast Name Resolution (a multicast variant of DNS designed for on-link use and that runs on a different port number than DNS; used for local service and node discovery)
- LMQI Last Member Query Interval (in IGMP and MLD, the time between group-specific query messages)
- LMQT Last Member Query Time (in IGMP and MLD, the total spent after sending a last member query and possible transmissions; represents the “leave latency”)
- LNP Local Network Protection (a collection of techniques suggested for use in IPv6 deployments making NATs unnecessary)
- LoST Location-to-Service Translation (a framework for offering services based on location—e.g., indication of the nearest hospital)
- LQR Link Quality Reports (in PPP, reports of link quality measurements including number of packets received, sent, and rejected due to errors)

- LTE** Long-Term Evolution (3GPP wireless broadband standard; an evolution of HSPA)
- LW-MLD** Lightweight MLD (variant of MLD with simpler join/leave semantics)
- MAC** Media Access Control (controls for mediating access to a shared network medium, usually a portion of the link layer protocol)
- MAC** Message Authentication Code (a mathematical function used to help verify the integrity of a message)
- MAN** Metropolitan Area Network (a network spanning a modest geographical extent, such as a city or region)
- MCS** Modulation and Coding Scheme (combination of modulation and coding, many combinations are available in 802.11n)
- MD** Message Digest Algorithms (mathematical functions giving a short numeric “fingerprint” for a larger message)
- mDNS** Multicast DNS (local variant of name service developed by Apple)
- MIH** Media-Independent Handoff (mechanisms to support change of network attachment point between heterogeneous networks; the IEEE 802.21 standard covers MIH for 802.3, 802.11, 802.15, 802.16, 3GPP, and 3GPP2 network types)
- MII** Media-Independent Interface (in hardware, the interface between the MAC implementation and PHY protocol implementation, which is PHY-independent)
- MIME** Multipurpose Internet Mail Extensions (method for labeling and encoding various object types in electronic mail)
- MIMO** Multiple Input, Multiple Output (wireless antenna scheme with multiple antennas offering performance superior to single-antenna systems but requiring more sophisticated signal processing)
- MIP** Mobile IP (IP addressing and routing extensions to support movement of network attachment point without address change)
- MITM** Man-in-the-Middle attack (the typical form of an MSM attack, carried out by an interposer)
- MLD** Multicast Listener Discovery (used by IPv6 routers to discover multicast receivers on a link; provides similar capabilities as IGMP for IPv4)
- MLPP** Multilevel Precedence and Preemption (telephone scheme to prioritize calls—e.g., for military use)
- MMRP** Multiple MAC Registration Protocol (part of MRP used for registering multicast interest)
- MN** Mobile Node (the moving node in a MIP scenario)

-
- MOBIKE** Mobile version of IKE (enhancements to IKE to support mobility and change of addressing information)
- MODP** Modulo-P groups (groups based on modular arithmetic, in the abstract algebraic sense, used with key establishment protocols)
- MoS** Mobility Services (portion of the IEEE 802.21 standard supporting media-independent handoff services)
- MP** Mesh Point (name of a node in IEEE 802.11s operating in a mesh configuration)
- MP, MPPP, MLP, MLPPP** Multi-link PPP (using PPP over multiple links simultaneously)
- MPDU** MAC Protocol Data Unit (name of the frame used in 802.11 standards)
- MPE** Manchester Phase Encoding (bit encoding scheme where a voltage transition indicates one bit)
- MPLS** Multi-Protocol Label Switching (architecture that switches frames based on tag values, not IP addresses)
- MPPC** Microsoft's Point-to-Point Compression (used with PPP)
- MPPE** Microsoft's Point-to-Point Encryption (used with PPP)
- MPV** Maximum Pad Value (in PPP, maximum number of pad bytes)
- MRD** Multicast Router Discovery (protocol to discover on-link multicast router neighbors)
- MRP** Multiple Registration Protocol (IEEE 802.1ak standard for registering attributes)
- MRRU** Multilink Maximum Received Reconstructed Unit (MRU after reconstruction from parts on multiple MP links)
- MRU** Maximum Receive Unit (largest packet/message size a receiver will accept)
- MS-CHAP** Microsoft's Challenge-Handshake Authentication Protocol (an authentication protocol involving a request/replay and validated response, with two versions: MS-CHAPv1 and MS-CHAPv2)
- MSDU** MAC Services Data Unit (802.11 frame type available to layers above MAC)
- MSK** Master Session Key (a key derived after an EAP session using methods supporting key derivation)
- MSL** Maximum Segment Lifetime (in TCP, the maximum time a segment can exist in the network before being determined invalid)
- MSM** Message Stream Modification (active modification of messages; usually a type of attack)

- MSS** Maximum Segment Size (in TCP, the largest segment a receiver is willing to receive; usually provided in an option during connection establishment)
- MTU** Maximum Transmission Unit (maximum frame size a network will transport)
- MVRP** Multiple VLAN Registration Protocol (part of MRP used for registering VLANs)
- MX** Mail Exchanger (DNS RR indicating a priority order of hosts willing to use SMTP to exchange mail)
- NAC** Network Access Control (process employed to determine whether a device should receive access rights to use a network)
- NACK** Negative Acknowledgment (an indication of non-receipt or non-acceptance)
- NAP** Network Access Protection (Microsoft's variant of NAC; first available with Windows Server 2008)
- NAPT** NAT with Port Translation (NAT with port re-writing, the most common form of NAT)
- NAPTR** Name Authority Pointer (DNS RR used with a DNS-based DDDS for holding re-writing rules)
- NAR** New Access Router (in FMIPv6, router that is expected to be used soon)
- NAT** Network Address Translation (mechanism to re-write addresses in IP datagrams; used primarily to reduce the usage of globally routable IP addresses; usually used in conjunction with private IP addresses; also supports a type of firewall capability)
- NAT64** IPv6/IPv6 NAT (a NAT that translates between IPv4/ICMPv4 and IPv6/ICMPv6 and vice versa; proposed for IPv6/IPv4 interoperability and coexistence)
- NAT-PMP** NAT Port Mapping Protocol (an alternative to IGD developed by Apple for configuring some NAT devices; provides the ability to remotely set up port forwarding)
- NAT-PT** NAT with Protocol Translation (now-deprecated approach to IPv4/IPv6 translation)
- NAV** Network Allocation Vector (time delay before sending due to other stations' channel use in 802.11 DCF)
- NBMA** Non-Broadcast Multiple Access (multi-user networks lacking broadcasting capability)
- NCoA** New Care-of Address (in FMIPv6, CoA to be obtained from NAR)
- NCP** Network Control rotocol (in PPP, used to establish the network-layer protocol)

- ND, NDP Neighbor Discovery (IPv6 method to discovery and obtain MAC address of on-link neighbors; works like ARP; implemented as part of ICMPv6)
- NEMO Network Mobility (mobility where a router and network changes attachment point)
- NIC Network Interface Card (the device interfacing a computer with a network)
- NONCE number used once (a random value used in many cryptographic protocols to combat replay attacks)
- NPT66 IPv6-to-IPv6 NAPT (NAT with algorithmic address and port translation)
- NRO Number Resource Organization (the Address Supporting Organization to ICANN)
- NS Name Server (DNS RR carrying the name of another name server)
- NS Neighbor Solicitation (part of IPv6 ND; similar to an IPv4 ARP request but uses IPv6 multicast addressing; implemented using ICMPv6)
- NSCD Name Services Cache Daemon (process to provide caching for DNS and other resolutions popular on UNIX systems)
- NSEC Next Secure (DNS RR used with DNSSEC to indicate the next RR in an ordered list; used for authenticated denial of existence)
- NSEC3 Next Secure (version 3) (DNS RR like NSEC but including hash function to resist DNS name enumeration attacks)
- NSEC3PARAM NSEC Parameters (DNS RR used with DNSSEC holding NSEC3 hash function parameters)
- NTN Non-Terminal NAPTR (in DNS, a NAPTR pointing to another domain with records)
- NTP Network Time Protocol (a protocol for synchronizing clocks)
- NUD Neighbor Unreachability Detection (in IPv6 ND, to determine if a neighbor can still be reached)
- OCSF Online Certificate Status Protocol (a protocol for checking the validity of a certificate; an alternative to obtaining a CRL)
- OFDM Orthogonal Frequency Division Multiplexing (a sophisticated modulation scheme in which subcarriers of multiple frequencies are simultaneously modulated in a specified bandwidth to achieve high throughput; used by DSL, 802.11a/g/n, 802.16e, and advanced cellular data standards including LTE)
- OID Object Identifier (numeric identifier of a digital object; used in certificate encodings)
- OLSR Optimized Link State Routing (a standard protocol for on-demand routing in ad-hoc networks)

- OOB Out Of Band (information delivered outside a primary communication channel)
- ORO Option Request Option (in DHCP, an option indicating a systems interest in knowing which options are supported)
- OSI Open System Interconnect (an abstract reference model specified by ISO for open systems that helped form the basis of layered design in protocols)
- OUI Organizationally Unique Identifier (original MAC-layer address prefix format defined by IEEE)
- P2P Peer-to-Peer (participating systems are both clients and servers)
- PA Provider-Aggregatable (IP address space where a customer's prefix is given by their provider)
- PAA PANA Authentication Agent (PANA agent performing authentication, such as an AAA server)
- PaC PANA Client (PANA agent requesting authentication)
- PAD Peer Authentication Database (with IPsec, abstraction of database containing authentication information for each peer such as use of IKE or PSK and associated authentication data)
- PANA Protocol for Carrying Authentication for Network Access (UDP/IP carrier for EAP)
- PAP Password Authentication Protocol (protocol that carries cleartext password; vulnerable to MITM or eavesdroppers)
- PAWS Protection Against Wrapped Sequence Numbers (in TCP, method using TSOPT values to notice sequence number wrapping)
- PCF Point Coordinating Function (combined contention-free and contention-based MAC protocol for 802.11; not widely used)
- PCO Phased Coexistence Operation (method for an 802.11 AP to switch channel widths for less negative impact on legacy equipment)
- PCoA Previous Care-of Address (in FMIPv6, current or previous CoA obtained from PAR)
- PCP Port Control Protocol (current-generation draft IETF protocol for configuring NATs including SPNATs and NAT64)
- PDU Protocol Data Unit (describes a message at some protocol layer; sometimes used interchangeably and informally with packet, frame, datagram, segment, or message)
- PEAP Protected Extensible Authentication Protocol (a popular method to encapsulate EAP in TLS; similar to EAP-TTLS)
- PEN Private Enterprise Number (numbers assigned by IANA usable by an enterprise in forming OIDs)

-
- PFC Protocol Field Compression (in PPP, eliminating the Protocolfield to reduce overhead)
- PFS Perfect Forward Secrecy (in public key cryptography, the property by which compromise of one key leads at most to the compromise of data encrypted with that key and not other data or keys)
- PHB Per-Hop Behavior (abstract behavior at router used to implement DS)
- PHY Physical (a layer in the OSI; usually describes connectors, frequencies, coding, and modulation)
- PI Provider-Independent (IP address space owned by a customer; not derived from an ISP's address prefix)
- PIM Protocol Independent Multicast (non-local multicast routing protocol that can leverage unicast routing protocols' data and operations)
- PIO Prefix Information Option (in ICMPv6, an option carrying an IP address prefix)
- PKC Public Key Certificate (a digital object including a public key and signature from a CA, along with various usage policies and parameters)
- PKCS Public Key Cryptography Standards (methods to encode and represent public key and related material)
- PKI Public Key Infrastructure (system for managing and distributing public keys)
- PLCP Physical Layer Convergence Procedure (802.11 method for encoding and determining frame type and radio parameters)
- PMTU Path MTU (minimum MTU across links on the path from sender to receiver)
- PMTUD PMTU Discovery (process of determining the PMTU; usually depends on ICMP PTB messages)
- PNAC Port-Based NAC (a version of NAC wherein the physical port of attachment is used in making an authorization decision)
- PoE Power over Ethernet (carries device power over Ethernet wiring)
- POTS Plain Old Telephone Service (conventional analog telephone service)
- PPP Point-to-Point Protocol (a link-layer configuration and data encapsulating protocol capable of carrying multiple network layer protocols and using multiple underlying physical links)
- PPPoE PPP over Ethernet (methods to establish a PPP association over an Ethernet link)
- PPTP Point-to-Point Tunneling Protocol (Microsoft's link layer tunneling protocol)

- PRF** Pseudorandom Function Family (a set of functions that cannot be distinguished from truly random functions using a polynomial-time algorithm; also sometimes used less formally to refer to a single such function)
- PRNG, PRG** Pseudo-Random Generator (a mathematical function used to compute a series of random-appearing values)
- PSK** Pre-Shared Key (pre-placing encryption keys; no dynamic key exchange protocol used)
- PSM** Power Save Mode (a mode of 802.11 where devices may “sleep” when not busy and poll to receive their information from an AP at a later time)
- PSMP** Power-Save Multi-Poll (bi-directional version of APSD, part of 802.11n)
- PTB** Packet Too Big (a ICMP Destination Unreachable Fragmentation Required or IPv6 Packet Too Big message indicating a packet is too large for the next-hop MTU size)
- QAM** Quadrature Amplitude Modulation (combination of phase and amplitude modulation)
- QBSS** QoS BSS (an 802.11 BSS enhanced with 802.11e or 802.11n QoS features)
- QI** Query Interval (in IGMP and MLD, time between general queries)
- QoS** Quality of Service (general term describing how traffic can be handled differently, usually with better or worse latency or drop precedence, based on configuration parameters)
- QPSK** Quadrature Phase Shift Keying (typically, modulating two bits per symbol typically using four signal phases, although more advanced versions with more bits per symbol are possible)
- QQI** Querier's Query Interval (in IGMP and MLD, time between sending general query messages; current non-querier multicast routers adopt the most recently received QQI value as their QI value)
- QQIC** Querier's Query Interval Code (in IGMP and MLD messages, encoding of the QQI value)
- QRI** Query Response Interval (in IGMP and MLD, the maximum amount of time a receiver is permitted to send a response to a query)
- QRV** Querier Robustness Variable (in IGMP and MLD, sets number of retransmissions)
- QS** Quick Start (in TCP, an experimental modification for faster startup behavior provided devices on the path agree)
- QSTA** QoS STA (an 802.11 STA supporting QoS capabilities)
- RA** Router Advertisement (message indicating presence of an on-link router neighbor; uses ICMP)

- RADIUS** Remote Authentication Dial-In User Service protocol (a popular protocol for carrying AAA data)
- RAIO** Relay Agent Information Option (in DHCPv6, an option used by relays to insert various bits of information)
- RARP** Reverse ARP (protocol providing network layer to MAC layer address mappings)
- RAS** Remote Access Server (a server that handles remote users—authentication, access control, etc.)
- RC4** Rivest Cipher #4 (a popular symmetric key encryption scheme designed by Ron Rivest)
- RD** Router Discovery (procedure to locate a proximal router; uses ICMP)
- RDATA** Returned Data (part of the DNS protocol used to hold returned data)
- RDNSS** Recursive DNS Server (used in RAs; indicates address of DNS server)
- RED** Random Early Detection (an AQM scheme that marks or drops packets with increasing probability when persistent congestion appears to be growing)
- RFC** Request for Comments (documents published by IETF; some are standards)
- RGMP** Router-port Group Management Protocol (Cisco's protocol to enable IGMP snooping)
- RH** Routing Header (an IPv6 extension header that alters traffic delivery path)
- RHBP** Rate Halving with Bounded Pacing (in TCP, an evolved version of the FACK algorithm to help spread retransmissions more evenly across an RTT period after inferred packet loss)
- RIP** Routing Information Protocol (small organization routing protocol; the original version does not support subnet masks)
- RIR** Regional Internet Registry (allocates address space for some region of the world)
- RO** Route Optimization (improving routes from indirect “dogleg” paths used in simple MIP)
- ROAD** Running Out of Address Space (a problem motivating the creation of IPv6 and resulting in the creating of CIDR)
- ROHC** Robust Header Compression (current-generation standards for protocol header compression)
- RP** Rendezvous Point (used with multicast routing to exchange group information)
- RPC** Remote Procedure Call (a framework supporting a program's procedure calls to be handled remotely)

- RPF Reverse Path Forwarding (to avoid loops, an RPF check is performed by multicast routers to ensure a multicast datagram arrives on the same interface used to reach the sender)
- RPSL Routing Policy Specification Language (a language used to express routing policies such as which network prefix corresponds to which owning AS)
- RR Resource Record (a typed information block owned by a domain name and distributed via DNS)
- RRP, RR Return Routability/Procedure (a check used with MIPv6 to ensure a mobile node is authentic, and includes a HoA check and CoA check)
- RRset Resource Record Set (a collection of DNS RRs with same domain name owner and class)
- RRSIG Resource Record Signature (DNS RR used with DNSSEC holding a signature on an RRset)
- RS Router Solicitation (an ICMP message that induces a router to produce a response)
- RSA Rabin, Shamir, Adelman (the most popular public key cryptography algorithm)
- RSN Robust Security Network (improved security in IEEE 802.11i/WPA; included in 802.11 standard)
- RSNA RSN Association (full use/implementation of RSN)
- RST Reset (a TCP header bit and segment type that causes a TCP connection abort)
- RSTP Rapid Spanning Tree Protocol (decreased latency version of STP)
- RTO Retransmission Timeout (time before retransmitting data thought to be lost)
- RTS Request To Send (message indicating desire to send a subsequent message)
- RTT Round Trip Time (minimum time to expect a response from a communication peer)
- RTTM RTT Measurement (an instantaneous estimate of the RTT)
- RTTVAR RTT Variance (in TCP, time-averaged estimate of a connection's RTT deviation)
- RTX Retransmission (re-sending of data)
- RW Restart Window (in TCP, CWND value when TCP restarts sending after an idle period)
- SA Security Association (in IPsec, state pertaining to a unidirectional association between peers; includes agreed-upon keys, algorithms, etc.; an SA can be unicast or multicast)

-
- SACK** Selective Acknowledgment (in TCP, an option indicating correctly received out-of-sequence data)
- SAD** Security Association Database (in IPsec, abstraction of database containing information on each active SA; logically indexed by SPI)
- SAE** Simultaneous Authentication of Equals (form of authentication used with 802.11s)
- SAP** Session Announcement Protocol (carries experimental multicast session announcements; see also SDP)
- SCSV** Signaling Cipher Suite Value (in TLS, a CS value that indicates not a CS but a particular set of alternative functions or options)
- SCTP** Stream Control Transport Protocol (a reliable transport protocol alternative to TCP that does not enforce strict ordering and supports multiple sub-streams and endpoint address changes)
- SCVP** Server-Based Certificate Verification Protocol (a protocol supporting DPD and DPV for certificates)
- SDID** Signing Domain Identifier (with DKIM, name for the domain of the signer)
- SDLC** Synchronous Data Link Control (a precursor to HDLC, the link layer of SNA)
- SDO** Standards-Defining Organization (including IEEE, IETF, ISO, ITU, 3GPP, 3GPP2)
- SDP** Session Description Protocol (a protocol that describes multimedia sessions)
- SEND** Secure Neighbor Discovery (a secure variant of ND using CGAs)
- SEP** Secure Entry Point (in DNSSEC, indicates a DNSKEY RR contains a KSK)
- SFD** Start Frame Delimiter (bit pattern indicating the starting portion of frame in a link PDU)
- SG** Security Gateway (with IPsec, system terminating IPsec protocols, often at network edge)
- SHA** Secure Hash Algorithm (one of a set of hashing algorithms suitable for ensuring message integrity)
- SIFS** Shorts Inter-Frame Space (smallest amount of time between an 802.11 frame and its ACK)
- SIIT** Stateless IP/ICMP Translation (a framework for translation between IPv4 and IPv6, including special rules for ICMP translation, NAT64, and DNS64)
- SIP** Session Initiation Protocol (general signaling protocol; used with VoIP)
- SLAAC** Stateless Address Autoconfiguration (a mechanism whereby a node self-configures its own IP address; usually applies to IPv6 nodes)

- SLLAO** Source Link-Layer Address Option (in ICMPv6, an option carrying the sender's link layer address)
- SMSS** Sender's MSS (the MSS for a connection as viewed by the sender)
- SMTP** Simple Mail Transfer Protocol (a protocol to carry e-mail in transit among mail transfer agents)
- SNA** Systems Network Architecture (IBM's network architecture)
- SNAP** Subnetwork Access Protocol (IEEE terminology for 802.2 encapsulation; rare for TCP/IP networks)
- S-NAPTR** Straightforward NAPTR (simplified NAPTR where AUS maps directly to result without regular expression substitution)
- SNMP** Simple Network Management Protocol (status reporting and configuration settings for network equipment; usually used with UDP/IP)
- SOA** Start of Authority (DNS RR indicating meta-data about a zone)
- SOAP** (formerly) Simple Object Access Protocol (a web services application protocol using XML, which provides RPC-like capabilities; SOAP is no longer an acronym)
- SPD** Security Policy Database (with IPsec, abstraction of database containing security policies applying to how traffic is handled—e.g., discard, bypass, or protect)
- SPI** Security Parameter Index (in IPsec, a logical index into the SAD to indicate security parameters, either 32 or 64 bits)
- SPNAT, CGN, LSN** Service-Provider ("large scale") NAT (a NAT deployment arrangement where address translation is performed by a service provider instead of a customer)
- SRP** Secure Remote Password (a strong key agreement protocol based on passwords; being supported by various security protocols such as TLS and EAP)
- SRTP** Secure Real-Time Protocol (a secure variant of the UDP/IP based real-time protocol; typically used to carry multimedia information)
- SRTT** Smoothed RTT (in TCP, time-averaged estimate of a connection's RTT)
- SSDP** Simple Service Discovery Protocol (an IETF-specified distributed service discovery protocol designed for LANs and residential networks used by UPnP)
- SSH** Secure Shell Protocol (secure remote login/execution protocol; also supports tunneling of other protocols)
- SSID** Service Set Identifier (802.11 network name)
- SSL** Secure Sockets Layer (encrypted and integrity-protected layer above TCP; precursor to TLS)

-
- SSM Single-Source Multicast (multicast wherein only a single party can source traffic to a particular group)
- STA Station (IEEE 802.11 terminology for an access point or associated wireless host)
- STP Spanning Tree Protocol (protocol used among bridges and switches to avoid loops)
- STUN Session Traversal Utilities for NAT (a client/server protocol for helping to fix the address and port number of a traffic flow when passing through a NAT)
- SWS Silly Window Syndrome (in protocols using window-based flow control, an undesirable situation where small amounts of data are exchanged due to the use of small window sizes)
- SYN Synchronize (a TCP header bit and first segment type sent on a TCP connection)
- TCP Transmission Control Protocol (a connection-oriented reliable stream protocol lacking message boundaries, which includes flow and congestion control)
- TCP-AO TCP Authentication Option (in TCP, an algorithm-agile mechanism to combat MSM attacks)
- TDES, 3DES Triple DES (encryption using three rounds of DES encipherment, resulting in an effective key length of 112 bits)
- TDM Time Division Multiplexing (sharing by allocation of separate usage time slots)
- TFC Traffic Flow Confidentiality (in IPsec, methods to disguise the traffic flow even when encrypted, including padding and generation of dummy packets)
- TFRC TCP Friendly Rate Control (methods to control the sending rate of a protocol so as to not compete unfairly with a TCP flow in a similar operating environment)
- TFTP Trivial File Transfer Protocol (UDP/IP-based simple transfer protocol)
- TKIP Temporal Key Integrity Protocol (replaced the WEP encryption algorithm for WPA)
- TLD Top-Level Domain (a top-level domain name such as EDU, COM, UK, ZA)
- TLS Transport Layer Security (based on the SSL protocol developed by Netscape)
- TLV Type/Length Value (used in protocols; indicates a type, length of variable-length value, and the value)
- ToS Type of Service (older name for the IPv4 header byte indicating type of service; replaced with DS Field and ECN bits)

- TS Traffic Selector (with IKE, specifications for identifying traffic such as IP address range, port number, etc.)
- TSER, TSer Timestamp Echo Reply (in TCP, portion of TSOPT used to echo TSV value to peer)
- TSF Time Synchronization Function (establishes a common time in an 802.11 BSS)
- TSIG Transaction Signatures (signatures used to secure individual DNS transactions, not content from its origin)
- TSOPT Timestamps Option (in TCP, an option including the TSV and TSER values)
- TSPEC Traffic Specification (a structure indicating traffic parameters for 802.11 QoS)
- TSV Timestamp Value (in TCP, portion of TSOPT used to identify the sender's time—used in RTTM and PAWS)
- TTL Time-to-Live (IPv4 header field indicating number of remaining router hops allowed for a datagram)
- TURN Traversal Using Relay NAT (a protocol in which a third party relays information between hosts that are otherwise unable to communicate due to the presence of one or more NATs)
- TWA Time-Wait Assassination (in TCP, an erroneous condition caused by receiving certain segments during TIME-WAIT state)
- TXOP Transmission Opportunity (in 802.11, a form of “credit” allowing a station to send one or more frames)
- TXT Text (DNS RR carrying descriptive text; used by DKIM)
- UBM Unicast Prefix-based Multicast addressing (deriving multicast addresses based on assigned unicast prefixes)
- UDL Unidirectional Link (link providing communication in only one direction)
- UDP User Datagram Protocol (a best-effort message protocol with message boundaries and lacking congestion or flow control)
- UEQM Unequal Modulation (using different modulation schemes on different data streams simultaneously)
- ULA Unique Local IPv6 Unicast Addresses (private addresses used with IPv6, allocated from the fc00::/7 prefix)
- U-NAPTR URI-enabled NAPTR (simplified NAPTR allowing limited regular expression substitution)
- U-NII Unlicensed National Information Infrastructure (unlicensed radio spectrum in much of the world)

- UNSAF Unilateral Self-Address Fixing (heuristics used in an attempt to determine how a traffic flow is identified after passing through a NAT; a fragile process for which techniques like ICE are recommended alternatives)
- UP User Priority (802.11 priorities; based on same terminology from 802.1d)
- UPnP Universal Plug and Play (a protocol framework for device and service discovery aimed at the residential user; standardized by the UPnP Forum)
- URG Urgent Mechanism (in TCP, a method for marking and indentifying information as “urgent”; not recommended for use)
- URI Universal Resource Identifier (string of characters identifying a name or resource on the Internet, including URLs and URNs)
- URL Uniform Resource Locator (informally, a “WWW address”)
- URN Universal Resource Name (a URI using the urn scheme not implying availability of resource)
- USRK Usage-Specific Root Key (key derived from an EMSK intended to be used for certain purposes)
- UTC Coordinated Universal Time (standard time used by NTP and other protocols; effectively interchangeable with GMT but with some technical differences)
- UTO User Timeout (in TCP, the maximum time a TCP sender will wait attempting to retransmit before abandoning a connection)
- VC Virtual Circuit (a simulated dedicated communication path)
- VLAN Virtual LAN (used most often to simulate multiple distinct LANs on shared wiring)
- VLSM Variable-Length Subnet Masks (proximal use of subnet masks of differing lengths in same environment)
- VoIP Voice over IP (the carriage of voice traffic over IP networks, usually involves SIP signaling)
- VPN Virtual Private Network (virtually isolated network; often encrypted)
- W3C World Wide Web Consortium (SDO defining web standards such as XML)
- WAN Wide Area Network (a network connecting geographically distributed sites; usually involving multiple administrative authorities)
- WEP Wired Equivalent Privacy (original WiFi encryption; found to be catastrophically weak)
- WESP Wrapped ESP (in IPsec, a method to prepend ESP with a header to indicate if the following traffic is encrypted or only authenticated; useful for inspection by middleboxes)

- Wi-Fi Wireless Fidelity (IEEE 802.11 wireless LAN standard)
- WiMAX Worldwide Interoperability for Microwave Access (IEEE 802.16 wire - less broadband standard)
- WKP Well-Known Prefix (a checksum neutral IPv6 prefix, 64:ff9b::/96, used in algorithmic mappings between IPv4 and IPv6 addresses)
- WLAN Wireless LAN (a wireless LAN such as WiFi)
- WMM Wi-Fi Multimedia (subset of 802.11e QoS functions now available in 802.11n)
- WoL Wake on LAN (method to remain in “sleep” mode until a particular packet is received)
- WPA WiFi Protected Access (802.11 encryption method)
- WPAD Web Proxy Autodiscovery Protocol (a protocol to discover the presence of a proximate WWW proxy)
- WRED Weighted RED (RED where the mark/drop probability is a function of traffic class and weight assignment)
- WSALE, WOPT, WSOPT Window Scale Option (in TCP, an option indicating a scaling factor is to be applied to the Window Size field)
- WWW World Wide Web (networked data environment using the HTTP/TCP/IP protocol suite)
- X.25 ITU-T recommendation X.25 (an ITU-T standard packet switched network standard covering OSI layers 1-3; the most popular packet switched technology until widespread use of TCP/IP)
- XML Extensible Markup Language (a set of rules for encoding documents in machine-readable form; extensively used by web services)
- XMPP Extensible Messaging and Presence Protocol (an open, extensible, HTML-based protocol for the exchange of messages, presence, and contact list information)
- ZSK Zone Signing Key (a key used with DNSSEC for signing zone contents, usually signed by a KSK)

Index

Symbols

- * (Wildcard)
 - domain names and, 526
 - local IP address restrictions in server design, 500–501
- .in-addr.arpa
 - classless delegation, 539
 - special domain for IPv4, 537–538
- .ip6.arpa, 537–538

Numbers

- 2.4GHz band, Wi-Fi, 124–126
- 3DES. See Triple-DES
- 3GPP (3rd Generation Partnership Project), 275, 933
- 5GHz band, Wi-Fi, 124–126
- 6rd (IPv6 Rapid Deployment), 339, 933
- 6to4
 - definition of, 933
 - IPv4 to IPv6 transition, 482

A

- A (address) records
 - definition of, 529, 933
 - overview of, 529–530
 - querying, 531
 - translating DNS from IPv4 to IPv6, 569
- A-MPDU (aggregated MAC protocol data unit)
 - definition of, 934
 - frame aggregation support, 118–119
- A-MSDU (aggregated MAC service data unit)
 - definition of, 934
 - frame aggregation support, 118
- AAA (authentication, authorization, and accounting), 833–834, 933
- AAAA (address) records
 - definition of, 933
 - DNS resource record types, 529–530
 - translating DNS from IPv4 to IPv6, 569
- Abbreviated handshake, TLS, 881
- ABC (Appropriate Byte Counting), in TCP, 733, 933
- Abortive release, of TCP connections, 627
- Abstract Syntax Notation One (ASN.1), 935
- Access categories (ACs), in EDCA, 123

- Access control
 - NAC (Network Access Control), 833–837
 - RADIUS server for, 141
- Access control lists. See ACLs (access control lists)
- Access Network Discovery and Selection Function (ANDSF), 275, 934
- Access points. See APs (access points)
- ACCM (Asynchronous Control Character Map), in PPP
 - definition of, 933
 - escaping characters and, 134–135
- ACD (Address Conflict Detection), 176–177, 933
- ACFC (Address and Control Field Compression), in PPP, 132, 933
- ACK clock, in TCP, 731
- ACK division, attack against TCP, 785
- Acknowledge Number field, in GRE tunnels, 150
- ACKs (acknowledgement)
 - clocking congestion via, 730–731
 - combined with SYN segments (SACK), 607
 - cumulative in TCP, 586–587
 - definition of, 933
 - duplicate ACK threshold in fast retransmit, 667
 - establishing TCP connections and, 597, 602–603
 - NAT and TCP and, 307–308
 - requesting connection to nonexistent TCP port, 626
 - retransmission and, 580–581
 - retransmission timeout settings, 584
 - stretch ACKs in recovery from local congestion, 754–757
 - TCP header field, 588–589
 - TCP segments and, 701
 - in Wi-Fi control frames, 116
 - window update and, 583
- ACLs (access control lists)
 - definition of, 933
 - in packet-filtering firewalls, 300
 - rules in, 335
- ACs (access categories), in EDCA, 123
- ACs (attribute certificates)
 - as alternative to public key certificates, 831
 - definition of, 933

- ASCII characters, escaping in PPP operations, 134–135
- Actions, in ACL rules, 335
- Active attacks, threats to network communication, 807–809
- Active closer, FIN segments and, 597
- Active open, in TCP connections, 597
- Active opener (client)
 - RST segments, 631
 - simultaneous open and, 600
 - in TCP connections, 596, 599
- Active queue management (AQM), 782–785, 935
- Ad hoc mode, Wi-Fi, 112
- Ad-Hoc On-Demand Distance Vector (AODV)
 - definition of, 934
 - Wi-Fi mesh and, 130
- Additive increase/additive decrease (AIAD)
 - congestion control and, 777
 - definition of, 934
- Additive increase/multiplicative decrease (AIMD)
 - congestion control and, 769
 - definition of, 934
- Address (A) records. *See* A (address) records
- Address (AAAA) records. *See* AAAA (address) records
- Address (Addr) field, in PPP frames, 132
- Address and Control Field Compression (ACFC), in PPP, 132, 933
- Address autoconfiguration. *See* SLAAC (stateless address autoconfiguration)
- Address behavior, in NAT, 311–313
- Address Conflict Detection (ACD), 176–177, 933
- Address Family Transition Router (AFTR), in DS-Lite, 340, 934
- Address management, DHCP for, 235
- Address pools, DHCP, 235–236
- Address realms
 - IP addresses, 299
 - proxy firewalls supporting private address realms, 301
- Address Resolution Protocol (ARP), 165
- Address selection, in IP host models
 - destination address selection algorithm, 224–225
 - overview of, 222–223
 - source address selection algorithm, 223–224
- Address unreachable message, ICMPv6, 364
- Admin-scope boundaries, in router configuration, 53
- Administrative prohibition, ICMP messages and, 365
- Administrative scope, in multicast addresses, 53
- ADSP (Author Domain Signing Practices), in DKIM, 916–917, 933
- Advanced Encryption Standard. *See* AES (Advanced Encryption Standard)
- ADVVERTISE message, DHCPv6, 262–264
- Advertised window. *See* *w*nd (advertised window)
- Advertisement Interval option, neighbor discovery in IPv6, 412
- Advertisement messages, in MRD, 394–395
- AEAD (authenticated encryption with associated data), 820, 934
- AES (Advanced Encryption Standard)
 - definition of, 934
 - standardized for Internet use, 819
 - as symmetric encryption algorithm, 811
 - in Wi-Fi security, 129
- AES-MAC, 819
- AF (Assured Forwarding), 190, 934
- AFTR (Address Family Transition Router), in DS-Lite, 340, 934
- Aggregated MAC service data unit. *See* A-MSDU (aggregated MAC service data unit)
- Aggregation
 - route aggregation, 50
 - of Wi-Fi frames, 116–119
- Agile probing, in TCP, 779
- AH (Authentication Header)
 - authentication and integrity protection with, 856, 858
 - definition of, 934
 - fields in, 856
 - in IPSec, 841
 - NAT updates and, 866
 - overview of, 454–455
 - transport and tunnel modes, 856–857
- AIA (Authority Information Access)
 - certificate extension, 828
 - definition of, 934
- AIAD (additive increase/additive decrease)
 - congestion control and, 777
 - definition of, 934
- AIMD (additive increase/multiplicative decrease)
 - congestion control and, 769
 - definition of, 934
- Alert protocol, TLS handshaking, 880
- ALGs (application layer gateways). *See also* Gateways
 - definition of, 934
 - IP routers, 20
 - IPv4/IPv6 translation, 340–345
 - NAT Traversal as alternative to, 316
 - proxy firewalls and, 301
- Allocation of IP addresses
 - multicast addresses, 65
 - overview of, 62
 - unicast addresses, 62–65
 - to users and organizations, 31

- Alternate ports
 - RSTP, 110
 - STP, 104–105
- Amplification attacks, DNS-related attacks, 571
- ANDSF (Access Network Discovery and Selection Function), 275, 934
- Answer, authority, and additional information section, of DNS message, 526–527
- Any-source multicast (ASM)
 - definition of, 935
 - as multicast service model, 54
- Anycast addresses, 62
- AODV (Ad-Hoc On-Demand Distance Vector)
 - definition of, 934
 - Wi-Fi mesh and, 130
- APIPA (Automatic Private IP Addressing)
 - definition of, 934
 - SLAAC and, 276, 284
- APIs (Application Programming Interfaces)
 - definition of, 934
 - design and, 22
- Application design
 - APIs in, 22
 - client/server design pattern, 20–21
 - peer-to-peer design pattern, 21–22
- Application layer
 - full-duplex TCP service to, 587
 - of OSI model, 10
 - TCP and UDP services for, 585
- Application-managed keepalives, 794
- Application Programming Interfaces. *See* APIs (Application Programming Interfaces)
- Application protocols, NAT and, 304
- Application-unique strings. *See* AUS (application-unique strings)
- Appropriate Byte Counting (ABC), in TCP, 733, 933
- APs (access points)
 - definition of, 934
 - ICMP fast handover messages and, 388
 - Wi-Fi, 112
- APSD (automatic power save delivery), 120, 935
- AQM (active queue management), 782–785, 935
- Architecture, protocol
 - end-to-end argument, 6
 - error control and flow control, 7–8
 - rate sharing, 6–7
 - packets, connections, and datagrams, 3–6
 - principles of, 2–3
 - of protocol suite, 1
- ARM (ARPANET Reference Model), 1–2, 13–16
- ARP (Address Resolution Protocol)
 - ACD (Address Conflict Detection), 176–177
 - announcement packets, 176
 - arp command, 177–178
 - attacks related to, 178–179
 - cache, 169–170
 - cache timeout, 174
 - definition of, 935
 - determining MAC addresses, 442
 - direct delivery and, 167–169
 - example of use, 166–167
 - frame format, 170–171
 - gratuitous ARP, 175–176
 - interaction between IP fragmentation and ARP/ND, 496–497
 - introduction to, 165–166
 - IPv4 and, 13
 - operation of, 171–173
 - Proxy ARP, 174–175
 - request to nonexistent host, 173–174
 - setting IPv4 address for embedded device, 178
 - summary and references, 179–180
 - TCP connection timeouts and, 604
- arp command
 - ARP cache timeout and, 174
 - examining ARP cache, 169–170
 - options, 177–178
- ARP hack, 175
- ARP poisoning, attacks on ICMP, 429
- ARP probe, ACD defining, 176
- ARP reply frames, 168
- ARP request frames
 - direct delivery and, 167
 - Proxy ARP and, 174–175
 - request to nonexistent host, 173–174
- ARPANET Reference Model (ARM), 1–2, 13–16
- ARQ (Automatic Repeat Request), 579–581, 935
- AS (Authentication Server), in PANA, 935
- AS (autonomous system)
 - definition of, 935
 - multicast addresses based on, 55
- ASM (any-source multicast)
 - definition of, 935
 - as multicast service model, 54
- ASN.1 (Abstract Syntax Notation One), 935
- Assignment of unicast addresses
 - to devices, 32
 - multiple providers/multiple networks/multiple addresses, 68–70
 - overview of, 65–66
 - single provider/multiple networks/multiple addresses, 67–68
 - single provider/no network/single address, 66–67
 - single provider/single network/single address, 67
- Assignment policies, IA (Identity Association) based on, 255–256

Assured Forwarding (AF), 190, 934
Asymmetric (public) key ciphers. *See also* Public key cryptography, 809–812
Asynchronous Control Character Map (ACCM)
 definition of, 933
 escaping characters and, 134–135
Attacks
 ARP, 178–179
 DNS, 571–572
 ICMP, 428–429
 Internet architecture, 25–26
 IP address, 70–71
 IP protocol, 226
 link layer, 154–156
 on NAT and firewall, 345–346
 system configuration, 292
 TCP, 640–643
 TCP congestion control and, 785–786
 TCP keepalive and, 802
 TCP timeout/retransmission and, 687
 TCP window management and, 723
 UDP, 507–508
Attribute certificates (ACs)
 as alternative to public key certificates, 831
 definition of, 933
Attribution, of datagrams, 26
Auditability, ESP and, 863
Augmented message, CRC, 86
AUS (application-unique strings)
 definition of, 935
 ENUM records and, 551–552
 NAPTR records and, 549
AUTH (authentication packets), in IKE, 935
Authenticated encryption, 814–815
Authenticated encryption with associated data (AEAD), 820, 934
Authenticated nonexistence, DNSSEC, 895
Authentication
 AH (Authentication Header). *See* AH (Authentication Header)
 basic principles of security, 806–807
 DHCP and, 271–273
 EAP methods for, 838
 PPP and, 140–141
 PSKs (preshared keys) for, 129–130
 SAE (Simultaneous Authentication of Equals), 130
 SHA-1 algorithm in, 268
 spoofing attacks and, 226
 TCP-AO (Authentication Option), 612
Authentication, authorization, and accounting (AAA), 833–834, 933

Authentication Header. *See* AH (Authentication Header)
Authentication Option (TCP-AO)
 definition of, 959
 TCP header, 612
Authentication Server (AS), in PANA, 55, 935
Author Domain Signing Practices (ADSP), 916–917, 933
Authorities, in allocation of IP addresses, 62
Authority Information Access (AIA)
 certificate extension, 828
 definition of, 934
Authority Key Identifier, for identifying public keys, 828, 830
Auto-proxy ARP, 175
Automatic power save delivery (APSD), 120, 935
Automatic Repeat Request (ARQ), 579–581, 935
Autonegotiation, in Ethernet
 duplex mismatch and, 96
Autonomous system (AS)
 definition of, 935
 multicast addresses based on, 55
Autotuning TCP receive windows, 715–716
Availability, in CIA triad, 806
awnd (advertised window) in TCP
 overview of, 729–730
 slow start algorithm and, 733–734, 736
AXFR (full zone transfer) messages, in DNS, 559–561, 935

B

B4 (Bridging Broadband), 340, 935
Backoff factor, RTO and, in TCP, 655
Backoff time, in MAC, 121–122
Backup ports, STP, 104–105
BACP (Bandwidth Allocation Control Protocol), 139, 935
Bandwidth Allocation Protocol (BAP), 139
Bandwidth (capacity)
 allocating in MP, 139
 buffer bloat and, 781
 connections and, 3
Bandwidth-delay product. *See* BDP (bandwidth-delay product)
Bandwidth on demand (BOD), 139, 936
Bandwidth-scalable TCPs, 773
Bank teller's algorithm, 138
BAP (Bandwidth Allocation Protocol), 139
Baran, Paul, 1
Basic Encoding Rules (BER), 935
Basic service set *See* BSS (basic service set)
BCMCS (Broadcast and Multicast Service Controller), 935

- BCP (best current practice) category, RFCs and, 23
- BDP (bandwidth-delay product)
 - congestion control and, 730
 - high-speed networks and, 770
 - HSTCP (HighSpeed TCP) and, 772
- BER (Basic Encoding Rules), 935
- BER (bit error rate)
 - data frame fragmentation, 117–118
 - definition of, 935
- Berkeley Internet Name Domain v. 9 (BIND9), 935
- Berkeley sockets
 - half-close support, 598
 - incoming connection queue and, 636, 639–640
 - popular APIs, 22
 - restrictions on foreign endpoints, 635
 - state transitions, 618
 - TCP ports, 588
 - TCP_NODELAY option for disabling Nagle algorithm, 700
- Berkeley Software Distribution. *See* BSD (Berkeley Software Distribution)
- Best current practice (BCP) category, RFCs and, 23
- Best-effort delivery, of packets, 7
- Better-than-Nothing Security (BTNS), 852, 936
- BGP (Border Gateway Protocol), 935
- BI (binary increase), 774
- BIC (Binary Increase Congestion Control)
 - BIC-TCP, 773–774
 - overview of, 772–773
- Bidirectional tunneling, in mobile IP, 216–217
- Big endian byte ordering, 183
- Binary additive increase algorithm, 773–774
- Binary exponential backoff, retransmission and, 650
- Binary increase (BI), 774
- Binary Increase Congestion Control (BIC)
 - BIC-TCP, 773–774
 - overview of, 772–773
- Binary notation
 - expressing IP addresses in, 32–33
 - prefixes, 48
 - of subnet masks, 39
- Binary phase shift keying (BPSK)
 - definition of, 936
 - higher throughput (802.11n) support and, 128
- Binary search increase algorithm, BIC-TCP and, 773–774
- BIND9 (Berkeley Internet Name Domain v. 9), 935
- Binding method, in STUN, 321
- Binding, MNs (mobile nodes), 216–217
- Binding Update (BU), in MIP, 936
- Bit error rate (BER)
 - data frame fragmentation, 117–118
 - definition of, 935
- Bit flipping attack, 918
- Bit stuffing, in PPP frames, 132
- BITS (Bump in the Stack), in IPsec, 840, 935
- BITW (Bump in the Wire), in IPsec, 840, 936
- Bitwise AND operation, used with subnet masks, 40
- BL (Bulk Leasequery)
 - DCHP relay agents, 269–270
 - definition of, 936
- Black hats, attacks related to Internet architecture, 26
- Black holes, in PMTUD, 613
- Blackhole route messages, ICMPv6, 372
- Block ciphers, 811
- Blocking route messages, ICMPv6, 372
- Bloop attacks, 429
- BOD (bandwidth on demand), 139, 936
- Bombs, ICMP attacks, 428
- Bonding, link aggregation and, 92–93
- Boot Server Discovery Protocol (BSDP), 246, 936
- BOOTP (Internet Bootstrap Protocol)
 - compatibility with DHCP, 236–238
 - definition of, 936
 - DHCP based on, 235
 - options, 238–239
 - relay agents, 268
- BOOTREQUEST, 239, 242
- Border Gateway Protocol (BGP), 209, 935
- Bot attacks, 26
- Bot herders, 806
- Botnets
 - attacks related to Internet architecture, 26
 - taking control of computers, 806
- BPDU (Bridge PDU)
 - building the spanning tree, 107
 - definition of, 936
 - RSTP (Rapid Spanning Tree Protocol), 110–111
 - STP and, 104
 - structure of, 105–107
 - viewing with Wireshark, 109
- BPSK (binary phase shift keying)
 - definition of, 936
 - higher throughput (802.11n) support and, 128
- Bridge PDUs. *See* BPDU (Bridge PDU)
- Bridges
 - layer 2 relay agents, 270
 - overview of, 98–102
 - STP. *See* STP (Spanning Tree Protocol)
- Bridging Broadband (B4), in DS-Lite, 340, 935
- Broadcast addresses
 - overview of, 15
 - setting/finding, 437–439
 - structure of, 42–43
- Broadcast and Multicast Service Controller (BCMCS), in cellular networks, 239, 935

Broadcast domain, link-layer broadcast, 167

Broadcasting

- introduction to, 435–436

- overview of, 436–437

- sending broadcast datagrams, 439–441

- setting/finding broadcast addresses, 437–439

Brute-force attacks, 816

BSD (Berkeley Software Distribution)

- definition of, 936

- standards and, 24

- Tahoe release, 737

BDSP (Boot Server Discovery Protocol), 246, 936

BSS (basic service set)

- definition of, 936

- QoS BBS, 122

- Wi-Fi, 112

BTNS (Better-than-Nothing Security), in IPsec, 852, 936

BU (Binding Update), in MIP, 936

Buffer bloat, TCP congestion control and, 781–782

Buffer overflow, worms, 805

Buffers

- large buffers and auto-tuning, 715–719

- packets stored in, 4

Bulk data, in TCP communication, 692

Bulk Leasequery (BL)

- DCHP relay agents, 269–270

- definition of, 936

Bump in the Stack (BITS), in IPsec, 840, 935

Bump in the Wire (BITW), in IPsec, 840, 936

Bundles, of PPP links, 137, 139

Byte stuffing, in PPP frames, 132

C

Cache

- ARP cache, 169–170

- ARP cache timeouts, 174

Cache poisoning, DNS-related attacks, 572

Caching servers, DNS, 517–518

CALIPSO (Common architecture Label IPv6 Security Option), 199, 936

Callback Control Protocol (CBCP), in PPP

- definition of, 936

- negotiation of callbacks in LCP, 136

Callback, PPP supporting, 136

CAMELLIA, standardized for Internet use, 819

Candidate sets (CS), in source address selection, 223–224

Candidate transport addresses, in ICE, 333

Canonical name records. See CNAME (canonical name) records

Canonical ordering, of RRset in DNSSEC, 902–903

Capture, network communication, 808

Capturing portals, link layer attacks, 155

Care-of address (CoA)

- definition of, 937

- in Mobile IP, 216–217

Care-of Test (CoT)

- definition of, 937

- mobility messages in RRP, 218–219

Care-of Test Init (CoTI)

- definition of, 937

- mobility messages in RRP, 218–219

Carrier-grade NAT (CGN), 315

Carrier sense, 120

Carrier sense, multiple access with collision avoidance. See CSMA/CA (carrier sense, multiple access with collision avoidance)

Carrier sensed, multiple access with collision detection (CSMA/CD)

- definition of, 938

- Ethernet interface and, 80–81

CAs (certification authorities). See also Certificates (public key)

- definition of, 936

- PKI (Public Key Infrastructure) and, 821–822

Catenet. See also Internet network, 1

CBC (cipher block chaining)

- block ciphers in encryption algorithms, 820

- definition of, 936

CBC-MAC (cipher block chaining message authentication code), 129

CBCP (Callback Control Protocol)

- definition of, 936

- negotiation of callbacks in LCP, 136

CCA (clear channel assessment), in Wi-Fi

- definition of, 936

- for physical carrier sense, 121

CCITT (Comité Consultatif International Télégraphique), 24, 936

CCM (counter mode)

- CCMP algorithm based on, 129

- definition of, 936

CCMP (counter mode with CBC Message Authentication Code)

- definition of, 936

- in Wi-Fi security, 129–130

CCP (Compression Control Protocol), in PPP,

- definition of, 936

- MPPE and, 145

- overview of, 139–140

ccTLDs (country code TLDs)

- definition of, 937

- in DNS name space, 512–514

CDN (content delivery networks), 535

CDP (CRL Distribution Point), 828–829, 937

-
- CE (Congestion Experienced) bit, in IP header, 783–784
 - CERT (Computer Emergency Response Team), 937
 - Certificate Policies (CP), 828
 - Certificate Request (CERTREQ) payload
 - definition of, 937
 - in IKE, 869–870
 - Certificate revocation lists. *See* CRLs (certificate revocation lists)
 - Certificates, ACs (attribute certificates), 831
 - Certificates option, in ND, 417
 - Certificates (public key)
 - CAs and PKIs and, 821–822
 - extensions, 827–828
 - for identifying four types of Internet entities, 823
 - validating, 826
 - validating and revoking certificates, 828–831
 - viewing preconfigured, 823–826
 - Certificates, SEND, 403
 - Certification authorities. *See* CAs (certification authorities)
 - Certification Path Advertisement message, ICMP Send messages, 407
 - Certification Path Solicitation message, ICMP Send messages, 406–407
 - Certification Practice Statement (CPS), 937
 - CERTREQ (Certificate Request) payload
 - definition of, 937
 - in IKE, 869–870
 - CGAs (cryptographically generated addresses)
 - definition of, 937
 - Handover Key Request/Reply options, 422–423
 - neighbor discovery options in IPv6, 414–415
 - RSA Signature option, 415–416
 - securing IPv6 Neighbor Discovery, 292
 - SEND (Secure Neighbor Discovery) and, 403–406
 - verification of, 405
 - CGN (carrier-grade NAT), 315
 - Chaddr (Client Hardware Address) field
 - DHCP/BOOTP message format, 238
 - MAC addresses in, 244
 - Challenge-response protocols, 816
 - Change Cipher Spec protocol, 878
 - Channels
 - in SSM multicast service model, 54
 - TURN, 327
 - Wi-Fi, 124
 - CHAP (Challenge-Handshake Authentication Protocol)
 - definition of, 937
 - for PPP authentication, 140–141
 - Character stuffing, in PPP frames, 132
 - Checkpointing, saving work, 10
 - Checksums
 - (Generic Routing Encapsulation), 150
 - applying at application layer, 601
 - for detecting packet errors, 580
 - IP header fields, 185
 - TCP, 586, 590
 - UDP, 475–478
 - UDP-Lite, 487–488
 - verifying message integrity, 817
 - WKP checksum neutrality, 341
 - CIA (confidentiality, integrity, and availability)
 - AH and, 856, 858
 - definition of, 937
 - ESP and, 860
 - overview of, 806
 - ciaddr (Client IP address) field, 237
 - CIDR (Classless Inter-Domain Routing)
 - definition of, 937
 - developed to alleviate pressure on available IPv4 addresses, 47–48
 - masks, 47
 - routing scalability addressed by, 303
 - Cipher-based MAC (CMAC), 819–820, 937
 - Cipher block chaining (CBC)
 - block ciphers in encryption algorithms, 820
 - definition of, 936
 - Cipher block chaining message authentication code (CBC-MAC), 129
 - Cipher Change protocol, TLS handshaking, 880
 - Cipher suite rollback attacks, 919
 - Cipher suites. *See* CS (cipher suites)
 - Ciphertext
 - encrypting cleartext message, 810
 - TLS, 878–879
 - Civic location, location information in DHCP, 274
 - Clark, D., 3
 - Class of Service (CoS), 937
 - Class selector code points, 190
 - Class Selector (CS), 938
 - Classes, IP address
 - Class D addresses reserved for IPv4 multicast, 54–55
 - overview of, 34–36
 - prefix length and, 47–48
 - Classic RTO method, 651–652
 - Classless Inter-Domain Routing. *See* CIDR (Classless Inter-Domain Routing)
 - Classless routes, DHCP and, 246
 - Classless Static Route (CSR) parameter, 246
 - Clear channel assessment (CCA)
 - definition of, 936
 - for physical carrier sense, 121
 - Clear to send. *See* CTS (clear to send)

- Client alive messages, 802
- Client Hardware Address (Chaddr) field
 - DHCP/BOOTP message format, 238
 - MAC addresses in, 244
- Client IP address (Ciaddr) field, in DHCP, 237
- Client/server design pattern, 20–21
- Client state machine, DHCP, 251–252
- ClientHello message, in TLS, 887
- Clients
 - keepalives detecting state of client host, 795
 - setting keepalive time for Windows client, 797–799
- Clock granularity, RTO bounds and, 654
- Clock recovery, Ethernet frames, 84
- CLOSED state, in TCP,
 - sharing connection state, 768
 - simultaneous open and close transitions, 625
 - TCP state transitions, 618
- CLOSE_WAIT state, TCP state transitions, 618
- CMAC (cipher-based MAC), 819–820, 937
- CNAME (canonical name) records
 - definition of, 937
 - DNS resource record types, 534–536
 - translating DNS from IPv4 to IPv6, 569
- CNs (correspondent nodes), in MIP, 216–218, 937
- CoA (care-of address)
 - definition of, 937
 - in Mobile IP, 216–217
- Coding theory, 579
- Collision Count field, CGAs, 404–405
- Comité Consultatif International Télégraphique (CCITT), 24, 936
- Common architecture Label IPv6 Security Option (CALIPSO), 199, 936
- Communication protocols
 - ARQ (Automatic Repeat Request), 579–581
 - congestion control, 583–584
 - flow control, 583
 - introduction to, 579
 - retransmission settings, 584
 - sliding windows, 582
 - TCP. *See* TCP (Transmission Control Protocol)
 - windows of packets, 581–582
- Compound TCP (CTCP) algorithm, 779–781, 938
- Compression
 - ACFC (Address and Control Field Compression), 132
 - CCP (Compression Control Protocol), 139–140
 - header compression, 139, 142–143
 - MPPC (Microsoft Point-to-Point Compression Protocol), 140
 - PFC (Protocol Field Compression), 133
 - VJ (Van Jacobson) compression, 141
- Compression Control Protocol. *See* CCP (Compression Control Protocol)
- Compression labels, DNS names and, 523–524
- Compression-optional attributes, STUN, 321
- Computer Emergency Response Team (CERT), 937
- Computer Systems Research Group (CSRG), 24, 938
- Concurrent servers, 21
- Confidentiality, in CIA triad, 806
- Confidentiality, integrity, and availability. *See* CIA (confidentiality, integrity, and availability)
- Configuration data delivery, DHCP for, 235
- Configuration Payload (CP)
 - definition of, 937
 - IKE, 849
- Congestion avoidance algorithm
 - classic algorithms for TCP congestion, 734–736
 - comparing with slow start, 736–737
- Congestion collapse state, 728
- Congestion control. *See also* flow control
 - in communication protocols, 583–584
 - in TCP. *See* TCP congestion control
 - in UDP server design, 505
- Congestion Experienced (CE) indicator, in IP header, 783–784
- Congestion indicator, ECN, 188
- Congestion Manager, 768
- Congestion window. *See* cwnd (congestion window)
- Congestion Window Reducing. *See* CWR (Congestion Window Reducing)
- Congestion Window Validation (CWW), in TCP, 742–744, 938
- Connection completion, TCP congestion control, 766–767
- Connection-oriented networks, 5
- Connection-oriented protocols, 595
- Connection-oriented service, 585
- Connection refused error, in TCP and UDP, 626
- Connection state, TCP, 595
- Connectionless networks, 5, 181
- Connectionless protocols, 595
- Connections, in protocol architecture, 3–6
- Connections, TCP
 - aborting, 627–628
 - attacks related to, 640–643
 - establishing and terminating, 595–598
 - example of PMTUD with, 613–616
 - example showing packet-level details, 602–604
 - FIN_WAIT_2 state, 625
 - half-close operation, 598–599
 - half-open connections, 628–630
 - header options, 605–606

- incoming connection queue, 636–640
- introduction to, 595
- ISN (initial sequence number), 601–602
- MSS (Maximum Segment Size) option, 606–607
- PAWS (Protection against Wrapped Sequence Numbers), 610–611
- PMTUD (Path MTU Discovery) and, 612–613
- port numbers and, 632–634
- quiet time concept, 624
- requesting connection to nonexistent port, 626
- reset segments, 625–626
- restrictions on foreign endpoints, 635–636
- restrictions on local IP addresses, 634–635
- SACK (selective acknowledgement) option, 607
- server operation and, 631–632
- simultaneous open and close, 599–601
- simultaneous open and close transitions, 625
- state transition diagrams, 617–618
- summary and references, 643–645
- TCP-AO (Authentication Option), 612
- timeout settings, 604–605
- Timestamps option, 608–610
- TIME_WAIT state (2MSL), 618–624
- translating addresses and port numbers, 605
- TWA (TIME-WAIT Assassination), 630–631
- UTO (User Timeout) option, 611–612
- WSALE (Window Scale) option, 608
- Conservation of packets, 731
- Content delivery networks (CDN), 535
- Content filters, web proxies operating as, 302
- Contention window (CWV)
 - definition of, 938
 - in MAC, 122
- Control field, in PPP frames, 132
- Control frames, Wi-Fi, 115–116
- Cooks, C., 814
- Coordinated Universal Time (UTC), 961
- Correspondent nodes (CNs), in MIP, 216–218, 937
- Correspondent registration, in RO, 218
- CoS (Class of Service), 937
- CoT (Care-of Test)
 - definition of, 937
 - mobility messages in RRP, 218–219
- CoTI (Care-of Test Init)
 - definition of, 937
 - mobility messages in RRP, 218–219
- Counter (CTR) mode
 - definition of, 938
 - operating modes of encryption algorithms, 820
- Counter mode (CCM)
 - CCMP algorithm based on, 129
 - definition of, 936
- Counter mode with CBC Message Authentication Code (CCMP)
 - definition of, 936
 - in Wi-Fi security, 129–130
- Counters, IGMP/MLD, 467–468
- Country code TLDs (ccTLDs)
 - definition of, 937
 - in DNS name space, 512–514
- CP (Certificate Policies), 828
- CP (Configuration Payload)
 - definition of, 937
 - IKE, 849
- CPS (Certification Practice Statement), 937
- CRCs (Cyclic Redundancy Checks)
 - applying at application layer, 601
 - compared with Internet checksum, 186
 - definition of, 937
 - for detecting errors in packets, 580
 - host address filtering and, 449, 451
 - integrity checking in Ethernet frames, 86–88
 - TCP reliability and, 586
- CREATE_CHILD_SA exchange, IKE protocol, 852–853
- CRL Distribution Point (CDP), 828–829, 937
- CRLs (certificate revocation lists)
 - definition of, 937
 - distribution point for, 828–829
 - extensions, 830
- Cryptographic suites, 819–821
- Cryptographically generated addresses. *See* CGAs (cryptographically generated addresses)
- Cryptographically strong PRNGs (CSPRNGs), 816, 938
- Cryptography. *See also* Encryption
 - attacks, 918
 - cryptographic and cipher suites, 819–821
 - cryptosystems, 809–812
 - DH (Diffie-Hellman-Merkle Key Agreement), 813–814
 - ECC (Elliptic Curve Cryptography), 815
 - hash functions and message digests, 817–818
 - message authentication codes, 818–819
 - message syntax, 823
 - nonces and salt, 816
 - overview of, 809
 - PFS (Perfect Forward Secrecy), 815
 - pseudorandom numbers, generators, and function families, 815–816
 - RSA (Rivest, Shamir, and Adleman) public key cryptography, 812–813
 - signcryption, 814–815
 - CS (candidate sets), in source address selection, 223–224

- CS (cipher suites)
 - definition of, 938
 - overview of, 819–821
 - in TLS, 878
 - CS (Class Selector), 938
 - CSMA/CA (carrier sense, multiple access with collision avoidance)
 - DCF as form of, 120
 - definition of, 938
 - WLANs (wireless LANs), 84
 - CSMA/CD (carrier sensed, multiple access with collision detection)
 - definition of, 938
 - Ethernet interface and, 80–81
 - CSPRNGs (cryptographically strong PRNGs), 816, 938
 - CSR (Classless Static Route) parameter, 246
 - CSRG (Computer Systems Research Group), 24, 938
 - CTCP (Compound TCP) algorithm, 779–781, 938
 - CTR (Counter) mode
 - definition of, 938
 - operating modes of encryption algorithms, 820
 - CTS (clear to send)
 - carrier sense and, 121
 - definition of, 938
 - Wi-Fi control frames, 115
 - CUBIC algorithm, for
 - TCP congestion control, 775–776
 - Current-state records, IGMP/MLD group member - ship reports, 457
 - CW (contention window)
 - definition of, 938
 - in MAC, 122
 - cwnd (congestion window) in TCP
 - comparing slow start with congestion avoidance, 736–737
 - congestion avoidance algorithm and, 734–736
 - CWV (Congestion Window Validation), 742–744
 - definition of, 938
 - Eifel Response Algorithm and, 744–745
 - FAK (forward acknowledgment) and, 741
 - overview of, 729–730
 - SACK congestion control and, 740–741
 - slow start algorithm and, 732–734
 - standard TCP algorithm and, 738
 - Tahoe, Reno, and Fast Recovery and, 737
 - undoing changes in, 762–766
 - CWR (Congestion Window Reducing)
 - CWR bit, 784
 - definition of, 938
 - fast retransmit events, 761–762
 - local congestion events, 764
 - sender pause and, 753
 - CWV (Congestion Window Validation), 742–744, 938
 - Cyclic Redundancy Checks. *See* CRCs (Cyclic Redundancy Checks)
- ## D
- DAD (duplicate address detection)
 - definition of, 938
 - DHCPv6, 259–260
 - IPv6 addresses and, 277–278
 - MLD messages and, 457
 - Neighbor Discovery protocol and, 253
 - RA and RS messages and, 280–282
 - Daemen, Joan, 811
 - Data Encryption Standard. *See* DES (Data Encryption Standard)
 - Data flow, TCP. *See* TCP data flow
 - Data frames
 - fragmentation and aggregation, 94–95
 - Wi-Fi, 116–119
 - Data labels, DNS names and, 523
 - Data-link layer, of OSI model, 9
 - Data types, resource record categories, 528
 - Datagram Congestion Control Protocol. *See* DCCP (Datagram Congestion Control Protocol)
 - Datagram TLS. *See* DTLS (Datagram TLS)
 - Datagrams
 - attribution of, 26
 - fragmenting, 148
 - important concepts in development of network architecture, 5–6
 - in protocol architecture, 3–6
 - receiving multicast, 447–449
 - sending multicast, 446–447
 - spoofing attacks and, 25–26
 - TLS with. *See* DTLS (Datagram TLS)
 - Datagrams, IP
 - direct delivery of, 167
 - DNS messages using IPv4 datagrams, 525
 - fragmentation of, 488
 - of ICMP messages within, 354–355
 - IPv4, 182
 - TCP encapsulation in, 587
 - Datagrams, UDP
 - DNS messages using, 525
 - encapsulation, 474
 - fragmentation of, 488–492
 - maximum size, 497–498
 - translating UDP/IPv4 and UDP/IPv6 datagrams, 505–506
 - truncation of, 498
 - Davies, Donald, 1
 - Day, J., 2

- DCCP (Datagram Congestion Control Protocol)
 - definition of, 938
 - NAT and, 309
 - transport protocols in TCP/IP suite, 16
- DCF (distributed coordinating function), in Wi-Fi
 - collision avoidance/backoff procedure, 121–122
 - definition of, 938
 - options for controlling sharing of wireless medium, 120
- DDDS (Dynamic Delegation Discovery System)
 - definition of, 938
 - ENUM records and, 551–552
 - DNS NAPTR records and, 549
 - URI/URN resolution and, 553
- DDNS (Dynamic DNS)
 - mapping DNS to DHCP addresses, 286
 - supporting DNS Update, 567
- DDoS (distributed DoS)
 - attacks related to Internet architecture, 26
 - definition of, 939
- Deadlock, Nagle algorithm resulting in, 699
- Decimal notation, expressing IP addresses in, 32
- Default router, IP forwarding, 208
- Deferred authentication, in DHCP, 272
- Defragmentation, of data frames. *See also* Fragmentation, 117
- Delay-based congestion control
 - buffer bloat and, 782
 - CTCP (Compound TCP) algorithm, 779–781
 - FAST TCP algorithm, 778–779
 - overview of, 777
 - TCPW (TCP Westwood) algorithm, 779
 - Vegas TCP algorithm, 777–778
- Delayed ACKs
 - interaction with Nagle algorithm, 699
 - with piggybacking, 692
 - in TCP data flow, 695–696
- Delegated path discovery (DPD)
 - certificate validation and, 831
 - definition of, 940
- Delegated path validation (DPV)
 - certificate validation and, 831
 - definition of, 940
- Delegation
 - classless .in-addr.arpa delegation, 539
 - DNS zones and, 516
- Delegation signer (DS) resource record
 - definition of, 940
 - DNSSEC, 897–898
- Delivery to multiple locations, broadcasting and multicasting for, 435
- Demilitarized zones. *See* DMZ (demilitarized zones)
- Demultiplexing
 - identifiers in, 11
 - implementation and design and, 10–13
 - TCP/IP suite and, 16–17
- Denial-of-service. *See* DoS (denial-of-service)
- DER (Distinguished Encoding Rules), 869, 939
- DES (Data Encryption Standard). *See also* triple-DES
 - definition of, 939
 - dictionary attacks and, 816
 - as symmetric encryption algorithm, 811
- Destination address selection algorithm, in IP host models, 224–225
- Destination cache, 403
- Destination (DST) address, in Ethernet frame format, 85
- Destination IP address
 - host processing of IP datagrams, 220–221
 - in IP datagrams, 186
 - in IP forwarding, 209
 - IPv6 header fields, 196
 - Routing header fields, 201
 - selection by hosts, 222–223
 - Teredo tunneling and, 485
- Destination metrics, TCP timeout/retransmission, 685–686
- Destination Options, IPv6, 196
- Destination unreachable messages, ICMP
 - overview of, 364
 - PTB (Packet Too Big), 612
 - requesting connection to nonexistent ports, 626
 - UDP datagram and, 480
- Detecting Network Attachment (DNA), 241, 939
- Detection algorithm, for spurious timeouts and retransmissions, 677
- Detection, of congestion, 728–729
- DF (Don't Fragment), 939
- DH (Diffie-Hellman-Merkle Key Agreement), 813–814, 939
- DHCP (Dynamic Host Configuration Protocol)
 - address pools and leases, 235–236
 - attacks related to, 292
 - authentication, 271–273
 - automatic address assignment, 67
 - BOOTP message format, 236–238
 - BOOTP options, 238–239
 - definition of, 939
 - DHCP/DNS interaction, 285–286
 - DHCPACK message, 250
 - DHCPDISCOVER message, 244–247
 - DHCPNAK message, 243
 - DHCPOFFER message, 247–248
 - DHCPREQUEST message, 241–243, 248–249

- DHCP (Dynamic Host Configuration Protocol),
 - continued
 - location information, 274–275
 - manual configuration, 265–266
 - mobility and handoff information, 275
 - operation of, 239–241
 - overview of, 234–235
 - PPPoE (PPP over Ethernet) and, 286–291
 - Rapid Commit option, 273–274
 - reconfigure extension, 273
 - relays, 267–271
 - snooping, 276
 - state machine, 251–252
 - subnet mask configuration, 39
 - summary and references, 292–298
- DHCP Unique Identifier. *See* DUID (DHCP Unique Identifier)
- DHCPACK message, 241, 250
- DHCPDECLINE message, 241
- DHCPDISCOVER message, 240, 244–247
- DHCPINFORM message, 241
- DHCLEASEQUERY message, 269
- DHCPNAK message, 241, 243
- DHCPOFFER message, 240, 247–248
- DHCPRELEASE message, 241
- DHCPREQUEST message, 239–243, 248–249
- DHCPv6
 - DAD (Neighbor Solicitation), 259–260
 - DUID (DHCP Unique Identifier), 256–257
 - IA (Identity Association), 255–256
 - IPv6 address lifecycle, 252–253
 - manual configuration, 250–251
 - message format, 253–255
 - operation of, 257–258
 - overview of, 252
 - prefix delegation, 266–267
 - REQUEST message, 264–265
 - router solicitation and advertisement, 260–263
- Dictionary attacks
 - DES and, 816
 - security protocol-related, 918
- Differentiated Services Code Point (DSCP), 188–190, 940
- Differentiated Services field. *See* DS (Differentiated Services) field
- Diffie-Hellman-Merkle Key Agreement (DH), 813–814, 939
- DIFS (distributed inter-frame space), in Wi-Fi
 - carrier sense and, 120–121
 - definition of, 939
- Digest challenge, STUN mechanisms and, 325
- Digests, message digests, 817–818
- Digital, Intel, Xerox (DIX)
 - definition of, 939
 - Ethernet, 82
- Digital Living Network Alliance (DLNA), 939
- Digital Object Identifier (DOI), 939
- Digital Signature Algorithm (DSA), 821, 940
- Digital Signature Standard (DSS), 821, 940
- Digital signatures
 - in cipher suites, 821
 - in public key cryptography, 812
 - RSA Signature option, 416
- Digital subscriber line. *See* DSL (digital subscriber line)
- Direct delivery
 - IP forwarding, 210–212
 - with IPv4, 167–169
- Directed broadcast, 43
- Direction specification, in firewall rules, 335
- Discard Request messages, in LCP operation, 134
- Discovery problem, in p2p networks, 22
- Discrete log problem, in DH (Diffie-Hellman)
 - encryption, 814
- Distinguished Encoding Rules (DER), 869, 939
- Distributed coordinating function. *See* DCF (distributed coordinating function)
- Distributed DoS (DDoS)
 - attacks related to Internet architecture, 26
 - definition of, 939
- Distributed inter-frame space (DIFS)
 - carrier sense and, 120–121
 - definition of, 939
- Distribution service (DS)
 - definition of, 940
 - Wi-Fi, 112
- Distributions, TCP/IP suite, 24–25
- DIX (Digital, Intel, Xerox)
 - definition of, 939
 - Ethernet, 82
- DKIM (DomainKeys Identified Mail)
 - definition of, 939
 - DKIM signatures, 916
 - example using, 916–918
 - overview of, 915–916
- DLNA (Digital Living Network Alliance), 939
- DMZ (demilitarized zones)
 - definition of, 939
 - DNS queries and, 565–567
 - packet-filtering firewalls and, 300
 - unicast addresses and, 67–68
- DNA (Detecting Network Attachment), 241, 939
- DNAME resource records, DNS, 536, 939
- DNS-0x20, 572

DNS (Domain Name System)

- address and name server records, 529–530
- answer, authority, and additional information section formats, 526–527
- attacks related to, 571–572
- AXFR (full zone transfer) messages, 559–561
- caching and, 517–518
- CNAME (canonical name) records, 534–536
- definition of, 939
- DHCP and, 233, 285–286
- DNS notify, 564–565
- dynamic updates, 555–558
- ENUM records, 551–552
- example using resource record types, 530–534
- extension format (EDNSO), 524–525
- introduction to, 511–512
- IXFR (incremental zone transfer) messages, 561–563
- LDAP and, 570–571
- LLMNR and mDNS, 569–570
- mDNS (Multicast DNS), 444–445
- message format, 520–524
- MX (mail exchanger) records, 544–545
- name resolution process, 518–520
- name servers and zones, 516–517
- name space, 512–514
- naming syntax, 514–516
- NAPTR (name authority pointer) records, 549–551
- open DNS servers and DDNS, 567
- OPT (option) pseudo records, 547–548
- PTR (pointer) records, 536–541
- question (query) and zone section format, 526
- resource record types, 527–529
- S-NAPTR and U-NAPTR, 554–555
- security. *See* DNSSEC (DNS Security)
- SIP records, 552
- SOA (start of authority) records, 541–544
- sort lists, round-robin, and split DNS, 565–567
- SPF (sender policy framework) and TXT records, 545–547
- SRV (service) records, 548–549
- summary and references, 572–578
- TCP/IP suite and, 19
- translating DNS from IPv4 to IPv6, 568–569
- transparency and extensibility, 567–568
- URI/URN resolution, 553–554
- well-known ports, 18, 525–526
- zone transfers, 558–559

DNS Notify

- initiating zone transfers, 525
- necessity of zone transfers, 518

DNS proxy, 568

DNS Security. *See* DNSSEC (DNS Security)

DNS servers

- caching, 517–518
- gTLD servers, 519–520
- primary and secondary, 517
- response to DNS queries, 565–567
- root servers, 518
- zones, 516–517

DNS Update, DDNS support for, 567

DNS64

- definition of, 939
- DNSSEC with, 915
- translating DNS from IPv4 to IPv6, 568–569

DNSKEY resource record

- definition of, 939
- DNSSEC and, 896–897
- signed zones and zone cuts, 903

DNSSEC (DNS Security)

- canonical orderings and forms, 902–903
- definition of, 939
- DNS-related attacks and, 571
- DNS64 and, 915
- DNSKEY resource record in, 896–897
- DS (delegation signer) resource record in, 897–898
- NSEC (NextSECure) resource record in, 898–901
- operation of, 902
- overview of, 894–896
- resolver operation example, 903–911
- resource records, 896
- RRSIG (Resource Record Signature) resource record in, 901–902
- signed zones and zone cuts, 903
- transaction authentication, 911–915

DNSSSL (DNS Search List) option, in ND, 422–423

DOI (Digital Object Identifier), 939

Domain hacks, 514

Domain Keys Identified Mail. *See* DKIM (Domain Keys Identified Mail)

Domain Name System. *See* DNS (Domain Name System)

Domain names, DNS Search List option, 422–423

Domain-specific keys (DSRK)

- definition of, 940
- key derivation in EAP, 838

Domain-specific usage-specific root keys (DSUSRKs)

- definition of, 940
- key derivation in EAP, 838

Domains

- DNS and, 19
- in DNS name space, 512

Done message, ICMP, 388–390

Don't Fragment (DF), 939

- DoS (denial-of-service)
 - definition of, 939
 - DNS attacks, 571
 - DTLS (Datagram TLS) protection, 894
 - IGMP or MLD attacks, 469–470
 - Internet architecture attacks, 26
 - system configuration attacks, 292
 - TCP attacks, 640
 - TCP timeout/retransmission attacks, 686
 - types of threats to network communication, 808
 - UDP attacks, 506
- Dotted-decimal notation
 - IP addresses in, 32, 537
 - of subnet masks, 39
- DPD (delegated path discovery)
 - certificate validation and, 831
 - definition of, 940
- DPV (delegated path validation)
 - certificate validation and, 831
 - definition of, 940
- Drive-by attacks, 806
- Drop precedence, assigned to datagrams, 191
- DS (delegation signer) resource record
 - definition of, 940
 - DNSSEC, 897–898
- DS (Differentiated Services) field
 - definition of, 940
 - ICMP Parameter Problem and, 379
 - in IP header, 183
 - in IP protocol, 188–192
- DS (distribution service)
 - definition of, 940
 - Wi-Fi, 112
- DS-Lite (Dual-Stack Lite)
 - definition of, 940
 - IPv4/IPv6 translation, 339–340
- DSA (Digital Signature Algorithm), 821, 940
- DSACK (duplicate SACK) extension
 - definition of, 940
 - Eifel Detection Algorithm and, 679
 - Eifel Response Algorithm and, 681
 - overview of, 677–679
- DSCP (Differentiated Services Code Point), 188–190, 940
- DSL (digital subscriber line)
 - buffer bloat and, 781
 - definition of, 940
 - overview of, 4
 - PPPoE and, 286–287
- DSRK (domain-specific keys)
 - definition of, 940
 - key derivation in EAP, 838
- DSS (Digital Signature Standard), 821, 940
- DST (Destination) address, in Ethernet frame format, 85
- DSUSRKs (domain-specific usage-specific root keys)
 - definition of, 940
 - key derivation in EAP, 838
- DTCP (Dynamic Tunnel Configuration Protocol), 154
- DTLS (Datagram TLS)
 - definition of, 940
 - DoS protection, 894
 - example of use of, 884–891
 - handshake protocol, 892–894
 - overview of, 876–877
 - record layer, 891–892
- Dual-Stack Lite (DS-Lite)
 - definition of, 940
 - IPv4/IPv6 translation, 339–340
- DUID (DHCP Unique Identifier)
 - definition of, 940
 - DHCPDISCOVER message and, 246
 - types of, 256–257
- DupACK spoofing, TCP congestion control attacks, 785–786
- Duplex modes
 - duplex mismatch, 96
 - overview of, 94–96
- Duplicate ACK threshold (dupthresh), in TCP
 - in fast retransmit, 667
 - NewReno algorithm and, 739
 - packet reordering and, 683
- Duplicate acknowledgements, congestion control and, 589
- Duplicate address detection. *See* DAD (duplicate address detection)
- Duplicate SACK. *See* DSACK (duplicate SACK) extension
- Dupthresh. *See* Duplicate ACK threshold (dupthresh)
- Duration field, in frame transmission, 121
- DWORD value, 518
- Dynamic Delegation Discovery System. *See* DDS (Dynamic Delegation Discovery System)
- Dynamic DNS (DDNS)
 - mapping DNS to DHCP addresses, 286
 - supporting DNS Update, 567
- Dynamic Host Configuration Protocol. *See* DHCP (Dynamic Host Configuration Protocol)
- Dynamic/private ports, 18
- Dynamic Tunnel Configuration Protocol (DTCP), 154
- Dynamic updates, DNS, 555–558

- E**
- E-mail, PGP (Pretty Good Privacy) encryption for, 822
 - EAP (Extensible Authentication Protocol)
 - attacks related to, 918
 - definition of, 940
 - ERP (EAP Re-authentication Protocol), 839
 - IKE protocol and, 851–852
 - key derivation, 838
 - layers of, 836–837
 - methods, 837–838
 - network security and, 833–834
 - packet format, 835
 - for PPP authentication, 141
 - request/response messages, 835–836
 - in Wi-Fi security, 129
 - EAP-FAST (EAP-Flexible Authentication via Security Tunneling), 940
 - EAP Re-authentication Protocol (ERP), 839, 941
 - EAP-TTLS (EAP-Tunneled Transport Layer Security), 838, 940
 - EAPoL (EAP over LAN), 833–834, 940
 - Eavesdropping
 - link layer attacks, 155
 - types of threats to network communication, 808
 - EC2N (Elliptic Curve groups modulo a power 2), 821, 941
 - ECC (Elliptic Curve Cryptography), 815, 821
 - ECC (Error Correcting Code), 579, 941
 - ECDSA (Elliptic Curve Digital Signature Algorithm), 941
 - ECE (ECN Echo), 589, 941
 - Echo Request/Reply (ping) messages
 - example of broadcasting, 438–439
 - ICMP, 380–383
 - in LCP operation, 134
 - Redirect message and, 374
 - sending from link-local unicast address, 445
 - translating ICMPv6 to ICMPv4, 426
 - ECN-Capable Transport (ECT), 783, 941
 - ECN Echo (ECE), 589, 941
 - ECN (Explicit Congestion Notification)
 - delay-based congestion control and, 777
 - detecting congestion, 728
 - ECN-Echo bit, 784
 - ICMP Parameter Problem and, 379
 - IP header fields, 183, 188–192
 - TCP congestion control attacks and, 786
 - TCP header fields, 782–785
 - ECP (Elliptic Curve groups modulo a Prime), 821, 941
 - EDCA (enhanced DCF channel access)
 - definition of, 941
 - UPs (user priorities), 123
 - Wi-Fi mesh and, 130
 - Edge ports, RSTP (Rapid Spanning Tree Protocol), 110
 - Editors, NAT, 315
 - EDNS0 (Extension format for DNS)
 - definition of, 941
 - DNS (Domain Name System), 524–525
 - DNSSEC, 895–896
 - EF (Expedited Forwarding), 191, 941
 - EFO (Extended Flags option)
 - definition of, 941
 - Router Advertisement Flags Extension option, 420–421
 - Eifel Detection Algorithm, 679–680
 - Eifel Response Algorithm
 - handling spurious RTOs in congestion control, 744–745
 - responding to spurious transmissions, 680–682
 - EIFS (extended interframe space), in Wi-Fi
 - carrier sense and, 121
 - definition of, 941
 - Eligible rate estimate (ERE)
 - definition of, 941
 - in TCPW congestion control, 779
 - Elliptic Curve Cryptography (ECC), 815, 821
 - Elliptic Curve Digital Signature Algorithm (ECDSA), 941
 - Elliptic Curve groups modulo a power 2 (EC2N), 821, 941
 - Elliptic Curve groups modulo a Prime (ECP), 821, 941
 - Elliptic curves, 815
 - Embedded devices, setting IPv4 addresses for, 178
 - EMSKs (extended MSKs)
 - definition of, 941
 - key derivation in EAP, 838
 - Encapsulating Security Payload. *See* ESP (Encapsulating Security Payload)
 - Encapsulation
 - definition of, 10
 - of ICMP messages within IP datagrams, 354–355
 - IGMP/MLD, 453–454
 - implementation and design and, 10–13
 - TCP encapsulation in IP datagrams, 587
 - TCP/IP suite and, 16–17
 - tunnel encapsulation limits in IPv6, 198
 - of UDP datagram, 474
 - Encryption. *See also* Cryptography
 - MPPE (Microsoft Point-to-Point Encryption), 145
 - spoofing attacks and, 226
 - TCP-AO (Authentication Option), 612
 - in Wi-Fi security, 129

- End system, for protocol suites, 12
- End-to-end argument, in protocol architecture, 6
- End-to-end checksum, in UDP, 473, 475
- Endpoint discriminator, LCP options, 138
- Endpoints, TCP, 588, 595–596
- Enforcement Point (EP), in PANA, 839, 941
- Enhanced DCF channel accessSeeEDCA (enhanced DCF channel access)
- ENUM records
 - definition of, 941
 - DNS resource record types, 551–552
- Enumeration attacks, 919
- EOL (End of List), TCP header options, 605
- EP (Enforcement Point), in PANA, 941
- Ephemeral port numbers, 18
- EQM (equal modulation)
 - definition of, 941
 - higher throughput (802.11n) support and, 127
- Equation-based rate control, 768
- ERE (eligible rate estimate)
 - definition of, 941
 - in TCPW congestion control, 779
- ERP (EAP Re-authentication Protocol), 839, 941
- Error control, in protocol architecture, 7–8
- Error Correcting Code (ECC), 579, 941
- Error messages, ICMP
 - destination unreachable, 364–372
 - extended and multipart messages, 363–364
 - overview of, 309, 361–363
 - Parameter Problem message, 379–380
 - Redirect message, 372–375
 - time exceeded message, 375–378
 - translating ICMPv4 to ICMPv6, 424–425
 - translating ICMPv6 to ICMPv4, 427
- ESN (Extended Sequence Number), in IPsec, 856, 942
- ESP (Encapsulating Security Payload)
 - definition of, 942
 - ESP-NULL, WESP, and traffic visibility, 863–864
 - in IPsec, 217, 841
 - overview of, 858
 - transport and tunnel modes, 858–863
- ESP-NULL, 863–864
- ESS (extended service set), Wi-Fi, 112
- ESSID (extended service set identifiers)
 - definition of, 942
 - Wi-Fi, 112
- Established connections, TCP, 596
- ESTABLISHED state in TCP
 - half-open connections and, 628
 - incoming connection queue and, 636
 - simultaneous open and close transitions, 625
 - TCP port numbers and, 632–634
 - TCP state transitions, 618
- Ethernet (IEEE 802.3)
 - autonegotiation in, 95
 - converting IP multicast addresses to MAC addresses, 442–444
 - flow control, 98
 - frames, 84–86
 - frames sizes, 88–89
 - integrity checking on frames, 86–88
 - LAN/MAN standards, 82–84
 - MAC addresses, 16
 - MTU (maximum transmission unit), 506
 - overview of, 80–82
 - power saving, 96–97
 - speeds, 81
 - supporting broadcasting at link layer, 437
- Ethernet interfaces, Promiscuous mode, 155
- Ethernet type field, 16
- ethtool, Linux program for checking full duplex support, 94
- EUI (extended unique identifier)
 - definition of, 942
 - formats of IPv6 addresses, 44–45
- EV-DO (Evolution, Data Optimized (or Only)), 942
- EV (Extended Validation), 942
- Exchange of database recordsSeeZone transfers
- Expedited Forwarding (EF), 191, 941
- Experimental category, RFCs and, 23
- Experimental Values, neighbor discovery in IPv6, 423
- Explicit Congestion Notification. SeeECN (Explicit Congestion Notification)
- Explicit sending, in congestion control, 583
- Explicit signaling, in congestion control, 728
- Exponential backoff
 - binary, 650
 - SWS (silly windows syndrome) and, 713
 - in TCP connection timeout, 604
- Extended and multipart messages, ICMP, 363–364
- Extended Flags option (EFO)
 - definition of, 941
 - Router Advertisement Flags Extension option, 420–421
- Extended interframe space (EIFS)
 - carrier sense and, 121
 - definition of, 941
- Extended MSKs (EMSKs)
 - definition of, 941
 - key derivation in EAP, 838
- Extended Sequence Number (ESN), in IPsec, 856, 942
- Extended service set (ESS), Wi-Fi, 112
- Extended service set identifiers (ESSID)
 - definition of, 942
 - Wi-Fi, 112

- Extended unique identifier (EUI)
 - definition of, 942
 - formats of IPv6 addresses, 44–45
- Extended Validation (EV), certificates, 942
- Extensibility, DNS, 567–568
- Extensible Authentication Protocol. *See* EAP (Extensible Authentication Protocol)
- Extensible Markup Language (XML)
 - common use with Web pages, 338
 - definition of, 962
- Extensible Messaging and Presence Protocol (XMPP), 333, 962
- Extension data structure, appended to ICMP messages, 363–364
- Extensions
 - DNS. *See* EDNS0
 - identifiers, 44
 - IP header, 182–183
 - IPv6 header, 194–196
 - TLS, 883–884
- Extranets, 20
- F**
- F-RTO (Forward-RTO Recovery), in TCP
 - definition of, 942
 - for detecting spurious transmissions, 680
- FACK (forward acknowledgment), in TCP
 - definition of, 942
 - for TCP congestion control, 741–742
- Fast Recovery algorithm, 737–738
- Fast retransmit
 - event, 761
 - example of, 668–671
 - introduction to, 647
 - local congestion and, 759–762
 - overview of, 667–668
 - SACK recovery and, 757–759
- FAST TCP algorithm, 778–779
- Fate sharing, in protocol architecture, 6–7
- FCFS (first-come-first-served)
 - definition of, 942
 - packet processing, 4
- FCS (Frame Check Sequence)
 - definition of, 942
 - integrity checking in Ethernet frames, 88
 - verifying message integrity, 817
- FEC (forward error correction)
 - definition of, 942
 - higher throughput (802.11n) support and, 128
- FIFO (first-in-first-out)
 - definition of, 942
 - queue management and, 782
 - scheduling packets, 4
- File (Boot File Name) field, DHCP/BOOTP message format, 238–239
- File Transfer Protocol. *See* FTP (File Transfer Protocol)
- Filter-mode-change records, IGMP/MLD group membership reports, 457
- Filters
 - host address filtering, 449–451
 - IGMP/MLD processing and, 456
 - iptables, 335
 - NAT, 312–313
 - packet-filtering firewalls, 300
 - web proxies operating as content filters, 302
- FIN segments, in TCP
 - active and passive closers and, 597–598
 - connection completion and, 766
 - definition of, 942
 - half-close operation and, 598–599
 - half-open connections and, 628
 - sequence numbers in, 603
- Fingerprinting, TCP keepalive attacks, 802
- Fingerprints, or digest of message, 817–818
- FIN_WAIT_1 state, TCP state transitions, 618, 625
- FIN_WAIT_2 state, TCP state transitions, 618, 625
- Firewalls
 - attacks related to, 345–346
 - configuring, 334
 - direct interaction with, 338–339
 - IP addresses and, 67
 - overview of, 300
 - packet-filtering firewalls, 300–301
 - proxy firewalls, 301–303
 - rules for packet-filtering, 335–336
 - summary and references, 345–346
- First-come-first-served (FCFS)
 - definition of, 942
 - packet processing, 4
- First-in-first-out. *See* FIFO (first-in-first-out)
- Flags
 - DHCP/BOOTP message format, 236
 - GRE tunnels and, 150
 - IPv6 multicast addresses, 57, 58
- Flooding attacks, 102, 428
- Flow control. *See also* Congestion control
 - link layer and, 98
 - in protocol architecture, 7–8
 - rate-based and window-based, 583
 - in UPD server design, 505
- Flow control, in TCP
 - example of dynamic window size adjustment and flow control, 705–708
 - large buffers and auto-tuning, 715–719
 - overview of, 700–701

- Flow control, in TCP, continued
 - sliding window protocol, 701–704
 - SWS (silly windows syndrome), 708–715
 - TCP header and, 590
 - zero windows and TCP persistent timer, 704–705
 - FMIP (Mobile IP with Fast Handovers), 388, 942
 - Foreign IP addresses
 - TCP port numbers and, 632
 - TCP server restrictions on foreign endpoints, 635–636
 - Forgery, message authentication codes protecting against, 818
 - Forward acknowledgment (FACK), in TCP
 - definition of, 942
 - for TCP congestion control, 741–742
 - Forward error correction (FEC)
 - definition of, 942
 - higher throughput (802.11n) support and, 128
 - Forward-RTO Recovery (F-RTO)
 - definition of, 942
 - for detecting spurious transmissions, 680
 - Forwarding actions, IP forwarding, 209–210
 - Forwarding datagrams, 14
 - Forwarding tables, IP forwarding, 208–209
 - Four-message exchange operations, of DHCPv6, 265–266
 - FQDN (fully qualified domain names)
 - definition of, 942
 - DHCPv6 and, 260
 - vs. unqualified domain names, 515
 - Fraggle attacks, UDP-related attacks, 506
 - Fragment header, IPv6 protocol, 203–208
 - Fragment number, data frame fragmentation, 117
 - Fragment Offset field
 - in IPv6 Fragment header, 203–205
 - in UDP fragmentation, 489
 - Fragmentation
 - of datagrams, 14
 - of IGMP packets, 470
 - of Internet traffic, 506
 - IP fragmentation. *See* P fragmentation
 - UDP/IPv4, 488–492
 - of Wi-Fi frames, 116–119
 - Frame Check Sequence*See* FCS (Frame Check Sequence)
 - Frame Control Word
 - in MPDU, 113
 - PSM (power save mode) and, 119
 - Retry bit and, 116
 - Frame Relay, best-effort delivery, 7
 - Frames, Ethernet
 - 802.3 standard, 84–86
 - ARP frame format, 170–171
 - ARP reply frames, 168
 - ARP request frames, 167, 173–174
 - integrity checking on, 86–88
 - link-layer PDUs, 14
 - payload of, 16
 - sizes, 88–89
 - Frames, PPP, 131–132
 - Frames, Wi-Fi
 - control frames, 115–116
 - data frames, 116–119
 - management frames, 113–115
 - overview of, 113
 - FreeBSD
 - broadcast addresses, 441
 - incoming connection queue and, 638
 - standards and, 24
 - Frequencies, Wi-Fi, 124
 - Freshness property, authentication protocols and, 816
 - FTP (File Transfer Protocol)
 - definition of, 942
 - NAT and, 304
 - TCP/IP suite and, 13
 - Full duplex
 - support, 94–95
 - TCP service to application layer, 587
 - Fully qualified domain names. *See* FQDN (fully qualified domain names)
- ## G
- Galois/Counter Mode. *See* GCM (Galois/Counter Mode)
 - Galois MAC (GMAC), 819–820, 943
 - Gateway or Router IP Address (Giaddr) field
 - DHCP/BOOTP message format, 238
 - LDRAs and, 271
 - Gateways
 - application layer. *See* ALGs (application layer gateways)
 - between packet-switching networks, 1
 - between protocols, 20
 - RED (Random Early Detection) gateways, 783–785
 - GCKSs (group controller/key servers)
 - definition of, 942
 - multicast support in IPsec, 864–865
 - GCM (Galois/Counter Mode)
 - AES and, 819
 - definition of, 942
 - operating modes of encryption algorithms, 820
 - GDOI (Group Domain of Interpretation), 942
 - GENA (General Event Notification Architecture), 338, 942
 - Generator polynomial, CRC and, 86–87
 - Generic Attribute Registration Protocol (GVRP), 943

- Generic Multicast Registration Protocol (GMRP), 943
 - Generic Routing Encapsulation. *See* GRE (Generic Routing Encapsulation)
 - Generic Security Services API (GSS-API), 943
 - Generic top-level domains (gTLD), 512–514, 943
 - GI (guard interval)
 - definition of, 942
 - higher throughput (802.11n) support and, 127
 - Giaddr (Gateway or Router IP Address) field
 - DHCP/BOOTP message format, 238
 - LDRAs and, 271
 - GKM (group key management)
 - definition of, 943
 - multicast support in IPSec, 864
 - Global Internet. *See* Internet
 - Global scope
 - configuring global addresses with SLAAC, 278
 - of IPv6 addresses, 43
 - of multicast addresses, 53
 - GLOP addressing, IP multicast, 54–56
 - GMAC (Galois MAC), 819–820, 943
 - GMRP (Generic Multicast Registration Protocol), 943
 - GPAD (Group PAD), 864, 943
 - Granularity, of TCP clock, 654
 - Gratuitous ARP, 175–176
 - GRE (Generic Routing Encapsulation)
 - definition of, 943
 - establishing tunneling with, 149–153
 - link layer attacks and, 156
 - NAT and tunneled packets, 310
 - Greenfield mode, 802.11n operating modes, 128
 - Group addresses. *See* Multicast addresses
 - Group controller/key servers (GCKSs)
 - definition of, 942
 - multicast support in IPSec, 864–865
 - Group Domain of Interpretation (GDOI), 942
 - Group key management (GKM)
 - definition of, 943
 - multicast support in IPSec, 864
 - Group members (Group Member Part), IGMP/MLD
 - processing by, 454–457
 - Group membership
 - displaying IP group membership, 446–447
 - IGMP membership reports, 455
 - Group PAD (GPAD), 864, 943
 - Group Secure Association Key Management (GSAKMP), 943
 - Group security associations (GSAs)
 - definition of, 943
 - multicast support in IPSec, 864
 - Group SPD (GSPD), 943
 - GSAKMP (Group Secure Association Key Management), 943
 - GSAs (group security associations)
 - definition of, 943
 - multicast support in IPSec, 864
 - GSPD (Group SPD), 943
 - GSS-API (Generic Security Services API), 943
 - gTLD (generic top-level domains), 512–514, 943
 - Guard interval (GI)
 - definition of, 942
 - higher throughput (802.11n) support and, 127
 - GVRP (Generic Attribute Registration Protocol), 943
- ## H
- HAIO (Home Agent Information Option), 943
 - Hairpinning (HP), 314, 486
 - Half-close operation, TCP connections, 598–599
 - Half-open connections, keepalives detecting, 794
 - Handoff information, DHCP, 275
 - Handover Key Request/Reply options, in ND, 422–423
 - Handshaking protocols
 - DTLS (Datagram TLS), 892–894
 - three-way handshake in TCP, 597, 640
 - TLS, 880–883
 - HAs (home agents)
 - definition of, 943
 - Home Agent Information option in ND, 412–413
 - ICMP Home Agent Discovery Request message, 386
 - ICMP Mobile Prefix Solicitation message, 387–388
 - in Mobile IP, 216–217
 - Hash-based addresses (HBAs), 405
 - Hash functions
 - CGAs (cryptographically generated addresses) and, 404–405
 - cryptographic, 817–818
 - initial sequence numbers in TCP and, 601–602
 - TCP-AO (Authentication Option), 612
 - HBAs (hash-based addresses), 405
 - HC (hybrid coordinator), in HCCA, 123
 - HCCA (HFCA-controlled channel access), 123
 - HCF (hybrid coordination function)
 - definition of, 943
 - options for controlling sharing of wireless medium, 120
 - QoS and, 122–123
 - HDLC (High-Level Data Link Control)
 - Address and Control fields, 132
 - definition of, 943
 - LCP links based on, 131
 - Header compression, in PPP
 - CCP (Compression Control Protocol), 139
 - PPP (Point-to-Point Protocol), 142–143

- Header, ICMP
 - ICMP Parameter Problem and, 379–380
 - Redirected Header option in ND, 411
- Header, IP
 - CE (Congestion Experienced) indicator in, 783–784
 - Checksum field, 186–188
 - DS and ECN fields, 188–192
 - extensions, 182–183
 - header compression, 142–143
 - IPv6 extensions, 194–196
 - IPv6 Fragment header, 203–208
 - IPv6 RH (Router header), 200–203, 955
 - Mobile IP, 216
 - overview of, 183–186
- Header, TCP
 - connection options, 605–606
 - fields in, 588–590
 - MSS (Maximum Segment Size) option, 606–607
 - PAWS (Protection against Wrapped Sequence Numbers), 610–611
 - SACK (selective acknowledgement) option, 607
 - TCP-AO (Authentication Option), 612
 - Timestamps option, 608–610
 - UTO (User Timeout) option, 611–612
 - WSALE (Window Scale) option, 608
- Header, UDP, 474–476, 481–482
- HELD (HTTP-enabled Location Delivery)
 - definition of, 943
 - location information in DHCP, 274
- Hexadecimal notation
 - expressing IP addresses in, 32–33
 - IPv6 addresses in, 537
- HCCA-controlled channel access (HCCA), 123
- Hierarchical routing, 48
- High-Level Data Link Control. *See* HDLC (High-Level Data Link Control)
- High-speed environments, congestion control in, 770
- High-Speed Packet Access (HSPA), 944
- High Throughput (HT), 128, 944
- HighSpeed TCP (HSTCP), 770–772, 944
- Hijacking attacks
 - firewalls and NATs and, 345
 - TCP-related attacks, 641
- HIP (Host Identity Protocol)
 - definition of, 943
 - Identifier/locator separating protocols, 70
- Historic category, RFCs and, 23
- HK-LIFETIME, Handover Keys, 422–423
- Hlen (HW Len) field, DHCP/BOOTP message format, 236
- HMAC (keyed-hash message authentication code), 818–819, 943
- HoA (home address)
 - definition of, 944
 - of IPv6 node, 199
 - in Mobile IP, 216–218
- Hole punching, in NAT traversal, 317
- Home address. *See* HoA (home address)
- Home Agent Discovery Request message, ICMP, 386
- Home Agent Information Option (HAIO), 943
- Home Agent Information option, in ND, 412–413
- Home agents. *See* HAs (home agents)
- Home Test (HoT)
 - definition of, 944
 - mobility messages in RRP, 218–219
- Home Test Init (HoTI)
 - definition of, 944
 - mobility messages in RRP, 218–219
- Hop-by-hop
 - IP forwarding, 209
 - protocols, 12
- Hop-by-hop options (HOPOPTs)
 - definition of, 944
 - IPv6, 196
- Hop Limit field
 - ICMP Time Exceeded message, 375
 - IPv6, 199
 - MRD (Multicast Router Discovery) and, 394
 - ND messages and, 396
- HOPOPTs (hop-by-hop options)
 - definition of, 944
 - IPv6, 196
- Hops field, DHCP/BOOTP message format, 236
- Host addresses, 35
- Host fields, in IP addresses, 37
- Host Identity Protocol (HIP)
 - definition of, 943
 - Identifier/locator separating protocols, 70
- Host models, IP
 - address selection, 222–223
 - destination address selection algorithm, 224–225
 - overview of, 220–222
 - source address selection algorithm, 223–224
- Host names, 19
- Host number, in IP addresses, 35
- Host Requirements RFCs, 23
- Host unreachable message, ICMP, 364
- Hosts
 - ARP request to nonexistent, 173–174
 - host address filtering, 449–451
 - keepalives detecting state of peer host, 795
 - server host crashes and does not reboot (keepalive scenarios), 796
 - server host crashes and reboots (keepalive scenarios), 797–799

- server host unreachable (keepalive scenarios), 799–800
 - in small networks, 11–13
 - HoT (Home Test)
 - definition of, 944
 - mobility messages in RRP, 218–219
 - HoTI (Home Test Init)
 - definition of, 944
 - mobility messages in RRP, 218–219
 - HP (Hairpinning), 314, 486
 - HSPA (High-Speed Packet Access), 944
 - HSTCP (HighSpeed TCP), 770–772, 944
 - HT (High Throughput), 128, 944
 - HTML (Hyper-Text Markup Language), 944
 - HTTP-enabled Location Delivery (HELD)
 - definition of, 943
 - location information in DHCP, 274
 - HTTP (Hypertext Transfer Protocol)
 - definition of, 944
 - proxy firewalls, 302–303
 - well-known port for, 18
 - HTTP over SSL/TLS (HTTPS), 944
 - HTTP using UDP (HTTPMU), 338, 944
 - HTTPMU (HTTP using UDP), 338, 944
 - HTTPS (HTTP over SSL/TLS), 944
 - Htype (HW Type) field, DHCP/BOOTP message
 - format, 236
 - HWRP (Hybrid Wireless Routing Protocol)
 - definition of, 944
 - Wi-Fi mesh and, 130
 - Hybrid coordination function. *See* HCF (hybrid coordination function)
 - Hybrid coordinator (HC), in HCCA, 123
 - Hybrid cryptosystems, 812
 - Hybrid Wireless Routing Protocol (HWRP)
 - definition of, 944
 - Wi-Fi mesh and, 130
 - Hyper-Text Markup Language (HTML), 944
 - Hypertext Transfer Protocol. *See* HTTP (Hypertext Transfer Protocol)
- I
- IA (Identity Association)
 - definition of, 944
 - in DHCPv6, 255–256
 - IAB (Internet Architecture Board), 22, 944
 - IAID (Identity Association Identifier)
 - definition of, 944
 - in DHCP, 246, 255–256
 - IANA (Internet Assigned Numbers Authority)
 - allocation of IP addresses and, 62
 - allocation of IPv6 multicast addresses, 58
 - definition of, 944
 - IKE registry of values, 849
 - OUI (Organizationally Unique Identifier), 442–444
 - port number assignments, 18, 632
 - registry for Sec values, 405
 - registry for SRV values, 548–549
 - URI, 553
 - IANA Service Name and Transport Protocol Port Number (ISPR), 548
 - IBSS (independent basic service set)
 - definition of, 944
 - Wi-Fi, 112
 - ICANN (Internet Corporation for Assigned Names and Numbers)
 - definition of, 944
 - DNSSEC zones list, 904
 - TLD management, 512
 - ICE (Interactive Connectivity Establishment), 332–334, 944
 - ICMP fix-up, 309
 - ICMP (Internet Control Message Protocol)
 - Advertisement Interval option in ND, 412
 - attacks related to, 428–429
 - Certificate option in ND, 417
 - CGA options in ND, 414–415
 - definition of, 944
 - destination unreachable, 364–372
 - DNS Search List option in ND, 422–423
 - Echo Request/Reply messages, 380–383
 - encapsulation of messages within IP datagrams, 354–355
 - error messages, 361–363
 - Experimental Values in ND, 423
 - extended and multipart messages, 363–364
 - Handover Key Request/Reply options in ND, 422–423
 - Home Agent Discovery Request message, 386
 - Home Agent Information option in ND, 412–413
 - ICMPv4 messages, 356–357
 - ICMPv6 messages, 358–360
 - IND (Inverse Neighbor Discovery), 401–402
 - introduction to, 353–354
 - IP Address/Prefix option in ND, 417–418
 - Link-Layer Address (LLA) option in ND, 418–419
 - MIPv6 fast handover messages, 388
 - MLD extension messages, 390–394
 - Mobile Prefix Solicitation message, 387–388
 - MRD (Multicast Router Discovery), 394–395
 - MTU option in ND, 411–412
 - NAT and, 309
 - ND options, 407–409
 - ND support, 395–396
 - Nonce option in ND, 416–417
 - NS (Neighbor Solicitation) message, 398–401

- ICMP (Internet Control Message Protocol), continued
 - NUD (Neighbor Unreachability Detection), 402–403
 - Parameter Problem message, 379–380
 - Prefix Information option in ND, 410–411
 - processing messages, 360–361
 - queries/informational messages, 380
 - Recursive DNS Server option in ND, 420
 - Redirected Header option in ND, 411
 - redirection of messages, 372–375
 - Route Information option in ND, 420
 - Router Advertisement Flags Extension option in ND, 420–421
 - router solicitation and advertisement messages in ICMPv4, 383–385
 - router solicitation and advertisement messages in ICMPv6, 396–398
 - RSA Signature option in ND, 415–416
 - SEND (Secure Neighbor Discovery), 403–407
 - sending broadcast datagrams, 439
 - Source Link-Layer Address option in ND, 409–410
 - Source Quench messages, 785
 - Source/Target Address List options in ND, 413–414
 - summary and references, 430–434
 - TCP-related attacks, 641–642
 - time exceeded message, 375–378
 - Timestamp option in ND, 416
 - translating ICMPv4 to ICMPv6, 424–426
 - translating ICMPv6 to ICMPv4, 426–428
 - Trust Anchor option in ND, 417
 - use in layers of TCP/IP suite, 15
- ICS (Internet Connection Sharing)
 - assignment of unicast addresses, 67
 - definition of, 944
 - NAT in Windows OS context, 337
- ICV (Integrity Check Value), 856, 945
- ID (identification payload), in IKE, 945
- Id/loc split protocols, 70
- Identification field
 - IP header fields, 185, 203–204
 - in UDP fragmentation, 489
- Identification (ID payload), in IKE, 945
- Identification messages, in LCP operation, 134
- Identifier/locator separating protocols, 70
- Identifiers
 - in demultiplexing, 11
 - IP addresses as, 70
 - in multiplexing, 10
- Identity Association (IA)
 - definition of, 944
 - in DHCPv6, 255–256
- Identity Association Identifier (IAID)
 - definition of, 944
 - in DHCP, 246, 255–256
- IDN ccTLS (Internationalized ccTLDs), 512
- IDNs (internationalized domain names), 512, 945
- IEEE (Institute of Electrical and Electronics Engineers)
 - definition of, 945
 - Ethernet. *See* Ethernet (802.3)
 - interface standards, 44
 - LAN/MAN standards (802), 82–84
 - link aggregation (802.1AX), 92–93
 - Logical Link Control (802.2), 84
 - Multiple Registration Protocol (802.1ak), 111
 - network security (802.1x), 833–834
 - quality of service (802.1p), 90
 - standards of, 24
 - for VLANs (802.1q), 89–92
 - wireless (802.11). *See* Wi-Fi (wireless fidelity-802.11)
- IESG (Internet Engineering Steering Group), 22–23, 945
- IETF (Internet Engineering Task Force)
 - definition of, 945
 - for Internet standards, 22–23
 - ROAD (ROuting and Addressing) group, 47
 - securing IPv6 Neighbor Discovery, 292
- ifconfig command, in UNIX and Linux
 - setting/finding broadcast addresses, 437
 - viewing active multicast addresses, 66
 - viewing format of link-local IPv6 address, 45–46
- IGD (Internet Gateway Device), in UPnP
 - definition of, 945
 - NAT and, 338–339
- IGDDC (Internet Gateway Device Discovery and Control), 337
- IGMP (Internet Group Management Protocol)
 - attacks related to, 469–470
 - counters and variables, 467–468
 - definition of, 945
 - examples, 459–464
 - lightweight IGMP3, 464–465
 - MLD as translation of IGMPv3 to IPv6, 390
 - MRD (Multicast Router Discovery) and, 394
 - in multicast addressing, 15
 - overview of, 451–453
 - processing by group members (Group Member Part), 454–457
 - processing by multicast routers (Multicast Router Part), 457–459
 - robustness of, 465–467
 - snooping, 468–469
 - summary and references, 470–472

- IHL (Internet Header Length), in IPv4
 - attacks related to, 919
 - definition of, 945
 - ICMP Parameter Problem and, 379
 - IP header and, 183
- IIDs (interface identifiers)
 - as basis for unicast IPv6 addresses, 43–46
 - definition of, 945
 - for link-scoped IPv6 addresses, 58
- IKE (Internet Key Exchange), in IPsec
 - algorithm selection and application, 849–850
 - BTNS (Better-than-Nothing Security), 852
 - CREATE_CHILD_SA exchange, 852–853
 - definition of, 945
 - EAP and, 851–852
 - example using, 867–876
 - IKE_AUTH exchange, 850–851
 - IKE_SA_INIT exchange, 846–847
 - INFORMATIONAL exchange, 853–854
 - KE (Key Exchange) and Ni, Nr (Nonce) payloads, 848
 - message formats, 844–846
 - MOBIKE (Mobile IKE), 854
 - N (Notification) and CP (Configuration) payloads, 849
 - NAT updates and, 866
 - overview of, 842–843
 - SA (Security Association) payloads and proposals, 847–848
 - traffic selectors, 851
- IKE_AUTH exchange, 850–851, 871–874
- IKE_SA_INIT exchange, 846–847, 867–870
- IMAP (Interactive Mail Access Protocol)
 - definition of, 945
 - SRV record providing IMAP service, 549
 - well-known port for, 18
- IMAPS (IMAP over SSL/TLS), 18, 945
- Implementation architecture
 - design and, 8
 - layering, 8–10
 - multiplexing, demultiplexing, and encapsulation, 10–13
- Implementations, TCP/IP suite, 24–25
- Implicit sending, in congestion control, 583
- IN (Internet class name), 945
- Incoming connection queue, TCP servers, 636–640
- Incremental zone transfer (IXFR) messages, in DNS, 561, 946
- IND (Inverse Neighbor Discovery)
 - definition of, 945
 - neighbor discovery in IPv6, 401–402
 - Source/Target Address List options in ND, 413–414
- Independent basic service set (IBSS)
 - definition of, 944
 - Wi-Fi, 112
- Indication transactions, STUN, 320
- Indirect delivery, example of IP forwarding, 212–215
- Industrial, Scientific, and Medical (ISM), 124
- Information disclosure attacks, ICMP and, 428
- Information security. *See also* Security, 806
- Information theory, 579
- Informational category, RFCs and, 23
- INFORMATIONAL exchange, IKE protocol, 853–854, 873, 875
- Informational messages, ICMP, 309
- Initial sequence number. *See* SN (initial sequence number)
- Initial window (IW) value
 - definition of, 946
 - in slow start algorithm, 732
- Institute of Electrical and Electronics Engineers (IEEE). *See* IEEE (Institute of Electrical and Electronics Engineers)
- Integrated Services Digital Network (ISDN), 946
- Integrity Check Value (ICV), 856, 945
- Integrity, in CIA triad, 806
- Integrity protection, AH (Authentication Header), 856, 858
- Inter-Packet Gap (IPG), in Ethernet, 89, 945
- Inter-Switch Link (ISL)
 - definition of, 946
 - VLAN trunking, 90
- Interactive communication, TCP data flow, 692–695
- Interactive Connectivity Establishment (ICE), 332–334, 944
- Interactive data, in TCP communication, 692
- Interactive keystrokes, TCP and, 692–693
- Interactive Mail Access Protocol. *See* IMAP (Interactive Mail Access Protocol)
- Interface address, IP addresses and, 35
- Interface identifiers. *See* IIDs (interface identifiers)
- Interface, in IP forwarding, 209
- Intermediate system, for protocol suites, 12
- Intermediate System to Intermediate System (IS-IS), 946
- International Organization for Standardization (ISO), 8–9
- International Telecommunication Union. *See* ITU (International Telecommunication Union)
- Internationalized ccTLDs (IDN ccTLS), 512
- Internationalized domain names (IDNs), 512, 945
- Internet
 - attacks related to Internet architecture, 25–26
 - internationalization of, 512
 - overview of, 19–20

Internet, continued

- TCP/IP suite forming basis of, 2
- UDP in, 506–507
- WWW compared to, 2–3
- Internet Architecture Board (IAB), 22, 944
- Internet Assigned Numbers Authority. *See* ANA (Internet Assigned Numbers Authority)
- Internet checksum
 - algorithm for computing Internet-related checksum, 185
 - mathematics of, 187–188
 - overview of, 186–187
- Internet class name (IN), 945
- Internet Connection Sharing. *See* CS (Internet Connection Sharing)
- Internet Control Message Protocol. *See* CMP (Internet Control Message Protocol)
- Internet Corporation for Assigned Names and Numbers. *See* CANN (Internet Corporation for Assigned Names and Numbers)
- Internet Engineering Steering Group (IESG), 22–23, 945
- Internet Engineering Task Force. *See* ETF (Internet Engineering Task Force)
- Internet Gateway Device Discovery and Control (IGDDC), 337
- Internet Gateway Device (IGD)
 - definition of, 945
 - NAT and, 338–339
- Internet Group Management Protocol. *See* GMP (Internet Group Management Protocol)
- Internet Header Length. *See* HL (Internet Header Length)
- Internet Key Exchange. *See* KE (Internet Key Exchange)
- Internet Protocol Control Protocol (IPCP)
 - MPPE and, 145
 - types of NCPs used on PPP links, 141
- Internet Registry Information Service (IRIS), 554, 946
- Internet Research Task Force (IRTF), 23, 946
- Internet Security Association and Key Management Protocol (ISAKMP), 867, 946
- Internet Service Providers. *See* SPs (Internet Service Providers)
- Internet Society (ISOC), 23, 946
- Internetwork layer. *See* Network (internetwork) layer
- Intra-Site Automatic Tunnel Addressing Protocol (ISATAP), 440, 946
- Intranets, 20
- Inverse Neighbor Discovery. *See* ND (Inverse Neighbor Discovery)
- IP address pooling behavior, NAT and, 312
- IP Address/Prefix option, in ND, 417–418

IP addresses

- Address realms, 299
- allocation of, 62
- allocation of multicast addresses, 65
- allocation of unicast addresses, 62–65
- anycast addresses, 62
- assigning unicast addresses, 65–66
- attacks involving, 70–71
- broadcast addresses, 42–43
- CIDR, 47–48
- classful addressing, 34–36
- converting IP multicast addresses to MAC addresses, 442–444
- definition of, 14
- in DHCP/BOOTP message format, 237–238
- expressing, 32–34
- foreign IP address restrictions in server design, 502–503
- host names and, 19
- introduction to, 31–32
- IPv4/IPv6 translators, 52–53
- IPv4 multicast addresses, 54–57
- IPv6 addresses and interface identifiers, 43–46
- IPv6 multicast addresses, 57–61
- multicast addresses, 53–54
- multiple addresses in UDP server design, 501–502
- multiple providers/multiple networks/multiple addresses, 68–70
- route aggregation, 48–50
- single provider/multiple networks/multiple addresses, 67–68
- single provider/no network/single address, 66–67
- single provider/single network/single address, 67
- spanning IP address families in server design, 504
- special use addresses, 50–52
- spoofing attacks and, 226
- structure of, 34
- subnet addressing, 36–39
- subnet masks, 39–41
- summary and references, 71–77
- TCP/IP suite and, 19
- TCP server restrictions on local, 634–635
- unicast and broadcast, 15
- in UDP server design, 499–501
- VSLM (variable-length subnet masks), 41–42
- IP datagrams. *See also* packets
 - fragmenting, 184
 - of ICMP messages within IP datagrams, 354–355
 - LCP (Link Control Protocol) and, 131–137
 - PPP and, 130–131

- source and destination IP addresses in, 186
- in TCP/IP suite, 14
- IP forwarding
 - direct delivery, 210–212
 - forwarding actions, 209–210
 - forwarding table, 208–209
 - indirect delivery, 212–215
 - overview of, 208
- IP fragmentation
 - example of UDP/IPv4 fragmentation, 488–492
 - interaction between IP fragmentation and ARP/ND, 496–497
 - of IP datagrams, 184
 - IP performing, 148
 - IPv6 Fragment header, 203–208
 - overview of, 488
 - reassembly timeout, 492
 - UDP-related attacks, 506
- IP (Internet Protocol)
 - address selection by hosts, 222–223
 - attacks, 226
 - best-effort delivery, 7
 - bidirectional tunneling in mobile IP, 216–217
 - destination address selection algorithm, 224–225
 - discussion of issues in mobile IP, 220
 - DS field and ECN, 188–192
 - examples of forwarding, 210–215
 - forwarding. *See* P forwarding
 - Fragment header, 203–208
 - header. *See* Header, IP
 - host models, 220–222
 - ICMP addressing limits in, 353
 - Internet checksum, 186–188
 - introduction to, 181–183
 - IPv6 extension headers, 194–196
 - IPv6 options, 196–199
 - mobile IP. *See* MIP (Mobile IP)
 - options, 192–194
 - RO (route optimization) in mobile IP, 217–219
 - Routing header, 200–203
 - source address selection algorithm, 223–224
 - summary and references, 226–231
- IP masquerading
 - ipchains command for configuring, 345
 - NAT in Linux context, 337
- IP routers, 20
- ipchains command, configuring IP masquerading in Linux, 345
- ipconfig command in Windows
 - manual management of DHCP information, 250–251
 - manual management of DHCPv6 information, 265–266
- IPCP (Internet Protocol Control Protocol), in PPP
 - MPPE and, 145
 - types of NCPs used on PPP links, 141
- IPG (Inter-Packet Gap), in Ethernet, 89, 945
- IPSec (IP Security)
 - attacks related to, 918
 - Authentication Header. *See* AH (Authentication Header)
 - definition of, 945
 - Encapsulating Security Payload. *See* ESP (Encapsulating Security Payload)
 - example using IKE, 867–876
 - GRE tunnels and, 150
 - Internet Key Exchange protocol. *See* KE (Internet Key Exchange)
 - L2TP/IPSec, 865
 - layer 3 security with, 840–842
 - multicast support, 864
 - NAT traversal, 865–867
- iptables, 335
- IPv4 addresses
 - allocation of multicast addresses, 65
 - allocation of unicast addresses, 62–65
 - configuring with SLAAC, 276
 - example of subnet addressing, 37
 - examples of subnet masks, 39
 - expressing, 32–33
 - foreign IP address restrictions in server design, 502–503
 - IPv4-converted addresses, 341
 - IPv4-embedded IPv6 address, 52
 - IPv4/IPv6 translators, 52–53
 - IPv4-translatable addresses, 341
 - local address restrictions in UDP server design, 500–501
 - multicast addresses, 54–57
 - multiple addresses in UDP server design, 501–502
 - multiple servers per port in UDP server design, 503–504
 - setting for embedded devices, 178
 - spanning IP address families in UDP server design, 504
 - special-use addresses, 50–51
 - tunneling IPv6 packets over UDP/IPv4 packets, 154
 - using ARP to map to hardware addresses, 165
- IPv4/IPv6 translation
 - DS-Lite (Dual-Stack Lite), 339–340
 - IPv4-converted and IPv4 translatable addresses, 341–342
 - NAT and ALG for, 340–345
 - overview of, 340–341

- IPv4/IPv6 translation, continued
 - stateful translation, 344–345
 - stateless translation, 342–344
 - IPv4 protocol
 - ARP and, 13
 - Checksum field, 186–188
 - computing Internet checksum, 186–187
 - direct delivery with, 167–169
 - DS and ECN fields, 188–192
 - encapsulation of ICMP messages in, 354–355
 - header, 182–183
 - header fields, 183–186
 - ICMP messages related to, 356–357
 - ToS (Type of Service) byte in, 188–189
 - translation to/from IPv6. *See* IPv4/IPv6 translation
 - tunneling IPv6 through, 482–487
 - IPv6 addresses
 - allocation of multicast addresses, 65
 - allocation of unicast addresses, 62–65
 - configuring global addresses with SLAAC, 278
 - configuring with SLAAC, 276–277
 - DAD (Duplicate Address Detection), 277–278
 - examples of subnet masks, 40
 - expressing, 32–34
 - interface identifiers and, 43–46
 - IPv4/IPv6 translators, 52–53
 - lifecycle of, 252–253
 - multicast addresses, 57–61
 - multiple servers per port in UDP server design, 503–504
 - spanning IP address families in UDP server design, 504
 - special-use addresses, 51–52
 - tunneling IPv6 packets over UDP/IPv4 packets, 154
 - ULAs (Unique Local IPv6 Unicast Addresses), 225
 - wildcard address, 632
 - IPv6 protocol
 - Checksum field, 186–188
 - DS and ECN fields, 188–192
 - encapsulation of ICMP messages in, 354–355
 - extension headers, 194–196
 - Fragment header, 203
 - header, 182–183
 - header fields, 183–186
 - ICMP messages related to, 358–360
 - jumbogram support, 481
 - NAT and, 310–311
 - neighbor discovery. *See* ND (Neighbor Discovery Protocol)
 - options, 196–199
 - Rapid Deployment (6rd), 339, 933
 - Routing header, 200–203
 - Traffic Class byte in, 188–189
 - translating to/from IPv4. *See* IPv4/IPv6 translation
 - tunneling through IPv4 networks, 482–487
 - UDP and, 481–482
 - IPv6 Remote-ID, DHCP relay agents, 268
 - IPv6CP (IPv6 Control Protocol), in PPP, 141, 945
 - IRIS (Internet Registry Information Service), 554, 946
 - IRTF (Internet Research Task Force), 23, 946
 - IS-IS (Intermediate System to Intermediate System), 946
 - ISAKMP (Internet Security Association and Key Management Protocol), 867, 946
 - ISATAP (Intra-Site Automatic Tunnel Addressing Protocol), 440, 946
 - ISDN (Integrated Services Digital Network), 946
 - ISL (Inter-Switch Link)
 - definition of, 946
 - VLAN trunking, 90
 - ISM (Industrial, Scientific, and Medical), 124
 - ISN (initial sequence number)
 - definition of, 946
 - SYN segment containing, 597
 - in TCP connection establishment, 601–602
 - TCP header and, 589
 - ISO (International Organization for Standardization), 8–9
 - ISOC (Internet Society), 23, 946
 - ISPR (IANA Service Name and Transport Protocol Port Number), 548
 - ISPs (Internet Service Providers)
 - ACs (access concentrator), 287
 - allocation of IP addresses and, 32, 62–63
 - definition of, 946
 - MTU (maximum transmission unit), 494
 - Iterative servers, 21
 - ITU (International Telecommunication Union)
 - definition of, 946
 - standards organizations, 24
 - X.509 standard. *See* X.509 standard
 - IW (initial window) value, in TCP
 - definition of, 946
 - in slow start algorithm, 732
 - iwconfig command, setting Wi-Fi control frame variables, in UNIX and Linux, 115
 - IXFR (incremental zone transfer) messages, in DNS, 561–563, 946
- ## J
- Jacobson, V., 652–654, 731
 - Jumbograms
 - IPv6 options for, 198
 - IPv6 supporting, 481

-
- K**
- Kaminsky attacks, DNS-related, 572
 - Kamoun, F., 48
 - Karn's algorithm, 655
 - KDF (key derivation function), 815
 - KE (Key Exchange)
 - definition of, 946
 - IKE payloads, 848
 - Keepalive interval
 - changing values of, 796
 - definition of, 795
 - Keepalive probes
 - changing values of, 796
 - definition of, 795
 - overview of, 793
 - Keepalive time
 - changing values of, 796–797
 - definition of, 795
 - server host crashes and reboots, 799–800
 - server host unreachable, 800–802
 - Keepalive timers, 793
 - Keepalives, TCP. *See* TCP keepalive
 - Key derivation function (KDF), 815
 - Key Exchange (KE)
 - definition of, 946
 - IKE payloads, 848
 - Key Hash field, RSA Signature option, 416
 - Key management, in cryptography, 821
 - Key signing keys. *See* KSKs (key signing keys)
 - Keyed-hash message authentication code (HMAC), 818–819, 943
 - Keys, cryptographic, 810
 - Kleinrock, Leonard, 1, 48
 - KSKs (key signing keys)
 - definition of, 946
 - DNSSEC, 897, 905
- L**
- L2TP/IPSec, 865
 - L2TP (Layer 2 Tunneling Protocol)
 - definition of, 946
 - establishing tunneling with, 149
 - LaBrea tarpit, attacks related to window management, 723
 - LACP (Link Aggregation Control Protocol), 92–93, 947
 - LAGs (link aggregation groups), 93, 947
 - Land attacks, ICMP attacks, 428
 - LANs (local area networks)
 - definition of, 947
 - Ethernet standards, 82–84
 - virtual. *See* VLANs (virtual LANs)
 - Large-scale NAT (LSN), IPv6 transition, 315
 - Last Member Query Interval (LMQI), in IGMP/MLD, 468, 947
 - Last Member Query Time (LMQT), 466–467, 947
 - LAST_ACK state
 - simultaneous open and close transitions, 625
 - TCP state transitions, 618
 - Latency, connections and, 4
 - Layer 2 devices, DHCP relay agents, 270–271
 - Layer 2 Tunneling Protocol (L2TP)
 - definition of, 946
 - establishing tunneling with, 149
 - Layering
 - implementation and design and, 8–10
 - security protocols, 832–833
 - TCP/IP suite and, 14
 - Layering violation, transport layer, 476
 - LCG (Linear Congruential Generator), 816, 947
 - LCI (Location Configuration Information), 274, 947
 - LCI (logical channel identifier), 5
 - LCN (logical channel number), 5, 947
 - LCP (Link Control Protocol)
 - definition of, 947
 - operation of, 133–134
 - options, 134–137
 - overview of, 131–134
 - LDAP (Lightweight Directory Access Protocol)
 - definition of, 947
 - DNS and, 570–571
 - SRV record providing LDAP service, 548–549
 - well-known port for, 18
 - LDRAs (lightweight DHCP relay agents), 271, 947
 - LEAP (Lightweight EAP), 947
 - Leasequery, DHCP relay agents, 269–270
 - Leases
 - in BOOTP, 235
 - duration of DHCP leases, 235–236
 - Length field
 - in Ethernet frame format, 85–86
 - UDP header, 475, 481–482
 - Payload Length field, in IPv6 header, 184, 198, 204–205
 - Total Length field, in IPv4 header, 183–184, 207
 - Licklider, J.C.R., 2
 - Lightweight DHCP relay agents (LDRAs), 271, 947
 - Lightweight Directory Access Protocol. *See* LDAP (Lightweight Directory Access Protocol)
 - Lightweight EAP (LEAP), 947
 - Lightweight IGMP3, 464–465
 - Lightweight MLD (LW-MLD), 394, 948
 - Lightweight MLDv2, 464–465
 - Limited slow start, 772
 - Limited transmit approach, to congestion control in TCP, 742

- Linear Congruential Generator (LCG), 816, 947
- Link Address field, in DHCPv6 message format, 254
- Link aggregation (802.1AX), 92–93
- Link Aggregation Control Protocol (LACP), 92–93, 947
- Link aggregation groups (LAGs), 93, 947
- Link Control Protocol. *See* LCP (Link Control Protocol)
- Link discriminator, LCP options, 139
- Link layer
 - address resolution and, 165–166
 - attacks on, 154–156
 - autonegotiation in Ethernet interface, 95
 - bridges and switches, 98–102
 - broadcasting ARP frames, 167
 - duplex mismatch, 96
 - Ethernet (802.3) frames, 84–86
 - Ethernet frame sizes, 88–89
 - Ethernet supporting broadcasting at, 437
 - flow control (802.1X) in Ethernet interface, 98
 - full duplex Ethernet, 94–95
 - IEEE 802 LAN/MAN standards for, 82–84
 - integrity checking on Ethernet frames, 86–88
 - introduction to, 79
 - link aggregation (802.1AX), 92–93
 - MRP (Multiple Registration Protocol), 111
 - MTU (maximum transmission unit), 148
 - Point-to-Point Protocol. *See* PPP (Point-to-Point Protocol)
 - standards, 80–82
 - STP (Spanning Tree Protocol). *See* STP (Spanning Tree Protocol)
 - summary and references, 156–163
 - tunneling, 149–153
 - UDLs (unidirectional links), 153–154
 - VLANs and QoS tagging, 89–92
 - wireless LANs. *See* Wi-Fi (wireless fidelity-802.11)
 - WoL (Wake-on LAN), power saving, and magic packets, 96–97
- Link-Layer Address. *See* LLA (Link-Layer Address)
- Link Local Multicast Name Resolution (LLMNR), 445, 569–570, 947
- Link-local scope
 - configuring IPv4 addresses with SLAAC, 276
 - configuring IPv6 addresses with SLAAC, 276–277
 - of IPv6 addresses, 43
 - IPv6 multicast addresses, 58–59
 - multicast addresses, 53
 - viewing format of link-local IPv6 address, 45
- Link Quality Reports (LQRs)
 - definition of, 947
 - in PPP operations, 136
- Links, unidirectional, 153–154
- Linux OSs
 - autotuning TCP receive windows, 715
 - `ethtool` program for checking full duplex support, 94
 - IP masquerading, 337
 - IPSec implementations, 867
 - quick acknowledgments, 733
 - rate limiting of ICMP messages, 369–370
 - RTT estimation, 657–661
 - standards and, 24
 - `vconfig` command for manipulating 802.1p/q information, 90–91
- LISTEN state, in TCP
 - TCP port numbers and, 632–633
 - TCP state transitions and, 618
- LLA (Link-Layer Address)
 - definition of, 947
 - Link-Layer Address (LLA) option in ND, 418–419
 - Source Link-Layer Address option in ND, 409–410
 - Source/Target Address List options in ND, 413–414
- LLC (Logical Link Control)
 - 802.2 standard defining, 84
 - definition of, 947
 - relationship of link layer frames to data frames, 116
- LLMNR (Link Local Multicast Name Resolution), 445, 569–570, 947
- LMQI (Last Member Query Interval), in IGMP/MLD, 468, 947
- LMQT (Last Member Query Time), 466–467, 947
- LNP (Local Network Protection), 310, 947
- Local area networks. *See* LANs (local area networks)
- Local congestion, in Linux TCP example,
 - fast retransmit and, 759–762
 - sender pause and, 750–754
 - stretch ACKs and recovery from, 754–757
- Local IP addresses
 - restrictions in UDP server design, 500–501
 - TCP server restrictions on local, 634–635
- Local net (limited) broadcast, 43
- Local Network Protection (LNP), 310, 947
- Location Configuration Information (LCI), 274, 947
- Location-to-Service Translation (LoST), 274–275
- Locators, IP addresses as, 70
- Logical channel identifier (LCI), 5
- Logical channel number (LCN), 5, 947
- Logical Internet addresses, translating to physical hardware addresses, 167
- Logical Link Control. *See* LLC (Logical Link Control)
- Long options, DHCP/BOOTP options, 239

- Long-term credential mechanism, STUN, 325–326
- Long-Term Evolution (LTE), 948
- Longest matching prefix algorithm, 69, 209–210
- Lookup, IP addresses from host names, 19
- Loopback
 - hairpinning (NAT loopback), 314, 486
 - implementing loopback capacity, 145–148
 - PPP issues and, 134
- LoST (Location-to-Service Translation), 274–275
- Low-rate DoD attacks, TCP timeout/retransmission attacks, 686
- LQRs (Link Quality Reports), in PPP
 - definition of, 947
 - in PPP operations, 136
- LSN (large-scale NAT), IPv6 transition, 315
- LTE (Long-Term Evolution), 948
- LW-MLD (Lightweight MLD), 948
- M**
- MAC layer, sublayer of link layer, 84
- MAC (Media Access Control)
 - addresses in Ethernet frames, 16
 - chaddr (Client Hardware Address) field, 244
 - DCF collision avoidance/backoff procedure, 121–122
 - definition of, 948
 - HCF (hybrid coordination function), 122–123
 - multicast addresses, 437
 - overview of, 120–121
 - protocols, 81
 - virtual carrier sense and physical carrier sense, 121
- MAC (message authentication codes), 818–820, 948
- MAC PDU (MPDU), 113, 949
- MAC Services Data Unit (MSDU), in 802.11n, 949
- Magnification attacks, UDP-related attacks, 506
- Mail exchanger (MX) records, in DNS
 - definition of, 950
 - DNS resource record types, 544–545
- Mailboxes, port numbers as, 474
- Malware (malicious software)
 - attacks related to Internet architecture, 26
 - definition of, 805–806
- Man-in-the-middle attacks. *See* MITM (man-in-the-middle) attacks
- Management frames, Wi-Fi, 113–115
- Manchester Phase Encoding (MPE)
 - clock recovery in Ethernet frames, 85
 - definition of, 949
- MANs (metropolitan area networks)
 - cable TV and DSL, 79
 - definition of, 948
 - Ethernet standards, 82–84
- Mapping timer, clearing NAT state, 308–309
- MAPs (mesh APs), 130
- Masks, in IP forwarding, 209
- Masquerading attacks
 - firewalls and NATs and, 345
 - ICMP attacks and, 429
 - link layer attacks, 155
 - types of threats to network communication, 808
- Master session keys (MSK), in EAP
 - definition of, 949
 - key derivation in EAP, 838
- Maximum pad value (MPV), 137, 949
- Maximum probing, 774
- Maximum received unit (MRU)
 - definition of, 949
 - length of LCP packet and, 134
- Maximum segment life (MSL). *See also* TIME_WAIT state (2MSL)
 - SYN segments, 610
 - TIME_WAIT state (2MSL) and, 618
- Maximum Segment Size. *See* MSS (Maximum Segment Size)
- Maximum transmission unit. *See* MTU (maximum transmission unit)
- MCS (Modulation and coding scheme)
 - definition of, 948
 - higher throughput (802.11n) support and, 127
- MD (Message Digest Algorithms)
 - definition of, 948
 - MD-5 (Message Digest Algorithm 5), 817–818
 - TLS extensions, 883
- mDNS (Multicast DNS), 444–445, 570, 948
- Mean deviation, in RTT estimation, 653
- Mechanisms
 - DNS TXT records and DKIM, 546
 - STUN, 325–326
- Media Access Control. *See* MAC (Media Access Control)
- Media independent handoff (MIH)
 - definition of, 948
 - mobility and handoff information in DHCP, 275
- Member links, in PPP bundles, 137
- Mesh (802.11s), Wi-Fi, 130
- Mesh APs (MAPs), 130
- Mesh deterministic access, 130
- Mesh Point (MP), 949
- Mesh points (MPs), 130, 949
- Mesh STAs (mesh stations), 130
- Message authentication codes (MAC), 818–820, 948
- Message boundaries, datagrams and, 5–6
- Message digests
 - Message Digest Algorithms. *See* MD (Message Digest Algorithms)
 - overview of, 817–818

Message formats

- DHCP and BOOTP, 236–238
- DHCPv6, 253–255
- DNS, 520–524
- ESP, 862
- IKE protocol, 844–846
- STUN, 320

Message stream modification (MSM)

- definition of, 949
- types of threats to network communication, 808

Meta types, resource record categories, 528**Metcalfe's Law, 19****Metropolitan area networks. See MANs (metropolitan area networks)****MF (More Fragments) field, in UDP fragmentation, 489****Michael, for message integrity checking, 129****Microsoft CHAP (MS-CHAP), 949****Microsoft Point-to-Point Compression Protocol (MPPC), 140, 949****Microsoft Point-to-Point Encryption (MPPE), 145, 150****MIH (media independent handoff)**

- definition of, 948
- mobility and handoff information in DHCP, 275

MIME (Multipurpose Internet Mail Extensions), 948**MIMO (multiple input, multiple output)**

- definition of, 948
- higher throughput (802.11n) support, 126–127

MIP (Mobile IP)

- bidirectional tunneling, 216–217
- definition of, 948
- DHCP and, 233
- discussion of issues in, 220
- handling IP nodes, 199
- ICMP fast handover message in MIPv6, 388
- ICMP Home Agent Discovery Request message, 386
- ICMP Mobile Prefix Solicitation message, 387–388
- Mobile IP with Fast Handovers (FMIP), 942
- Mobile IPv6, 70
- overview of, 215–216
- RO (route optimization), 217–219
- Routing header and, 200

Misbehaving receivers, TCP congestion control attacks, 785**MITM (man-in-the-middle) attacks**

- ICMP attacks and, 428
- security protocol-related, 918–919
- types of threats to network communication, 808

MLD (Multicast Listener Discovery)

- attacks related to, 469–470
- counters and variables, 467–468

DAD and, 278

- definition of, 948
- examples, 459–464
- ICMP extension messages, 390–394
- lightweight MLDv2, 464–465
- overview of, 15, 451–453
- processing by group members (Group Member Part), 454–457
- processing by multicast routers (Multicast Router Part), 457–459
- query/report/done messages in ICMP, 388–390
- robustness of, 465–467
- snooping, 468–469
- summary and references, 470–472

MLPP (Multilevel Precedence and Preemption), 189, 948**MMRP (Multiple MAC Registration Protocol), 111, 948****MNs (mobile nodes)**

- definition of, 948
- in Mobile IP, 216–218

MOBIKE (Mobile IKE), 854, 865, 949**Mobile IP. See MIP (Mobile IP)****Mobile nodes (MNs)**

- definition of, 948
- in Mobile IP, 216–218

Mobile Prefix Advertisement messages, ICMP, 410–411**Mobile Prefix Solicitation message, ICMP, 387–388****Mobility header, 216****Mobility messages, in RRP, 218–219****Mobility Services (MoS)**

- definition of, 949
- mobility and handoff information in DHCP, 275

Modifiers, DNS TXT records, 546**MODP (Modulo-P groups), in DH, 821, 949****Modulation and coding scheme (MCS), in 802.11n**

- definition of, 948
- higher throughput (802.11n) support and, 127

Modulo-P groups (MODP), in DH, 821, 949**More Frag field, data frame fragmentation, 117****MoS (Mobility Services)**

- definition of, 949
- mobility and handoff information in DHCP, 275

MP (Multilink PPP)

- definition of, 949
- overview of, 137–139

MPDU (MAC PDU), 113, 949**MPE (Manchester Phase Encoding)**

- clock recovery in Ethernet frames, 85
- definition of, 949

MPLS (Multi-Protocol Label Switching), 215, 949**MPPC (Microsoft Point-to-Point Compression Protocol), 140, 949**

-
- MPPE (Microsoft Point-to-Point Encryption), 145, 150
 - MPs (mesh points), 130, 949
 - MPV (maximum pad value), 137, 949
 - MRD (Multicast Router Discovery)
 - definition of, 949
 - designing location of multicast routers, 469
 - overview of, 394–395
 - MRP (Multiple Registration Protocol)
 - 802.1ak, 111
 - definition of, 949
 - MRRU (multilink maximum received reconstructed unit), 138, 949
 - MRU (maximum received unit)
 - definition of, 949
 - length of LCP packet and, 134
 - MS-CHAP (Microsoft CHAP), 949
 - MSDU (MAC Services Data Unit), in 802.11n, 949
 - MSK (master session keys)
 - definition of, 949
 - key derivation in EAP, 838
 - MSL (maximum segment life), in TCP. *See also*
 - TIME_WAIT state (2MSL)
 - SYN segments, 610
 - TIME_WAIT state (2MSL) and, 618
 - MSM (message stream modification) attack
 - definition of, 949
 - types of threats to network communication, 808
 - MSS (Maximum Segment Size), in TCP
 - definition of, 950
 - SWS (silly windows syndrome) and, 709
 - TCP header, 590, 605–607
 - MSTP (Multiple Spanning Tree Protocol), in bridges, 111
 - MTU (maximum transmission unit)
 - definition of, 950
 - for Ethernet, 86, 506
 - frame formats and, 79
 - ISPs (Internet Service Providers) and, 494
 - link layer and, 148
 - neighbor discovery options in IPv6, 411–412
 - PPPoE (PPP over Ethernet) and, 614
 - preventing fragmentation of TCP datagrams, 612–613
 - PTB messages in ICMPv6, 370–371
 - Multi-access networks, 9
 - Multi-Protocol Label Switching (MPLS), 215, 949
 - Multi6 architecture, 70
 - Multicast addresses
 - allocation of, 65
 - IPv4 multicast addresses, 54–57
 - IPv6 multicast addresses, 57–61
 - NAT and, 310
 - overview of, 53–54
 - Multicast DNS (mDNS), 444–445, 570, 948
 - Multicast groups
 - overview of, 15
 - RP (rendezvous point), 60
 - Multicast Listener Discovery. *See* MLD (Multicast Listener Discovery)
 - Multicast Listener Query, Report, and Done messages, ICMP messages, 388–390
 - Multicast Router Discovery. *See* MRD (Multicast Router Discovery)
 - Multicast routers
 - designing location of, 469
 - IGMP/MLD processing by, 457–459
 - overview of, 452–454
 - querier election, 466
 - query message options, 459
 - Multicast state, 441
 - Multicasting
 - converting IP multicast addresses to MAC addresses, 442–444
 - example of, 444–446
 - host address filtering, 449–451
 - introduction to, 435–436
 - IPSec supporting, 864–865
 - overview of, 441–442
 - receiving multicast datagrams, 447–449
 - sending multicast datagrams, 446–447
 - Multihomed systems
 - IPv6 and, 70
 - overview of, 12–13
 - unicast addresses and, 67–68
 - Multilevel Precedence and Preemption (MLPP), 189, 948
 - Multilink maximum received reconstructed unit (MRRU), 138, 949
 - Multilink PPP (MP), 137–139, 949
 - Multiple input, multiple output (MIMO)
 - definition of, 948
 - higher throughput (802.11n) support, 126–127
 - Multiple MAC Registration Protocol (MMRP), 948
 - Multiple Registration Protocol (MRP)
 - 802.1ak, 111
 - definition of, 949
 - Multiple Spanning Tree Protocol (MSTP), 111
 - Multiple VLAN Registration Protocol (MVRP), 111, 950
 - Multiplexing
 - implementation and design and, 10–13
 - important concepts in development of network architecture, 4
 - TCP/IP suite and, 16–17
 - Multipurpose Internet Mail Extensions (MIME), 948
 - MVRP (Multiple VLAN Registration Protocol), 111, 950

MX (mail exchanger) records, in DNS
 definition of, 950
 DNS resource record types, 544–545

N

N (Notification), IKE payloads, 849, 873

NA (Neighbor Advertisement)

 ICMPv6 messages, 277

 IND (Inverse Neighbor Discovery), 401

 main components of ND, 396

 neighbor discovery in IPv6, 398–401

NAC (Network Access Control), 833–837, 950

NACK (Negative ACKs), 111, 950

Nagle algorithm, in TCP

 Delayed ACK interaction with, 699

 disabling, 699–700

 overview of, 696–698

 reducing number of packets across WANs, 692

Nagle, John, 696

Name authority pointer records. *See* NAPTR (name authority pointer) records

Name resolution. *See also* DNS (Domain Name System)

 of host names into IP addresses, 511

 process of, 518–519

Name server (NS) records

 definition of, 951

 DNS resource record types, 529–530

Name servers

 caching, 517–518

 DNS, 516–517

Name Service Caching Daemon (NSCD), 518, 951

Name space, DNS, 512–514

Naming syntax, DNS, 514–516

NAP (Network Access Protection)

 definition of, 950

 DHCP leases and, 246

NAPT (Network Address Port Translation), 305–306, 950

NAPTR (name authority pointer) records

 definition of, 950

 DNS resource record types, 549–551

 NTN (non-terminal NAPTR), 551–552

 S-NAPTR and U-NAPTR, 554–555

 URI/URN resolution and, 553–554

NAR (New Access Router), 417, 950

NAT (network address translation)

 address and port translation behavior, 311–313

 address pools, 312

 attacks related to, 345–346

 configuring, 334

 definition of, 950

 direct interaction with, 338–339

 DS-Lite (Dual-Stack Lite) and, 339–340

 editors, 315

 filtering behavior, 312–313

 firewall rules, 335–336

 hairpinning (NAT loopback), 314, 486

 ICE and, 332–334

 ICMP and, 309

 IPSec NAT traversal, 865–867

 IPv4/IPv6 translation, 340–345

 IPv6 and, 310–311

 mapping, 307

 multicast and, 310

 NAPT (Network Address Port Translation), 305–306

 other transport protocols and, 309

 overview of, 303–305

 pinholes and hole punching, 317

 port forwarding and port mapping and, 314

 private addresses and, 51

 rules, 337–338

 session, 307–308

 SPNAT (service provider NAT), 315–316

 STUN (Session Traversal Utilities for NAT), 319–326

 summary and references, 346–352

 TCP and, 306–308

 Teredo tunneling and, 485–486

 translating TCP and UDP addresses and port numbers, 605

 traversal, 316

 tunneled packets and, 310

 TURN (Traversal Using Relays around NAT), 326–332

 UDP and, 308–309

 UNSAF (unilateral self-address fixing), 317–319

NAT-PMP (NAT Port Mapping Protocol)

 definition of, 950

 direct interaction with NAT and firewalls, 338–339

NAT Port Mapping Protocol (NAT-PMP)

 definition of, 950

 direct interaction with NAT and firewalls, 338–339

NAT-PT (NAT with Port Translation), 950

NAT Traversal

 ICE (Interactive Connectivity Establishment), 332–334

 overview of, 316

 pinholes and hole punching, 317

 STUN (Session Traversal Utilities for NAT), 319–326

 TURN (Traversal Using Relays around NAT), 326–332

 UNSAF (unilateral self-address fixing), 317–319

NAT with Port Translation (NAT-PT), 950

- NAT64, IPv6/IPv4 translation, 344, 950
- NAV (Network Allocation Vector), 121, 950
- NBMA (non-broadcast multiple access)
 - ICMP Redirect message used with, 375
 - ND and, 396
 - overview of, 167
- NCoA (New Care-of Address), 419, 950
- NCPs (Network Control Protocols), 131, 141–142, 950
- ND (Neighbor Discovery Protocol)
 - Advertisement Interval option, 412
 - Certificate option, 417
 - CGA options, 414–415
 - definition of, 951
 - DNS Search List option, 422–423
 - Experimental Values in, 423
 - Handover Key Request/Reply options, 422–423
 - Home Agent Information option, 412–413
 - IND (Inverse Neighbor Discovery), 401–402
 - interaction between IP fragmentation and ARP/ND, 496–497
 - IP Address/Prefix option, 417–418
 - Link-Layer Address (LLA) option, 418–419
 - MTU option in, 411–412
 - Nonce option, 416–417
 - NS (Neighbor Solicitation) message, 398–401
 - NUD (Neighbor Unreachability Detection), 402–403
 - options, 407–409
 - overview of, 395–396
 - Prefix Information option, 410–411
 - Recursive DNS Server option, 420
 - Redirected Header option, 411
 - Route Information option, 420
 - Router Advertisement Flags Extension option, 420–421
 - router solicitation and advertisement messages in ICMPv6, 396–398
 - RSA Signature option, 415–416
 - SEND (Secure Neighbor Discovery), 403–407
 - Source Link-Layer Address option, 409–410
 - Source/Target Address List options, 413–414
 - Teredo tunneling and, 486–487
 - Timestamp option, 416
 - Trust Anchor option, 417
- NDP. See ND (Neighbor Discovery Protocol)
- Negative ACKs (NACK), 133, 950
- Negative caching, DNS servers and, 517
- Neighbor Advertisement. See NA (Neighbor Advertisement)
- Neighbor Discovery Protocol. See ND (Neighbor Discovery Protocol)
- Neighbor Solicitation messages, ICMPv6, 277
- Neighbor Unreachability Detection (NUD)
 - definition of, 951
 - neighbor discovery in IPv6, 402–403
- NEMO (Network Mobility), 216, 951
- Net number, in IP addresses, 35
- NetBoot service, from Apple, 246
- netsh command, viewing in-use multicast groups in Windows OSs, 447–448
- netstat command
 - displaying IP group membership, 446–447
 - restricting local IP addresses, 634
 - viewing active multicast addresses, 66
 - viewing forwarding table with, 446–447
 - viewing IPv4 UDP servers, 500–501
- Network Access Control (NAC), 833–837, 950
- Network Address Port Translation (NAPT), 305–306, 950
- Network address translation. See NAT (network address translation)
- Network Allocation Vector (NAV), in Wi-Fi, 121, 950
- Network architecture
 - APIs, 22
 - ARM (ARPANET Reference Model), 1–2, 13–16
 - attacks related to Internet architecture, 25–26
 - client/server design pattern, 20–21
 - end-to-end argument, 6
 - error control and flow control, 7–8
 - fate sharing, 6–7
 - implementation and design, 8
 - implementations and distributions, 24–25
 - Internet, intranets, extranets, 19–20
 - layering, 8–10
 - multiplexing, demultiplexing, and encapsulation, 10–13, 16–17
 - names, addresses, and DNS, 19
 - packets, connections, and datagrams, 3–6
 - peer-to-peer design pattern, 21–22
 - port numbers, 17–19
 - principles of, 2–3
 - standardization of, 22–24
 - summary and references, 26–30
 - TCP/IP suite, 13
- Network byte order, TCP/IP headers, 183
- Network communication, threats to, 807–809
- Network Control Protocols (NCPs), 131, 141–142, 950
- Network File System (NFS), 478
- Network interface cards. See NICs (network interface cards)
- Network (internetwork) layer
 - address resolution and, 165–166
 - of OSI model, 9
 - passing datagram to transport layer, 17
 - TCP and UDP in, 585

- Network Mobility (NEMO), 216, 951
 - Network Time Protocol. *See* NTP (Network Time Protocol)
 - New Access Router (NAR), 417, 950
 - New Care-of Address (NCoA), 419, 950
 - NewReno algorithm, 739–740
 - Next Header field
 - ICMP Parameter Problem and, 379–380
 - IPv6, 194–195
 - Mobile IP, 216
 - UDP header, 476
 - UDP-Lite, 487
 - Next-hop, in IP forwarding, 209
 - Next Server IP Address (Siaddr) field, DHCP/BOOTP message format, 238, 246
 - NextSECure resource record. *See* NSEC (NextSECure) resource record
 - NFS (Network File System), 478
 - NICs (network interface cards)
 - definition of, 951
 - host address filtering, 449–451
 - overview of, 92
 - No Route to Destination, ICMPv6 messages, 365
 - Node-local scope
 - of IPv6 addresses, 43
 - multicast addresses, 53
 - Node Requirements RFCs, 23
 - Nominees, in ICE, 333
 - Non-broadcast multiple access. *See* NBMA (non-broadcast multiple access)
 - Non-HT mode, 802.11n operating modes, 128
 - Non-terminal NAPTR (NTN), 551, 951
 - Nonce (number used once)
 - cryptographic nonces and, 816
 - definition of, 951
 - IKE payloads, 848
 - in ND, 416–417, 486–487
 - Nonportable addresses, allocation of IP addresses and, 62
 - Nonrepudiation, basic principles of security, 806–807
 - NOP (No Operation), TCP header options, 605
 - Notification (N), IKE payloads, 849, 873
 - NPTv6, 310, 951
 - NRO (Number Resource Organization)
 - allocation of IP addresses and, 62–63
 - definition of, 951
 - NS (name server) records
 - definition of, 951
 - DNS resource record types, 529–530
 - NS (Neighbor Solicitation)
 - definition of, 951
 - IND (Inverse Neighbor Discovery), 401–402
 - main components of ND, 396
 - neighbor discovery in IPv6, 398–401
 - NSCD (Name Service Caching Daemon), 518, 951
 - NSEC (NextSECure) resource record
 - canonical ordering of, 902
 - definition of, 951
 - DNSSEC, 898–901
 - signed zones and zone cuts, 903
 - NSEC Parameters (NSEC3PARAM), in DNSSEC, 898, 951
 - NSEC3PARAM (NSEC Parameters), in DNSSEC, 900, 951
 - NTN (non-terminal NAPTR), 551, 951
 - NTP (Network Time Protocol)
 - definition of, 951
 - multicast group in, 54
 - variable-scope IPv6 multicast addresses, 58
 - NUD (Neighbor Unreachability Detection)
 - definition of, 951
 - neighbor discovery in IPv6, 402–403
 - Nuke class, bombs attacks on ICMP, 428
 - NULL encryption algorithm, 819
 - Number Resource Organization (NRO)
 - allocation of IP addresses and, 62–63
 - definition of, 951
 - Number used once. *See* Nonce (number used once)
- O**
- Object Identifier (OID), 951
 - OCSP (Online Certification Status Protocol)
 - definition of, 951
 - validating and revoking certificates, 829–831
 - OFDM (Orthogonal frequency division multiplexing)
 - definition of, 951
 - MIMO and, 127
 - OID (Object Identifier), 951
 - OLSR (Optimized Link State Routing)
 - definition of, 951
 - Wi-Fi mesh and, 130
 - Online Certification Status Protocol (OCSP)
 - definition of, 951
 - validating and revoking certificates, 829–831
 - OOB (Out of Band), 719, 952
 - Op field, DHCP/BOOTP message format, 236
 - Open DNS servers, 567
 - Open Systems Interconnection (OSI)
 - definition of, 952
 - layering and, 8
 - Open systems, TCP/IP suite as, 2
 - OpenPGP, 822, 883
 - OPT (option) pseudo records, DNS resource record types, 547–548

- Optimal window size, 730
- Optimistic ACKing, TCP congestion control attacks, 785–786
- Optimistic DAD, 253
- Optimistic state, IPv6 addresses, 253
- Optimized Link State Routing (OLSR)
 - definition of, 951
 - Wi-Fi mesh and, 130
- Option overloading, DHCP/BOOTP, 239
- Option Request Option (ORO), in DHCP, 952
- Orderly release, aborting TCP connections, 627
- Organizationally Unique Identifier. *See* OUI (Organizationally Unique Identifier)
- ORO (Option Request Option), in DHCP, 952
- Orthogonal frequency division multiplexing (OFDM)
 - definition of, 951
 - MIMO and, 127
- OSI (Open Systems Interconnection)
 - definition of, 952
 - layering and, 8
- OUI (Organizationally Unique Identifier)
 - definition of, 952
 - formats of IPv6 addresses, 44–45
 - IP multicasting and, 442–444
- Out of Band (OOB), 719, 952
- Overlay networks
 - p2p and, 22
 - tunneling allowing formation of, 149
- P**
- P2P (Peer-to-Peer)
 - definition of, 952
 - design pattern, 21–22
 - discovery problem in, 22
- PA (provider-aggregatable) addresses
 - allocation of IP addresses and, 62
 - definition of, 952
- PAA (PANA Authentication Agent), 839, 952
- PaC (PANA client), 839, 952
- Packet duplication, TCP timeout/retransmission, 684–685
- Packet-filtering firewalls
 - overview of, 300–301
 - rules, 335–336
- Packet-filtering, NAT functions, 305
- Packet reordering, TCP timeout/retransmission, 682–684
- Packet size threshold, in Wi-Fi control frames, 115
- Packet sniffing, 26, 156
- Packet-switching, 4
- Packet-switching networks, 1
- Packet Too Big. *See* PTB (Packet Too Big)
- Packetization, TCP reliability and, 586
- Packets
 - AQM (active queue management), 782–783
 - conservation of, 731
 - IEEE, 84
 - in protocol architecture, 3–6
 - retransmission and, 580–581
 - sliding window protocol, 582
 - in TCP/IP suite, 14
 - windows of packets, 581–582
- PACP (port access control protocol), 835
- PAD (peer authorization database)
 - definition of, 952
 - in IPSec, 841
- PAD (PPPoE Activity Discovery) messages, 288
- Padded payload, in Ethernet frame format, 86
- Padding
 - block size in PPP, 136–137
 - IPv6 options for, 197–198
- Padding oracle, security protocol-related attacks and, 919
- PANA Authentication Agent (PAA), 839, 952
- PANA client (PaC), 839, 952
- PANA (Protocol for Carrying Authentication for Network Access), 839–840, 952
- PAP (Password Authentication Protocol)
 - definition of, 952
 - for PPP authentication, 140
- Parameter Problem messages
 - ICMP, 379–380
 - translating ICMPv4 to ICMPv6, 426
 - translating ICMPv6 to ICMPv4, 426–427
- Partial ACKs, in TCP, 739
- Passive attacks, types of threats to network communication, 807–809
- Passive closer, FIN segments and, 597
- Passive open, in TCP connections, 597
- Passive opener (server)
 - RST segments, 631
 - in TCP connections, 599
- Password Authentication Protocol (PAP)
 - definition of, 952
 - for PPP authentication, 140
- Path MTU, 148
- Path MTU Discovery. *See* PMTUD (Path MTU Discovery)
- Path MTU (PMTU), 953
- Pattern-match-criteria, in ACL rules, 335
- PAWS (Protection against Wrapped Sequence Numbers)
 - definition of, 952
 - TCP header, 610–611

Payloads

- of frames, 16, 86
- IKE protocol, 847–849
- IP header, 184–185, 203–206
- jumbogram options, 198
- UDP header, 475, 482

PCF (point coordination function), in Wi-Fi

- definition of, 952
- options for controlling sharing of wireless medium, 120

PCO (phased coexistence operation)

- definition of, 952
- higher throughput (802.11n) support and, 128

PCoA (Previous Care-of Address), 419, 952**PCP (Port Control Protocol)**

- definition of, 952
- direct interaction with NAT and firewalls, 339

PD (prefix delegation), 266–267**PDU (protocol data units)**

- definition of, 952
- encapsulation and, 10
- link layer. *See* Frames, Ethernet

PEAP (Protected EAP), 838, 952**Peer Address field, in DHCPv6 message format, 254****Peer authorization database (PAD)**

- definition of, 952
- in IPsec, 841

Peer-to-Peer (P2P)

- definition of, 952
- design pattern, 21–22
- discovery problem in, 22

PEN (Private Enterprise Number), 257, 952**Per-association (per-connection) basis, for NAT connections, 303–304****Per-flow state, LCI and LCN and, 5****Per-hop behavior (PHB)**

- definition of, 953
- forwarding and, 189

Perfect Forward Secrecy (PFS), 815, 953**PFC (Protocol Field Compression)**

- definition of, 953
- PPP and, 133

PFS (Perfect Forward Secrecy), 815, 953**PGP (Pretty Good Privacy), 822****Phased coexistence operation (PCO)**

- definition of, 952
- higher throughput (802.11n) support and, 128

PHB (per-hop behavior)

- definition of, 953
- forwarding and, 189

Phishing attacks, 806**PHY (physical) layer, 953****Physical addresses, translating logical addresses to, 167****Physical carrier sense, 121****Physical layer**

- 802.11 standard describing, 123–124
- channels and frequencies, 124–126
- of OSI model, 9

PI (provider-independent) addresses

- allocation of IP addresses and, 63
- definition of, 953

Piggybacking, Delayed ACKs and, 692, 695**PIM (Protocol Independent Multicast), 953****ping. *See* Echo Request/Reply (ping) messages****Ping of death attacks**

- ICMP attacks, 428
- UDP attacks, 506–507

Pinholes, in NAT traversal, 317**PIO (Prefix Information option)**

- definition of, 953
- neighbor discovery options in IPv6, 410–411

PKCs (public key certificates). *See also* Certificates, 831, 953**PKCS (Public Key Cryptography Standards), 953****PKI (Public Key Infrastructure)**

- CGAs not requiring, 404
- definition of, 953
- overview of, 822

Plain old telephone service (POTS)

- definition of, 953
- DSL and, 287

Plaintext

- encrypting cleartext message, 810
- TLS, 878

PLCP (Physical Layer Convergence Procedure), 113, 953**PLPMTUD (Packetization Layer Path MTU Discovery), 612–613****Plug and Play, 337–339****PMTU (Path MTU), 148, 953****PMTUD (Path MTU Discovery)**

- definition of, 953
- example of use with TCP, 613–616
- example using UDP, 493–496
- link layer and, 148
- PTB messages in ICMPv6, 370–371
- TCP connections and, 612–613
- TCP-related attacks, 641–642
- using UDP for, 493

PNAC (Port-Based Network Access Control), 833, 953**PoE (power-over-Ethernet)**

- definition of, 953
- higher throughput (802.11n) support and, 128

- Point coordination function (PCF), in Wi-Fi
 - definition of, 952
 - options for controlling sharing of wireless medium, 120
- Point-to-Point Protocol. *See* PPP (Point-to-Point Protocol)
- Point-to-Point Tunneling Protocol. *See* PPTP (Point-to-Point Tunneling Protocol)
- Pointer (PTR) records, DNS resource record types, 536–541
- POP3, SRV record providing POP3 service, 549
- Port access control protocol (PACP), 835
- Port-Based Network Access Control (PNAC), 833, 953
- Port Control Protocol (PCP)
 - definition of, 952
 - direct interaction with NAT and firewalls, 339
- Port forwarding, in NAT, 314
- Port mapping, in NAT, 314
- Port numbers
 - binding options available to TCP server, 635
 - as mailboxes, 474
 - TCP/IP suite and, 17–19
 - TCP servers, 632–634
 - in UDP server design, 499–500
- Port overloading, in NAT, 313
- Port parity, in NAT, 313
- Port preservation, NAT and TCP and, 307
- Port-Preserving Symmetric NAT (PP), 486
- Port states, STP, 104–105
- Port translation behavior, in NAT, 311–313
- Port unreachable message, ICMP, 365–370
- Ports
 - multiple UDP servers per port, 503–504
 - requesting connection to nonexistent TCP port, 626
 - in TCP header, 588
- POTS (plain old telephone service)
 - definition of, 953
 - DSL and, 287
- Pouzin, Louis, 1
- Power-over-Ethernet (PoE)
 - definition of, 953
 - higher throughput (802.11n) support and, 128
- Power save mode (PSM), 119–120, 954
- Power Save Multi-Poll (PSMP), 120, 954
- Power saving, Ethernet (802.3) and, 96–97
- PP (Port-Preserving Symmetric NAT), 486
- PPP over Ethernet. *See* PPPoE (PPP over Ethernet)
- PPP (Point-to-Point Protocol)
 - authentication, 140–141
 - CCP (Compression Control Protocol) and, 139–140
 - definition of, 953
 - example of, 143–145
 - header compression, 142–143
 - MP (Multilink PPP), 137–139
 - NCPs (Network Control Protocols), 141–142
 - overview of, 130–131
- PPPMux, 137
- PPPMuxCP (PPP Mux Control Protocol), 137
- PPPoE Activity Discovery (PAD) messages, 288
- PPPoE (PPP over Ethernet)
 - definition of, 953
 - Discovery and PPP Session phases, 288
 - DSL and, 287
 - example of use of, 289–291
 - message format, 288–289
 - MTU (maximum transmission unit) and, 614
 - overview of, 286
- PPTP (Point-to-Point Tunneling Protocol)
 - definition of, 953
 - establishing sessions, 151–153
 - establishing tunneling with, 149–153
 - NAT editors and, 315
- Preamble, of Ethernet frame, 84
- Preferred lifetime
 - IA (Identity Association) and, 255
 - IPv6 addresses, 252
- Prefix delegation (PD), 266–267
- Prefixes
 - in IPv4 and IPv6 address management, 47–48
 - route aggregation and, 50
 - subnet mask formats, 39
- Presentation layer, of OSI model, 10
- Preshared keys. *See* PSKs (preshared keys)
- Pretty Good Privacy (PGP), 822
- Previous Care-of Address (PCoA), 419, 952
- PRFs (pseudorandom functions), 816, 954
- PRGs (pseudorandom generators), 815–816, 954
- Primary DNS servers, 517
- Priority fields, QoS 802.1p, 90
- Private Enterprise Number (PEN), 257, 952
- PRNGs (pseudorandom numbers)
 - definition of, 954
 - overview of, 815–816
- Probing, NAT session, 308
- Promiscuous ARP, 175
- Promiscuous mode, Ethernet interfaces, 155
- Protected EAP (PEAP), 838, 952
- Protection against Wrapped Sequence Numbers (PAWS)
 - definition of, 952
 - TCP header, 610–611
- Protocol data units. *See* PDUs (protocol data units)

- Protocol field
 - IP header, 185
 - in PPP frames, 132–133
 - UDP header, 476
 - Protocol Field Compression (PFC)
 - definition of, 953
 - PPP and, 133
 - Protocol for Carrying Authentication for Network Access (PANA), 839–840, 952
 - Protocol identifiers, in multiplexing, 10
 - Protocol Independent Multicast (PIM), 953
 - Protocol multiplexing, 10
 - Protocol suites
 - definition of, 1
 - end and intermediate systems, 12
 - Protocols, 1
 - Provider-aggregatable (PA) addresses
 - allocation of IP addresses and, 62
 - definition of, 952
 - Provider-independent (PI) addresses
 - allocation of IP addresses and, 63
 - definition of, 953
 - Proxy ARP, 174–175
 - Proxy firewalls
 - overview of, 301–303
 - types of firewalls, 300
 - Proxy Router Solicitation (RtSolPr), 388
 - Proxy routers, 388
 - PrRtAdv (Proxy Router Advertisement), 388
 - Pseudorandom functions (PRFs), 816, 954
 - Pseudorandom generators (PRGs), 815–816, 954
 - Pseudorandom numbers. *See* PRNGs (pseudorandom numbers)
 - PSH bit
 - sender pause and local congestion (event 1), 750
 - in TCP communication, 694–695
 - PSKs (preshared keys)
 - definition of, 954
 - encryption and, 129–130
 - vulnerability to dictionary attacks, 918
 - PSM (power save mode), 119–120, 954
 - PSMP (Power Save Multi-Poll), 120, 954
 - PTB (Packet Too Big)
 - definition of, 954
 - destination unreachable and, 612
 - ICMP attacks and, 429
 - ICMPv6 messages, 364, 370–371
 - TCP-related attacks, 641
 - PTR (pointer) records, DNS resource record types, 536–541
 - Public key certificates. *See also* Certificates
 - DNSKEY resource record and, 896
 - overview of, 822
 - Public key cryptography
 - asymmetric (public) key ciphers, 809–812
 - Handover Key Request/Reply options, 422–423
 - RSA (Rivest, Shamir, and Adleman), 812–813
 - Public Key Infrastructure. *See* PKI (Public Key Infrastructure)
 - Pure ACK, TCP header, 590
- ## Q
- QAM (quadrature amplitude modulation), 128, 954
 - QAPs (QoS access points), 122
 - QBSS (QoS BSS), 122, 954
 - QI (Query Interval), in IGMP and MLD, 468, 954
 - QoS access points (QAPs), 122
 - QoS BSS (QBSS), 122, 954
 - QoS (quality of service)
 - 802.1p standard, 90
 - APSD and, 120
 - definition of, 954
 - HCF (hybrid coordination function) and, 122–123
 - tagging, 89–92
 - in VLANs, 86
 - QoS stations (QSTAs), 122–123, 954
 - QPSK (quadrature phase shift keying), 128, 954
 - QQI (Querier's Query Interval), in IGMP/MLD, 954
 - QQIC (Querier's Query Interval Code)
 - definition of, 954
 - in MLD, 390
 - MLD and, 459
 - QRI (Query Response Interval), in IGMP/MLD, 468, 954
 - QRV (querier robustness variable), in IGMP/MLD, 466–467, 954
 - QS (Quick-Start)
 - definition of, 954
 - IPv6 options for, 199
 - QSTAs (QoS stations), 122–123, 954
 - Quadrature amplitude modulation (QAM), 128, 954
 - Quadrature phase shift keying (QPSK), 128, 954
 - Qualifiers, DNS TXT records, 546
 - Quality of service. *See* QoS (quality of service)
 - Querier election, multiple multicast routers and, 466
 - Querier robustness variable (QRV), 466–467, 954
 - Querier Robustness Variable (QVR), 390
 - Querier's Query Interval Code. *See* QQIC (Querier's Query Interval Code)
 - Querier's Query Interval (QQI), in IGMP/MLD, 954
 - Queries, DNS, 526
 - Queries/informational messages, ICMP
 - Echo Request/Reply messages, 380–383
 - Home Agent Discovery Request message, 386
 - MIPv6 fast handover messages, 388
 - MLD extension messages, 390–394

- MLD query/report/done messages, 388–390
 - Mobile Prefix Solicitation message, 387–388
 - MRD (Multicast Router Discovery), 394–395
 - overview of, 380
 - router solicitation and advertisement messages
 - in ICMPv4, 383–385
 - Query Interval (QI), in IGMP and MLD, 468, 954
 - Query message
 - ICMP, 388–390
 - IGMP, 457–459
 - Query/response, in DNS protocol, 518
 - Query Response Interval (QRI), in IGMP/MLD, 468, 954
 - Query types, resource record categories, 528
 - Question (query) and zone section format, 526
 - Queueing theory, in congestion control, 583
 - Queues
 - packets stored in, 4
 - TCP server incoming connection queue, 636–640
 - Quick acknowledgments, Linux, 733
 - Quick-Start (QS)
 - definition of, 954
 - IPv6 options for, 199
 - Quiet time concept, TCP state transitions, 624
 - QVR (Querier Robustness Variable), 390
- R**
- RA (Router Advertisement)
 - Advertisement Interval option in ND, 412
 - definition of, 954
 - DHCPv6, 260–263
 - Home Agent Information option in ND, 412–413
 - ICMP attacks and, 429
 - ICMP messages, 383–385
 - ICMPv6 messages, 280–281
 - link with NA (Neighbor Advertisement), 396
 - MTU option in ND, 411–412
 - neighbor discovery in IPv6, 396–398
 - Prefix Information option, 410–411
 - Route Information option in ND, 420
 - Router Advertisement Flags Extension option, 420–421
 - Trust Anchor option, 417
 - RADIUS (Remote Authentication Dial-In User Service)
 - for access control, 141
 - definition of, 955
 - RAIO (Relay Agent Information Option)
 - definition of, 955
 - DHCP, 268
 - Random Early Detection (RED) gateways
 - AQM (active queue management) and, 783–785
 - definition of, 955
 - Random numbers, in ND, 416–417
 - Rapid Commit option, DHCP/BOOTP message
 - format, 273–274
 - Rapid Spanning Tree Protocol (RSTP), 103, 110–111, 956
 - RARP (reverse ARP), 166, 955
 - RASs (remote access servers)
 - control decisions by, 141
 - definition of, 955
 - Rate-based flow control, 583
 - Rate halving, for TCP congestion control, 741–742
 - Rate-Halving with Bounding Parameters (RHBP)
 - definition of, 955
 - for TCP congestion control, 741–742
 - Rate limiting, of ICMP messages in Linux server, 369–370
 - RC4 algorithm
 - definition of, 955
 - in Wi-Fi security, 129–130
 - RD (Router Discovery)
 - definition of, 955
 - overview of, 383–385
 - RDATA, in DNS resource record, 527, 955
 - RDNSS (Recursive DNS Server)
 - definition of, 955
 - neighbor discovery in IPv6, 420
 - Real-Time Protocol (RTP), 313
 - Reassembly
 - fragmentation and, 488
 - of fragmented datagrams, 14, 205
 - timeout, 492
 - Rebinding time (T2), for DHCP messages, 240
 - Receive window structure, sliding window protocol, 701
 - Reconfigure extension, DHCP, 273
 - Record layer, in TLS
 - DTLS (Datagram TLS), 891–892
 - TLS (Transport Layer Security), 877
 - Record markers, datagrams and, 5–6
 - Record protocol, TLS, 878–880
 - Recovery point, in TCP retransmission, 671
 - Recur field, GRE tunnels and, 150
 - Recursive DNS Server (RDNSS)
 - definition of, 955
 - neighbor discovery in IPv6, 420
 - RED (Random Early Detection) gateways
 - AQM (active queue management) and, 783–785
 - definition of, 955
 - Redirect messages, ICMP
 - ICMP attacks and, 428
 - overview of, 372–375
 - Redirected Header option in ND, 411
 - Reference model, of protocol suite, 1

- Referenced connections, TCP reset segments and, 625
- regedit program in Windows, setting keepalive time with, 797
- Regional Internet registries (RIRs)
 - allocation of IP addresses and, 62–63
 - definition of, 955
- Registered ports, 18
- Reject route message, ICMPv6, 372
- Relative fairness, congestion control schemes and, 769
- Relay Agent Information Option (RAIO)
 - definition of, 955
 - DHCP, 268
- Relay agents, DHCP
 - layer 2 devices, 270–271
 - leasequery and bulk leasequery, 269–270
 - overview of, 267
 - RAIO (Relay Agent Information Option), 268
 - Remote-ID and IPv6 Remote-ID, 268
 - Server Identifier Override, 268–269
- Relayed transport address, TURN, 326
- Reliability, TCP, 586–587
- Remote access servers (RASs)
 - control decisions by, 141
 - definition of, 955
- Remote Authentication Dial-In User Service (RADIUS)
 - for access control, 141
 - definition of, 955
- Remote-ID, DHCP relay agents, 268
- Remote procedure call (RPC)
 - definition of, 955
 - SOAP and, 338
- Rendezvous point (RP)
 - definition of, 955
 - for multicast groups, 60
- Renegotiation, of cryptographic connection parameters in TLS, 884
- Renewal time (T1), for DHCP messages, 240
- Reno algorithm, 737–738
- Renumbering, allocation of IP addresses and, 63
- Repacketization, in TCP
 - overview of, 586
 - TCP timeout/retransmission, 686–687
- Replay attacks
 - Nonce option in ND countering, 416–417
 - types of threats to network communication, 808
- Report message, ICMP, 388–390
- REQUEST message, DHCPv6, 264–265, 269
- Request/response transactions, in STUN, 320
- Request to send. *See* RTS (request to send)
- Reserved addresses, IPv6 multicast, 61
- Reserved field, in IPv6 Fragment header, 203–204
- Reset (RST) segments, TCP
 - aborting connections, 627–628
 - definition of, 956
 - half-open connections, 628–630
 - incoming connection queue and, 637
 - overview of, 625–626
 - requesting connection to nonexistent port, 626
 - TWA (TIME-WAIT Assassination), 630–631
- Resolver
 - accessing DNS with, 511
 - DNSSEC example of operation, 903–911
 - UDP and, 525–526
 - validating security aware resolver, 895
- Resource Record Set. *See* RRSet (Resource Record Set)
- Resource Record Signature resource record. *See* RRSIG (Resource Record Signature) resource record
- Resource records. *See* RRs (resource records)
- Resource utilization attacks
 - attacks related to IGMP or MLD, 469–470
 - UDP-related attacks, 506
- Response algorithm, for spurious timeouts and retransmissions, 677
- Restart Window (RW), 739, 956
- Retransmission
 - ARQ and, 580
 - of packets, 7
 - in TCP. *See* TCP timeout/retransmission
 - timeout settings in communication protocols, 584
- Retransmission ambiguity problem, 655, 679
- Retransmission (RTX), 894, 956
- Retransmission timeout. *See* RTO (retransmission timeout)
- Retry bit, Frame Control Word, 116
- Return Routability Procedure (RRP), in MIP
 - definition of, 956
 - in RO, 218–219
- Reverse ARP (RARP), 166, 955
- Reverse DNS queries, 536
- Reverse lookup, host names from IP addresses, 19
- Reverse Path Forwarding (RPF), 956
- RFC (Request for Comments), 23–24, 955
- RGMP (Router-port Group Management Protocol), 469, 955
- RH (Routing Header)
 - definition of, 955
 - in IPv6, 200–203
- RHBP (Rate-Halving with Bounding Parameters)
 - definition of, 955
 - for TCP congestion control, 741–742
- RIID field, IPv6 multicast addresses, 60–61
- Rijmen, Vincent, 811

- Rijndael algorithm. See AES (Advanced Encryption Standard)
- RIP (Router Information Protocol), 955
- RIRs (regional Internet registries)
allocation of IP addresses and, 62–63
definition of, 955
- Rivest, Shamir, and Adleman. See RSA (Rivest, Shamir, and Adleman)
- rlogin (UNIX), precursor to SSH, 692
- RO (route optimization), in MIP
definition of, 955
in mobile IP, 217–219
- ROAD (Running Out of Address Space), 955
- Robust Header Compression (ROHC), 143, 955
- Robust Security Network access (RSNA)
definition of, 956
in Wi-Fi security, 129
- Robust Security Network (RSN)
definition of, 956
in Wi-Fi security, 129
- Robustness/reliability, of IGMP and MLD, 465–467
- ROHC (Robust Header Compression), 143, 955
- Roles, STP, 104–105
- Root bridge, building the spanning tree in STP, 107
- Root certificates, 822
- Root ports, STP, 104–105
- Rouge RAs, ICMP attacks and, 429
- Round-robin, DNS, 565–567
- Round-trip time. See RTT (round-trip time)
- Round-trip-time
estimation, 584
traceroute measuring, 377
- Route aggregation, 48–50
- Route Information option, in ND, 420
- Route optimization (RO), in MIP
definition of, 955
in mobile IP, 217–219
- Route Type identifiers, 201
- Router Advertisement. See RA (Router Advertisement)
- Router Alert, IPv6 options for, 198
- Router Discovery (RD)
definition of, 955
overview of, 383–385
- Routing Header (RH)
definition of, 955
IPv6, 200–203
- Router Information Protocol (RIP), 955
- Router Requirements RFC, 23
- Router Solicitation. See RS (Router Solicitation)
- Router solicitation and advertisement messages in ICMPv4, ICMP, 383–385
- Routers
congestion of, 727–728
crashes, 226
default router, 208
IGMP/MLD processing by multicast routers, 457–459
IP routers, 20
multicast routing, 452–454
between packet-switching networks, 1
in small networks, 11–13
- Routing Policy Specification Language (RPSL), 65, 956
- Routing protocols, 209
- Routing tables, 208, 439–441
- RP (rendezvous point), in IP Multicast
definition of, 955
for multicast groups, 60
- RPC (remote procedure call)
definition of, 955
SOAP and, 338
Reverse Path Forwarding (RPF), 956
- RPSL (Routing Policy Specification Language), 65, 956
- RRP (Return Routability Procedure), in MIP
definition of, 956
in RO, 218–219
- RRs (resource records), in DNS
address and name server records, 529–530
CNAME (canonical name) records, 534–536
definition of, 956
in DNS message format, 520–521
dynamic DNS updates and, 555–557
ENUM records, 551–552
example using resource record types, 530–534
MX (mail exchanger) records, 544–545
NAPTR (name authority pointer) records, 549–551
OPT (option) pseudo records, 547–548
overview of, 527–529
PTR (pointer) records, 536–541
S-NAPTR and U-NAPTR, 554–555
SIP records, 552
SOA (start of authority) records, 541–544
SPF (sender policy framework) and TXT records, 545–547
SRV (service) records, 548–549
translating DNS from IPv4 to IPv6, 569
transparency and, 568
URI/URN resolution, 553–554
- RRs (resource records), DNSSEC
DNSKEY resource record, 896–897
DS (delegation signer) resource record, 897–898
NSEC (NextSECure) resource record, 898–901

- RRs (resource records), DNSSEC
 - overview of, 896
 - RRSIG (Resource Record Signature) resource record, 901–902
 - RRset (Resource Record Set)
 - canonical ordering of, 902–903
 - definition of, 956
 - dynamic DNS updates and, 555–557
 - overview of, 527
 - RRSIG (Resource Record Signature) resource record
 - definition of, 956
 - DNSSEC, 901–902
 - signed zones and zone cuts, 903
 - RS (Router Solicitation)
 - definition of, 956
 - DHCPv6, 260–263
 - ICMP attacks and, 429
 - ICMP messages, 383–385
 - ICMPv6 messages, 280
 - link with NS (Neighbor Solicitation), 396
 - neighbor discovery in IPv6, 396–398
 - RSA (Rivest, Shamir, and Adleman)
 - in cipher suites, 821
 - definition of, 956
 - ECC as alternative to, 815
 - overview of, 812–813
 - TLS extensions, 883
 - RSA Signature option, in ND, 415–416
 - RSN (Robust Security Network)
 - definition of, 956
 - in Wi-Fi security, 129
 - RSNA (Robust Security Network access)
 - definition of, 956
 - in Wi-Fi security, 129
 - RST. See Reset (RST) segments, TCP
 - RSTP (Rapid Spanning Tree Protocol), 103, 110–111, 956
 - RTO (retransmission timeout), in TCP
 - classic method, 651–652
 - clock granularity and RTO bounds, 654
 - definition of, 956
 - initial values, 654
 - introduction to, 647
 - Linux RTT estimation, 657–661
 - retransmission ambiguity and Karn's algorithm, 655
 - robustness of RTTM to loss and reordering, 662–664
 - RTT estimation behaviors, 661–662
 - RTTM (RTT Measurement) with Timestamps option, 656–657
 - setting, 651
 - slow start algorithm and, 732
 - spurious. See Spurious timeouts and retransmissions, in TCP
 - standard method, 652–654
 - TCP connections and, 611
 - RTP (Real-Time Protocol), 313
 - RTS (request to send)
 - carrier sense and, 121
 - definition of, 956
 - Wi-Fi control frames, 115
 - RtSolPr (Proxy Router Solicitation), 388
 - RTT (round-trip time)
 - classic method of RTT estimation, 651–652
 - clock granularity and RTO bounds, 654
 - definition of, 956
 - estimation behaviors, 661–662
 - HSTCP (HighSpeed TCP) and, 773
 - initial values in RTO, 654
 - Linux estimation of, 657–661
 - Nagle algorithm and, 696–697
 - retransmission timeout settings and, 584–585
 - RTO based on, 648, 651
 - standard method of estimating, 652–654
 - "stop and wait" protocol and, 581
 - STUN messages, 320
 - TCP Timestamp option and, 610
 - RTTM (RTT Measurement)
 - robustness to loss and reordering, 662–664
 - with Timestamps option, 656–657, 956
 - RTTVAR (RTT Variance), in TCP, 685–686, 956
 - RTX (Retransmission), 894, 956
 - RW (Restart Window), in TCP, 739, 956
- ## S
- S-NAPTR (straightforward NAPTR)
 - definition of, 958
 - DNS resource record types, 554
 - SACK (selective acknowledgement)
 - definition of, 957
 - DSACK (duplicate SACK) extension, 677–679
 - example of retransmission with, 673–676
 - fast retransmit and SACK recovery, 757–759
 - receiver behavior, 672
 - retransmission with, 647, 671–672
 - sender behavior, 673
 - for TCP congestion control, 740–741
 - in TCP header, 589, 607
 - SAD (security association database)
 - definition of, 957
 - in IPSec, 841–842
 - SAE (Simultaneous Authentication of Equals)
 - definition of, 957
 - Wi-Fi mesh and, 130
 - Salt, in cryptography, 816

- SAP (Session Announcement Protocol)
 - definition of, 957
 - for multicast sessions, 55
- SAs (Security Associations), in IPsec
 - CREATE_CHILD_SA exchange, 852–853
 - definition of, 956
 - GSAs (group security associations), 864
 - in IPsec, 841
 - payloads and proposals, 847–848
 - proposed algorithms, 867–869
- Scalability, of DNS, 516
- Scope
 - ICMPv6 error (Beyond Scope of Source Address), 371
 - of IPv6 addresses, 43
 - IPv6 multicast addresses, 57–58
 - of multicast addresses, 53
- SCSV (Signaling Cipher Suite Value), 884, 957
- SCTP (Stream Control Transmission Protocol)
 - definition of, 957
 - NAT and, 309
 - transport protocols in TCP/IP suite, 16
- SCVP (Server-Based Certificate Validation Protocol)
 - certificate validation and, 831
 - definition of, 957
- SDID (Signing Domain Identifier), 916, 957
- SDLC (Synchronous Data Link Control)
 - based on HDLC, 131
 - definition of, 957
- SDOs (standards-defining organizations), 23, 957
- SDP (Session Description Protocol)
 - definition of, 957
 - ICE and, 332–333
 - IP multicast and, 55
- Secondary DNS servers, 517
- Secret Key Transaction Authentication for DNS (TSIG), 911–914
- Secs field, DHCP/BOOTP message format, 236
- Secure Entry Point (SEP) bit, DNSSEC, 896, 905, 957
- Secure Hash Algorithm 1. See SHA 1 (Secure Hash Algorithm 1)
- Secure hash function, 404
- Secure Neighbor Discovery. See SEND (Secure Neighbor Discovery)
- Secure Real-Time Protocol (SRTP), 883, 958
- Secure Remote Password (SRP), 883, 958
- Secure Shell. See SSH (Secure Shell)
- Secure Sockets Layer (SSL). See also TLS (Transport Layer Security), 876–877, 958
- Security
 - ACs (attribute certificates), 831
 - basic principles, 806–807
 - certificates, CAs, and PKIs, 821–822
 - cryptographic and cipher suites, 819–821
 - cryptographic nonces and salt, 816
 - cryptosystems, 809–812
 - DH (Diffie-Hellman-Merkle Key Agreement), 813–814
 - ECC (Elliptic Curve Cryptography), 815
 - hash functions and message digests, 817–818
 - introduction to, 805–806
 - message authentication codes, 818–819
 - PFS (Perfect Forward Secrecy), 815
 - protocols. See Security protocols
 - pseudorandom numbers, generators, and function families, 815–816
 - public key certificates, CAs, and X.509, 822–828
 - RSA (Rivest, Shamir, and Adleman) public key cryptography, 812–813
 - signcryption, 814–815
 - summary and references, 919–932
 - threats to network communication, 807–809
 - validating and revoking certificates, 828–831
 - Wi-Fi, 129–130
- Security association database (SAD)
 - definition of, 957
 - in IPsec, 841–842
- Security Associations. See SAs (Security Associations)
- Security Gateway (SG), in IPsec, 840, 957
- Security Parameter Index (SPI), in IPsec
 - definition of, 958
 - IKE protocol, 844
- Security policy database (SPD), in IPsec
 - definition of, 958
 - in IPsec, 841–842
- Security protocols
 - attacks on, 918–919
 - DKIM (Domain Keys Identified Mail), 915–918
 - DNS. See DNSSEC (DNS Security)
 - EAP methods, 837–838
 - ERP (EAP Re-authentication Protocol), 839
 - Internet Key Exchange. See IKE (Internet Key Exchange)
 - IPsec (IP Security), 840–842
 - IPsec NAT traversal, 865–867
 - L2TP/IPsec, 865
 - layering and, 832–833
 - NAC (Network Access Control), 833–837
 - PANA (Protocol for Carrying Authentication for Network Access), 839–840
 - transport layer. See TLS (Transport Layer Security)
- Segments Left field, in Routing header, 201–202

- Segments, TCP, 586
- Selective acknowledgement. *See* SACK (selective acknowledgement)
- Selective retransmission, 673
- Self-clocking
 - ACKs and, 731
 - Nagle algorithm and, 696
- Self-describing padding, 137
- Send maximum segment size. *See* MSS (send maximum segment size)
- SEND (Secure Neighbor Discovery)
 - Certificate option, 417
 - certification path solicitation/advertisement, 406–407
 - CGAs (cryptographically generated addresses), 403–406
 - definition of, 957
 - Handover Key Request/Reply options, 422–423
 - ICMP attacks and, 429
 - neighbor discovery options in IPv6, 414–415
 - Nonce option in ND, 416–417
 - overview of, 403
 - RSA Signature option, 415–416
 - securing IPv6 Neighbor Discovery, 292
 - Timestamp option, 416
 - Trust Anchor option, 417
 - as variant on ND, 396
- Send window structure, sliding window protocol, 701
- Sender pause and local congestion (event 1), TCP congestion control, 750–754
- Sender policy framework (SPF) records, DNS resource record types, 545–547
- SEP (Secure Entry Point) bit, DNSSEC, 896, 905, 957
- Sequence Control field, data frame fragmentation, 117
- Sequence numbers
 - for avoiding duplicate packets, 580
 - data frame fragmentation and, 117
 - GRE, 150
 - PPP, 138
 - TCP, 587–588
 - TCP-related attacks, 641
 - TCP segments, 701
 - URG, 590
- Sequencing header, in MP, 138
- Sequential Port-Symmetric NAT (SP), 486
- Server alive messages, TCP keepalive attacks, 802
- Server-Based Certificate Validation Protocol (SCVP), 831
- Server Identifier Override, DHCP relay agents, 268–269
- Server Load Reduction (SLR), 486
- ServerHello message, in TLS, 887–889
- Servers
 - accessing servers behind NAT, 314
 - iterative and concurrent, 21
 - server host crashes and does not reboot (keepalive scenarios), 796
 - server host crashes and reboots (keepalive scenarios), 797–799
 - server host unreachable (keepalive scenarios), 799–800
- Service model, TCP, 585–586
- Service provider NAT. *See* SPNAT (service provider NAT)
- Service set identifiers (SSID)
 - definition of, 958
 - Wi-Fi, 112
- Service sets, Wi-Fi, 112
- Service (SRV) records, DNS resource record types, 548–549
- Session Announcement Protocol (SAP)
 - definition of, 957
 - for multicast sessions, 55
- Session Description Protocol. *See* SDP (Session Description Protocol)
- Session Initiation Protocol (SIP)
 - definition of, 957
 - ENUM records and, 551–552
- Session keys, in public key cryptography, 812
- Session layer, of OSI model, 10
- Session timers, NAT, 307–308
- Session Traversal Utilities for NAT. *See* STUN (Session Traversal Utilities for NAT)
- SFD (start frame delimiter), in link layer protocols
 - clock recovery in Ethernet frames, 84
 - definition of, 957
- SG (Security Gateway), in IPsec, 840, 957
- SHA 1 (Secure Hash Algorithm 1)
 - for authentication in DHCP, 268
 - definition of, 957
 - overview of, 817–818
 - TLS extensions, 883
- Shannon, Claude, 579
- Sharing connection state, 767–768
- Shim6 protocol, 70
- Short Interframe Space (SIFS), in Wi-Fi
 - definition of, 957
 - in MAC, 122
- Short sequence number, LCP options, 138
- Short-term credential mechanism, STUN, 325
- Siaddr (Next Server IP Address) field, DHCP/BOOTP message format, 238, 246

- SIFS (Short Interframe Space), in Wi-Fi
 - definition of, 957
 - in MAC, 122
- Signaling Cipher Suite Value (SCSV), 884, 957
- Signature verification, CGAs for, 405–406
- Signed zones, DNSSEC, 903
- Signing Domain Identifier (SDID), 916, 957
- SIIT (Stateless IP/ICMP Translation)
 - definition of, 957
 - IPv4/IPv6 translation, 342–344
- Silly windows syndrome. *See* SWS (silly windows syndrome)
- Simple Mail Transfer Protocol. *See* SMTP (Simple Mail Transfer Protocol)
- Simple Network Management Protocol (SNMP)
 - definition of, 958
 - well-known port for, 18
- Simple Object Access Protocol (SOAP)
 - definition of, 958
 - GENA using, 338
- Simple Service Discovery Protocol. *See* SSDP (Simple Service Discovery Protocol)
- Simple Tunneling of UDP through NATs, 319
- Simultaneous Authentication of Equals (SAE)
 - definition of, 957
 - Wi-Fi mesh and, 130
- Simultaneous close, in TCP connections
 - overview of, 600–601
 - state transition, 625
- Simultaneous open, in TCP connections
 - defined, 597
 - overview of, 599–600
 - state transition, 625
- SIP Outbound mechanism, in ICE, 333
- SIP records, DNS resource record types, 552
- SIP (Session Initiation Protocol)
 - definition of, 957
 - ENUM records and, 551–552
- SLAAC (stateless address autoconfiguration)
 - configuring IPv4 link-local addresses, 276
 - configuring IPv6 link-local addresses, 276–277
 - deciding whether to use, 244
 - definition of, 957
 - example of, 278–283
 - IPv6 DAD (Duplicate Address Detection), 277–278
 - IPv6 global addresses, 278
 - overview of, 276
 - stateless DHCP and, 283–284
 - utility/benefit of, 284–285
- Sliding window protocol
 - movement of windows, 702–704
 - in packet communication, 582
 - send and receive structures, 701
 - TCP as, 589
- SLLAO (Source Link-Layer Address Option), 409–410, 958
- Slot time, in MAC, 122
- Slow start algorithm, in TCP
 - classic algorithms for TCP congestion, 732–734
 - comparing with congestion avoidance, 736–737
 - limited, 772
 - viewing slow start behavior with Wireshark, 749–750
- slow start threshold. *See* ssthresh (slow start threshold)
- SLR (Server Load Reduction), 486
- Smack attacks, ICMP attacks and, 429
- Smoothed RTT. *See* SRTT (smoothed RTT)
- SMSS (send maximum segment size)
 - definition of, 958
 - SWS (silly windows syndrome) and, 709
 - TCP connections and, 613
- SMTP (Simple Mail Transfer Protocol)
 - definition of, 958
 - MX (mail exchanger) records and, 544
 - SRV record providing SMTP service, 549
 - well-known port for, 18
- Smurf attacks, ICMP, 428
- SNA (System Network Architecture)
 - definition of, 958
 - SDLC in, 131
- Sname (Server Name) field, DHCP/BOOTP message
 - format, 238–239
- SNAP (Subnetwork Access Protocol), 958
- Sniffing, 808
- SNMP (Simple Network Management Protocol)
 - definition of, 958
 - well-known port for, 18
- Snooping
 - DHCP, 276
 - IGMP/MLD, 468–469
- SNS (Symmetric NAT Support), 486
- SOA (start of authority) records
 - definition of, 958
 - DNS resource record types, 541–544
- SOAP (Simple Object Access Protocol)
 - definition of, 958
 - GENA using, 338
- sock program
 - creating UDP datagram, 493, 496
 - generating UDP datagram with, 478–481
 - restricting local IP addresses, 634
- Sockets
 - popular APIs, 22
 - in TCP connections, 595–596
 - TCP ports, 588

- SOCKS proxy firewalls, 302–303
- Soft state
 - ARP cache timeout and, 174
 - multicast information and, 441
- SOLICIT message, DHCPv6, 260, 269
- Solicitation messages, in MRD, 394–395
- Solicitation of servers, by clients, 435
- Sort lists, DNS, 565–567
- Source address selection algorithm, in IP host models, 223–224
- Source IP addresses
 - address selection by hosts, 222–223
 - host processing of IP datagrams, 220–221
 - ICMPv6 errors, 371–372
 - in IP datagrams, 186
- Source Link-Layer Address Option (SLLAO), 409–410, 958
- Source Quench messages, TCP congestion control attacks, 785
- Source/Target Address List options, in ND, 413–414
- SP (Sequential Port-Symmetric NAT), 486
- Spam
 - DNS resource record for fighting, 545–547
 - as malware, 806
- Spanning tree, building, 107
- Spanning Tree Protocol. *See* STP (Spanning Tree Protocol)
- Spatial multiplexing, power save mode, 120
- Spatial streams, higher throughput (802.11n) support, 126
- SPD (security policy database)
 - definition of, 958
 - in IPsec, 841–842
- Spear phishing attacks, 806
- Special-use IP addresses
 - for IPv4, 50–51
 - for IPv6, 51–52
 - local net (limited) broadcast, 43
- SPF (sender policy framework) records, DNS resource record types, 545–547
- SPI (Security Parameter Index)
 - definition of, 958
 - IKE protocol, 844
- Split DNS, 565–567
- SPNAT (service provider NAT)
 - definition of, 958
 - DS-Lite and, 339
 - overview of, 315–316
- Spoofing attacks
 - ICMP, 429
 - Internet architecture, 25
 - IP addresses, 70, 226
 - TCP, 640–642
 - TCP keepalive attacks, 802
 - Spurious association attacks, 808
 - Spurious timeouts and retransmissions, in TCP congestion control and, 744–745
 - DSACK (duplicate SACK) extension, 677–679
 - Eifel Detection Algorithm, 679–680
 - Eifel Response Algorithm, 680–682
 - F-RTO (Forward-RTO Recovery), 680
 - overview of, 677
- SRC (Source) address, in Ethernet frame format, 85
- SRP (Secure Remote Password), 883, 958
- SRTP (Secure Real-Time Protocol), 883, 958
- SRTT (smoothed RTT)
 - classic method of RTT estimation, 651–652
 - definition of, 958
 - destination metrics and, 685–686
- SRV (service) records, DNS resource record types, 548–549
- SSDP (Simple Service Discovery Protocol)
 - definition of, 958
 - direct interaction with NAT and firewalls, 338
 - viewing in-use multicast groups in Windows OSs, 448
- SSH (Secure Shell)
 - for application-managed keepalives, 794
 - definition of, 958
 - TCP data flow and, 692
 - tracing RTT of TCP connection, 697–698
 - well-known port for, 18
 - well-known ports for, 632
- SSID (service set identifiers)
 - definition of, 958
 - Wi-Fi, 112
- SSL (Secure Sockets Layer). *See also* TLS (Transport Layer Security), 876–877, 958
- SSM (source-specific multicast)
 - attacks related to IGMP or MLD, 470
 - definition of, 959
 - IGMP and MLD supporting, 452
 - MLD supporting, 390
 - as multicast service model, 54
- ssthresh (slow start threshold), in TCP congestion control
 - comparing slow start with congestion avoidance, 736
 - Eifel Response Algorithm and, 744–745
 - overview of, 733
 - standard TCP algorithm and, 738
- Standard RTO method, in TCP, 652–654
- Standard TCP congestion control algorithm, 728–739
- Standards
 - IETF (Internet Engineering Task Force) in, 22–23
 - link layer, 80–82
 - other organizations in, 23–24
 - RFC (Request for Comments) and, 23–24

- Standards-defining organizations (SDOs), 23, 957
- Standards-track category, RFCs and, 23
- Start frame delimiter (SFD)
 - clock recovery in Ethernet frames, 84
 - definition of, 957
- Start of authority (SOA) records, in DNS
 - definition of, 958
 - DNS resource record types, 541–544
- STAs (stations), in Wi-Fi
 - definition of, 959
 - Wi-Fi, 112
- State-change records, IGMP/MLD group member -
 - ship reports, 457
- State machine, DHCP, 251–252
- State, storing in connection switches, 5
- State transitions, TCP
 - FIN_WAIT_2 state, 625
 - overview of, 616
 - quiet time concept, 624
 - simultaneous open and close transitions, 625
 - state transition diagrams, 617–618
 - TIME_WAIT state (2MSL), 618–624
- Stateful translation, IPv4/IPv6, 344–345
- Stateless address autoconfiguration. *See* SLAAC (stateless address autoconfiguration)
- Stateless IP/ICMP Translation (SIIT)
 - definition of, 957
 - IPv4/IPv6 translation, 342–344
- Stateless mode, DHCPv6, 283–284
- Static multiplexing, 4
- Station-to-Station (STS) protocol, relation to DH (Diffie-Hellman), 814
- Statistical multiplexing, 4
- STODER, repacketization and, 686
- “Stop and wait” protocol
 - communication protocols and, 581
 - Nagle algorithm and, 697
 - TCP and, 696
- STP (Spanning Tree Protocol), in bridges
 - BPDU structure, 105–107
 - building the spanning tree, 107
 - definition of, 959
 - example of, 107–109
 - handling topology changes, 107
 - overview of, 102–104
 - port states and roles, 104–105
 - RSTP (Rapid Spanning Tree Protocol), 110–111
- Straightforward NAPTR (S-NAPTR)
 - definition of, 958
 - DNS resource record types, 554
- Stream ciphers, symmetric key ciphers, 811
- Stream Control Transmission Protocol. *See* SCTP (Stream Control Transmission Protocol)
- Stretch ACKs, 754–757
- Strong host model, 220
- STS (Station-to-Station) protocol, relation to DH (Diffie-Hellman), 814
- STUN (Session Traversal Utilities for NAT)
 - attributes defined by TURN, 328
 - binding method, 321
 - definition of, 959
 - ICE making use of, 332–334
 - mechanisms, 325–326
 - message formats, 320
 - Teredo servers compared with, 482
- Subdomains, in DNS hierarchy, 514
- Subnet addressing, 36–39
- Subnet broadcast addresses. *See* Broadcast addresses
- Subnet fields, in IP addresses, 37
- Subnet masks
 - overview of, 39–41
 - VLSM (variable-length subnet masks), 41–42
- Subnetwork Access Protocol (SNAP), 105, 958
- Subnetworks, 37
- Switches and bridges
 - attacks on, 155
 - layer 2 relay agents and, 270
 - link layer and, 98–102
 - in small networks, 11–13
 - VLAN, 90
- SWS (silly windows syndrome)
 - definition of, 959
 - example of avoiding, 709–715
 - overview of, 708
 - rules for avoiding, 708–709
- Symmetric key encryption
 - cryptographic algorithms, 809–811
 - KDF (key derivation function) in, 815
- Symmetric NAT Support (SNS), 486
- SYN bit field, TCP header, 589–590
- SYN cookies, in TCP
 - attacks related to window management and, 723
 - TCP-related attacks, 640–641
- SYN floods, TCP-related attacks, 640
- SYN segments, in TCP
 - combined with ACKs (SACK), 607
 - definition of, 959
 - establishing TCP connections and, 602–603
 - MSL (maximum segment life), 610
 - NAT and TCP, 307–308
 - requesting connection to nonexistent TCP port, 626
 - in TCP connections, 596–597
 - TCP header and, 589
 - WSCALE (Window Scale) option and, 608

Synchronous Data Link Control (SDLC)

- based on HDLC, 131

- definition of, 957

SYN_RCVD state, in TCP

- incoming connection queue and, 636

- simultaneous open and close transitions, 625

- TCP state transitions, 618

SYN_SENT state, in TCP

- simultaneous open and close transitions, 625

- TCP state transitions, 618

System configuration options

- attacks related to system configuration, 292

- autoconfiguration. *See* SLAAC (stateless address autoconfiguration)

- DHCP (Dynamic Host Configuration Protocol). *See* DHCP (Dynamic Host Configuration Protocol)

- introduction to, 233–234

- summary and references, 292–298

System Network Architecture (SNA) from IBM

- definition of, 958

- SDLC in, 131

T**T1 (Renewal time), for DHCP messages, 240****Tahoe algorithm, TCP congestion control, 737–738****Tarbits, attacks related to window management, 723****Taylor, Bob, 2****tc program, for packet scheduling and traffic control subsystem in Linux, 752****TC (Topology Change), in BPDUs, 106****TCA (Topology Change Acknowledgment), 106****TCN (topology change notification), 107****TCP-AO (Authentication Option)**

- definition of, 959

- TCP header, 612

TCP congestion control

- active queue management and ECN, 782–785

- attacks related to, 785–786

- BIC (Binary Increase Congestion Control), 772–774

- buffer bloat, 781–782

- classic algorithms for, 730–732

- comparing slow start with congestion avoidance, 736–737

- congestion avoidance algorithm, 734–736

- connection completion and, 766–767

- CTCP (Compound TCP) algorithm, 779–781

- CUBIC, 775–776

- CWV (Congestion Window Validation), 742–744

- delay-based, 777

- example of handling, 745–749

- FAK (forward acknowledgment) and rate halving for, 741–742

- fast retransmit and local congestion, 759–762

- fast retransmit and SACK recovery, 757–759

- FAST TCP algorithm, 778–779

- handling spurious RTOs, 744–745

- in high-speed environments, 770

- HSTCP (HighSpeed TCP), 770–772

- introduction to, 727–728

- limited transmit approach to, 742

- NewReno algorithm for, 739–740

- SACK (selective acknowledgement) for, 740–741

- sender pause and local congestion (event 1), 750–754

- sharing connection state, 767–768

- slow start algorithm, 732–734

- slow start behavior, 749–750

- slowing down TCP senders, 729–730

- standard TCP algorithm, 728–739

- stretch ACKs and recovery from local congestion, 754–757

- summary and references, 786–792

- Tahoe, Reno, and Fast Recovery algorithms, 737–738

- TCPW (TCP Westwood) algorithm, 779

- TFRC (TCP Friendly Rate Control), 768–770

- timeouts, retransmissions, and undoing cwnd changes, 762–766

- Vegas TCP algorithm, 777–778

TCP data flow

- attacks related to window management, 723

- delayed ACK interaction with Nagle algorithm, 699

- delayed ACKs, 695–696

- disabling Nagle algorithm, 699–700

- example of dynamic window size adjustment and flow control, 705–708

- example using urgent mechanism, 720–722

- flow control, 700–701

- interactive communication, 692–695

- introduction to, 691

- large buffers and auto-tuning, 715–719

- Nagle algorithm, 696–698

- sliding window protocol, 701–704

- summary and references, 723–725

- SWS (silly windows syndrome), 708–715

- urgent mechanism, 719–720

- zero windows and TCP persistent timer, 704–705

TCP Friendly Rate Control (TFRC), 768–770, 959**TCP/IP suite**

- ARPANET Reference Model, 13–16

- based on ARPANET, 1

- implementations and distributions, 24–25
- layering, 14
- multiplexing, demultiplexing, and encapsulation, 16–17
- names, addresses, and DNS, 19
- OSI model compared with, 8–9
- overview of, 13
- port numbers, 17–19
- TCP keepalive
 - attacks related to, 802
 - description of, 795–797
 - introduction to, 793–794
 - server host crashes and does not reboot, 797–799
 - server host crashes and reboots, 799–800
 - server host unreachable, 800–802
 - summary and references, 802–803
- TCP segments, 15
- TCP servers
 - incoming connection queue, 636–640
 - overview of, 631–632
 - port numbers and, 632–634
 - restrictions on foreign endpoints, 635–636
 - restrictions on local IP addresses, 634–635
- TCP timeout/retransmission
 - attacks related to, 687
 - classic RTO method, 651–652
 - clock granularity and RTO bounds, 654
 - congestion control and, 762–766
 - connection establishment and, 604–605
 - destination metrics, 685–686
 - DSACK (duplicate SACK) extension, 677–679
 - Eifel Detection Algorithm, 679–680
 - Eifel Response Algorithm, 680–682
 - example of, 648–651
 - example of fast retransmit, 668–671
 - example of retransmission with SACK, 673–676
 - example of timer-based retransmission, 665–667
 - F-RTO (Forward-RTO Recovery), 680
 - fast retransmit, 667–668
 - introduction to, 647–648
 - Linux RTT estimation, 657–661
 - packet duplication, 684–685
 - packet reordering, 682–684
 - repacketization, 686–687
 - retransmission ambiguity and Karn's algorithm, 655
 - retransmission with SACK, 671–672
 - robustness of RTTM, 662–664
 - RTO (retransmission timeout) setting, 651
 - RTT estimation behaviors, 661–662
 - RTTM (RTT Measurement) with Timestamps option, 656–657
 - SACK receiver behavior, 672
 - SACK sender behavior, 673
 - spurious timeouts and retransmissions, 677
 - standard RTO method, 652–654
 - summary and references, 688–690
 - timer-based retransmission, 664–665
- TCP (Transmission Control Protocol)
 - ARQ as basis of, 579
 - connection management. *See* Connections, TCP
 - definition of, 959
 - encapsulation in IP datagrams, 587
 - flow control and, 7–8
 - header fields, 588–590
 - introduction to, 584–585
 - NAT and, 306–308
 - reliability, 586–587
 - service model, 585–586
 - STUN and, 320
 - summary and references, 591–593
 - transport protocols in TCP/IP suite, 15
 - well-known ports for, 525–526
- TCP Westwood+ (TCPW+) algorithm, 777
- TCP Westwood (TCPW) algorithm, 779
- tcpdump command
 - connecting to Web server on host, 171
 - ICMP destination unreachable messages, 480
 - not converting IP addresses to machine names, 479
 - viewing UDP fragmentation, 490–491
- TCP_NODELAY option, for disabling Nagle algorithm, 700
- tcptrace, connection statistics with, 745–747
- TCPW+ (TCP Westwood+) algorithm, 777
- TCPW (TCP Westwood) algorithm, 779
- TDM (time-division multiplexing), 4, 959
- Teardrop attacks
 - ICMP, 428
 - UDP, 506
- telnet command
 - connecting to Web server on host, 171
 - establishing TCP connections, 602
- Telnet program
 - SSH replacing, 692
 - well-known port for, 18
- Temporal Key Integrity Protocol (TKIP), in Wi-Fi, 129–130
- Temporary addresses, in DHCPv6, 255–256
- Tentative state, IPv6 addresses, 253
- Teredo, tunneling IPv6 over IPv4
 - IPv4/IPv6 translation, 339
 - relays and servers, 482
 - tunneling, 154, 482–487

- Translation functions, NAT, 305
 - Translators, TCP connections, 605
 - Transmission Control Protocol. *See* TCP (Transmission Control Protocol)
 - Transmit opportunities (TXOPs), in Wi-Fi QoS
 - in DCF, 123
 - definition of, 960
 - Transparency, DNS, 567–568
 - Transport layer
 - layering violation, 476
 - of OSI model, 9–10
 - security. *See* TLS (Transport Layer Security)
 - transport protocols in TCP/IP suite, 15–16
 - UDP checksum, 475–476
 - Transport Layer Security. *See* TLS (Transport Layer Security)
 - Transport PDU (TPDU), 10
 - Transport protocols, 309
 - Traversal, NAT, 316
 - Traversal Using Relays around NAT. *See* TURN (Traversal Using Relays around NAT)
 - Tribe Flood Network (TFN), 429
 - Triple-DES (3DES)
 - definition of, 959
 - standardized for Internet use, 819
 - as symmetric encryption algorithm, 811
 - Trivial File Transfer Protocol (TFTP)
 - definition of, 959
 - ICMP port unreachable messages and, 366–370
 - Trunking, VLAN switches and, 90
 - Trust anchors
 - CAs (certification authorities) and, 822
 - in ND, 417
 - SEND (Secure Neighbor Discovery), 403
 - TS (traffic selectors), in IPsec
 - definition of, 960
 - IKE, 851, 873
 - TSER (Timestamp Echo Reply), in TCP
 - definition of, 960
 - Eifel Detection Algorithm and, 679
 - TCP Timestamp option and, 609
 - timer-based retransmission and, 665–666
 - TSF (time sync function), in Wi-Fi
 - in 802.11 specification, 119–120
 - definition of, 960
 - Wi-Fi frames and, 114
 - TSIG (Transaction Signatures), in DNS
 - definition of, 960
 - transaction authentication in DNS, 911–914
 - TSKs (transient session keys), 838
 - TSOPT (timestamps option), in TCP
 - definition of, 960
 - Eifel Detection Algorithm using, 679
 - Linux RTT estimation and, 657
 - robustness of RTTM to loss and reordering, 662–664
 - RTTM (RTT Measurement) with, 656–657
 - TCP header, 608–610
 - TSPEC (traffic specification), in Wi-Fi QoS
 - definition of, 960
 - in HCCA, 123
 - TSV (Timestamp Value), in TCP
 - definition of, 960
 - Eifel Detection Algorithm and, 679
 - RTTM with Timestamps option, 656
 - TCP Timestamp option and, 608–609
 - TTL (Time-to-live)
 - definition of, 960
 - ICMP Time Exceeded message, 375, 378
 - IP header fields, 184
 - MRD (Multicast Router Discovery) and, 394
 - name servers, 517
 - QS (Quick-Start) TTL, 199
 - SYN segments, 611
 - Tunnel endpoint, IPv6 traffic and, 46
 - Tunneled packets, NAT and, 310
 - Tunneling
 - IPv4/IPv6 translation, 339
 - IPv6 options for, 198
 - link layer and, 149–153
 - link layer attacks and, 156
 - Tunneling proxy servers, 302
 - TURN (Traversal Using Relays around NAT)
 - definition of, 960
 - ICE making use of, 332–334
 - overview of, 326–332
 - Teredo relays compared with, 482
 - TWA (TIME-WAIT Assassination), 630–631, 960
 - TXOPs (transmit opportunities), in Wi-Fi QoS
 - in DCF, 123
 - definition of, 960
 - TXT records
 - definition of, 960
 - DNS resource record types, 545–547
 - Type field, in Ethernet frame format, 85–86
 - Type-length-value (TLV) sets
 - definition of, 959
 - IPv6 options held as, 196–197
 - Type of Service byte. *See* ToS (Type of Service) byte
- ## U
- U-NAPTR (URI-enabled NAPTR)
 - definition of, 960
 - DNS resource record types, 555

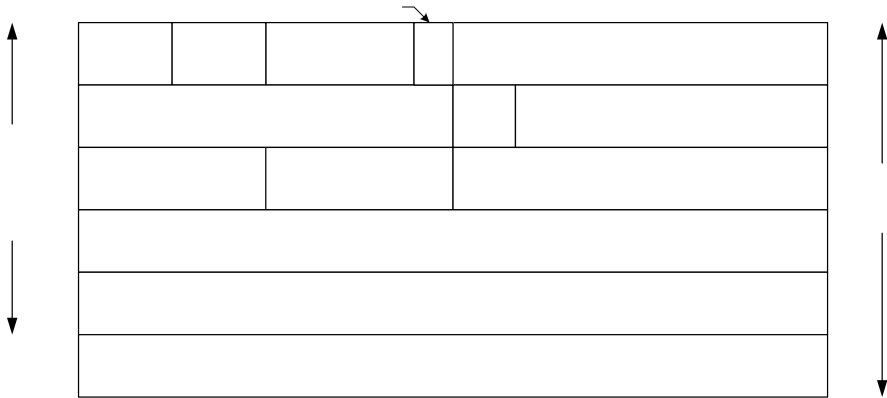
- U-NII (Unlicensed National Information Infrastructure)
 - 5GHz band for, 124
 - definition of, 960
- UBM (unicast-prefix-based multicast)
 - allocation of IPv4 addresses, 56
 - definition of, 960
- UDLs (unidirectional links)
 - definition of, 960
 - link layer and, 153–154
- UDP-Lite, 487–488
- UDP servers
 - designing, 498–499
 - flow control and congestion control in server design, 505
 - foreign IP address restrictions in server design, 502–503
 - IP addresses and port numbers in server design, 499–500
 - local IP address restrictions in server design, 500–501
 - multiple addresses in server design, 501–502
 - multiple servers per port, 503–504
 - spanning IP address families in server design, 504
- UDP (User Datagram Protocol)
 - attacks related to, 507–508
 - broadcast overhead and, 451
 - checksum, 475–478
 - connection refused error, 626
 - as connectionless protocols, 595
 - definition of, 960
 - examples, 478–481
 - flow control and congestion control in server design, 505
 - foreign IP address restrictions in server design, 502–503
 - header, 474–475
 - ICE and, 332
 - interaction between IP fragmentation and ARP/ND, 496–497
 - in the Internet, 506–507
 - introduction to, 473–474
 - IP addresses and port numbers in server design, 499–500
 - IP fragmentation and, 488–492
 - IPv6 and, 481–482
 - local IP address restrictions in server design, 500–501
 - maximum UDP datagram size, 497–498
 - multiple addresses in server design, 501–502
 - multiple servers per port, 503–504
 - NAT and, 308–309
 - PMTUD (Path MTU Discovery) with, 493–496
 - reassembly timeout, 492
 - sending broadcast datagrams, 439
 - server design, 498–499
 - spanning IP address families in server design, 504
 - STUN and, 320
 - summary and references, 508–510
 - Teredo tunneling and, 482–487
 - translating UDP/IPv4 and UDP/IPv6 datagrams, 505–506
 - transport protocols in TCP/IP suite, 15
 - UDP-Lite, 487–488
 - well-known ports for, 525–526
- UEQM (unequal modulation), in 802.11n
 - definition of, 960
 - higher throughput (802.11n) support and, 127
- ULAs (Unique Local IPv6 Unicast Addresses)
 - definition of, 960
 - NAT and, 310
 - overview of, 225
- Unauthorized access attacks, 26
- Unequal modulation (UEQM), in 802.11n
 - definition of, 960
 - higher throughput (802.11n) support and, 127
- Unicast addresses
 - allocation of, 62–65
 - anycast addresses, 62
 - assigning, 65–66
 - C class spaces for, 35
 - definition of, 34
 - Echo Request message sent from link-local unicast address, 445–446
 - IIDs as basis for unicast IPv6 addresses, 43–46
 - multiple providers/multiple networks/multiple addresses, 68–70
 - overview of, 15
 - single provider/multiple networks/multiple addresses, 67–68
 - single provider/no network/single address, 66–67
 - single provider/single network/single address, 67
- Unicast-prefix-based IPv6 multicast addresses, 58
- Unicast-prefix-based multicast (UBM)
 - allocation of IPv4 addresses, 56
 - definition of, 960
- Unicode, internationalization of Internet, 512
- Unidirectional links (UDLs)
 - definition of, 960
 - link layer and, 153–154
- Uniform Resource Locator (URL), 961
- Unilateral self-address fixing. See UNSAF (unilateral self-address fixing)

-
- Unique Local IPv6 Unicast Addresses. *See* ULAs (Unique Local IPv6 Unicast Addresses)
 - Universal Plug and Play (UPnP) framework
 - definition of, 961
 - direct interaction with NAT and firewalls, 337–339
 - Universal Resource Identifier. *See* URI (Universal Resource Identifier)
 - UNIX
 - Berkeley version. *See* BSD (Berkeley Software Distribution)
 - `rlogin`, 692
 - Unlicensed National Information Infrastructure (U-NII)
 - 5GHz band for, 124
 - definition of, 960
 - Unreachable hosts, keepalives detecting, 795–796
 - UNSAF (unilateral self-address fixing)
 - definition of, 961
 - overview of, 317–319
 - STUN (Session Traversal Utilities for NAT), 319–326
 - Updates
 - DNS Update, 567
 - dynamic DNS updates, 555–558
 - UPnP (Universal Plug and Play) framework
 - definition of, 961
 - direct interaction with NAT and firewalls, 337–339
 - Upper layer, TLS (Transport Layer Security), 877
 - UPs (user priorities), in Wi-Fi QoS, 123, 961
 - URG (Urgent Mechanism), in TCP
 - definition of, 961
 - example working with urgent data, 720–722
 - overview of, 719–720
 - TCP header, 590
 - URI-enabled NAPTR (U-NAPTR)
 - definition of, 960
 - DNS resource record types, 555
 - URI (Universal Resource Identifier)
 - definition of, 961
 - ENUM records and, 551–552
 - NAPTR records and, 549
 - URI/URN resolution, 553–554
 - URL (Uniform Resource Locator), 961
 - URN resolution, 553–554
 - Usage-specific keys (USRK), in EAP
 - definition of, 961
 - key derivation in EAP, 838
 - User Datagram Protocol. *See* UDP (User Datagram Protocol)
 - User priorities (UPs), in Wi-Fi QoS, 123, 961
 - User Timeout (UTO) option, in TCP
 - definition of, 961
 - TCP header, 611–612
 - USRK (usage-specific keys)
 - definition of, 961
 - key derivation in EAP, 838
 - UTC (Coordinated Universal Time), 961
 - UTO (User Timeout) option, in TCP
 - definition of, 961
 - TCP header, 611–612
- ## V
- Valid lifetime
 - IA (Identity Association) and, 255
 - IPv6 addresses, 252
 - Validating certificates, 828–831
 - Validating security aware resolver, in DNSSEC, 895
 - Variable-length subnet masks (VLSM), 41–42, 961
 - Variable-scope addresses, IPv6 multicast, 58
 - Variables, IGMP/MLD, 467–468
 - `vconfig` command, for manipulating 802.1p/q information in Linux, 90–91
 - VCs (virtual circuits)
 - definition of, 961
 - multiplexing and, 4
 - Vegas TCP algorithm, 777–778
 - Vendor Extension field, DHCP/BOOTP message format, 238, 246
 - VENONA system, 918
 - Virtual carrier sense, 121
 - Virtual circuits (VCs)
 - definition of, 961
 - multiplexing and, 4
 - Virtual LANs. *See* VLANs (virtual LANs)
 - Virtual private networks. *See* VPNs (virtual private networks)
 - Viruses, 806
 - VJ (Van Jacobson) compression, 141–142
 - VLAN identifier, 90
 - VLAN tag, 90
 - VLANs (virtual LANs)
 - definition of, 961
 - multicast routing, 452
 - overview of, 89–92
 - QoS tagging and, 145–148
 - VLSM (variable-length subnet masks), 41–42, 961
 - VoIP (Voice over IP), 961
 - VPNs (virtual private networks)
 - connecting to Internet via, 20
 - definition of, 961
 - tunneling, 149
- ## W
- W3C (World Wide Web Consortium), 24, 961
 - Wake-on LAN (WoL), 96–97, 962

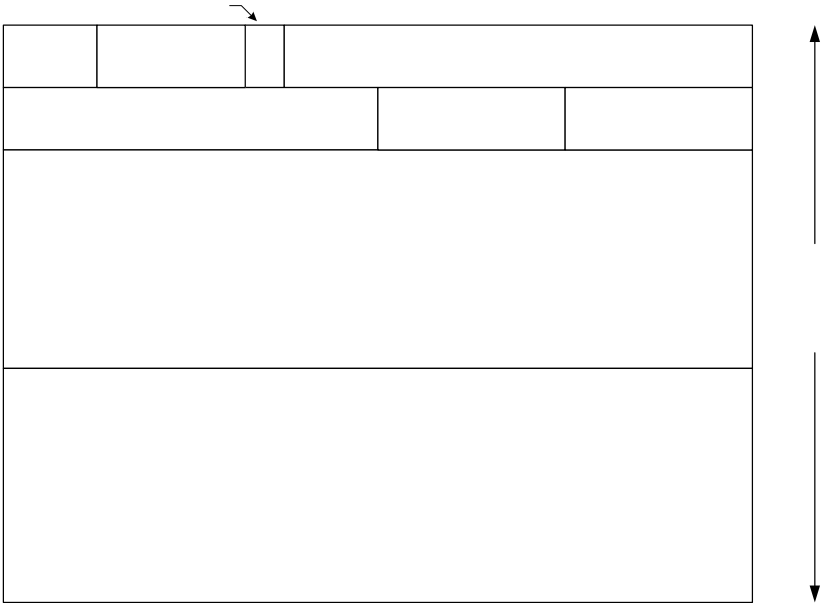
- WANs (wide area networks)
 - definition of, 961
 - Internet as, 2
 - PPPoE and, 286–287
- War driving, link layer attacks, 155
- Weak host model, 220
- Web caches, web proxies operating as, 302
- Web of trust, public key certificates and, 822
- Web proxies, 302
- Web Proxy Auto-Discovery Protocol (WPAD), 302, 962
- Web Proxy Autodiscovery Protocol (WRED), 962
- Weighted RED (WRED), 783, 962
- Well-known ports
 - overview of, 18
 - for SSH (Secure Shell), 632
 - for UDP or TCP, 525–526
- Well-Known Prefix (WKP), in algorithmic address translation, 341, 962
- WEP (wired equivalent privacy) in Wi-Fi
 - attacks related to, 918
 - definition of, 961
 - Wi-Fi attacks and, 155
 - for Wi-Fi security, 129–130
- WESP (Wrapped ESP), in IPsec, 863–864, 961
- White hats, 26
- WHOIS service, 63–64
- Wi-Fi Multimedia (WMM), in Wi-Fi QoS, 122, 962
- Wi-Fi Protected Access. *See* WPA (Wi-Fi Protected Access)
- Wi-Fi (wireless fidelity- IEEE 802.11)
 - attacks, 155
 - attacks related to, 918
 - channels and frequencies (802.11b/g), 124–125
 - control frames, 115–116
 - CSMA/CA and, 84
 - data frames, fragmentation, and aggregation, 116–119
 - definition of, 962
 - frames, 113–115
 - higher throughput (802.11n), 126–128
 - mesh (802.11s), 130
 - overview of, 111–112
 - physical layer, 123–124
 - physical layer channels and frequencies, 124–126
 - power save mode and time sync function, 119–120
 - RSTP (Rapid Spanning Tree Protocol), 120–123
 - security, 129–130
 - throughput (802.11n), 116, 126–128
- Wide area networks. *See* WANs (wide area networks)
- Wildcard (*)
 - domain names and, 526
 - local IP address restrictions in server design, 500–501
- Wildcard address, IPv6 addresses, 632
- WiMAX (Worldwide Interoperability for Microwave Access), 79, 82–83, 962
- Window advertisement (window update)
 - in window-based flow control, 583
 - WSCALE (Window Scale) option in TCP, 608
- Window-based flow control, 583
- Window management, TCP
 - example of dynamic window size adjustment and flow control, 705–708
 - flow control and, 700–701
 - large buffers and auto-tuning, 715–719
 - sliding window protocol, 701–704
 - SWS (silly windows syndrome), 708–715
 - zero windows and TCP persistent timer, 704–705
- Window probes, in TCP window management, 704
- Window Size field
 - cwnd (congestion window), 729
 - flow control and, 727
 - TCP segments, 701
- Window update
 - SWS (silly windows syndrome) and, 711
 - TCP header and, 590
 - window management and, 706
- Windows of packets, in communication protocols, 581–582
- Windows OS (Microsoft)
 - autotuning TCP receive windows, 715
 - ICS (Internet Connection Sharing), 337
 - IPSec implementations, 867
- Wired equivalent privacy. *See* WEP (wired equivalent privacy)
- Wireless fidelity. *See* Wi-Fi (wireless fidelity-802.11)
- Wireless LANs (WLANs). *See also* Wi-Fi (wireless fidelity-802.11), 962
- Wireshark
 - Flow graph, 749
 - monitoring TCP keepalives, 797–798
 - TCP Stream Graph, 707, 747–748
 - TCP ZeroWindowProbe, 710
 - viewing BPDUs with, 109
- WKP (Well-Known Prefix), 341, 962
- WLANs (wireless LANs). *See also* Wi-Fi (wireless fidelity-802.11), 962
- WMM (Wi-Fi Multimedia), in Wi-Fi QoS, 122, 962
- WoL (Wake-on LAN), 96–97, 962
- WOPT. *See* WSOPT (Window Scale Option)
- World Wide Web Consortium (W3C), 24, 961

-
- World Wide Web (WWW)
 - definition of, 962
 - Internet compared to, 2–3
 - Worldwide Interoperability for Microwave Access (WiMAX), 79, 82–83, 962
 - Worms
 - attacks related to window management, 723
 - buffer overflow and, 805
 - types of malware, 806
 - WPA (Wi-Fi Protected Access)
 - attacks related to, 918
 - definition of, 962
 - Wi-Fi attacks and, 155
 - for Wi-Fi security, 129–130
 - WPAD (Web Proxy Auto-Discovery Protocol), 302, 962
 - Wrapped ESP (WESP), in IPsec, 863–864, 961
 - WRED (Weighted RED), 783, 962
 - WSOPT (Window Scale Option)
 - definition of, 962
 - TCP header, 608, 610
 - WWW (World Wide Web)
 - definition of, 962
 - Internet compared to, 2–3
 - X
 - X.25 protocol
 - definition of, 962
 - VCs (virtual circuits) and, 4–5
 - X.509 standard
 - certificate extensions, 827–828
 - file or encoding formats in, 822–823
 - TLS extensions, 883
 - validating and revoking certificates, 828–831
 - viewing preconfigured certificates, 823–826
 - XML (Extensible Markup Language)
 - common use with Web pages, 338
 - definition of, 962
 - XMPP (Extensible Messaging and Presence Protocol), 333, 962
 - Y
 - Yiaddr (Your IP address) field, DHCP/BOOTP message format, 237, 247
 - Your IP address (Yiaddr) field, DHCP/BOOTP message format, 237, 247
 - Z
 - Zero window advertisement
 - example of dynamic window size adjustment and flow control, 705–708
 - large buffers and auto-tuning, 717–719
 - probes, 710
 - SWS (silly windows syndrome) and, 711–713
 - TCP persistent timer and, 704–705
 - Zombie attacks, 26
 - Zone cuts, DNS and DNSSEC, 903
 - Zone enumeration, NSEC chain and, 900
 - Zone signing key (ZSK)
 - definition of, 962
 - DNSSEC, 896, 905
 - Zone transfers, in DNS
 - AXFR (full zone transfer) messages, 559–561
 - DNS notify and, 558–559, 564–565
 - initiating, 525
 - IXFR (incremental zone transfer) messages, 561–563
 - overview of, 517–518
 - Zones, DNS
 - dynamic updates, 555–558
 - overview of, 516–517
 - Zones, DNSSEC, 903
 - ZSK (zone signing key)
 - definition of, 962
 - DNSSEC, 896, 905

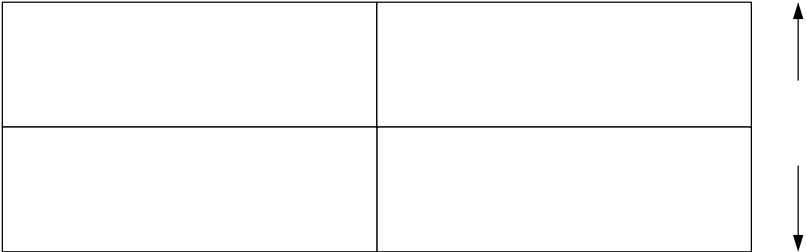
IPv4 Header



IPv6 Header



UDP Header



TCP Header

